Finanzas Computacionales

De los Fundamentos a los Derivados: Un Enfoque Práctico

Diego Onna

29/10/2026

Tabla de contenidos

Bienvenida **3**
Cómo leer este libro 3
Los cuatro módulos 4
¿A quién va dirigido? 5
Convenciones 6

I. Módulo 1: Fundamentos y Renta Fija **7**

Módulo 1: Fundamentos y Renta Fija **8**
Objetivos de Aprendizaje 9
Recorrido Capítulo a Capítulo 11
Herramientas y Librerías 11
Lecturas Recomendadas 12

1. Capítulo 1: Pensamiento Computacional en Finanzas **13**

Introducción **14**
1.1. Concepto de Variable: Gestión de Memoria y Referencias . 14
1.2. Variables y Tipos de Datos Primitivos 16
1.3. Conversión de Tipos (Casting) 19
1.4. Operadores 20
1.5. Entrada y Salida de Datos 21
1.6. Estructuras de Control: `if` / `elif` / `else` 22
1.7. Problemática Financiera: Valor Tiempo del Dinero (TVM) . 24
1.8. Referencias 28

2. Capítulo 2: Iteraciones y Estructuras de Datos Nativas **29**

Introducción **30**
2.1. Bucles `while`: repetir mientras algo sea cierto 31
2.2. Bucles `for` y la función `range()` 36
2.3. Listas: almacenar múltiples valores 37
2.4. Tuplas: la lista que no cambia 45
2.5. Diccionarios: datos etiquetados 47
2.6. Caso integrador: valuar un bono y encontrar su TIR 51
2.7. Errores comunes 56
2.8. Referencias 57

3. Capítulo 3: Funciones, Excepciones y Módulos **58**

Introducción **59**
3.1. Definición de Funciones 59
3.2. Scope (Alcance) de las Variables 64
3.3. Manejo de Errores: `try` / `except` 69
3.4. Módulos y librerías en Python 76
3.5. Problemática Financiera: Librería `finanzas_basicas.py` . 78
3.6. Principio DRY: Don't Repeat Yourself 82
3.7. Errores comunes 83
3.8. Referencias 84

4. Capítulo 4: Programación Orientada a Objetos (POO) **85**

Introducción **86**
4.1. Clases y objetos: la plantilla y sus ejemplares 87
4.2. La clase `Bono`: datos + comportamiento en un solo lugar . . 91
4.3. Interacción y composición: objetos que contienen objetos . . 95
4.4. Herencia: `Bono` → `BonoAmortizable` 99
4.5. Buenas prácticas de diseño 103
4.6. Errores comunes 104
4.7. Referencias 105

5. Capítulo 5: NumPy y Cálculo Vectorizado **106**

Introducción **107**
5.1. El problema: las listas no fueron hechas para calcular 108
5.2. El `ndarray`: qué es y por qué es distinto 110
5.3. Números aleatorios: el generador moderno 113
5.4. Indexado, slicing y máscaras booleanas 115
5.5. Vectorización: pensar en vectores, no en bucles 116
5.6. Reducciones: resumir un array 119
5.7. Broadcasting: operar arrays de distinta forma 121
5.8. Álgebra lineal: la antesala de Markowitz 124
5.9. De NumPy a Pandas 127
5.10. Errores comunes 127
5.11. Referencias 128

6. Capítulo 6: Series de Tiempo y Análisis de Datos con Pandas **129**

Introducción **130**
6.1. Las estructuras: Series y DataFrames 130
6.2. Lectura, Escritura y Alineación de Datos 135
6.3. Selección y Filtrado 138
6.4. Manejo de Fechas y Series Temporales 139
6.5. Limpieza y Transformación de Datos 141
6.6. Operaciones Vectorizadas y Cálculos Financieros 143
6.7. Agrupación (Groupby) 145
6.8. Visualización: cómo graficar (mini-tutorial de Matplotlib) . 147
6.9. Con datos reales: el dataset del libro 155
6.10. Caso de Uso: Construcción de la Curva de Rendimientos . . 159
6.11. Errores comunes 165
6.12. Referencias 165

II. Módulo 2: Teoría de Portafolios y Optimización **166**

Módulo 2: Teoría de Portafolios y Optimización **167**
Objetivos de Aprendizaje 169
Recorrido Capítulo a Capítulo 169
Herramientas y Librerías 170
Concepto Clave: Media-Varianza 170
Lecturas Recomendadas 171

7. Capítulo 7: Estadística Descriptiva y Optimización Matemática **172**

Introducción **173**
7.1. Estadística descriptiva de retornos 174
7.2. Funciones lambda: el pegamento de la optimización 180
7.3. Optimización con `scipy.optimize` 182
7.4. Errores comunes 187
7.5. Practicá en el navegador 188
7.6. Referencias 188

8. Capítulo 8: Teoría Moderna de Portafolios (Markowitz) **189**

Introducción **190**
8.1. El problema en lenguaje matricial 191
8.2. La frontera eficiente 194
8.3. ¿Qué métrica optimizar? Sharpe, Sortino y compañía 201
8.4. Los inputs importan más que el optimizador 203
8.5. Las limitaciones del modelo (y adónde vamos) 209
8.6. Con datos reales 210
8.7. Errores comunes 213
8.8. Practicá en el navegador 214
8.9. Referencias 214

9. Capítulo 9: Hierarchical Risk Parity (HRP) **215**

Introducción: el talón de Aquiles de Markowitz **216**
9.1. Paso 1: Tree clustering — agrupar activos por similitud . . 219
9.2. Paso 2: Quasi-diagonalización — reordenar la matriz 222
9.3. Paso 3: Recursive bisection — repartir el capital por el árbol 224
9.4. HRP vs. Markowitz: la prueba fuera de muestra 228
9.5. Con datos reales 231
9.6. Errores comunes 234
9.7. Practicá en el navegador 235
9.8. Referencias 235

10. Capítulo 10: El Modelo de Black-Litterman **236**

Introducción: por qué Markowitz solo no alcanza **237**
10.1. La intuición: combinar una creencia previa con nueva información 238
10.2. Paso 1: los retornos implícitos de equilibrio (optimización inversa) 240
10.3. Paso 2: expresar las views del inversor 242
10.4. Paso 3: la fórmula maestra de Black-Litterman 244
10.5. Paso 4: del retorno ajustado a la cartera óptima 246
10.6. Visualizando el efecto de la confianza 248
10.7. Recapitulando el flujo 250
10.8. Errores comunes 250
10.9. Practicá en el navegador 252
10.10Referencias 252

III. Módulo 3: Riesgo, Simulación y Econometría **253**

Módulo 3: Riesgo, Simulación y Econometría **254**
Objetivos de Aprendizaje 255
Recorrido Capítulo a Capítulo 257
Herramientas y Librerías 257
Concepto Clave: VaR y Expected Shortfall 258

Lecturas Recomendadas 259

11. Capítulo 11: Valor en Riesgo (VaR) y Expected Shortfall 260

Introducción 261

11.1. Generación de Datos 261
11.2. VaR Paramétrico (Normal) 262
11.3. VaR Histórico 264
11.4. Comparación Visual: Paramétrico vs. Histórico 265
11.5. Expected Shortfall (CVaR): mirar dentro de la cola 267
11.6. Backtesting de VaR: ¿el modelo dice la verdad? 274
11.7. EWMA: cuando la volatilidad tiene memoria 277
11.8. Test de Normalidad 281
11.9. Resumen de Métricas 282
11.10Con datos reales 284
11.11Errores comunes 286
11.12Practicá en el navegador 287
11.13Referencias 288

12. Capítulo 12: Riesgo y Regulación 289

Introducción 290

12.1. Stress testing: preguntar "¿y si…?" 290
12.2. Basilea III: la arquitectura regulatoria 292
12.3. Caso 1 — LTCM: cuando los genios fallan 299
12.4. Caso 2 — 2008: la correlación falsa 303
12.5. Errores comunes 307
12.6. Practicá en el navegador 307
12.7. Referencias 308

13. Capítulo 13: Simulación de Monte Carlo 309

Introducción 310

13.1. El Movimiento Browniano Geométrico (GBM) 311
13.2. VaR y Expected Shortfall por Monte Carlo 317

13.3. ¿Cuántas simulaciones son suficientes? Convergencia 318
13.4. Puente al Módulo 4: valuar una opción por Monte Carlo . . 321
13.5. Errores comunes 324
13.6. Practicá en el navegador 325
13.7. Referencias 325

14. Capítulo 14: Procesos Estocásticos del Precio de un Activo 326

Introducción 327
14.1. El diagnóstico: qué le falta al GBM 328
14.2. Jump-diffusion de Merton: difusión + saltos de Poisson . . 330
14.3. Procesos de Lévy: la familia general 336
14.4. ¿Qué modelo usar? Un mapa 339
14.5. Errores comunes 341
14.6. Practicá en el navegador 341
14.7. Referencias 342

15. Capítulo 15: Econometría y Modelos Lineales en Finanzas 343

Introducción 344
15.1. El modelo lineal: qué afirma exactamente 345
15.2. Aplicación 1: el CAPM y la estimación del beta 347
15.3. Leer una regresión: errores estándar, t y p-values 350
15.4. Los supuestos: cuándo confiar en MCO 354
15.5. Regresión múltiple: más de una explicativa 357
15.6. Con datos reales: los betas del dataset 360
15.7. Errores comunes 362
15.8. Practicá en el navegador 363
15.9. Referencias 363

16. Capítulo 16: Series Temporales — ARIMA y VAR 364

Introducción 365
16.1. Estacionariedad: la condición de entrada 366
16.2. Modelos ARMA: la gramática de una serie estacionaria . . 369

16.3. ARIMA en acción: modelar y pronosticar la inflación 373
16.4. VAR: varias series que se influyen mutuamente 377
16.5. Con datos reales: retornos sin memoria, volatilidad con memoria 381
16.6. Errores comunes 383
16.7. Practicá en el navegador 383
16.8. Referencias 384

17. Capítulo 17: Backtesting de Estrategias **385**

Introducción **386**
17.1. El pecado original: evaluar donde diseñaste 387
17.2. El p-value de una estrategia 390
17.3. Walk-forward: el estándar profesional 393
17.4. Métricas para juzgar el track record OOS 395
17.5. El protocolo, en síntesis 399
17.6. Con datos reales 400
17.7. Errores comunes 403
17.8. Practicá en el navegador 403
17.9. Referencias 404

IV. Módulo 4: Derivados y Volatilidad **405**

Módulo 4: Derivados y Volatilidad **406**
Objetivos de Aprendizaje 408
Recorrido Capítulo a Capítulo 408
Herramientas y Librerías 409
Concepto Clave: Black-Scholes 409
Lecturas Recomendadas 410

18. Capítulo 18: Valuación de Opciones y Volatilidad Implícita **411**

Introducción **412**
18.1. La fórmula de Black-Scholes 414

18.2. Volatilidad implícita: leer la fórmula al revés 418
18.3. La sonrisa de volatilidad 420
18.4. Las griegas: el tablero de control 425
18.5. Errores comunes 429
18.6. Practicá en el navegador 430
18.7. Referencias 430

19. Capítulo 19: Simulación de Cobertura Dinámica (Delta Hedging) 431

Introducción 432
19.1. La mecánica de un día de trading 435
19.2. El error de cobertura: de dónde viene y cómo se achica . . . 440
19.3. Cuando el gamma muerde: cerca del strike, cerca del vencimiento 446
19.4. Errores comunes 448
19.5. Practicá en el navegador 448
19.6. Referencias 449

20. Capítulo 20: Modelos de Volatilidad Estocástica 450

Introducción 451
20.1. La evidencia: la volatilidad no se queda quieta 452
20.2. El modelo de Heston 453
20.3. SABR: el estándar de la industria para la sonrisa 464
20.4. El arco completo del libro 467
20.5. Errores comunes 469
20.6. Practicá en el navegador 469
20.7. Referencias 470

Appendices 471

A. Introducción a Python y Puesta en Marcha 471
A.1. Parte 1: Poner a punto tu entorno 471
A.2. Parte 2: ¿Qué es Python? 474

B. Obtención de Datos Financieros **476**
B.1. El patrón profesional: descargar una vez, trabajar en local . 476
B.2. El dataset del libro: `datos/precios.csv` 477
B.3. yfinance: precios de mercado, gratis 478
B.4. FRED: datos macroeconómicos, gratis 480
B.5. Bloomberg: el estándar institucional 481
B.6. ¿Cuál usar? Comparación y mapa de uso 482
B.7. Referencias 483

Bienvenida

Este es el sitio web del libro **"Finanzas Computacionales con Python"**, desarrollado por la Cátedra de Finanzas Computacionales de la **Universidad Argentina de la Empresa (UADE)**.

La idea que organiza todo el libro es simple: **las finanzas cuantitativas se aprenden construyendo**. Acá no vas a encontrar fórmulas para memorizar, sino problemas reales —valuar un bono, armar un portafolio, medir un riesgo, cubrir una opción— resueltos con código que podés ejecutar, romper y modificar. Cada concepto de programación aparece cuando un problema financiero lo necesita, y cada modelo financiero llega con su implementación completa y con una discusión honesta de sus límites.

Cómo leer este libro

El libro está pensado para leerse **en orden**: cada capítulo usa lo construido en los anteriores, y los cuatro módulos forman un arco que va de la primera variable de Python hasta los modelos de volatilidad estocástica. Los bloques de código son ejecutables — la mejor forma de estudiar es correrlos a medida que leés, ya sea en el propio navegador (los capítulos iniciales lo permiten), en Jupyter local o en Google Colab. Al final de cada capítulo vas a encontrar los **errores comunes** del tema (léelos: son los que vas a cometer) y, en los módulos 2 a 4, una celda para practicar con el código en el propio navegador.

Los cuatro módulos

Módulo 1: Fundamentos y Renta Fija (Capítulos 1–6)

Aprendés a programar desde cero, con problemas financieros como excusa y motor. El arco: variables y decisiones → estructuras y bucles → funciones → objetos → NumPy → Pandas, terminando con una librería de valuación de renta fija y la construcción de curvas de tasas hecha por vos.

1. Pensamiento Computacional en Finanzas
2. Iteraciones y Estructuras de Datos
3. Funciones, Excepciones y Módulos
4. Programación Orientada a Objetos (POO)
5. NumPy y Cálculo Vectorizado
6. Series de Tiempo con Pandas

Módulo 2: Teoría de Portafolios y Optimización (Capítulos 7–10)

La pregunta central de la inversión —¿cómo reparto el capital?— respondida en tres tiempos: el edificio clásico de Markowitz, sus fallas prácticas, y las dos respuestas modernas (HRP y Black-Litterman) que la industria usa para repararlas.

7. Estadística Descriptiva y Optimización Matemática
8. Teoría Moderna de Portafolios (Markowitz)
9. Hierarchical Risk Parity (HRP)
10. El Modelo de Black-Litterman

Módulo 3: Riesgo, Simulación y Econometría (Capítulos 11–17)

El oficio del análisis de riesgo completo: medir (VaR y Expected Shortfall), contextualizar (Basilea, stress testing y las lecciones de LTCM y 2008),

simular (Monte Carlo y procesos con saltos), inferir (modelos lineales y series de tiempo) y validar (backtesting con walk-forward). El módulo donde se aprende a desconfiar de los modelos con método.

11. Valor en Riesgo (VaR) y Expected Shortfall
12. Riesgo y Regulación
13. Simulación de Monte Carlo
14. Procesos Estocásticos del Precio de un Activo
15. Econometría y Modelos Lineales
16. Series Temporales — ARIMA y VAR
17. Backtesting de Estrategias

Módulo 4: Derivados y Volatilidad (Capítulos 18–20)

El examen integrador: los derivados usan todo el libro a la vez. Black-Scholes y la volatilidad implícita, la réplica puesta en práctica con delta hedging, y los modelos de volatilidad estocástica (Heston, SABR) que capturan lo que la sonrisa de volatilidad venía señalando.

18. Valuación de Opciones y Volatilidad Implícita
19. Gestión de Coberturas Dinámicas (Delta Hedging)
20. Modelos de Volatilidad Estocástica

¿A quién va dirigido?

Este libro está diseñado para estudiantes de finanzas, economía y profesionales del sector que deseen transicionar hacia un enfoque más cuantitativo y computacional. No se requiere experiencia previa en programación, ya que empezamos desde los fundamentos.

Convenciones

- Los bloques de código son totalmente ejecutables.
- Las fórmulas se presentan en LaTeX para mayor claridad.
- Usamos **callouts** para resaltar tips, advertencias y conceptos clave.



Recomendamos seguir el libro de forma secuencial, ejecutando los ejemplos dentro del libro, en un entorno local o en Google Colab. En el apéndice encontrás la guía para poner a punto tu entorno.

Este sitio es y será siempre gratuito. Si encontrás algún error o tenés alguna sugerencia, podés abrir un *issue* en nuestro repositorio de GitHub. En el caso que quieras contribuir o sugerir cambios al libro, no dudes en contactarnos.

Parte I.

Módulo 1: Fundamentos y Renta Fija

Módulo 1: Fundamentos y Renta Fija

Aprender a programar resolviendo problemas financieros reales

Todo el mundo dice que "hay que aprender a programar", pero los cursos genéricos tienen un problema: te enseñan a invertir strings y a imprimir triángulos de asteriscos, y después no sabés qué hacer con eso. Este módulo toma el camino contrario: **desde el primer día, cada concepto de programación aparece resolviendo un problema financiero real**. Vas a aprender qué es una variable calculando interés compuesto, qué es un bucle valuando los flujos de un bono, y qué es una clase modelando ese bono como un objeto.

La secuencia de los seis capítulos es una escalera donde cada peldaño resuelve la limitación del anterior. Arrancás con **variables y decisiones** (Capítulo 1): suficiente para calcular el valor tiempo del dinero, pero incómodo apenas hay muchos números. Las **estructuras de datos y los bucles** (Capítulo 2) resuelven eso: ahora podés recorrer los treinta cupones de un bono sin escribir treinta líneas. Pero el código se repite, así que lo encapsulás en **funciones** reutilizables (Capítulo 3), y cuando las funciones y los datos que las acompañan empiezan a viajar juntos por todos lados, los unís en **objetos** (Capítulo 4): la clase `Bono` que sabe valuarse a sí misma.

Los últimos dos peldaños cambian de escala. **NumPy** (Capítulo 5) reemplaza los bucles por operaciones vectorizadas: la diferencia entre esperar horas y esperar segundos cuando los datos son miles. Y **Pandas** (Capítulo 6) le pone nombres y fechas a esos números para trabajar con series de tiempo reales — el pan de cada día del análisis financiero. Al cerrar el módulo vas a haber construido, con tus propias manos, una librería de valuación

de renta fija y una curva de tasas. La Figura 1 resume el recorrido: cada flecha es la limitación que empuja al capítulo siguiente, y en cursiva está lo que construís en cada uno.

> **¿Y si nunca programaste?**
>
> Este módulo no asume ningún conocimiento previo de programación. Lo único que necesitás es paciencia para ejecutar cada bloque de código, romperlo, y volverlo a ejecutar. Programar se aprende con los dedos, no con los ojos.

Objetivos de Aprendizaje

Al completar este módulo, vas a ser capaz de:

1. **Programar en Python** con fluidez: variables, tipos, control de flujo, funciones y clases.
2. **Modelar instrumentos de renta fija** como objetos reutilizables.
3. **Valuar bonos** mediante flujos de fondos descontados y calcular su TIR (y, como extensión, su duración).
4. **Vectorizar cálculos** con NumPy: retornos, matrices de covarianzas, álgebra lineal.
5. **Manipular series de tiempo** con Pandas y construir curvas de tasas desde precios de mercado.

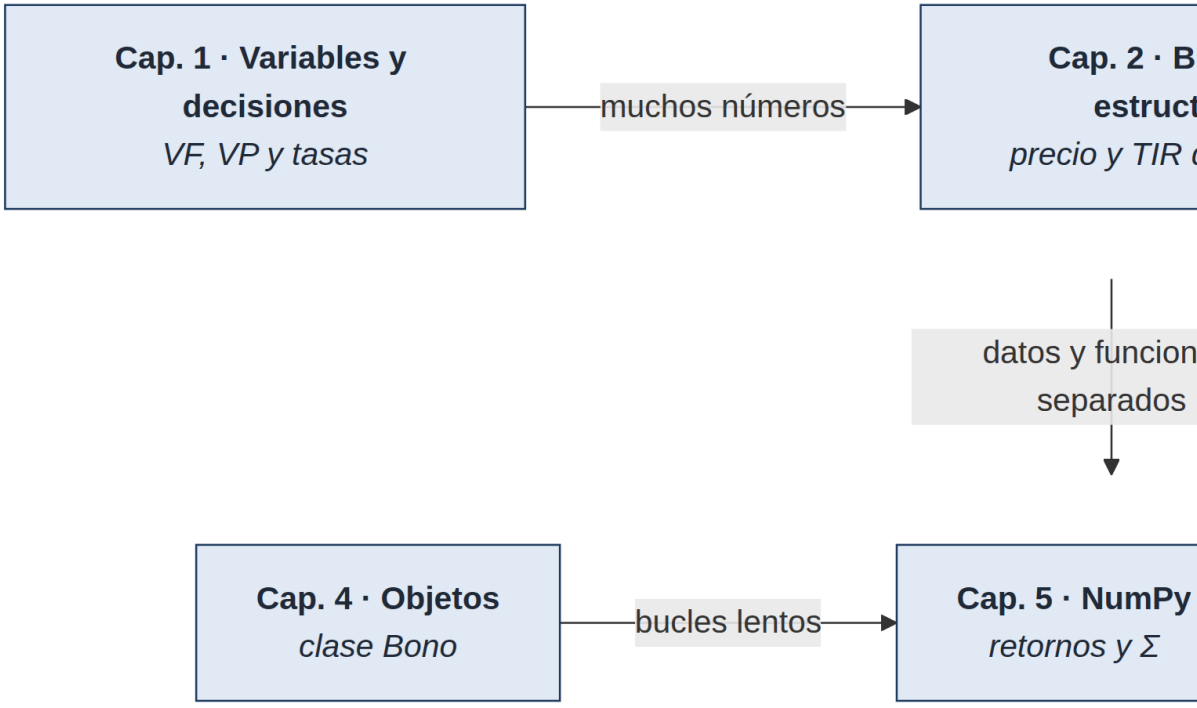


Figura 1.: La escalera del módulo: cada capítulo resuelve la limitación del anterior (flechas) y produce una pieza financiera concreta (en cursiva).

Recorrido Capítulo a Capítulo

Capítulo	Tema	Python	Finanzas
1	Pensamiento Computacional	Variables, tipos, `if/elif/else`	Interés compuesto, TVM
2	Iteraciones y Estructuras	Listas, dicts, `for`, `while`	Valuación de bonos, TIR
3	Funciones y Modularización	`def`, `*args`, `try/except`	Librería `finanzas_basicas.py`
4	Programación Orientada a Objetos	`class`, herencia, `@property`	Clase `Bono`, `BonoAmortizable`
5	NumPy y Cálculo Vectorizado	`ndarray`, vectorización, broadcasting	Retornos y covarianzas vectorizados
6	Series de Tiempo con Pandas	DataFrames, `pct_change()`, `ffill()`	ETTI, bootstrapping, curvas de tasas

---

Herramientas y Librerías

Python 3.11+

Lenguaje base

NumPy

Cálculo vectorizado

Pandas

DataFrames y series

Matplotlib

Visualización

math

Funciones matemáticas

Lecturas Recomendadas

- Benninga, S. (2014). *Financial Modeling*. MIT Press.
- Fabozzi, F. J. (2015). *Bond Markets, Analysis, and Strategies*. Pearson.
- Lutz, M. (2013). *Learning Python*. O'Reilly Media.
- McKinney, W. (2017). *Python for Data Analysis*. O'Reilly Media.

1. Capítulo 1: Pensamiento Computacional en Finanzas

Variables, Tipos de Datos, Estructuras de Control y Valor Tiempo del Dinero

Bases de Python aplicadas a finanzas: variables, tipos primitivos, operadores, condicionales, f-strings, interés compuesto, conversión de tasas y valor presente.

Introducción

En este primer capítulo se establecen las bases del pensamiento computacional y cómo traducir problemas financieros a instrucciones que Python pueda ejecutar. Se desarrollarán conceptos sobre variables, tipos de datos, operadores y estructuras de control, aplicándolos al cálculo de interés compuesto y al análisis de valor presente.

Entorno de Trabajo

Recomendamos usar **Jupyter Notebooks** (local) o **Google Colab** (en la nube). Ambos permiten ejecutar código Python de forma interactiva, celda por celda.

1.1. Concepto de Variable: Gestión de Memoria y Referencias

Es importante no pensar en las variables simplemente como cajas abstractas, sino comprender cómo la computadora administra sus recursos físicos (RAM). Cuando procesamos miles de datos financieros o realizamos millones de iteraciones en una simulación, la gestión de la memoria se vuelve un factor crítico para el rendimiento (latencia y capacidad).

Una variable se define a través de tres componentes fundamentales:

1. **Nombre (Identificador):** Es la etiqueta simbólica que el programador utiliza para referirse al dato (por ejemplo, `precio_bono` o `tasa_libre_riesgo`). Debe ser representativo del concepto financiero que modela.
2. **Espacio en Memoria:** Es la cantidad de bytes que el objeto físico ocupa en la Memoria RAM. Un entero pequeño no ocupa el mismo espacio que una matriz de covarianzas Σ.
3. **Dirección de Memoria:** Es el identificador único (una ubicación física o virtual en la RAM) donde residen los datos del objeto.

En Python, las variables son simples etiquetas (**referencias**) que apuntan a objetos en la memoria. Si reasignamos un objeto inmutable (como un precio representado por un `int` o `float`), Python crea un nuevo objeto en un nuevo espacio de memoria y mueve la etiqueta hacia allí.

```
#| label: concepto-variable-memoria
# Definimos una variable con un precio inicial
precio = 150.50
print(f"Precio inicial: {precio} | Dirección de memoria: {id(precio)}")

# Reasignamos el valor (simulando un cambio de mercado)
precio = 150.75
print(f"Precio nuevo:   {precio} | Dirección de memoria: {id(precio)}")
```

La Figura 1.1 muestra lo que acaba de pasar en memoria: la reasignación no "pisó" el número viejo, sino que **despegó la etiqueta** y la pegó sobre un objeto nuevo. Por eso cambió el `id`.

> **Mutabilidad e Inmutabilidad**
>
> Algunos objetos pueden ser modificados después de su creación (**mutables**, como las listas) y otros no (**inmutables**, como los números y strings). Desarrollaremos este concepto en profundidad al estudiar estructuras de datos más adelante.

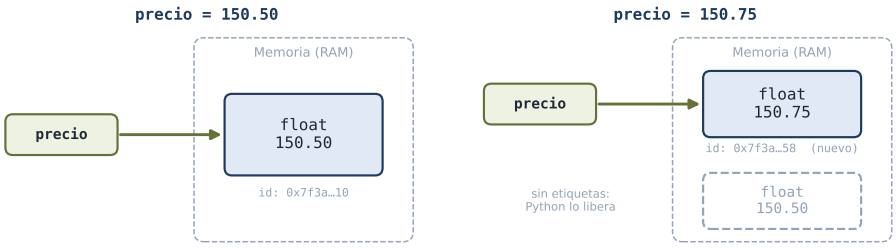


Figura 1.1.: Las variables son etiquetas. Reasignar `precio` no modifica el objeto 150.50: crea uno nuevo (150.75) y mueve la etiqueta. El objeto viejo, sin etiquetas, queda libre para que Python recupere esa memoria.

1.2. Variables y Tipos de Datos Primitivos

Python utiliza **tipado dinámico**: no necesitamos declarar el tipo de dato explícitamente. A continuación, exploraremos algunos de los tipos de datos fundamentales que utilizaremos para modelar instrumentos y variables financieras.

1.2.1. Tipos Numéricos

1.2.1.1. Enteros (`int`)

Representan números del conjunto $\mathbb{Z}$ y tienen precisión arbitraria.

Aplicación Financiera: Cantidades discretas como número de acciones, días al vencimiento, iteraciones de simulación.

```
#| label: tipos-int
# Cantidades discretas típicas de finanzas: no admiten decimales
dias_vencimiento = 365
cantidad_bonos = 100
lotes = 10

print(f"Días al vencimiento: {dias_vencimiento} | Tipo: {type(dias_vencimiento)}")
```

1.2.1.2. Punto Flotante (`float`)

Representan números reales mediante el estándar **IEEE 754**. Es el tipo estándar para precios, tasas y rendimientos — y esconde una sorpresa que en finanzas no podés permitirte ignorar. Ejecutá esto:

```
#| label: tipos-float
# La suma más famosa de la computación
precio_a = 0.1
precio_b = 0.2
resultado = precio_a + precio_b

print(f"0.1 + 0.2 = {resultado}")
print(f"¿Es exactamente 0.3? {resultado == 0.3}")
```

0.1 + 0.2 no da 0.3. No es un bug de Python: es una consecuencia de cómo las computadoras representan los números. Internamente todo se guarda en **binario** (base 2), y así como 1/3 no tiene representación decimal exacta (0.3333… infinito), **0.1 no tiene representación binaria exacta**: la computadora guarda la aproximación más cercana que entra en 64 bits. Los errores son minúsculos (del orden de 10^{-17}), pero existen, se acumulan al operar, y rompen las comparaciones con `==`.

Las dos consecuencias prácticas, que valen para todo el libro:

1. **Nunca compares floats con `==`**: preguntá si están *suficientemente cerca*, con una tolerancia. Python trae `math.isclose()` exactamente para esto (y NumPy, su análogo `np.isclose`).
2. **Para plata contable, no uses `float`**: un sistema que liquida saldos no puede tener centavos fantasma. La librería `decimal` calcula en base 10 exacta (más lenta, pero exacta), o directamente se trabaja con **enteros de centavos**.

```
#| label: float-comparacion
import math
from decimal import Decimal

# Comparación correcta entre floats: con tolerancia
resultado = 0.1 + 0.2
print(f"resultado == 0.3          → {resultado == 0.3}")
print(f"math.isclose(resultado, 0.3) → {math.isclose(resultado, 0.3)}")

# Para contabilidad: decimal calcula en base 10 exacta
suma_decimal = Decimal("0.1") + Decimal("0.2")
print(f"\nDecimal('0.1') + Decimal('0.2') = {suma_decimal}")
print(f"¿Es exactamente 0.3? {suma_decimal == Decimal('0.3')}")
```

> ⚠ La regla práctica: ¿float o decimal?
>
> **`float` para análisis** (retornos, volatilidades, simulaciones, optimización): el error de 10^{-17} es billones de veces menor que la incertidumbre de cualquier modelo financiero, y la velocidad importa. **`decimal` o centavos enteros para contabilidad** (saldos, liquidaciones, facturación): ahí cada centavo debe cerrar exacto. Todo este libro trabaja en el mundo del análisis, así que usaremos `float` — pero sabiendo lo que sacrificamos, y sin comparar jamás con `==`.

1.2.2. Booleanos (`bool`)

Valores lógicos `True` y `False`. Son la base de los algoritmos de decisión.

```
#| label: tipos-bool
mercado_abierto = True
volatilidad_alta = False

# True se comporta como 1, False como 0
señales_activas = True + True + False
print(f"Señales de compra activas: {señales_activas}")
```

1.2.3. Cadenas de Texto (`str`)

Secuencias inmutables de caracteres. Esenciales para tickers, códigos ISIN y limpieza de datos.

```
#| label: tipos-str
ticker = "GGAL"
mercado = "BYMA"
instrumento = f"{ticker}.{mercado}"
print(f"Instrumento: {instrumento}")
```

1.3. Conversión de Tipos (Casting)

Los datos del mundo real frecuentemente llegan como texto (desde CSVs o APIs), y lo mismo pasa con lo que ingresa el usuario. Por eso es necesario convertirlos explícitamente.

```
#| label: casting
# Dato que llega como string desde una fuente externa
volumen_str = "5000"
precio_cierre = 150.50

# Conversión y operación
volumen = int(volumen_str)
monto_operado = volumen * precio_cierre

print(f"Monto operado: ${monto_operado:,.2f}")
```

1.4. Operadores

1.4.1. Operadores Aritméticos

```
#| label: operadores-aritmeticos
capital = 1_000_000
tasa = 0.08

# Operadores básicos
interes = capital * tasa
print(f"Interés simple: ${interes:,.2f}")

# Potencia (**) - útil para interés compuesto
periodos = 5
valor_futuro = capital * (1 + tasa) ** periodos
print(f"Valor futuro a {periodos} años: ${valor_futuro:,.2f}")
```

1.4.2. Operadores de Comparación y Lógicos

Operador	Significado
`==`, `!=`	Igualdad / Desigualdad
`>`, `<`, `>=`, `<=`	Comparaciones
`and`, `or`, `not`	Lógica booleana

```
#| label: operadores-logicos
precio = 150
media_movil = 140
rsi = 65

señal_compra = precio > media_movil and rsi < 70
print(f"¿Señal de compra? {señal_compra}")
```

1.5. Entrada y Salida de Datos

1.5.1. F-Strings (Formato Moderno)

```
#| label: f-strings
tasa = 0.05238
capital = 1_000_000
interes = capital * tasa

# Formatos: :.2% (porcentaje), :, (miles), :,.2f (decimal con miles)
print(f"Tasa: {tasa:.2%}")
```

```
print(f"Capital: ${capital:,}")
print(f"Interés generado: ${interes:,.2f}")
```

1.5.2. Input del Usuario

> **Nota**
>
> `input()` siempre devuelve un `str`. Debe combinarse con `float()` o `int()` para obtener números.

```
# Ejemplo interactivo (no se ejecuta en renderizado estático)
# tasa_input = float(input("Ingrese la tasa anual (%): ")) / 100
```

1.6. Estructuras de Control: `if` / `elif` / `else`

Los algoritmos financieros requieren tomar decisiones basadas en condiciones de mercado.

Antes de leer el código, mirá su forma en la Figura 1.2: las condiciones se evalúan **en orden, de arriba hacia abajo**, y apenas una se cumple se ejecuta su rama y se ignora el resto. El `else` es la red de seguridad: atrapa todo lo que no cumplió ninguna condición.

1.6.1. Caso Práctico: Algoritmo de Decisión de Inversión

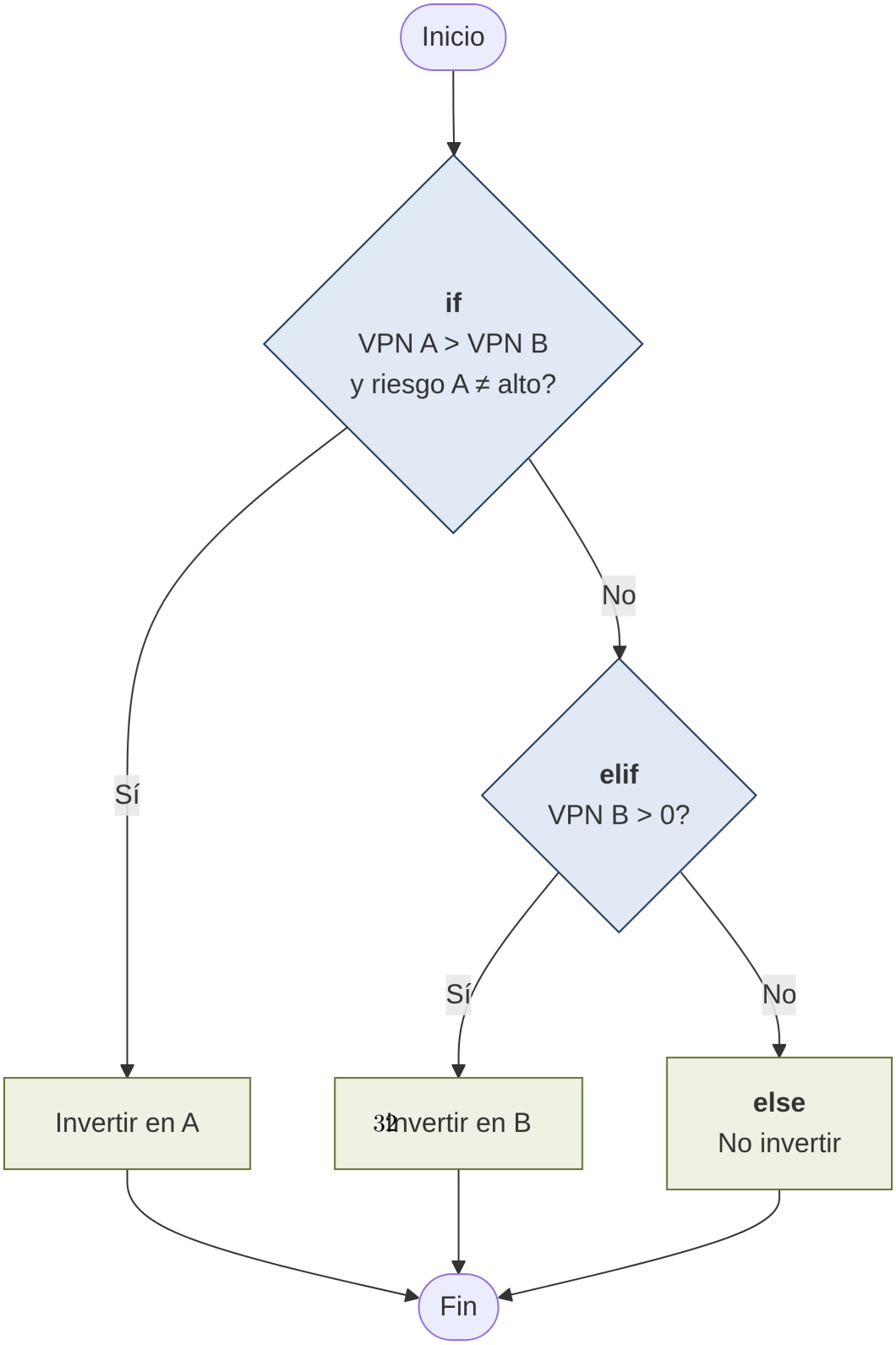


Figura 1.2.: El algoritmo de decisión como diagrama de flujo: `if`, `elif` y `else` se evalúan en orden y solo se ejecuta la primera rama cuya condición es verdadera.

```
#| label: decision-inversion
# --- Parámetros de dos proyectos ---
vpn_proyecto_a = 150_000
vpn_proyecto_b = 120_000
riesgo_a = "alto"
riesgo_b = "bajo"

print("=" * 50)
print("ALGORITMO DE DECISIÓN DE INVERSIÓN")
print("=" * 50)

# La decisión combina rentabilidad (VPN) y tolerancia al riesgo.
# Fijate qué pasaría si ambos VPN fueran negativos: cae al else.
if vpn_proyecto_a > vpn_proyecto_b and riesgo_a != "alto":
    print(f"→ Invertir en Proyecto A (VPN: ${vpn_proyecto_a:,})")
elif vpn_proyecto_b > 0:
    print(f"→ Invertir en Proyecto B (VPN: ${vpn_proyecto_b:,}, riesgo {riesgo_b})")
else:
    print("→ No invertir en ningún proyecto")
```

1.7. Problemática Financiera: Valor Tiempo del Dinero (TVM)

Todo el capítulo gira alrededor de una idea: **un peso hoy no vale lo mismo que un peso mañana**. La Figura 1.3 la resume en una línea de tiempo: capitalizar es llevar plata hacia el futuro, y descontar es traerla al presente. Las tres secciones que siguen son variaciones de esas dos flechas.

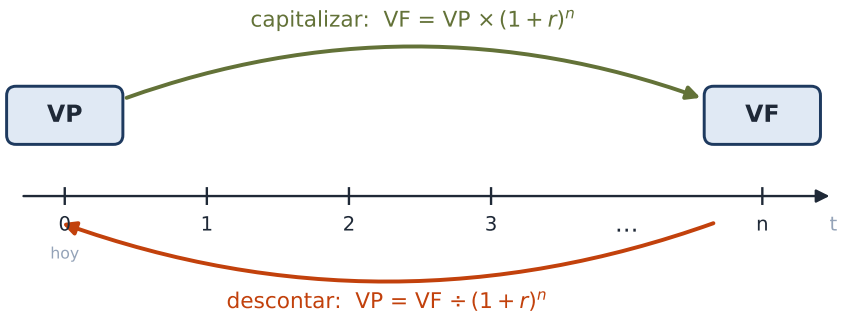


Figura 1.3.: Valor tiempo del dinero: capitalizar lleva un monto de hoy (VP) al período n multiplicando por (1+r)^n; descontar hace el camino inverso, dividiendo.

1.7.1. Interés Compuesto

La fórmula fundamental:

$$VF = VP \cdot (1 + r)^n$$

Donde:

- VF: Valor Futuro
- VP: Valor Presente
- r: Tasa de interés por período
- n: Número de períodos

```
#| label: interes-compuesto
# Cada año, los intereses se reinvierten y generan nuevos intereses
capital_inicial = 100_000
tasa_anual = 0.10
años = 10
```

```
valor_futuro = capital_inicial * (1 + tasa_anual) ** años
interes_ganado = valor_futuro - capital_inicial

print(f"Capital inicial:  ${capital_inicial:,.2f}")
print(f"Tasa anual:       {tasa_anual:.2%}")
print(f"Plazo:            {años} años")
print(f"Valor futuro:     ${valor_futuro:,.2f}")
print(f"Interés ganado:   ${interes_ganado:,.2f}")
```

1.7.2. Conversión de Tasas

```
#| label: conversion-tasas
# Convertimos entre las formas de expresar una misma tasa: TNA, TEA y continua
tna = 0.48  # Tasa Nominal Anual
n = 12      # capitalización mensual

# TNA a TEA
tea = (1 + tna / n) ** n - 1
print(f"TNA: {tna:.2%} → TEA: {tea:.2%}")

# TEA a tasa continua
import math
tasa_continua = math.log(1 + tea)
print(f"TEA: {tea:.2%} → Tasa continua: {tasa_continua:.4f}")
```

Las tres tasas describen **el mismo rendimiento** expresado de formas distintas. La Figura 1.4 muestra el mapa de conversiones: conviene tenerlo a mano, porque en Argentina las tasas se publican como TNA y casi todos los modelos piden TEA o tasa continua.

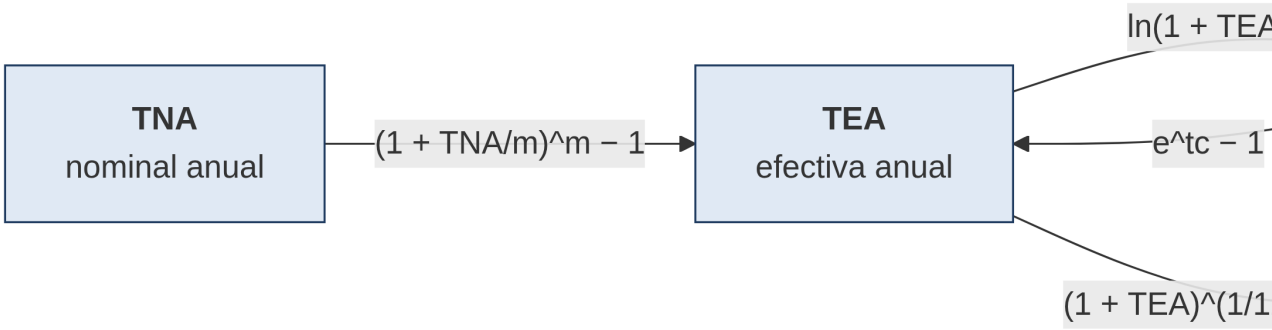


Figura 1.4.: Mapa de conversión de tasas. Todas expresan el mismo rendimiento; la fórmula de cada flecha pasa de una forma a la otra (m = capitalizaciones por año).

1.7.3. Valor Presente

$$VP = \frac{VF}{(1+r)^n}$$

```
#| label: valor-presente
# El valor presente "trae a hoy" un flujo que cobraremos en el futuro
flujo_futuro = 1_000_000
tasa = 0.08
años = 5

valor_presente = flujo_futuro / (1 + tasa) ** años
print(f"Recibir ${flujo_futuro:,} en {años} años")
print(f"equivale a ${valor_presente:,.2f} hoy (al {tasa:.0%})")
```

1.8. Referencias

2. Capítulo 2: Iteraciones y Estructuras de Datos Nativas

Bucles, listas y diccionarios aplicados a la valuación de bonos

Los problemas financieros reales involucran muchos números: treinta cupones de un bono, doscientas ruedas de cotización, una cartera de cincuenta activos. Este capítulo construye las dos herramientas que lo hacen manejable: los bucles (repetir sin copiar y pegar) y las estructuras de datos (listas y diccionarios para organizar colecciones). Cierra con el primer caso integrador del libro: valuar un bono y encontrar su TIR usando solo lo aprendido.

Introducción

En el capítulo anterior calculaste el interés compuesto de una inversión y el valor presente de un flujo. Pero fijate qué pasa si el problema crece un poco: un bono a 10 años paga 20 cupones semestrales — ¿vas a escribir 20 veces la fórmula de descuento, una por cupón? ¿Y si mañana el bono es a 30 años?

La programación empieza a rendir de verdad cuando aparece la **repetición**: le explicás a la computadora la operación *una* vez y ella la ejecuta mil. Para eso necesitamos dos cosas que este capítulo desarrolla en tándem:

- **Estructuras iterativas** (bucles `while` y `for`): repetir instrucciones de forma controlada.
- **Estructuras de datos** (listas y diccionarios): guardar colecciones de valores en una sola variable, para que los bucles tengan qué recorrer.

Al final del capítulo vas a combinar todo en el primer caso integrador del libro: **valuar un bono y encontrar su TIR** — el problema que en la práctica resuelve una calculadora financiera, reconstruido por vos desde cero.

2.1. Bucles `while`: repetir mientras algo sea cierto

El bucle `while` repite un bloque de instrucciones **mientras una condición sea verdadera**. Se lo llama *bucle indefinido* porque no sabemos de antemano cuántas vueltas va a dar: eso lo decide la condición.

¿Cuándo es la herramienta correcta? Cuando la pregunta es del tipo *"¿hasta cuándo…?"* o *"¿cuántas veces hace falta…?"*. Un ejemplo financiero clásico: **¿cuántos meses tengo que ahorrar para llegar a mi objetivo?** No sabés la respuesta de antemano — justamente es lo que querés calcular.

Antes del código, dos patrones que vas a usar toda la vida:

- **Contador:** una variable que suma un valor constante en cada vuelta (`meses = meses + 1`). Cuenta *cuántas veces* pasó algo.
- **Acumulador:** una variable que suma un valor que puede variar en cada vuelta. Acumula *cuánto* se juntó — capital, flujos, intereses.

```
#| label: while-ahorro
# ¿Cuántos meses de ahorrar $150 necesito para juntar $1000? (sin interés)
ahorro_mensual = 150
objetivo = 1000

capital_acumulado = 0  # ACUMULADOR: junta plata
meses = 0              # CONTADOR: cuenta vueltas

print("--- Inicio del ahorro ---")
while capital_acumulado < objetivo:
    capital_acumulado = capital_acumulado + ahorro_mensual
    meses = meses + 1
    print(f"Mes {meses}: Acumulado = ${capital_acumulado}")

print(f"Tomó {meses} meses alcanzar el objetivo de ${objetivo}.")
```

Leé la salida con atención: el bucle dio 7 vueltas y **paró solo**, cuando la condición `capital_acumulado < objetivo` dejó de ser cierta (1050 ya no es menor que 1000). Nadie le dijo "repetí 7 veces" — la condición lo decidió. Notá también que el resultado final (1050) *supera* el objetivo: el `while` verifica la condición **antes** de cada vuelta, así que la última suma se ejecuta completa. Este detalle —¿el bucle corta antes o después de pasarse?— es el tipo de pregunta que siempre conviene hacerse.

> ⚠ El bucle infinito: el error clásico del `while`
>
> Si la condición nunca se vuelve falsa, el bucle no termina jamás y el programa se cuelga. Probá mentalmente qué pasaría si olvidaras la línea `capital_acumulado = capital_acumulado + ahorro_mensual`: el capital quedaría en 0 para siempre, y la condición `0 < 1000` sería eternamente cierta. **Regla práctica**: antes de escribir un `while`, identificá qué línea de adentro acerca la condición hacia `False`.

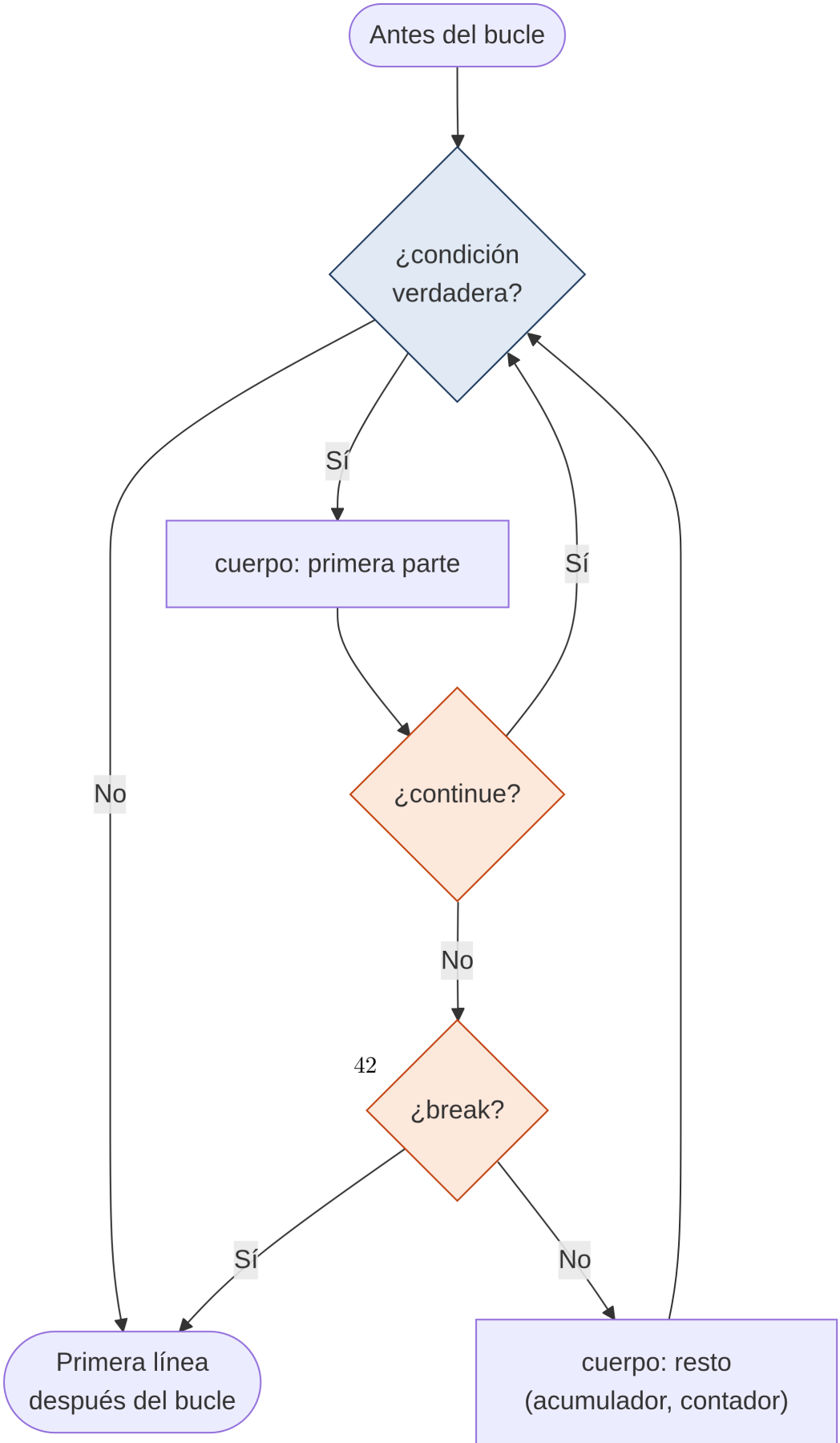
2.1.1. Control de flujo: `break` y `continue`

A veces necesitamos alterar el recorrido normal de un bucle:

- **`break`**: termina el bucle inmediatamente y sigue en la primera línea después del bloque. Útil cuando ya encontraste lo que buscabas — seguir iterando sería perder tiempo.
- **`continue`**: abandona la vuelta actual y salta directo a la siguiente. Útil para **ignorar** casos que no te interesan sin frenar el proceso.

La Figura 2.1 resume el recorrido de un `while` con sus dos desvíos. Fijate que `break` y `continue` hacen cosas opuestas: uno **sale** del bucle, el otro **vuelve** a la condición salteando lo que falta de la vuelta.

Un uso típico de `break` en finanzas: monitorear precios y **cortar apenas se dispara una alerta**. (Por ahora pensá `precios` como una secuencia


Antes del bucle
¿condición verdadera?
Sí
cuerpo: primera parte
Sí
¿continue?
No
No
¿break?
Sí
No
Primera línea después del bucle
cuerpo: resto (acumulador, contador)

de valores a la que se accede por posición con `precios[indice]`; las listas se formalizan más abajo en este capítulo.)

```
#| label: break-ejemplo
precio_objetivo = 120
precios = [110, 115, 122, 118, 125]
indice = 0

print("Iniciando monitoreo de precios...")
while indice < len(precios):
    precio_actual = precios[indice]
    if precio_actual > precio_objetivo:
        print(f"¡Alerta! Precio de ${precio_actual} detectado. Deteniendo proceso.")
        break  # sale del bucle: ya no hace falta mirar el resto

    print(f"Precio verificado: ${precio_actual} (Normal)")
    indice = indice + 1

print("Fin del monitoreo.")
```

Fijate que los precios 118 y 125 **nunca se revisaron**: el `break` cortó en 122, el primer valor que superó el umbral. Eso es exactamente lo que queríamos — y en una serie de un millón de precios, la diferencia de eficiencia importa.

Y el uso típico de `continue`: **saltear datos inválidos**. Los datos financieros reales vienen sucios (huecos, ceros por errores de carga, feriados), y procesarlos sin filtrar arruina los cálculos.

```
#| label: continue-ejemplo
# Supongamos que 0 representa un error en la carga del dato
precios_con_error = [105.0, 0, 106.5, 107.2, 0, 108.0]
suma = 0
contador_validos = 0
```

```
i = 0

while i < len(precios_con_error):
    p = precios_con_error[i]
    i = i + 1

    if p == 0:
        print("Dato inválido detectado. Saltando...")
        continue  # abandona ESTA vuelta; el bucle sigue con la próxima

    suma = suma + p
    contador_validos = contador_validos + 1

promedio = suma / contador_validos
print(f"El promedio de los {contador_validos} precios válidos es: ${promedio:.2f}")
```

El detalle importante está en el denominador: dividimos por `contador_validos` (4), **no** por la longitud de la lista (6). Si hubiéramos dividido por 6, los dos ceros habrían "diluido" el promedio hacia abajo — un error silencioso que produce un número con aspecto razonable pero incorrecto. Contar por separado los datos válidos es el antídoto.

> ¿Por qué incrementamos `i` ANTES del `continue`?
>
> Mirá el orden de las líneas: `i = i + 1` está *antes* del `if` con `continue`. Si estuviera después, al saltear un dato inválido el índice no avanzaría, el bucle volvería a leer el mismo cero… infinitamente. Es la versión sutil del bucle infinito: cuando uses `continue` dentro de un `while`, verificá que el avance del índice ocurra siempre.

2.2. Bucles `for` y la función `range()`

Cuando sabés de antemano **cuántas vueltas** hay que dar —"para cada uno de los 4 años", "para cada precio de la lista"— la herramienta natural es el bucle `for`. Recorre uno por uno los elementos de una secuencia, sin que tengas que manejar índices ni condiciones de corte a mano: menos código, menos lugares donde equivocarse.

La función `range()` genera secuencias de números para iterar: `range(1, 5)` produce 1, 2, 3, 4 (el extremo derecho **no** se incluye — una convención que vas a reencontrar en todo Python, slicing incluido).

```
#| label: for-interes-simple
# Proyección de un capital con interés SIMPLE:
# $500 al 5% anual durante 4 años. Los intereses NO se reinvierten.
capital_inicial = 500
tasa_anual = 0.05
anios = 4

print(f"Capital inicial: ${capital_inicial}")

# range(1, anios + 1) genera: 1, 2, 3, 4
for anio in range(1, anios + 1):
    interes_generado = capital_inicial * tasa_anual
    # En interés simple, el capital crece linealmente
    capital_total = capital_inicial + (interes_generado * anio)
    print(f"Fin del Año {anio}: El capital total es ${capital_total}")
```

Observá el patrón de la salida: el capital crece **$25 por año, siempre lo mismo** (525, 550, 575, 600). Es una recta. Compará mentalmente con el interés *compuesto* del Capítulo 1, donde cada año los intereses generan nuevos intereses y el crecimiento se acelera. Con estos números la diferencia a 4 años es chica ($600 vs ~$607.75), pero a 30 años es enorme ($1250 vs

~$2161): la diferencia entre una recta y una exponencial. Que puedas *ver* esa diferencia recorriendo año por año es exactamente el poder del bucle.

2.3. Listas: almacenar múltiples valores

Hasta acá, cada variable guardaba **un** valor. Pero "los precios de la última semana" no son un valor: son una colección. La **lista** es la estructura de Python para guardar varios elementos en una sola variable, **manteniendo el orden** — y en finanzas el orden casi siempre es tiempo, así que esto importa.

Lo esencial: se accede a los elementos por su **índice** (que arranca en 0), y se modifican con métodos como `.append()` (agregar al final), `.pop()` (quitar) o `del`.

El primer patrón de uso es el **filtrado**: recorrer una lista y quedarse solo con los elementos que cumplen una condición.

```
#| label: listas-filtrado
# Filtrado: quedarnos solo con los días en que el tipo de cambio superó $1000
cotizaciones = [980.5, 995.0, 1005.2, 1010.0, 998.0, 1020.5]
cotizaciones_altas = []  # lista vacía que iremos llenando

for precio in cotizaciones:
    if precio > 1000:
        cotizaciones_altas.append(precio)  # agrega al final

print("Cotizaciones originales:", cotizaciones)
print("Cotizaciones altas (mayores a 1000):", cotizaciones_altas)
```

El patrón completo —lista vacía + `for` + `if` + `append`— es probablemente el fragmento de código más repetido de todo el análisis de datos. De 6 cotizaciones sobrevivieron 3: las que superaron el umbral, en su orden original.

2.3.1. Comprensiones de listas: el mismo patrón, en una línea

Ese patrón es tan frecuente que Python le dedicó una sintaxis propia: la **comprensión de listas** (*list comprehension*). Dice lo mismo que el bucle, pero en el orden en que lo contarías en voz alta: *"quiero **el precio**, **para cada precio** de las cotizaciones, **si** supera 1000"*:

```
#| label: list-comprehension
cotizaciones = [980.5, 995.0, 1005.2, 1010.0, 998.0, 1020.5]

# Las cuatro líneas del filtrado anterior, en una:
cotizaciones_altas = [precio for precio in cotizaciones if precio > 1000]
print("Filtrar:      ", cotizaciones_altas)

# También sirve para TRANSFORMAR cada elemento (sin el if):
en_dolares = [precio / 1000 for precio in cotizaciones]
print("Transformar:  ", en_dolares)

# Y para las dos cosas a la vez: transformar lo que pasa el filtro
altas_en_usd = [precio / 1000 for precio in cotizaciones if precio > 1000]
print("Ambas:        ", altas_en_usd)
```

La regla de traducción mental: `[expresión for elemento in secuencia if condición]` equivale exactamente al bucle con `append` — la *expresión* es lo que se agrega, el *if* final es el filtro (y es opcional). Existe la versión para diccionarios con la misma lógica: `{clave: valor for ...}`.

¿Comprensión o bucle explícito?

Usá la comprensión cuando la operación se lee de un vistazo: filtrar, transformar, o ambas. Cuando la lógica crece —varios `if`, acumuladores, `break`— **volvé al bucle explícito**: una comprensión de tres renglones con condiciones anidadas es más corta pero mucho más difícil de leer, y la claridad siempre gana. Vas a ver comprensiones por todo el resto del libro; ahora ya sabés leerlas.

El segundo patrón es la **operación secuencial**: comparar cada elemento con el anterior. Es la esencia del análisis de series de tiempo — calcular variaciones, retornos, diferencias.

```
#| label: listas-variaciones
# ¿Cuánto varió el precio cada día respecto del anterior?
precios_accion = [50.0, 52.0, 51.0, 55.0]
variaciones = []

# Iteramos por índice. Llegamos hasta len() - 1 porque el último día
# no tiene un "día siguiente" con quien compararse.
for i in range(len(precios_accion) - 1):
    precio_hoy = precios_accion[i]
    precio_manana = precios_accion[i + 1]
    variaciones.append(precio_manana - precio_hoy)

print("Precios diarios:", precios_accion)
print("Variaciones absolutas diarias:", variaciones)
```

Dos lecturas del resultado. Primero, el contenido: la acción subió $2, bajó $1, subió $4 — con 4 precios obtenemos **3 variaciones**, siempre una menos (no hay variación "del primer día"). Segundo, el detalle técnico del `len() - 1`: si el rango llegara hasta el final, `precios_accion[i + 1]` intentaría leer un elemento que no existe y el programa explotaría con `IndexError`.

Este desfasaje entre una serie y sus diferencias te va a perseguir toda la carrera — en el Capítulo 6 lo va a manejar Pandas automáticamente con `pct_change()`, pero ahora sabés qué hace por dentro.

2.3.2. Listas anidadas: representando matrices

¿Y si los datos tienen **dos dimensiones**? Los precios de *varios* activos a lo largo de *varios* días forman una tabla: filas y columnas. La versión nativa de Python es la **lista de listas** — cada elemento de la lista principal es, a su vez, una lista que representa una fila.

```
#| label: listas-anidadas-matriz
# Matriz de precios: 3 activos (filas) x 3 días (columnas)
precios_activos = [
    [100.5, 101.2, 100.8],  # Activo A
    [45.0, 45.5, 46.2],     # Activo B
    [210.3, 209.5, 211.0]   # Activo C
]

print("Matriz completa de precios:")
for fila in precios_activos:
    print(fila)

# Acceso con dos índices: [fila][columna]
precio_b_dia2 = precios_activos[1][1]
print(f"\nPrecio del Activo B en el Día 2: ${precio_b_dia2}")

# Recorrido completo con bucles anidados
print("\nAcceso individual a cada precio:")
for i in range(len(precios_activos)):
    for j in range(len(precios_activos[i])):
        print(f"Activo {i}, Día {j}: ${precios_activos[i][j]}")
```

El doble índice `[1][1]` se lee de afuera hacia adentro: "de la lista principal, dame la fila 1 (Activo B); de esa fila, dame el elemento 1 (Día 2)". Y el **bucle anidado** —un `for` adentro de otro— recorre la tabla completa: por cada fila, todos sus elementos.

> **Spoiler del Capítulo 5**
>
> Las listas anidadas funcionan, pero se vuelven lentas e incómodas con datos grandes (probá calcular el promedio de cada columna: son varios renglones de bucles). NumPy resuelve exactamente este problema con el `ndarray`, donde esa operación es una línea. Cuando llegues ahí, vas a apreciar lo que la abstracción te ahorra — porque ya escribiste la versión manual.

2.3.3. Copiar no es lo mismo que apuntar

Es hora de cumplir una promesa del Capítulo 1. Ahí vimos que las variables de Python son **etiquetas** que apuntan a objetos en memoria, y que con los números (inmutables) eso no trae sorpresas: reasignar crea un objeto nuevo. Pero las listas son **mutables** —se pueden modificar sin crear un objeto nuevo— y ahí la historia cambia. Mirá este código y tratá de predecir la salida antes de ejecutarlo:

```
#| label: aliasing-listas
cartera_a = [100.0, 250.0, 80.0]
cartera_b = cartera_a          # ¿esto copia la cartera?

cartera_b.append(500.0)        # agregamos un activo a la "copia"

print("Cartera A:", cartera_a)
print("Cartera B:", cartera_b)
print(f"\n¿Son el MISMO objeto? id(a)={id(cartera_a)} | id(b)={id(cartera_b)}")
```

Sorpresa: agregamos un activo a `cartera_b`… **y apareció también en `cartera_a`**. El `id()` revela por qué: las dos variables apuntan a la **misma lista** en memoria. La línea `cartera_b = cartera_a` no copió nada — solo pegó una segunda etiqueta sobre el mismo objeto (a esto se lo llama *aliasing*). Es probablemente el bug más desconcertante que vas a encontrar en tus primeros meses de Python: "modifiqué una variable y cambió otra".

Para obtener una copia **de verdad** —un objeto nuevo e independiente— hay que pedirla explícitamente con `.copy()`:

```
#| label: copia-listas
cartera_original = [100.0, 250.0, 80.0]
cartera_escenario = cartera_original.copy()   # copia real: objeto nuevo

# Simulamos un escenario de estrés sobre la copia: todo cae 20%
for i in range(len(cartera_escenario)):
    cartera_escenario[i] = cartera_escenario[i] * 0.80

print("Original:  ", cartera_original)
print("Escenario: ", cartera_escenario)
print(f"\n¿Mismo objeto? {cartera_original is cartera_escenario}")
```

Ahora sí: el escenario de estrés vive en su propia lista y el original quedó intacto — que es exactamente lo que querés cuando simulás escenarios sobre una cartera.

Aprovechemos ese `is` que apareció en el último `print`, porque formaliza la distinción de toda esta sección. Python tiene **dos preguntas de comparación distintas**: `==` pregunta *"¿tienen el mismo contenido?"* (igualdad) e `is` pregunta *"¿son el mismo objeto en memoria?"* (identidad — equivale a comparar sus `id()`). Dos carteras recién copiadas dan `True` con `==` (mismos valores) y `False` con `is` (objetos distintos). La regla de uso: **para comparar valores, siempre `==`**; `is` se reserva para preguntar por

identidad — en la práctica, casi únicamente en `if x is None`, que es la forma idiomática de chequear "ausencia de valor".

Falta una vuelta de tuerca: ¿qué pasa con las **listas anidadas**? `.copy()` hace una copia **superficial** (*shallow*): copia la lista exterior, pero las filas interiores siguen siendo **las mismas** — etiquetas compartidas otra vez, un nivel más adentro:

```
#| label: copia-profunda
import copy

matriz_original = [
    [100.5, 101.2],   # Activo A
    [45.0, 45.5],     # Activo B
]

superficial = matriz_original.copy()       # copia la "cáscara", comparte las filas
profunda = copy.deepcopy(matriz_original)  # copia TODO, a todos los niveles

# Modificamos una fila a través de la copia superficial...
superficial[0][0] = 999.9

print("Original tras tocar la copia superficial:", matriz_original[0])
print("Copia profunda (intacta):                ", profunda[0])
```

Tocamos la copia superficial y **el original cambió** — las filas eran compartidas. La copia profunda (`copy.deepcopy`) duplica la estructura completa y queda verdaderamente aislada. La regla práctica: para listas "planas", `.copy()` alcanza; para estructuras anidadas (matrices, listas de diccionarios), `deepcopy`.

La Figura 2.2 pone en un solo dibujo las tres situaciones de esta sección. Volvé a ella cada vez que una variable "cambie sola": casi siempre hay dos etiquetas apuntando al mismo objeto.

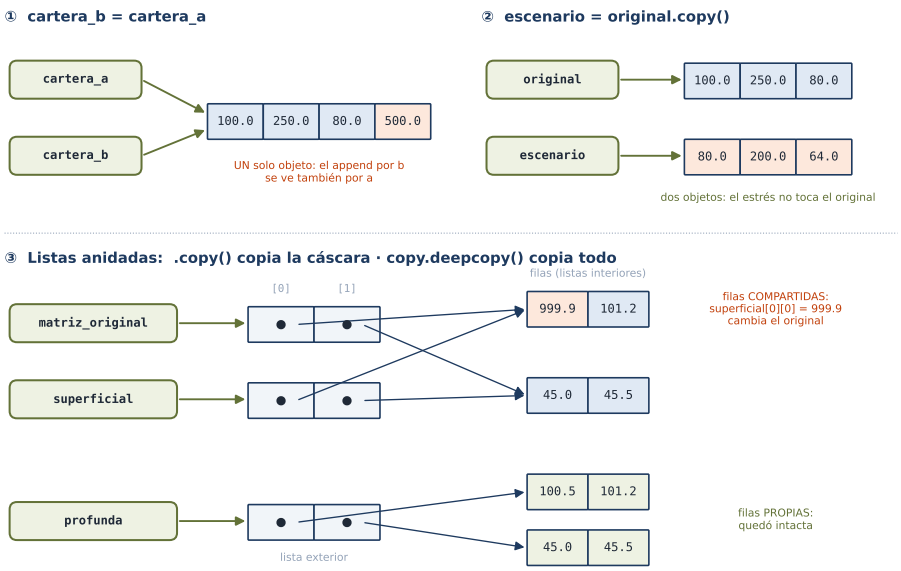


Figura 2.2.: Etiquetas y objetos. `b = a` no copia: pega una segunda etiqueta sobre la misma lista. `.copy()` crea una lista nueva e independiente. Con listas anidadas, `.copy()` duplica solo la lista exterior (las filas siguen compartidas) y `deepcopy` duplica todo.

⚠ Esto aplica a TODOS los objetos mutables

Los **diccionarios** (que ves en la sección siguiente) se comportan exactamente igual: `dict_b = dict_a` comparte el objeto, `dict_a.copy()` es superficial, y un diccionario con listas adentro necesita `deepcopy`. Lo mismo vale para cualquier objeto mutable, incluidas las clases que vas a crear en el Capítulo 4. Los inmutables (números, strings, tuplas) están exentos del problema: como no se pueden modificar, compartirlos es inofensivo — por eso el Capítulo 1 no necesitó esta advertencia.

2.4. Tuplas: la lista que no cambia

En el callout anterior se coló un tipo que todavía no presentamos: la **tupla**. Es la hermana inmutable de la lista — se escribe con paréntesis en lugar de corchetes, se indexa igual, pero **no se puede modificar** después de creada: ni `append`, ni reasignar elementos. ¿Para qué querrías una lista capada? Justamente por eso: para datos que **no deben** cambiar, la inmutabilidad es una garantía, no una limitación. Una operación ejecutada (ticker, cantidad, precio) es un hecho histórico: nada debería poder editarla.

```
#| label: tuplas
# Una operación ejecutada: un registro que no debe cambiar
operacion = ("GD30", 100, 92.5)

print("Operación completa:", operacion)
print("Ticker:", operacion[0])   # se indexa como una lista

# El "unpacking": desempaquetar la tupla en variables con nombre, de una
```

```
ticker, cantidad, precio = operacion
print(f"Compra de {cantidad} {ticker} a ${precio}")

# La inmutabilidad es real: descomentá para ver el error
# operacion[2] = 95.0   # TypeError: 'tuple' object does not support item assignment
```

La segunda línea destacable es el **desempaquetado** (*unpacking*): `ticker, cantidad, precio = operacion` asigna los tres elementos a tres variables de un golpe. Es la forma idiomática de "abrir" una tupla, y la vas a usar constantemente — sobre todo en bucles, gracias a dos funciones que producen tuplas:

- **`enumerate(lista)`**: al recorrer, te da `(indice, elemento)` — se acabó el `range(len(...))` manual.
- **`zip(lista_a, lista_b)`**: recorre dos listas **en paralelo**, dándote pares `(a, b)`.

```
#| label: enumerate-zip
precios = [50.0, 52.0, 51.0, 55.0]
tickers = ["GD30", "AL30", "GD35", "AE38"]

# enumerate: índice y elemento juntos (arrancando en 1 si querés)
for dia, precio in enumerate(precios, start=1):
    print(f"Día {dia}: ${precio}")

print()

# zip: dos listas en paralelo, sin índices a mano
for ticker, precio in zip(tickers, precios):
    print(f"{ticker}: ${precio}")
```

Fijate lo que desapareció: ni un solo `range(len(...))`, ni un índice `[i]`. El `for` desempaqueta cada tupla directamente en variables con nombre,

y el código se lee como la descripción del problema. Un beneficio extra de la inmutabilidad, que va a cobrar sentido enseguida: **una tupla sí puede ser clave de diccionario** (una lista no) — `precios[("GD30", "2024-01-15")]` es un patrón común para indexar por activo y fecha a la vez. El porqué te lo debe la sección de búsqueda, unas páginas más adelante.

2.5. Diccionarios: datos etiquetados

Las listas ordenan por **posición**: el elemento 0, el 1, el 2. Pero mirá cómo describirías un bono en la vida real: tiene un *ticker*, una *paridad*, una *legislación*. Nadie dice "el atributo 2 del bono" — cada dato tiene un **nombre**. El **diccionario** es la estructura que guarda pares `clave: valor`, y se busca por clave en lugar de por índice.

```
#| label: dict-bono
# Un bono descripto como diccionario: cada dato tiene su etiqueta
bono_soberano = dict()

bono_soberano['ticker'] = 'AL30'
bono_soberano['paridad'] = 45.5
bono_soberano['legislacion'] = 'Local'

print("Datos del bono:", bono_soberano)
print("Ticker específico:", bono_soberano['ticker'])

# Actualizar un valor existente: misma sintaxis que crear
bono_soberano['paridad'] = 48.2
print("Paridad actualizada:", bono_soberano['paridad'])
```

Compará `bono_soberano['ticker']` con `bono[0]`: el código con claves **se explica solo**. Seis meses después de escribirlo, seguís entendiendo qué hace. Ese es el criterio para elegir estructura: ¿tus datos son una *secuencia* (usá lista) o un *registro con campos con nombre* (usá diccionario)?

Un patrón enormemente útil: **contar frecuencias**. ¿Cuántos activos de cada nivel de riesgo tengo en la cartera?

```
#| label: dict-riesgos
# Contar cuántos activos hay por nivel de riesgo
activos = ["Bajo", "Alto", "Medio", "Bajo", "Bajo", "Medio", "Alto", "Bajo"]
conteo_riesgos = {}

for nivel in activos:
    # .get(clave, defecto): si la clave existe devuelve su valor; si no, el defecto.
    # Así la primera aparición de cada nivel arranca de 0 sin error.
    conteo_riesgos[nivel] = conteo_riesgos.get(nivel, 0) + 1

print("Distribución de riesgos de la cartera:")
print(conteo_riesgos)
```

El resultado dice que la cartera está cargada hacia el riesgo bajo (4 de 8 activos). El héroe silencioso es `.get(nivel, 0)`: sin él, la primera vez que apareciera cada nivel, `conteo_riesgos[nivel]` fallaría con `KeyError` porque la clave todavía no existe. `.get` con valor por defecto es la forma idiomática de "leer o inicializar" en un solo paso.

El tercer patrón: **combinar diccionarios** que comparten claves. Tenemos cantidades por commodity en un diccionario y precios en otro — la clave común (`'Soja'`, `'Maiz'`…) los conecta.

```
#| label: dict-inventario
# Valuar un inventario: cantidades x precios, conectados por la clave
cantidades = {'Soja': 150, 'Maiz': 300, 'Trigo': 200}
```

```
precios_mercado = {'Soja': 350.0, 'Trigo': 210.5, 'Maiz': 180.0}

valor_total_inventario = 0  # acumulador

for commodity in cantidades:          # recorre las CLAVES
    cantidad_toneladas = cantidades[commodity]
    precio_tonelada = precios_mercado[commodity]
    valor_total_inventario = valor_total_inventario + cantidad_toneladas * precio_to

print(f"El valor total del inventario agrícola es: ${valor_total_inventario:,.1f}")
```

Notá que los dos diccionarios listan los commodities **en distinto orden** — y no importa en absoluto, porque no accedemos por posición sino por clave. Esa independencia del orden es precisamente lo que los hace robustos para conectar datos de distintas fuentes (en la práctica: tu sistema de posiciones y el feed de precios del mercado nunca vienen en el mismo orden).

2.5.1. El costo de buscar: por qué existen los diccionarios

Hasta acá el diccionario parece solo una lista más cómoda. Pero hay una diferencia de fondo, **de ingeniería**, que explica por qué existe: el costo de *buscar*. Para saber si un ticker está en una **lista**, Python no tiene más remedio que recorrerla elemento por elemento — en el peor caso, la lista entera. Para buscarlo en un **diccionario**, calcula una función matemática sobre la clave (un *hash*) que le dice directamente *dónde* está guardado el valor: **una operación, sin recorrer nada**, tenga el diccionario diez claves o diez millones.

Con pocos datos la diferencia es invisible. Midámosla con un universo grande:

```
#| label: costo-busqueda
import time

# Un universo de 200.000 instrumentos
n = 200_000
tickers_lista = [f"TICKER_{i}" for i in range(n)]
tickers_dict = {t: i for i, t in enumerate(tickers_lista)}

buscado = f"TICKER_{n - 1}"   # el peor caso: está al final

# 200 búsquedas en la LISTA (recorre hasta encontrar)
t0 = time.perf_counter()
for _ in range(200):
    encontrado = buscado in tickers_lista
t_lista = time.perf_counter() - t0

# 200 búsquedas en el DICCIONARIO (salto directo por hash)
t0 = time.perf_counter()
for _ in range(200):
    encontrado = buscado in tickers_dict
t_dict = time.perf_counter() - t0

print(f"Lista:       {t_lista*1000:8.1f} ms")
print(f"Diccionario: {t_dict*1000:8.1f} ms")
print(f"El diccionario fue ~{t_lista/max(t_dict, 1e-9):,.0f}x más rápido")
```

La diferencia es de **miles de veces** — y crece con el tamaño: duplicar la lista duplica su tiempo de búsqueda, mientras el diccionario ni se inmuta. Ahora tenés el criterio completo para elegir estructura: lista cuando importa el **orden** y vas a recorrer todo; diccionario cuando vas a **buscar por clave**. Un sistema que consulta precios por ticker miles de veces por segundo *tiene* que ser un diccionario.

El precio del truco: claves inmutables

El salto directo funciona porque el hash de la clave se calcula una vez y no cambia. Por eso **las claves de un diccionario deben ser inmutables**: strings, números o tuplas sirven; una **lista no puede ser clave** (`precios[[1, 2]] = ...` lanza `TypeError: unhashable type`). Si la clave pudiera mutar, su hash cambiaría y el diccionario ya no sabría dónde la guardó. Fijate cómo se conecta todo: la distinción mutable/inmutable de la sección de copias no era solo una curiosidad — es la que decide qué puede ser clave.

2.6. Caso integrador: valuar un bono y encontrar su TIR

Hora de juntar todo. Un **bono bullet** paga un cupón fijo cada período y devuelve el capital al vencimiento. Su valor justo es la suma de todos sus flujos descontados:

$$P = \sum_{t=1}^{n} \frac{C}{(1+r)^t} + \frac{VN}{(1+r)^n}$$

Esa sumatoria $\sum$ que en el papel se lee de un vistazo es, computacionalmente, **un bucle `for` con un acumulador** — exactamente lo que venís practicando.

La Figura 2.3 muestra la sumatoria en acción: cada flujo futuro se "trae a hoy" dividiendo por $(1+r)^t$, y el precio del bono es la pila de esos valores presentes. Notá cuánto se achica el último flujo (1080 → ~671) por estar a 5 años.

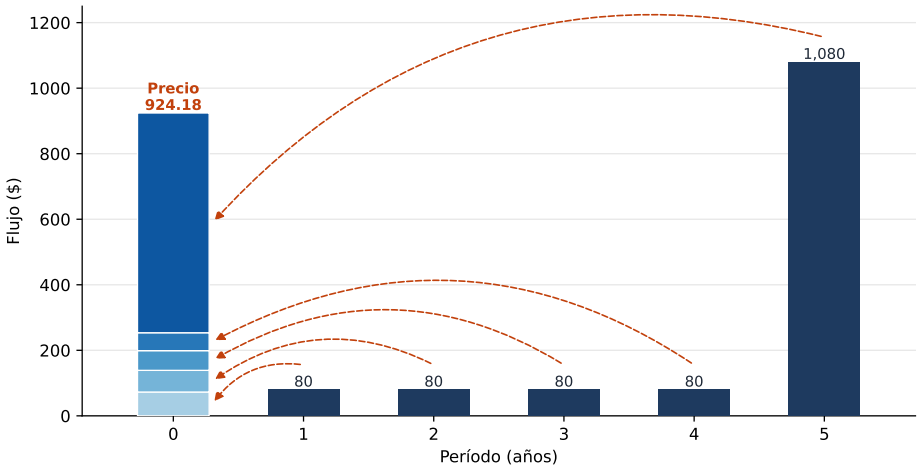


Figura 2.3.: Valuación del bono como suma de flujos descontados: a la derecha, los flujos que paga en cada año; en t = 0, el valor presente de cada uno apilado. La altura total de la pila es el precio.

```
#| label: caso-valuacion-bono
# Bono: valor nominal 1000, cupón 8% anual, 5 años, tasa de descuento 10%
valor_nominal = 1000
tasa_cupon = 0.08
anios = 5
tasa_descuento = 0.10

cupon = valor_nominal * tasa_cupon   # $80 por año

# La sumatoria de flujos descontados = bucle for + acumulador
valor_presente = 0
for t in range(1, anios + 1):
    flujo = cupon
    if t == anios:
        flujo = flujo + valor_nominal   # el último período devuelve el capital

    vp_flujo = flujo / (1 + tasa_descuento) ** t
    valor_presente = valor_presente + vp_flujo
    print(f"Año {t}: flujo ${flujo:>6.0f} → valor presente ${vp_flujo:>7.2f}")

print(f"\nPrecio justo del bono: ${valor_presente:.2f}")
```

El bono vale **menos que su valor nominal** (unos $924 contra $1000). Tiene sentido económico: paga cupones del 8 % pero el mercado exige 10 % — nadie paga $1000 por algo que rinde menos que la alternativa, así que el precio baja hasta compensar. Un bono cotiza *bajo la par* cuando su cupón está por debajo de la tasa de mercado.

Ahora la pregunta inversa, que es la que se hace el inversor real: el bono **cotiza** a un precio dado — ¿qué rendimiento me da si lo compro a ese precio? Esa tasa es la **TIR** (Tasa Interna de Retorno): la tasa de descuento que hace que el valor presente de los flujos iguale al precio de mercado. No se puede despejar algebraicamente, pero se puede **buscar por tanteo**:

probamos tasas cada vez más altas hasta que el valor presente cae por debajo del precio. Un `while` con búsqueda incremental — el mismo patrón del ejemplo del ahorro.

Antes del código, la estructura de la búsqueda en la Figura 2.4: dos bucles, uno adentro del otro, cada uno con su trabajo.

```
#| label: caso-tir
# El bono anterior cotiza a $950. ¿Qué TIR implica ese precio?
precio_mercado = 950

flujos = [cupon] * (anios - 1) + [cupon + valor_nominal]  # lista de flujos

tasa = 0.0
paso = 0.0001   # probamos de a 0.01%

while True:
    # Valuamos el bono a la tasa candidata (bucle interno con acumulador)
    vp = 0
    for t in range(1, anios + 1):
        vp = vp + flujos[t - 1] / (1 + tasa) ** t

    if vp <= precio_mercado:   # nos pasamos: el VP ya cayó hasta el precio
        break
    tasa = tasa + paso

print(f"Precio de mercado: ${precio_mercado}")
print(f"TIR aproximada: {tasa:.2%}")

# Verificación: valuar el bono a la TIR encontrada debe dar ~el precio
vp_verificacion = sum(flujos[t - 1] / (1 + tasa) ** t for t in range(1, anios + 1))
print(f"Verificación - VP a la TIR: ${vp_verificacion:.2f}")
```

La TIR da cerca del **9.3 %**: comprando a $950 un bono que promete

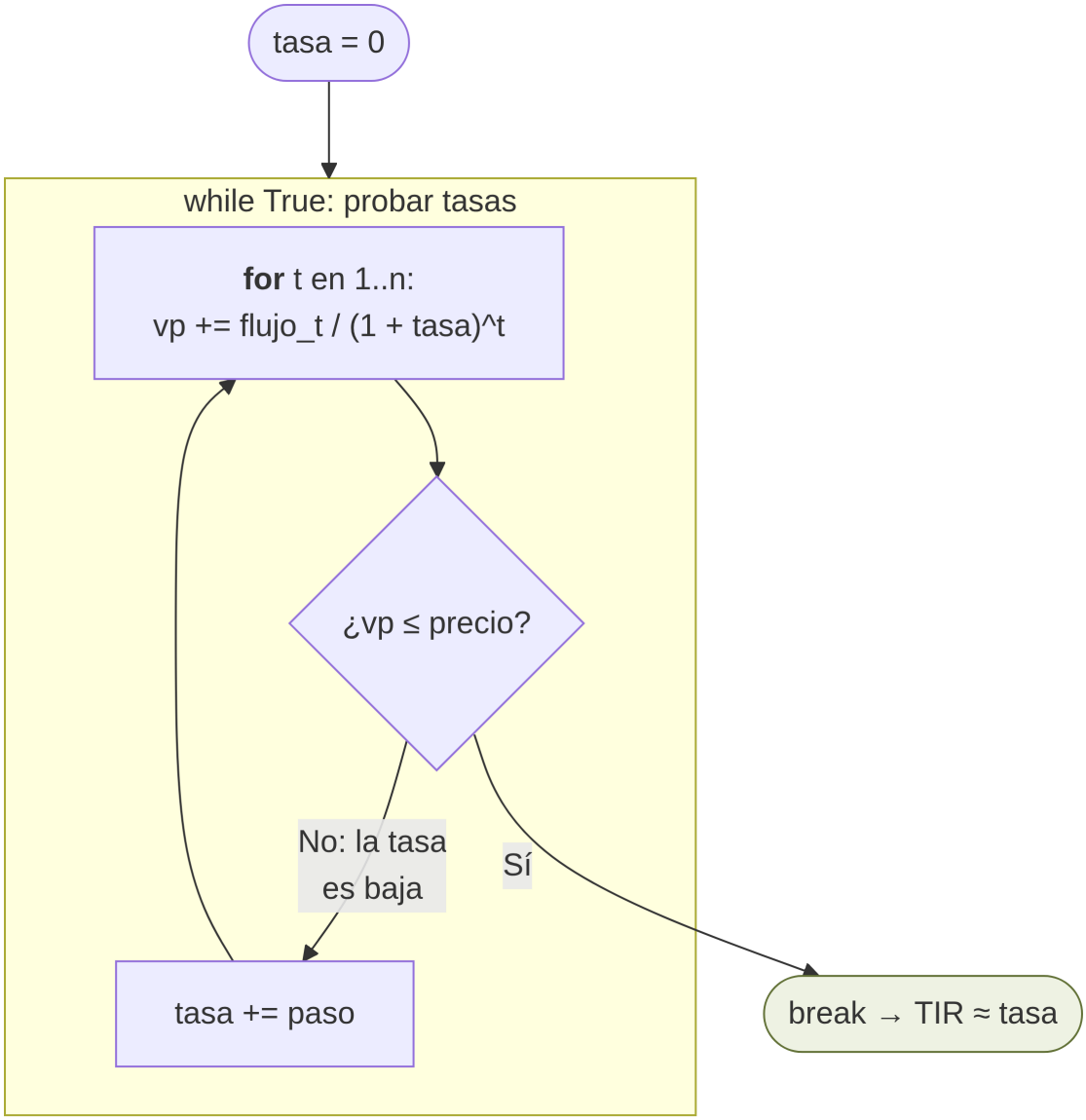


Figura 2.4.: Búsqueda incremental de la TIR: el `while` externo propone tasas cada vez más altas; el `for` interno valúa el bono a cada tasa candidata; el `break` corta cuando el valor presente cae hasta el precio de mercado.

cupones del 8 % sobre $1000, tu rendimiento efectivo es mayor que el cupón — el descuento en el precio es rendimiento extra. Fijate la estructura del código: un `while` (buscar la tasa) que contiene un `for` (valuar a cada tasa candidata), con un `break` como condición de salida. Todos los ingredientes del capítulo trabajando juntos.

Esto que hiciste tiene nombre

La búsqueda incremental es el método numérico más simple para resolver ecuaciones que no se pueden despejar. Es lento pero transparente. En el Capítulo 3 vas a empaquetar esta lógica en una función reutilizable, y en el Capítulo 18 vas a conocer a su primo profesional —Newton-Raphson— extrayendo volatilidades implícitas de opciones. La idea es la misma; solo mejora la puntería.

Simulador interactivo

En la versión web de este capítulo hay un simulador donde podés mover el cupón, el plazo, la tasa y el precio de mercado, y ver cómo cambian el precio y la TIR del bono.

2.7. Errores comunes

Trampas frecuentes

- **El bucle infinito**: un `while` cuya condición nunca se vuelve falsa (típicamente porque olvidaste actualizar el contador o lo actualizás después de un `continue`). Si tu programa "se cuelga", sospechá de esto primero.

- **`IndexError` por off-by-one**: pedir `lista[len(lista)]` no existe — el último índice válido es `len(lista) - 1`. Aparece muchísimo al comparar elementos consecutivos; por eso el `range(len(precios) - 1)` en el ejemplo de variaciones.
- **`KeyError` en diccionarios**: acceder a una clave que no existe (`conteo['Alto']` antes de crearla) rompe el programa. Usá `.get(clave, defecto)` o verificá con `if clave in diccionario`.
- **Modificar una lista mientras la recorrés**: hacer `lista.remove(x)` adentro de un `for` sobre esa misma lista saltea elementos de forma impredecible. El patrón seguro es construir una lista **nueva** con lo que querés conservar (como en el ejemplo de filtrado).
- **Creer que `b = a` copia la lista**: solo agrega una etiqueta al mismo objeto — modificás `b` y "cambia" `a`. Copia real: `a.copy()`; con estructuras anidadas, `copy.deepcopy(a)` (la sección "Copiar no es lo mismo que apuntar").
- **Dividir por el contador equivocado**: al promediar datos filtrados, dividí por la cantidad de datos *válidos*, no por el total de la lista (el error silencioso del ejemplo de `continue`).

2.8. Referencias

3. Capítulo 3: Funciones, Excepciones y Módulos

Creando una Librería Financiera Reutilizable

Funciones con parámetros, `*args`, scope, manejo de errores. Construimos la librería `finanzas_basicas.py` con VPN, valuación de bonos, TIR y conversión de tasas.

Introducción

Hasta ahora hemos escrito código de forma lineal. En este capítulo aprendemos a **encapsular lógica** en funciones reutilizables, siguiendo el principio **DRY** (*Don't Repeat Yourself*). Al final, tendremos nuestra propia librería `finanzas_basicas.py`.

3.1. Definición de Funciones

Una función es un bloque de código reutilizable que recibe **parámetros** y devuelve un **resultado**.

La mejor forma de pensar una función es como una **caja negra con un contrato** (Figura 3.1): entran parámetros, sale un resultado por `return`, y quien la usa no necesita saber qué pasa adentro.

```
#| label: funcion-basica
def valor_futuro(capital, tasa, periodos):
    """
    Calcula el valor futuro de una inversión.

    Parámetros:
    -----------
    capital : float - Monto inicial invertido
```

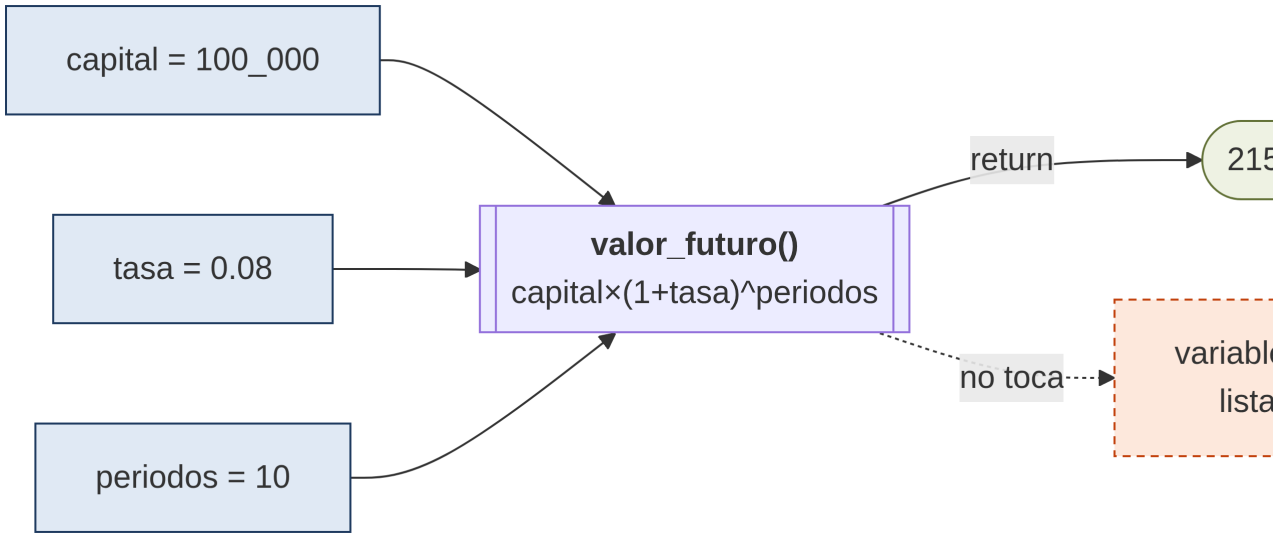


Figura 3.1.: Una función como caja negra: recibe datos por sus parámetros y entrega el resultado con `return`. Una función bien diseñada no lee ni modifica nada por fuera de ese contrato.

```
    tasa : float - Tasa de interés por período
    periodos : int - Número de períodos

    Retorna:
    --------
    float - Valor futuro
    """
    return capital * (1 + tasa) ** periodos

# Uso
vf = valor_futuro(100_000, 0.08, 10)
print(f"Valor futuro: ${vf:,.2f}")
```

3.1.1. Recibir y devolver múltiples valores

En Python, una función puede **recibir varios parámetros** y también **devolver más de un resultado**. Esto es útil en finanzas cuando, a partir de los mismos datos, queremos obtener más de un indicador.

```
#| label: multi-params-return
def resumen_inversion(capital, tasa, periodos):
    vf = capital * (1 + tasa) ** periodos
    interes_ganado = vf - capital
    rendimiento_total = interes_ganado / capital
    return vf, interes_ganado, rendimiento_total

# La función recibe 3 parámetros y devuelve 3 valores
vf, interes, rendimiento = resumen_inversion(100_000, 0.08, 3)

print(f"Valor futuro: ${vf:,.2f}")
print(f"Interés ganado: ${interes:,.2f}")
print(f"Rendimiento total: {rendimiento:.2%}")
```

Cuando devolvemos varios valores, Python los empaqueta en una tupla. Luego podemos "desempaquetarlos" en variables separadas para trabajar con cada resultado de forma clara.

3.1.2. Parámetros por Defecto

```
#| label: params-default
def valor_presente(flujo_futuro, tasa, periodos=1):
    """Calcula el valor presente. Por defecto, 1 período."""
    return flujo_futuro / (1 + tasa) ** periodos

# Con y sin el parámetro por defecto
vp1 = valor_presente(1000, 0.10)       # 1 período (default)
vp2 = valor_presente(1000, 0.10, 5)    # 5 períodos

print(f"VP a 1 año: ${vp1:,.2f}")
print(f"VP a 5 años: ${vp2:,.2f}")
```

3.1.3. Argumentos Variables (`*args`)

Hasta ahora vimos funciones con una cantidad fija de parámetros.
Pero en problemas financieros muchas veces no sabemos de antemano cuántos valores vamos a procesar (por ejemplo, una serie de flujos que puede tener 3, 10 o 30 períodos).

`*args` permite que una función reciba una **cantidad variable de argumentos posicionales**. Dentro de la función, esos argumentos se agrupan en una **tupla**.

Esto nos da flexibilidad sin tener que escribir versiones distintas de la misma función.

```
#| label: args
def sumar_flujos(*flujos):
    """Suma una cantidad variable de flujos de fondos."""
    total = 0
    for f in flujos:
        total += f
    return total

# Podemos pasar cualquier cantidad de argumentos
total_1 = sumar_flujos(100, 200, 300)
total_2 = sumar_flujos(80, 80, 80, 80, 1080)

print(f"Total 3 flujos: ${total_1:,}")
print(f"Total 5 flujos: ${total_2:,}")

# Si ya tenemos una lista, usamos * para desempaquetar
serie = [-1000, 300, 400, 500]
print(f"Suma serie: ${sumar_flujos(*serie):,}")
```

Con esto:

- `sumar_flujos(100, 200, 300)` envía 3 argumentos,
- `sumar_flujos(80, 80, 80, 80, 1080)` envía 5,
- y `sumar_flujos(*serie)` toma una lista y la transforma en argumentos separados.

Cuándo usar *args

Usá `*args` cuando la función necesite aceptar una cantidad variable de datos del mismo tipo (flujos, retornos, precios, etc.).
Si cada dato tiene un rol distinto y fijo (por ejemplo `capital`, `tasa`, `periodos`), conviene usar parámetros explícitos.

3.2. Scope (Alcance) de las Variables

El **scope** o alcance define en qué parte del programa una variable es "visible" y puede ser utilizada. En Python, distinguimos principalmente entre dos niveles:

1. **Scope Global (Main)**: Variables definidas fuera de cualquier función. Son visibles en todo el archivo, incluso dentro de las funciones.
2. **Scope Local**: Variables definidas dentro de una función. Solo existen mientras la función se está ejecutando y no pueden verse desde fuera.

La Figura 3.2 muestra los dos niveles como cajas anidadas. Cuando Python encuentra un nombre adentro de una función, lo busca **primero en la caja local** y, solo si no lo encuentra, sube a la global. Por eso `calcular_rendimiento` ve `tasa_mercado`, y por eso afuera no existe `variable_interna`.

3.2.1. Variables Globales vs. Locales

Una función puede **leer** una variable global, pero no puede modificarla directamente: para eso existe la declaración `global tasa_mercado` dentro de la función. Que exista no significa que convenga usarla — en código financiero es una mala práctica, porque una función que modifica estado externo se vuelve impredecible (el resultado deja de depender solo de sus parámetros). En todo este libro, las funciones reciben lo que necesitan por parámetro y devuelven resultados con `return`.

```
#| label: scope-basico
tasa_mercado = 0.05  # VARIABLE GLOBAL (Scope: Main)
```

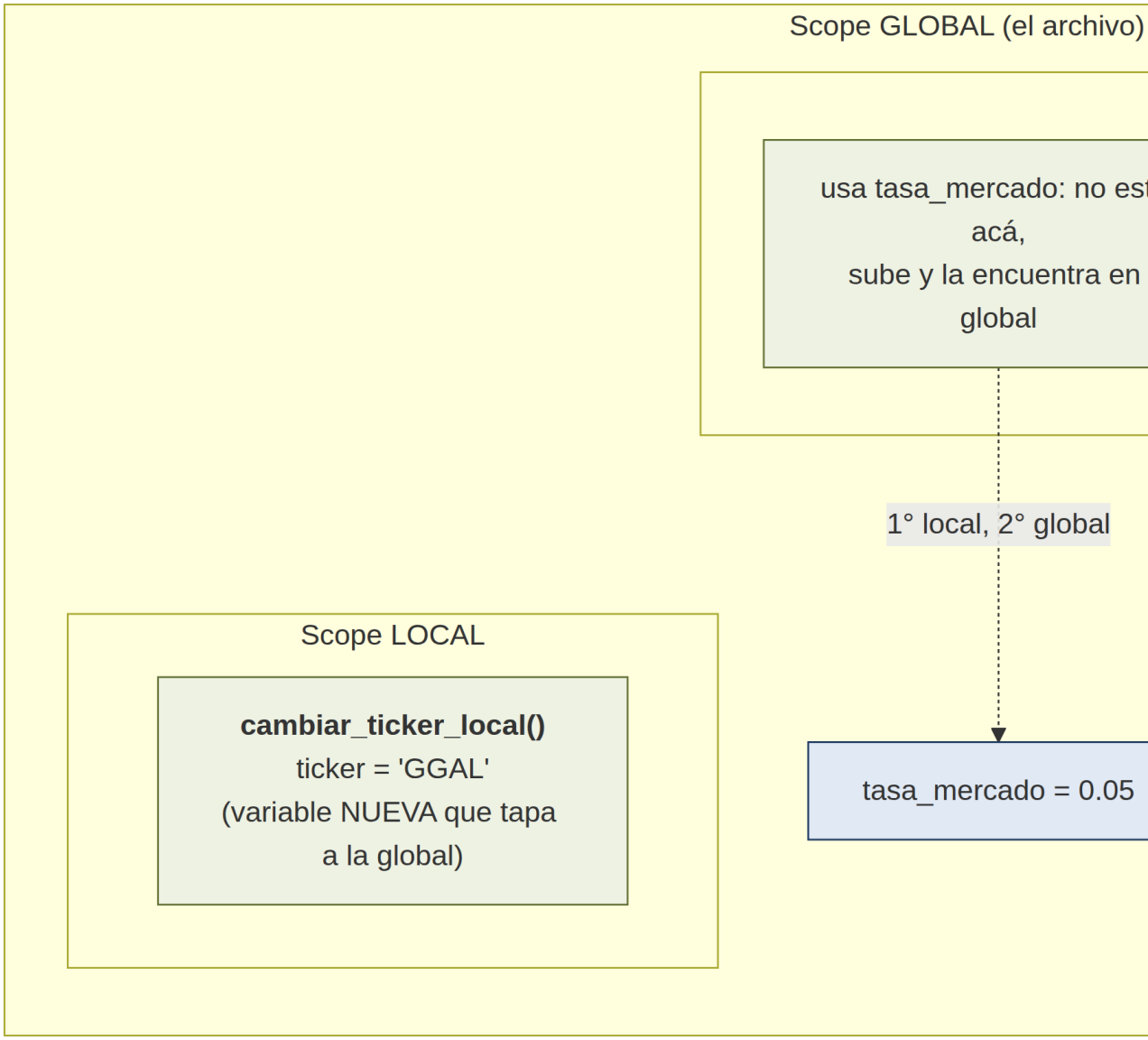


Figura 3.2.: Scope global y local. Cada llamada a una función crea su propia caja local, que desaparece al terminar. Para resolver un nombre, Python busca primero en la caja local y después en la global; desde afuera nunca se ve lo de adentro.

```
def calcular_rendimiento(monto):
    # tasa_mercado es visible aquí porque es global
    return monto * tasa_mercado

def operacion_interna():
    variable_interna = 100  # VARIABLE LOCAL (Scope: Función)
    print(f"Dentro de la función: {variable_interna}")

print(f"Rendimiento calculado: {calcular_rendimiento(1000)}")
operacion_interna()

# Intentar acceder a variable_interna aquí daría un NameError
# print(variable_interna)
```

3.2.2. El concepto de "Shadowing" (Sombreado)

Si definimos una variable dentro de una función con el mismo nombre que una global, la versión local "sombra" a la global durante la ejecución de esa función.

```
#| label: scope-shadowing
ticker = "AAPL"

def cambiar_ticker_local():
    ticker = "GGAL"  # Esto crea una variable LOCAL, no cambia la global
    print(f"Dentro de la función: {ticker}")

cambiar_ticker_local()
print(f"Fuera de la función (Global): {ticker}")
```

3.2.3. ¿Por qué evitar las variables globales?

En el desarrollo de software financiero, la **pureza** de las funciones es fundamental. Una función ideal es aquella que, para los mismos parámetros, siempre devuelve el mismo resultado sin depender de "estados externos".

```
#| label: scope-mala-practica
# MALA PRÁCTICA (Dependencia global)
tasa = 0.10
def calcular_interes_malo(capital):
    return capital * tasa

# BUENA PRÁCTICA (Parámetros explícitos)
def calcular_interes_bueno(capital, tasa_activa):
    return capital * tasa_activa

print(f"Malo: {calcular_interes_malo(100)}")
print(f"Bueno: {calcular_interes_bueno(100, 0.12)}")
```

> **!** Regla de Oro
>
> Siempre que necesites un dato dentro de una función, **pásalo como parámetro**. Evita depender de variables definidas en el "main" para que tu código sea modular y fácil de testear.

3.2.4. ¿Qué recibe una función cuando le pasás una lista?

Cerremos el círculo con el concepto de *aliasing* del Capítulo 2 ("copiar no es lo mismo que apuntar"), porque tiene una consecuencia directa sobre las funciones. Cuando llamás `mi_funcion(cartera)`, la función **no recibe una copia** de la cartera: recibe **otra etiqueta al mismo objeto**. Con parámetros inmutables (números, strings) esto es invisible — no se pueden

modificar. Pero con una **lista o diccionario**, si la función lo muta, *el llamador ve el cambio*:

```
#| label: mutar-argumentos
def aplicar_comision(flujos):
    """Descuenta 1 % de comisión a cada flujo... modificando la lista recibida."""
    for i in range(len(flujos)):
        flujos[i] = flujos[i] * 0.99
    return flujos

mis_flujos = [1000.0, 1000.0, 1000.0]
flujos_netos = aplicar_comision(mis_flujos)

print(f"Flujos netos:      {flujos_netos}")
print(f"¿Y mis originales? {mis_flujos}   ← ¡también cambiaron!")
```

La función "devolvió" los flujos netos, pero de paso **destruyó los originales** — `mis_flujos` y el parámetro `flujos` eran el mismo objeto. En un análisis real esto es venenoso: valuás la cartera dos veces y da distinto, porque la primera pasada la modificó. Hay dos diseños correctos, y ambos empiezan igual — **no mutar lo que te prestaron**:

```
#| label: no-mutar-argumentos
def aplicar_comision_bien(flujos):
    """Versión pura: trabaja sobre una copia y devuelve una lista nueva."""
    netos = []
    for f in flujos:
        netos.append(f * 0.99)
    return netos

mis_flujos = [1000.0, 1000.0, 1000.0]
flujos_netos = aplicar_comision_bien(mis_flujos)
```

```
print(f"Flujos netos:      {flujos_netos}")
print(f"Originales intactos: {mis_flujos}")
```

La convención del oficio

Una función debería **recibir datos, devolver resultados y no tocar nada más** — ni globales (la sección anterior) ni los objetos que le pasaron (esta). A las funciones que cumplen esto se las llama *puras*, y son las más fáciles de testear, reutilizar y razonar. Todas las funciones de la librería que construimos al final del capítulo son puras.

3.3. Manejo de Errores: `try` / `except`

En el mundo real, los datos financieros casi nunca llegan "perfectos": pueden venir incompletos, en formato incorrecto o fuera de rango. El manejo de excepciones con `try` / `except` permite que el programa sea **robusto**, evitando que se rompa ante errores esperables.

3.3.1. ¿Qué es una excepción?

Una **excepción** es un error que ocurre durante la ejecución del programa (runtime), por ejemplo:

- dividir por cero (`ZeroDivisionError`),
- convertir texto a número cuando no se puede (`ValueError`),
- acceder a una clave inexistente en un diccionario (`KeyError`),
- o intentar usar un tipo de dato incorrecto (`TypeError`).

Si no capturamos la excepción, el programa se detiene **e imprime un traceback** — el "informe forense" del error. Saber leerlo es probablemente la habilidad que más tiempo te va a ahorrar en toda tu carrera, así que detengámonos un minuto.

Cómo leer un traceback (la habilidad subvalorada)

Un traceback típico luce así:

```
Traceback (most recent call last):
  File "analisis.py", line 42, in <module>
    resultado = valuar_cartera(posiciones)
  File "analisis.py", line 17, in valuar_cartera
    total += valuacion_bono(bono, tasa)
  File "finanzas_basicas.py", line 8, in valuacion_bono
    return cupon / (1 + tasa) ** t
ZeroDivisionError: division by zero
```

La receta para leerlo:

1. **Empezá por la última línea**: ahí está el *tipo* de error (`ZeroDivisionError`) y el *mensaje* (`division by zero`). Es el "qué pasó".
2. **Subí una línea**: es el lugar exacto donde explotó (`finanzas_basicas.py`, línea 8). Es el "dónde".
3. **El resto, de abajo hacia arriba, es la cadena de llamadas**: quién llamó a quién hasta llegar ahí. El "cómo llegamos". Ojo: la línea que explotó no siempre es la culpable — acá el bug probablemente esté en quien pasó `tasa = -1`, no en la fórmula.

El error de principiante es leer solo la primera línea (que es la *menos* informativa) o, peor, no leer nada y adivinar. El traceback te está diciendo exactamente qué, dónde y cómo — gratis.

Con `try` / `except`, en cambio, podemos capturar el error y decidir cómo responder sin que el programa muera.

3.3.2. Flujo de `try` / `except`

La estructura general es:

```
try:
    # código que puede fallar
except TipoDeError:
    # qué hacer si ocurre ese error
```

La idea clave es separar:

- **camino normal** (cálculo esperado),
- de **camino de error** (qué hacemos cuando algo falla).

3.3.3. Ejemplo base: validación y captura en VPN

```
#| label: try-except
def calcular_vpn(flujos, tasa):
    """Calcula el VPN de una serie de flujos."""
    if len(flujos) == 0:
        raise ValueError("La lista de flujos no puede estar vacía")

    vpn = 0
    for t, cf in enumerate(flujos):
        vpn += cf / (1 + tasa) ** t
    return vpn

# Caso normal
```

```
print(f"VPN correcto: {calcular_vpn([-1000, 400, 400, 400], 0.10):.2f}")

# Caso con error controlado (lista vacía)
try:
    resultado = calcular_vpn([], 0.10)
except ValueError as e:
    print(f"Error capturado: {e}")
```

3.3.4. Capturar múltiples excepciones

Podemos manejar cada tipo de error de forma distinta:

```
#| label: try-except-multiple
def ratio_cobertura(ebit, intereses):
    return ebit / intereses

try:
    print(ratio_cobertura(500, 0))  # podría fallar por división por cero
except ZeroDivisionError:
    print("Error: intereses no puede ser 0")
except TypeError:
    print("Error: ebit e intereses deben ser numéricos")
```

Esto es útil porque no todos los errores deben tener el mismo mensaje o tratamiento.

3.3.5. Uso de `else` y `finally`

Además de `except`, Python ofrece dos bloques muy útiles:

- `else`: se ejecuta si **no** hubo excepción en `try`.

- `finally`: se ejecuta **siempre**, haya error o no.

La Figura 3.3 junta los cuatro bloques en un solo recorrido. La clave es que, de las dos ramas del medio, **se ejecuta exactamente una**, mientras que `finally` se ejecuta siempre.

```
#| label: try-except-else-finally
def parse_tasa(texto):
    return float(texto)

entrada = "0.12"

try:
    tasa = parse_tasa(entrada)
except ValueError:
    print("No se pudo convertir la tasa a número")
else:
    print(f"Tasa válida: {tasa:.2%}")
finally:
    print("Fin de la validación")
```

En análisis financiero real, `finally` suele usarse para cerrar archivos, conexiones o liberar recursos.

3.3.6. `raise`: generar errores de negocio explícitos

No solo capturamos errores: también podemos **lanzarlos** cuando una regla financiera no se cumple.

```
#| label: try-except-raise
def cuota_francesa(capital, tasa, periodos):
    if capital <= 0:
        raise ValueError("El capital debe ser positivo")
```

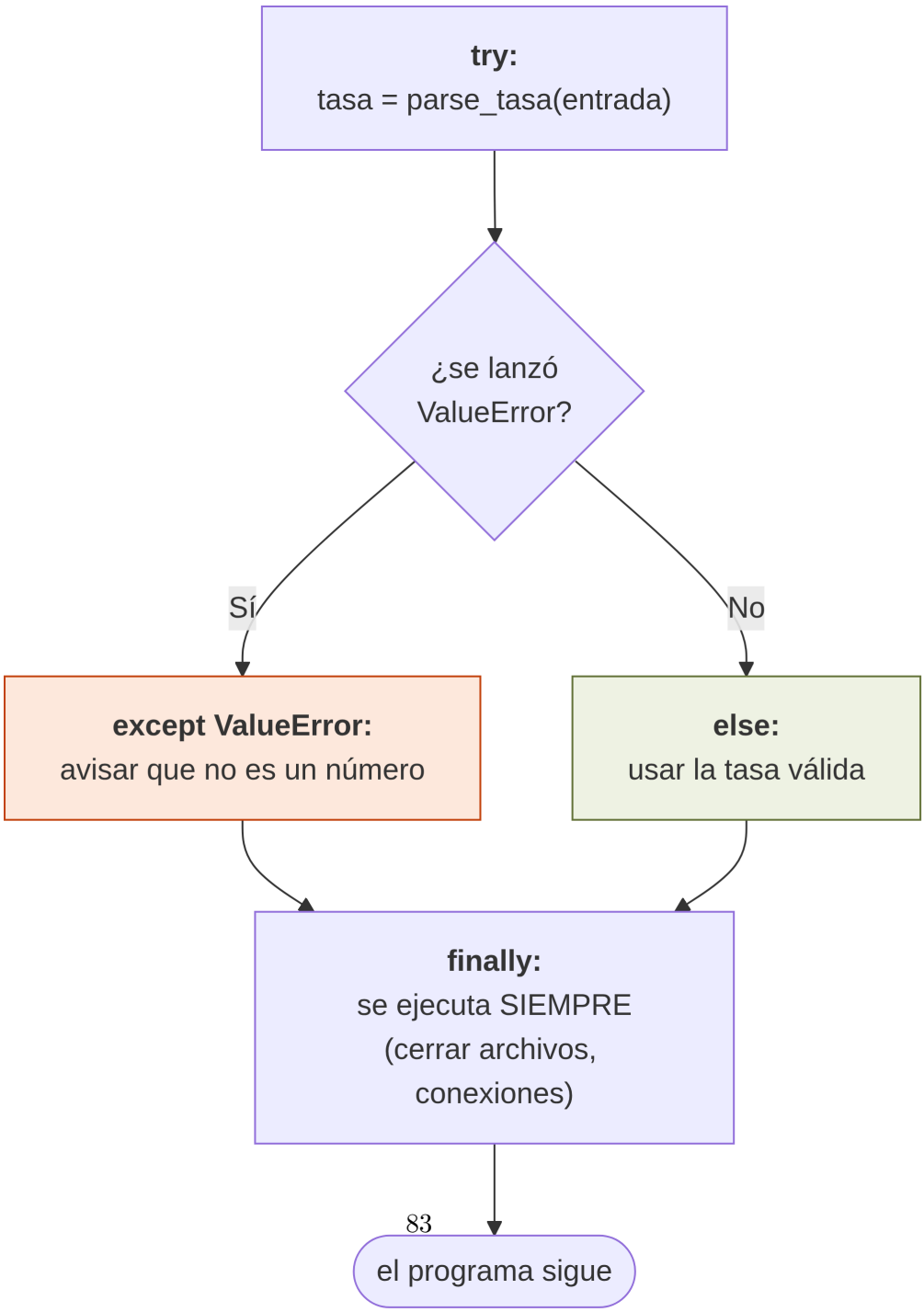


Figura 3.3.: Recorrido de `try`/`except`/`else`/`finally`. Si el bloque `try` lanza una excepción del tipo capturado, se ejecuta `except`; si no, `else`. En ambos casos, `finally` corre al final.

```
    if tasa <= 0:
        raise ValueError("La tasa debe ser positiva")
    if periodos <= 0:
        raise ValueError("Los periodos deben ser mayores a 0")
    return capital * (tasa * (1 + tasa) ** periodos) / ((1 + tasa) ** periodos - 1)

try:
    cuota = cuota_francesa(100000, 0.03, 24)
    print(f"Cuota: ${cuota:,.2f}")
except ValueError as e:
    print(f"Dato inválido: {e}")
```

3.3.7. Buenas prácticas en `try` / `except`

1. Capturar errores **específicos** (`ValueError`, `TypeError`, etc.), no usar `except:` genérico salvo casos muy controlados.
2. Mantener el bloque `try` **pequeño** (solo las líneas que pueden fallar).
3. Dar mensajes claros para quien usa el programa.
4. Validar supuestos de negocio con `raise` (tasas, plazos, signos de flujos).
5. No ocultar errores silenciosamente (`pass`) si eso dificulta detectar problemas.

> **!** Idea clave
>
> `try` / `except` no reemplaza una buena lógica: la complementa. Primero validamos reglas financieras, luego capturamos errores inevitables del mundo real.

3.4. Módulos y librerías en Python

Cuando un programa crece, no conviene dejar todo en un solo archivo. Python permite **organizar el código en módulos** para reutilizar funciones, mantener orden y facilitar el testing.

3.4.1. ¿Para qué sirve un módulo?

Un **módulo** es, en la práctica, un archivo `.py` con funciones, variables o clases.

Nos sirve para:

- separar responsabilidades (por ejemplo, valuación de bonos en un archivo y conversión de tasas en otro),
- reutilizar lógica en distintos scripts o notebooks,
- evitar duplicación de código (principio DRY),
- y mejorar mantenimiento y legibilidad.

3.4.2. Diferencia entre módulo y librería

- **Módulo**: un archivo Python individual, por ejemplo `finanzas_basicas.py`.
- **Librería**: un conjunto de módulos relacionados que resuelven un dominio o problema.
  Ejemplos: `numpy`, `pandas`, `matplotlib`.

En cursos introductorios, a veces se dice "librería" para referirse a un solo archivo propio. Técnicamente, ese archivo es un **módulo**, aunque puede ser el primer paso para construir una librería más completa.

3.4.3. Formas de importar: `import`, `from`, `as`

```
# 1) import modulo
import math
print(math.sqrt(16))  # accedemos con prefijo math.

# 2) import modulo as alias
import math as m
print(m.pi)  # mismo módulo, nombre corto

# 3) from modulo import nombre
from math import sqrt
print(sqrt(25))  # usamos sqrt directamente

# 4) from modulo import nombre as alias
from math import factorial as fact
print(fact(5))
```

i Buena práctica

En general, preferí `import modulo` o `import modulo as alias` porque hacen explícito de dónde viene cada función (`modulo.funcion`). `from modulo import *` no se recomienda en producción porque puede sobrescribir nombres y volver ambiguo el código.

3.5. Problemática Financiera: Librería `finanzas_basicas.py`

Hora de cosechar lo sembrado. En los dos capítulos anteriores resolviste problemas financieros reales —interés compuesto, conversión de tasas, valuación de bonos, TIR— pero cada solución quedó atrapada en su ejemplo. Vamos a **rescatar esa lógica y empaquetarla en una librería reutilizable**: el conjunto de funciones que vas a poder importar en cualquier análisis futuro. En un entorno real, este código viviría en un archivo `finanzas_basicas.py` y se importaría con `from finanzas_basicas import valuacion_bono, tir`.

3.5.1. Funciones de la Librería

```
#| label: libreria
# === finanzas_basicas.py ===
# En un proyecto real, este bloque viviría en su propio archivo .py
# y lo importarías con: from finanzas_basicas import valor_futuro, ...

def valor_futuro(capital, tasa, periodos):
    """Calcula VF = C * (1 + r)^n."""
    if tasa < -1:
        raise ValueError("La tasa no puede ser menor a -100%")
    return capital * (1 + tasa) ** periodos


def valor_presente_neto(flujos, tasa):
    """Calcula el VPN de una serie de flujos (el flujo 0 no se descuenta)."""
    if not flujos:
        raise ValueError("Lista de flujos vacía")
```

```
    vpn = 0
    for t, cf in enumerate(flujos):
        vpn += cf / (1 + tasa) ** t
    return vpn


def conversion_tasas(tna, capitalizaciones=12):
    """Convierte una TNA con n capitalizaciones anuales a TEA."""
    if capitalizaciones <= 0:
        raise ValueError("Las capitalizaciones deben ser positivas")
    return (1 + tna / capitalizaciones) ** capitalizaciones - 1


def valuacion_bono(valor_nominal, tasa_cupon, tasa_descuento, anios):
    """Precio de un bono bullet: cupones fijos + nominal al vencimiento.

    Es la lógica del Capítulo 2, ahora encapsulada y validada.
    """
    if tasa_cupon < 0 or anios <= 0:
        raise ValueError("Cupón no negativo y plazo positivo")

    cupon = valor_nominal * tasa_cupon
    precio = 0
    for t in range(1, anios + 1):
        flujo = cupon + (valor_nominal if t == anios else 0)
        precio += flujo / (1 + tasa_descuento) ** t
    return precio


def tir(valor_nominal, tasa_cupon, anios, precio_mercado, paso=0.0001):
    """TIR de un bono bullet por búsqueda incremental (Capítulo 2)."""
    if precio_mercado <= 0:
        raise ValueError("El precio debe ser positivo")
```

```
    tasa = 0.0
    while valuacion_bono(valor_nominal, tasa_cupon, tasa, anios) > precio_mercado:
        tasa += paso
        if tasa > 5:   # techo de seguridad: 500% anual
            raise ValueError("No se encontró TIR razonable: revisar inputs")
    return tasa
```

Fijate el detalle de diseño en `tir()`: **reutiliza `valuacion_bono()`** en lugar de repetir el bucle de descuento. La Figura 3.4 muestra cómo quedan conectadas las piezas: el script de análisis solo conoce los nombres de las funciones, y dentro de la librería unas se apoyan en otras.

Las funciones de una librería se apoyan unas en otras — si mañana mejorás la valuación, la TIR mejora gratis. Notá también el "techo de seguridad": una función de librería no puede colgarse en un bucle infinito porque alguien le pasó un precio absurdo; falla rápido y con un mensaje claro.

3.5.2. Usando la Librería

```
#| label: uso-libreria
# === Uso de las funciones ===

# 1. Valor futuro de una colocación
vf = valor_futuro(100_000, 0.08, 3)
print(f"Valor futuro: ${vf:,.2f}")

# 2. VPN de un proyecto de inversión
flujos = [-1000, 400, 400, 400]
vpn = valor_presente_neto(flujos, 0.10)
print(f"VPN: ${vpn:,.2f}")
```

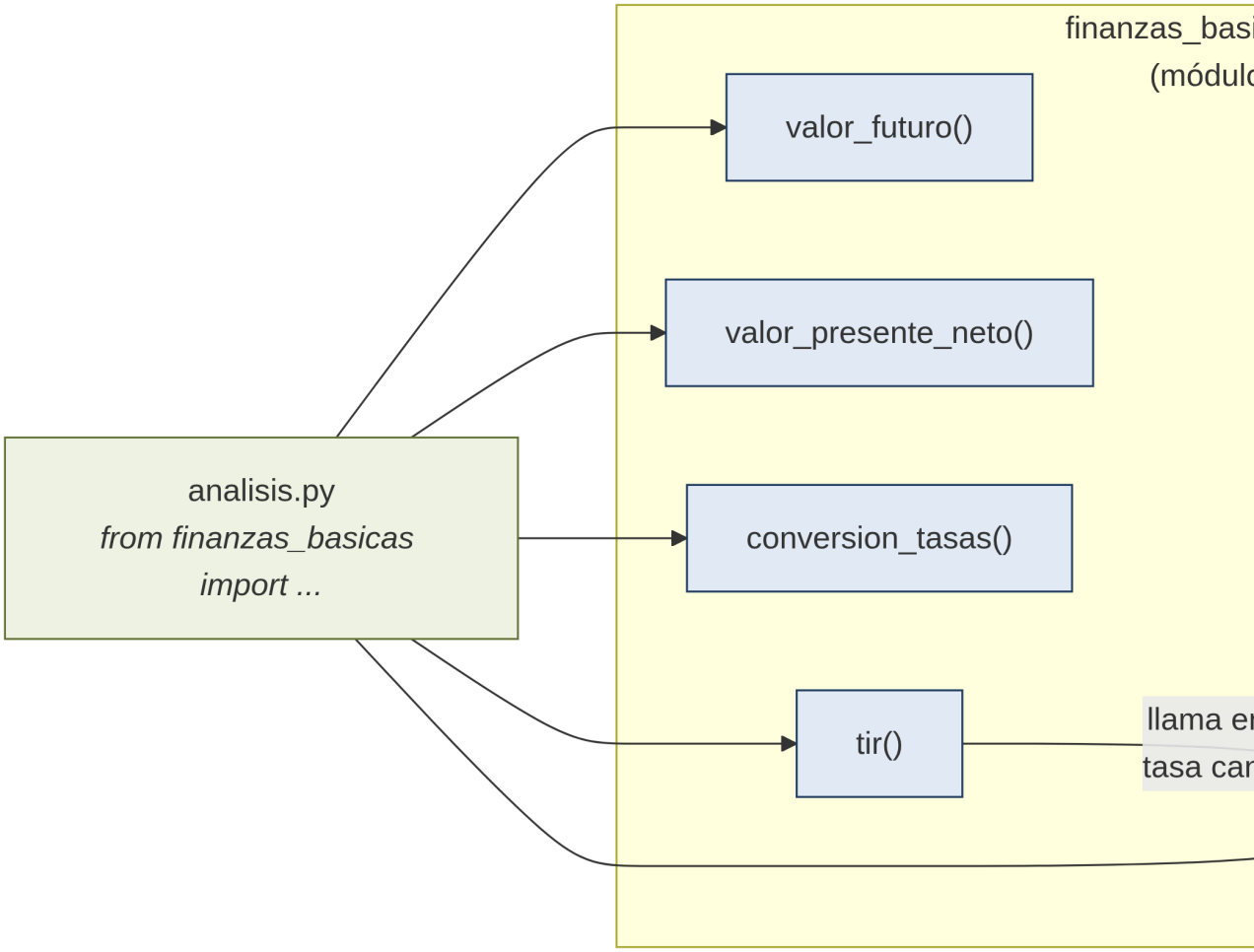


Figura 3.4.: Estructura de `finanzas_basicas.py`. El análisis importa las funciones que necesita; dentro del módulo, `tir()` delega la valuación en `valuacion_bono()`, así que cualquier mejora en esta última se hereda automáticamente.

```
# 3. Conversión de tasas: TNA 48% con capitalización mensual → TEA
tea = conversion_tasas(0.48, 12)
print(f"TNA 48% mensual → TEA: {tea:.2%}")

# 4. Valuación del bono del Capítulo 2, ahora en una línea
precio = valuacion_bono(1000, 0.08, 0.10, 5)
print(f"Precio del bono: ${precio:,.2f}")

# 5. Y su TIR comprándolo a $950
rendimiento = tir(1000, 0.08, 5, 950)
print(f"TIR al precio de $950: {rendimiento:.2%}")
```

Compará el punto 4 con las ~10 líneas de bucle que necesitaste en el Capítulo 2: la misma lógica, pero ahora **valuar un bono es una línea** — y con validación de inputs incluida. Ese salto de "código que escribo cada vez" a "herramienta que invoco" es exactamente lo que este capítulo vino a enseñar. En el próximo capítulo estas funciones van a dar su paso final: convertirse en *métodos* de una clase `Bono`.

3.6. Principio DRY: Don't Repeat Yourself

El principio **DRY** (*Don't Repeat Yourself*) propone evitar duplicar la misma lógica en distintos lugares del código. Cuando repetimos fórmulas o bloques manualmente, aumentamos el riesgo de inconsistencias: si cambia un supuesto (por ejemplo, la tasa o la frecuencia), debemos corregir múltiples líneas y es fácil olvidar alguna.

Aplicar DRY en finanzas significa **encapsular cálculos recurrentes en funciones** (por ejemplo, `valuacion_bono()` o `valor_presente_neto()`) para tener:

- **menos errores** de mantenimiento,
- **mayor claridad** del modelo,
- **reutilización** en distintos ejercicios o casos,
- y **validaciones centralizadas** en un solo lugar.

DRY en la Práctica

Antes de copiar y pegar código, preguntate: *¿puedo hacer una función?*
Sin DRY (malo):

```
precio_a = 80/(1.1) + 80/(1.1**2) + 1080/(1.1**3)
precio_b = 60/(1.08) + 60/(1.08**2) + 1060/(1.08**3)
```

Con DRY (bueno):

```
precio_a = valuacion_bono(1000, 0.08, 0.10, 3)
precio_b = valuacion_bono(1000, 0.06, 0.08, 3)
```

3.7. Errores comunes

Trampas frecuentes

- **Argumento por defecto mutable**: `def agregar(flujo, lista=[])` comparte la MISMA lista entre todas las llamadas — los flujos de un bono aparecen en otro. El idioma correcto: `def agregar(flujo, lista=None)` y crear la lista adentro (`if lista is None: lista = []`).
- **Olvidar el `return`**: una función que calcula pero no retorna devuelve `None` silenciosamente; el error explota recién más ade-

lante (“unsupported operand type NoneType”). Si la función produce un valor, terminá con `return`.

- **Capturar `Exception` a ciegas**: `except Exception: pass` esconde *todos* los errores, incluidos los bugs que necesitás ver. Capturá el error específico que esperás (`except ValueError:`) y dejá que el resto explote.
- **Modificar los argumentos recibidos**: si tu función hace `flujos.append(...)` sobre la lista que le pasaron, el llamador ve su lista alterada sin saberlo. Trabajá sobre una copia (`flujos = list(flujos)`) o no mutes.
- **Validar tarde (o nunca)**: validar los inputs al inicio de la función (como hace `valuacion_bono`) da errores claros y cercanos a la causa; sin validación, el error aparece diez llamadas más adentro con un mensaje incomprensible.

3.8. Referencias

4. Capítulo 4: Programación Orientada a Objetos (POO)

Modelar instrumentos financieros como objetos: la clase Bono

Un bono del mundo real tiene datos (ticker, nominal, cupón) y comportamiento (valuarse, pagar flujos). Hasta ahora en nuestro código esas dos mitades viajaban separadas: diccionarios por un lado, funciones por el otro. La POO las une en un solo objeto. Construimos la clase Bono completa —con constructor, propiedades, métodos de valuación— y la extendemos por herencia a BonoAmortizable, cerrando con composición (el Portafolio) y polimorfismo.

Introducción

Miremos el punto exacto donde quedamos al final del Capítulo 3. Para trabajar con un bono, teníamos **dos piezas separadas**: un diccionario con sus datos…

```
bono = {"ticker": "GD30", "valor_nominal": 1000, "tasa_cupon": 0.08, "anios": 5}
```

…y funciones sueltas que operaban sobre él, a las que había que pasarles cada dato por separado: `valuacion_bono(bono["valor_nominal"], bono["tasa_cupon"], tasa, bono["anios"])`. Funciona, pero tiene una fragilidad estructural: **nada une los datos con las funciones que saben operarlos**. Cualquier parte del programa puede meter mano en el diccionario y romper un invariante (¿una tasa de cupón de -8 %? ¿un diccionario al que le falta una clave?), y las funciones tienen que confiar en que el diccionario que reciben viene bien armado.

La **Programación Orientada a Objetos** propone otra organización: juntar en una misma entidad —el **objeto**— los datos y el comportamiento que les corresponde. Un objeto `Bono` *sabe* su ticker y su cupón, y *sabe* valuarse. Es la forma de programar que más se parece a cómo pensamos las finanzas: los instrumentos del mundo real **ya son objetos** — tienen propiedades y hacen cosas.

Los paradigmas, en contexto

Un **paradigma de programación** es un estilo general de estructurar el código. No hay uno "correcto"; hay herramientas para problemas distintos,

y en este libro los vas a usar todos:

1. **Imperativo**: secuencias de instrucciones que modifican el estado paso a paso — los bucles y acumuladores del Capítulo 2.
2. **Funcional**: organizar el código en funciones reutilizables con entradas y salidas claras — el Capítulo 3.
3. **Orientado a Objetos**: modelar entidades que agrupan datos y comportamiento — este capítulo.

La POO no reemplaza a los anteriores: se apoya en ellos. Adentro de cada método vas a seguir escribiendo bucles y llamando funciones.

4.1. Clases y objetos: la plantilla y sus ejemplares

Dos conceptos que conviene separar bien desde el principio:

- **Clase**: la *plantilla* (o plano, *blueprint*). Define qué atributos y qué métodos tendrán todos los objetos de ese tipo. Se escribe **una vez**.
- **Objeto** (o *instancia*): cada ejemplar concreto construido a partir de la plantilla, con sus propios valores.

La analogía financiera directa: la clase es "el concepto de bono" (todo bono tiene ticker, nominal, cupón, y sabe valuarse); los objetos son el GD30, el AL30, el GD35 — cada uno con sus números.

La Figura 4.1 lo muestra con los bonos del capítulo.

Empecemos con la clase más simple posible y una lectura línea por línea:

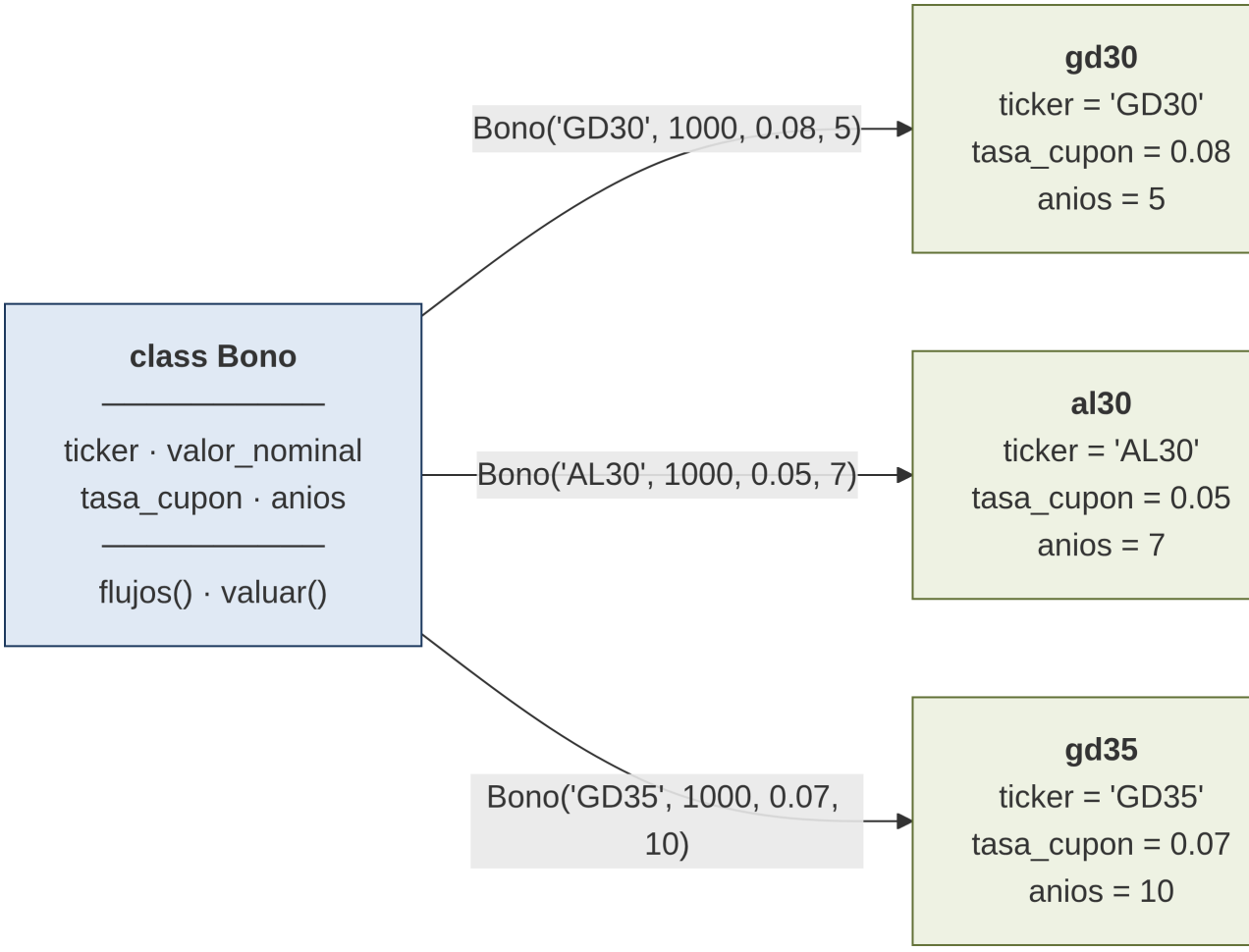


Figura 4.1.: Una clase es la plantilla; cada objeto (instancia) se construye a partir de ella con sus propios valores. Los tres bonos comparten la estructura y los métodos, pero no los datos.

```python
class Accion:
    """Una acción con su ticker y su último precio."""

    def __init__(self, ticker, precio):
        self.ticker = ticker   # atributo: dato del objeto (sustantivo)
        self.precio = precio

    def actualizar_precio(self, nuevo_precio):   # método: acción (verbo)
        variacion = nuevo_precio / self.precio - 1
        self.precio = nuevo_precio
        return variacion

# Creamos dos INSTANCIAS de la misma clase: cada una con su estado propio
apple = Accion("AAPL", 175.50)
galicia = Accion("GGAL", 4200.0)

print(f"{apple.ticker}: ${apple.precio}")
print(f"{galicia.ticker}: ${galicia.precio}")

# El objeto sabe hacer cosas con sus propios datos
var = apple.actualizar_precio(180.00)
print(f"\n{apple.ticker} actualizada a ${apple.precio} ({var:+.2%})")
print(f"{galicia.ticker} sigue en ${galicia.precio} - cada objeto tiene SU estado")
```

```
AAPL: $175.5
GGAL: $4200.0

AAPL actualizada a $180.0 (+2.56%)
GGAL sigue en $4200.0 - cada objeto tiene SU estado
```

Fijate el resultado clave del final: actualizamos el precio de `apple` y el de `galicia` **no se movió**. Cada instancia guarda su propio **estado** (el

conjunto de valores de sus atributos en un momento dado). Una plantilla, muchos ejemplares independientes: eso es una clase.

4.1.1. Las dos piezas con nombre raro: `__init__` y `self`

- `__init__` es el **constructor**: Python lo ejecuta automáticamente cada vez que creás una instancia (`Accion("AAPL", 175.50)`). Su trabajo es dejar al objeto recién nacido con todos sus atributos inicializados. Los guiones bajos dobles marcan que es un método "especial" del lenguaje.
- `self` es la referencia al **objeto concreto** que está ejecutando el método. Cuando escribís `apple.actualizar_precio(180)`, por dentro Python llama `Accion.actualizar_precio(apple, 180)`: `self` *es* `apple` en esa llamada, y `self.precio` es *su* precio, no el de otra acción. Por eso todo método de instancia lleva `self` como primer parámetro.

> **En Python, todo es un objeto**
>
> Esto no es una metáfora: un string, un entero o una lista son instancias de las clases `str`, `int` y `list`. Cuando escribís `"aapl".upper()` o `precios.append(105)` estás llamando métodos de un objeto, exactamente igual que `apple.actualizar_precio(180)`. Llevás tres capítulos usando POO sin saberlo.

4.2. La clase `Bono`: datos + comportamiento en un solo lugar

Vamos al objeto estrella del módulo. Un bono *bullet* tiene datos (ticker, valor nominal, tasa de cupón, plazo) y comportamientos (generar sus flujos, valuarse a una tasa dada). En el Capítulo 2 escribiste esa lógica con bucles; en el 3 la encapsulaste en funciones; ahora los datos y la lógica se mudan juntos a una clase.

```
class Bono:
    """Bono bullet: paga cupón fijo anual y devuelve el nominal al vencimiento."""

    def __init__(self, ticker, valor_nominal, tasa_cupon, anios):
        if tasa_cupon < 0:
            raise ValueError("La tasa de cupón no puede ser negativa")
        self.ticker = ticker
        self.valor_nominal = valor_nominal
        self.tasa_cupon = tasa_cupon
        self.anios = anios

    @property
    def cupon(self):
        """Monto del cupón anual, calculado a partir del estado del bono."""
        return self.valor_nominal * self.tasa_cupon

    def flujos(self):
        """Lista de flujos: cupones todos los años + nominal al final."""
        pagos = [self.cupon] * self.anios
        pagos[-1] = pagos[-1] + self.valor_nominal
        return pagos

    def valuar(self, tasa_descuento):
        """Valor presente de los flujos a la tasa dada (la lógica del Cap. 2)."""
```

```
        vp = 0
        for t, flujo in enumerate(self.flujos(), start=1):
            vp += flujo / (1 + tasa_descuento) ** t
        return vp

    def __repr__(self):
        return f"Bono({self.ticker}: VN ${self.valor_nominal}, cupón {self.tasa_cupo


gd30 = Bono("GD30", valor_nominal=1000, tasa_cupon=0.08, anios=5)

print(gd30)                          # usa __repr__: el objeto se presenta solo
print(f"Cupón anual:  ${gd30.cupon:,.0f}")
print(f"Flujos:       {gd30.flujos()}")
print(f"Valor al 10%: ${gd30.valuar(0.10):,.2f}")
print(f"Valor al 8%:  ${gd30.valuar(0.08):,.2f}")
print(f"Valor al 6%:  ${gd30.valuar(0.06):,.2f}")
```

```
Bono(GD30: VN $1000, cupón 8.0%, 5 años)
Cupón anual:  $80
Flujos:       [80.0, 80.0, 80.0, 80.0, 1080.0]
Valor al 10%: $924.18
Valor al 8%:  $1,000.00
Valor al 6%:  $1,084.25
```

Leé los tres precios del final: al 10 % el bono vale menos que el nominal (bajo la par), al 8 % vale exactamente $1000 (a la par — la tasa de descuento coincide con el cupón), y al 6 % vale más (sobre la par). Es la relación inversa precio-tasa del Capítulo 2, pero ahora **el bono se valúa solo**: `gd30.valuar(0.10)` en lugar de acordarte qué función suelta aplicarle a qué diccionario.

Tres detalles de diseño en esta clase que son marca de calidad:

1. **Validación en el constructor**: el `raise ValueError` impide que exista un bono con cupón negativo. Con el diccionario, nada lo impedía — esta es la **encapsulación** trabajando: el objeto protege sus propios invariantes.
2. **`@property`**: el decorador hace que `cupon` se use como un atributo (`gd30.cupon`, sin paréntesis) pero se **calcule** al momento a partir del estado. Si mañana cambia el nominal, el cupón se actualiza solo — no hay dos copias del dato que puedan desincronizarse.
3. **`__repr__`**: define cómo se muestra el objeto al imprimirlo. Un minuto de trabajo que paga cada vez que hagas debugging.

> Encapsulación: la idea de fondo
>
> Encapsular es ocultar la cocina y exponer el menú: el usuario de la clase llama `valuar(0.10)` sin saber (ni importarle) que adentro hay un bucle con acumulador. Eso te da libertad para mejorar la implementación interna —mañana la vectorizás con NumPy— sin romper el código de nadie que use la clase. Las **interfaces estables con interiores cambiables** son lo que permite construir sistemas grandes.

4.2.1. Atributos de instancia vs. atributos de clase

Todo lo que definimos con `self.` es un **atributo de instancia**: cada objeto tiene su copia. Pero a veces hay datos compartidos por *todos* los objetos de la clase — y para eso existen los **atributos de clase**, definidos directamente en el cuerpo:

```
class BonoLocal:
    mercado = "BYMA"   # atributo de CLASE: compartido por todas las instancias

    def __init__(self, ticker):
        self.ticker = ticker   # atributo de INSTANCIA: único de cada objeto
```

```python
b1, b2 = BonoLocal("GD30"), BonoLocal("AL30")
print(f"{b1.ticker} y {b2.ticker} cotizan en {BonoLocal.mercado}")

# Si el dato compartido cambia, cambia para todos a la vez
BonoLocal.mercado = "MAE"
print(f"Ahora ambos cotizan en: {b1.mercado} / {b2.mercado}")
```

```
GD30 y AL30 cotizan en BYMA
Ahora ambos cotizan en: MAE / MAE
```

Un solo cambio (`BonoLocal.mercado = "MAE"`) y las dos instancias lo ven — esa es la utilidad: constantes y configuración compartida viven en la clase, datos propios de cada objeto viven en la instancia.

Pero hay una sutileza que confunde a todo el mundo la primera vez: ¿qué pasa si asignás el atributo compartido **a través de una instancia**?

```python
# Asignamos "a través de" b1... ¿cambia para todos?
b1.mercado = "NYSE"

print(f"b1.mercado: {b1.mercado}")
print(f"b2.mercado: {b2.mercado}")
print(f"BonoLocal.mercado: {BonoLocal.mercado}")
```

```
b1.mercado: NYSE
b2.mercado: MAE
BonoLocal.mercado: MAE
```

No cambió para todos. La asignación `b1.mercado = "NYSE"` no modificó el atributo de clase: creó un atributo **de instancia** nuevo en `b1` que, a partir de ahí, *tapa* (hace *shadowing*) al de clase — solo para `b1`.

`b2` y la clase siguen en "MAE". La regla de búsqueda de Python lo explica: al leer `objeto.atributo`, primero busca en la instancia y, si no está, sube a la clase. Al **asignar** `objeto.atributo = ...`, siempre escribe en la instancia. Moraleja: los atributos de clase se modifican **por la clase** (`BonoLocal.mercado = ...`), nunca por una instancia.

La Figura 4.2 dibuja esa regla. Leer y escribir **no son simétricos**: la lectura sube a la clase si hace falta, la escritura nunca.

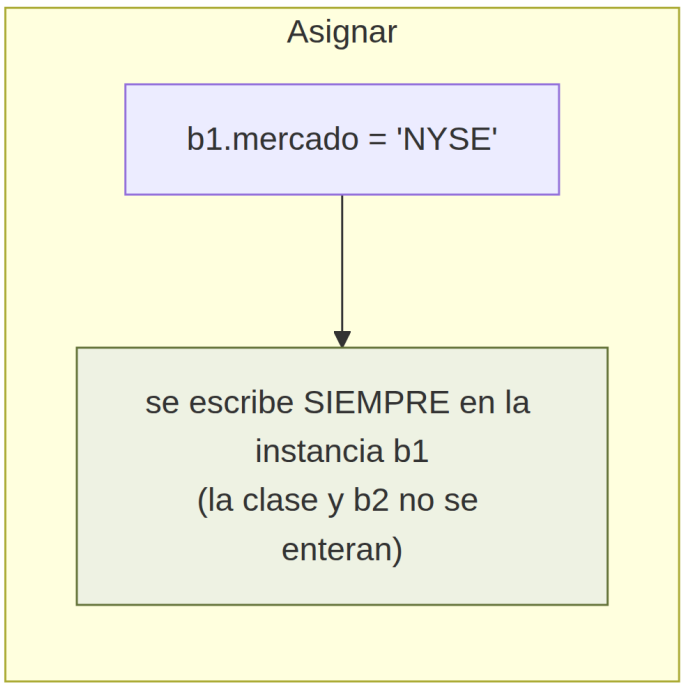
Esto también responde una pregunta natural: ¿da lo mismo definir un atributo en el cuerpo de la clase que dentro del `__init__`? No — son cosas distintas. En el cuerpo de la clase creás **un** atributo compartido; con `self.` dentro del `__init__` creás un atributo **por cada objeto**. Para valores inmutables la diferencia puede pasar inadvertida (por el shadowing, cada asignación posterior "individualiza" el valor), pero con valores **mutables** es un campo minado: la trampa del atributo de clase mutable te espera en los Errores comunes.

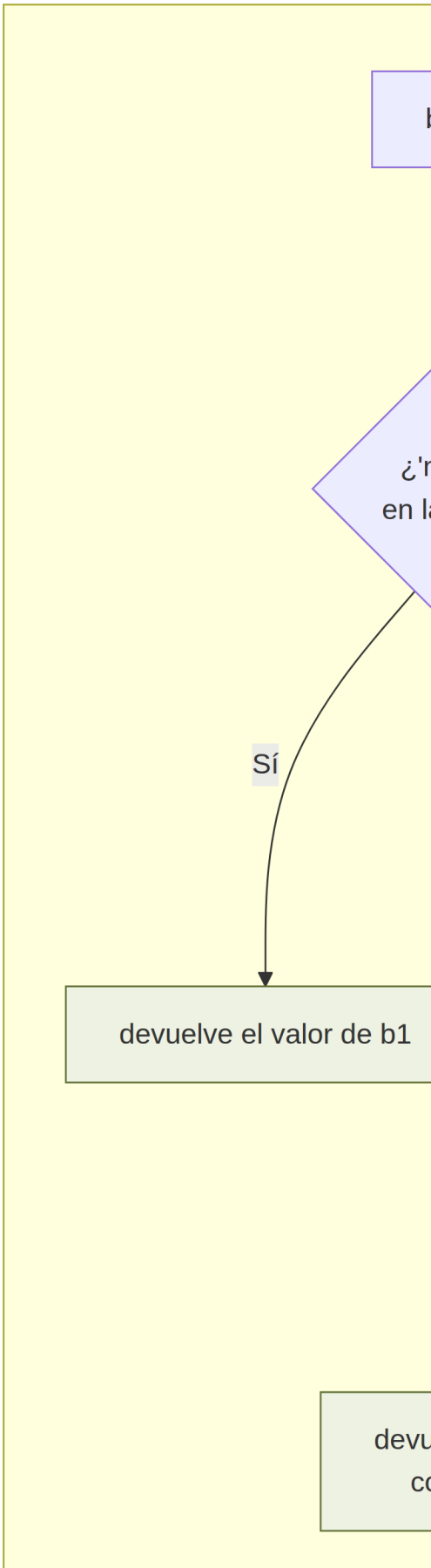
4.3. Interacción y composición: objetos que contienen objetos

Los objetos se vuelven realmente potentes cuando **colaboran**. Un objeto puede recibir a otro como parámetro, o directamente **contenerlo** como atributo — la relación llamada **composición** ("tiene un"): un portafolio *tiene* instrumentos.

```
class Portafolio:
    """Un portafolio TIENE una colección de posiciones (composición)."""

    def __init__(self, nombre):
```


Asignar
b1.mercado = 'NYSE'
se escribe SIEMPRE en la instancia b1
(la clase y b2 no se enteran)


Sí
devuelve el valor de b1

```
        self.nombre = nombre
        self.posiciones = []   # lista de tuplas (instrumento, cantidad)

    def agregar(self, instrumento, cantidad):
        self.posiciones.append((instrumento, cantidad))

    def valor_total(self, tasa_descuento):
        total = 0
        for instrumento, cantidad in self.posiciones:
            total += instrumento.valuar(tasa_descuento) * cantidad
        return total


cartera = Portafolio("Renta Fija Argentina")
cartera.agregar(Bono("GD30", 1000, 0.08, 5), cantidad=10)
cartera.agregar(Bono("AL35", 1000, 0.05, 10), cantidad=20)

print(f"Valor de '{cartera.nombre}' al 10%: ${cartera.valor_total(0.10):,.2f}")
```

```
Valor de 'Renta Fija Argentina' al 10%: $23,097.28
```

La Figura 4.3 muestra esa cadena como una conversación entre objetos, en el orden en que ocurre:

Mirá la cadena de delegación: le pedimos el valor al *portafolio*, el portafolio se lo pide a cada *bono*, y cada bono ejecuta su propio bucle de descuento. Cada clase resuelve **su** parte del problema y delega el resto — nadie sabe más de lo que necesita. Esta forma de partir problemas grandes en cajas chicas que colaboran es el corazón del diseño orientado a objetos.

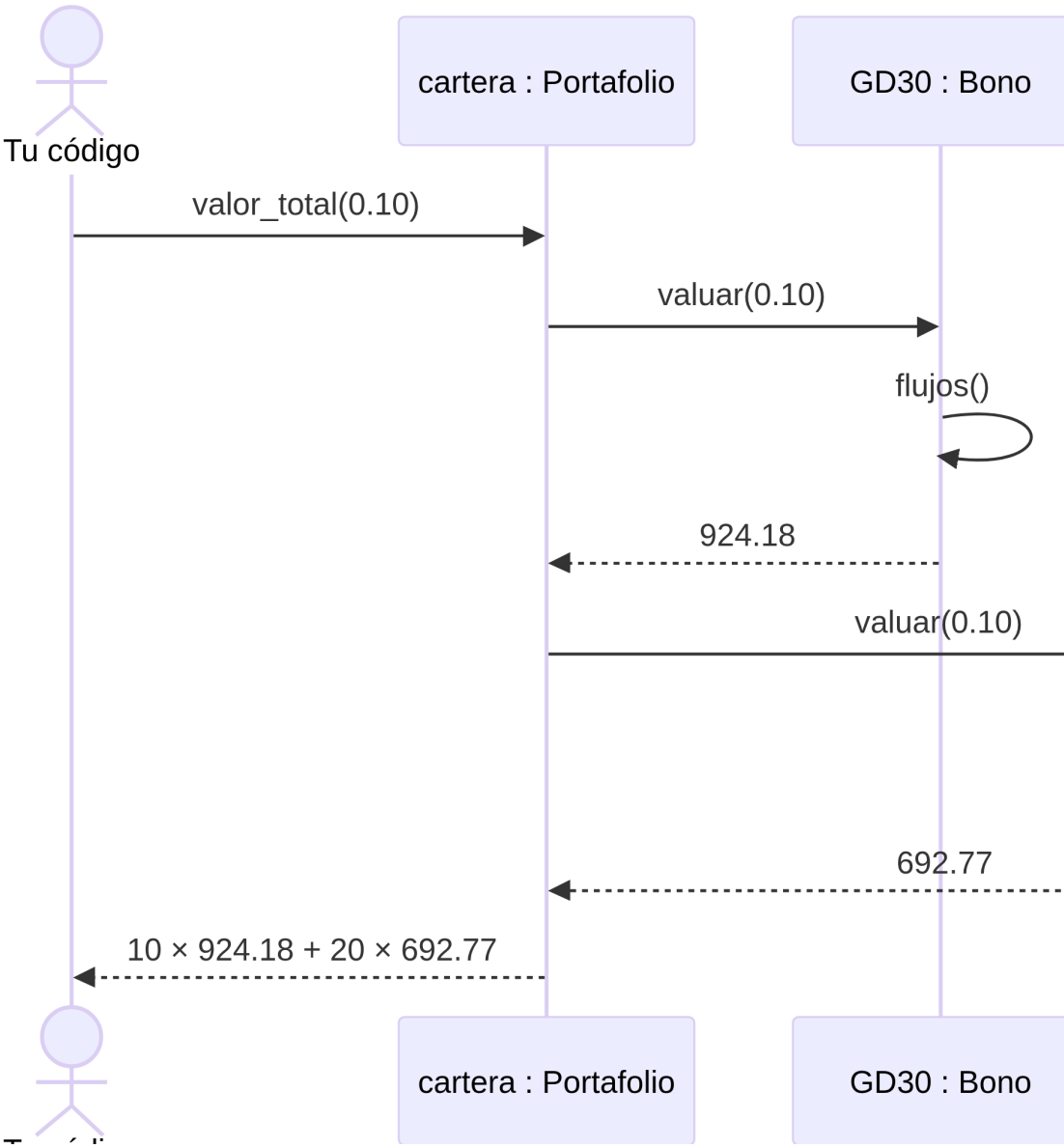


Figura 4.3.: Delegación en la composición: el portafolio no sabe valuar bonos, se lo pide a cada uno; cada bono le pide sus flujos a sí mismo. Cada objeto resuelve solo su parte.

4.4. Herencia: `Bono` → `BonoAmortizable`

La composición modela "tiene un". La **herencia** modela **"es un"**: un bono amortizable *es un* bono — comparte casi todo con el bono bullet, salvo el cronograma de pagos (devuelve el capital en cuotas, no todo al final). Sería absurdo reescribir la clase entera por esa única diferencia. La herencia permite que una **subclase** tome todo lo de su **superclase** y redefina solo lo que cambia:

Antes del código, el mapa completo de las clases del capítulo (Figura 4.4). Las flechas con triángulo son herencia ("es un"); el rombo es composición ("tiene un").

```
class BonoAmortizable(Bono):
    """Es un Bono, pero devuelve el capital en cuotas iguales anuales."""

    def flujos(self):
        # Redefinimos SOLO el cronograma; valuar() se hereda intacto
        amortizacion = self.valor_nominal / self.anios
        pagos = []
        saldo = self.valor_nominal
        for _ in range(self.anios):
            pagos.append(saldo * self.tasa_cupon + amortizacion)  # interés sobre sa
            saldo -= amortizacion
        return pagos


bullet = Bono("GD30", 1000, 0.08, 5)
amortizable = BonoAmortizable("AE38", 1000, 0.08, 5)

print("Flujos bullet:      ", [round(f) for f in bullet.flujos()])
print("Flujos amortizable: ", [round(f) for f in amortizable.flujos()])
print(f"\nValor bullet al 10%:      ${bullet.valuar(0.10):,.2f}")
print(f"Valor amortizable al 10%: ${amortizable.valuar(0.10):,.2f}")
```

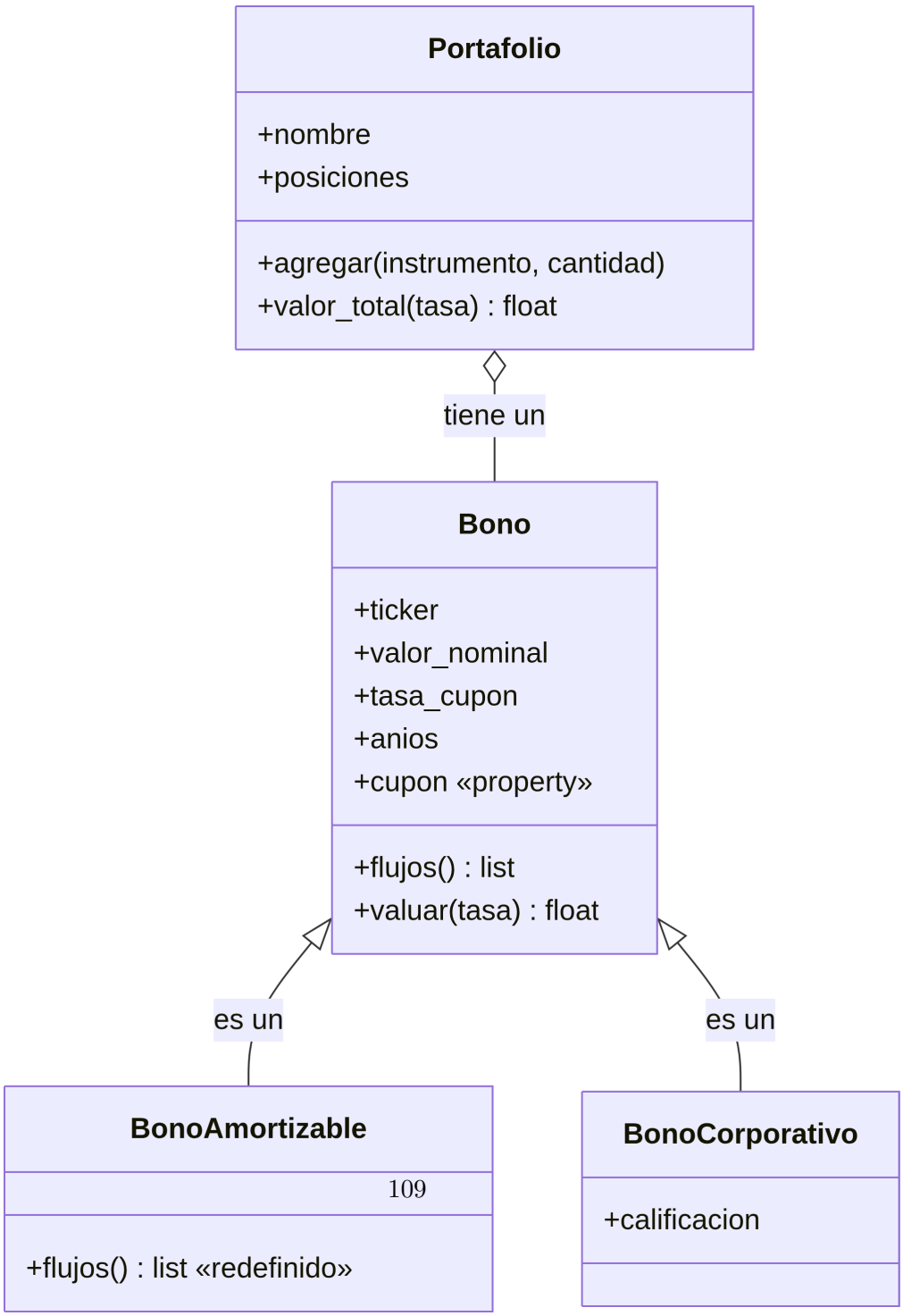


Figura 4.4.: Diagrama de clases del capítulo. `BonoAmortizable` y `BonoCorporativo` heredan de `Bono` (es un): reciben todo y redefinen o agregan solo lo distinto. `Portafolio` contiene bonos (tiene un).

```
Flujos bullet:       [80, 80, 80, 80, 1080]
Flujos amortizable:  [280, 264, 248, 232, 216]

Valor bullet al 10%:      $924.18
Valor amortizable al 10%: $951.63
```

La Figura 4.5 pone los dos cronogramas lado a lado: misma tasa de cupón, mismo plazo, mismo nominal; lo único que cambia es **cuándo** vuelve el capital.

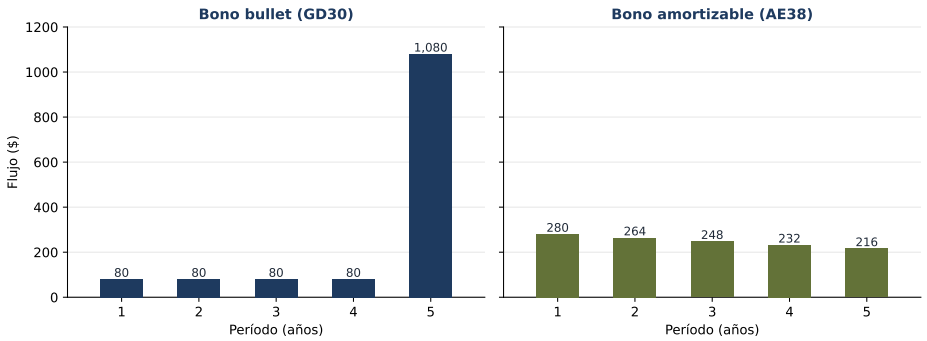


Figura 4.5.: Cronogramas de flujos del bono bullet y del amortizable (nominal 1000, cupón 8 %, 5 años). El bullet concentra el capital al final; el amortizable lo devuelve en cuotas, y sus intereses bajan porque se calculan sobre un saldo que decrece.

Compará los cronogramas: el bullet paga poco cada año y un pago gigante al final; el amortizable devuelve el capital de a partes (por eso sus cupones bajan: el interés se calcula sobre un saldo que decrece). Y ahora la observación importante: **no escribimos `valuar()` en la subclase** — se hereda de `Bono` y funciona perfecto, porque `valuar()` llama a `self.flujos()` y Python usa automáticamente la versión del objeto concreto. El amortizable vale más al 10 % porque recupera el capital antes: menos tiempo descontando a tasa alta. La clase te acaba de enseñar duración sin nombrarla (ese concepto se formaliza en renta fija como *duration*).

4.4.1. `super()`: extender sin reescribir

Cuando la subclase necesita **agregar** atributos propios, usa `super()` para invocar al constructor del padre y no duplicar la inicialización común:

```
class BonoCorporativo(Bono):
    """Es un Bono, con un dato extra: la calificación crediticia."""

    def __init__(self, ticker, valor_nominal, tasa_cupon, anios, calificacion):
        super().__init__(ticker, valor_nominal, tasa_cupon, anios)  # el padre inici
        self.calificacion = calificacion                             # y nosotros lo

corp = BonoCorporativo("YPF29", 1000, 0.09, 4, calificacion="B+")
print(f"{corp} - calificación {corp.calificacion}")
print(f"Valor al 12%: ${corp.valuar(0.12):,.2f}")
```

```
Bono(YPF29: VN $1000, cupón 9.0%, 4 años) - calificación B+
Valor al 12%: $908.88
```

4.4.2. Polimorfismo: una interfaz, muchos comportamientos

La consecuencia más elegante de la herencia: código que opera sobre "bonos en general" sin preguntar de qué tipo es cada uno. El mismo método (`valuar`) ejecuta la rutina correcta según el objeto — eso es el **polimorfismo**:

```
cartera_mixta = [
    Bono("GD30", 1000, 0.08, 5),
    BonoAmortizable("AE38", 1000, 0.08, 5),
    BonoCorporativo("YPF29", 1000, 0.09, 4, "B+"),
]
```

```python
print(f"{'Instrumento':<12}{'Tipo':<20}{'Valor al 10%':>14}")
for bono in cartera_mixta:
    print(f"{bono.ticker:<12}{type(bono).__name__:<20}${bono.valuar(0.10):>12,.2f}")
```

```
Instrumento Tipo                  Valor al 10%
GD30        Bono                $       924.18
AE38        BonoAmortizable     $       951.63
YPF29       BonoCorporativo     $       968.30
```

El bucle no tiene un solo `if` preguntando el tipo: le pide a cada objeto que se valúe, y cada uno usa *sus* flujos. Si el mes que viene agregás una clase `BonoCupónCero`, este bucle la valuará sin cambiar una línea. El polimorfismo es lo que hace que los sistemas orientados a objetos **crezcan sin romperse**.

4.5. Buenas prácticas de diseño

Tres convenciones que el ecosistema Python espera de tu código orientado a objetos:

1. **Nomenclatura**: clases en `PascalCase` y en singular (`Bono`, `CalculadoraRiesgo`); métodos y atributos en `snake_case` con verbos para las acciones (`calcular_tir()`, `obtener_precio()`).
2. **Visibilidad por convención**: un guion bajo inicial (`_saldo`) señala "uso interno, no toques desde afuera"; doble guion (`__clave`) activa el ocultamiento fuerte de Python. No hay `private` como en otros lenguajes — hay acuerdos entre adultos.

3. **Principio de Responsabilidad Única (SRP)**: cada clase debe tener **una** razón para cambiar. Si tu clase `Bono` empieza a descargar precios de internet, graficar y mandar emails, es hora de dividirla: el bono valúa, otra clase descarga, otra grafica.

> **i** ¿Y el destructor `__del__`?
>
> Python también tiene un método `__del__` que se ejecuta cuando el objeto es eliminado de memoria por el recolector de basura. En la práctica se usa poco y conviene evitarlo: **no sabés cuándo** va a ejecutarse (el recolector decide), lo que lo hace poco confiable para liberar recursos importantes. Para eso, el idioma moderno es el *context manager* (`with`), que vas a ver al trabajar con archivos.

4.6. Errores comunes

> **⚠** Trampas frecuentes
>
> - **Olvidar `self`**: escribir `def valuar(tasa)` en lugar de `def valuar(self, tasa)` produce el críptico `takes 1 positional argument but 2 were given`. Python siempre pasa el objeto como primer argumento — tenés que recibirlo.
> - **Olvidar `self.` al asignar**: dentro de `__init__`, `precio = precio` crea una variable local que muere al salir del método; el atributo se crea con `self.precio = precio`. El síntoma: `AttributeError` al usar el objeto.
> - **Atributo de clase mutable**: definir `posiciones = []` a nivel de clase hace que **todas** las instancias compartan la misma lista — agregás un bono a una cartera y aparece en todas. Las colec-

ciones siempre se inicializan en `__init__` con `self.posiciones = []`.

- **Redefinir un método y romper el contrato**: si tu subclase redefine `flujos()` pero devuelve otra cosa (un dict en vez de una lista), todos los métodos heredados que dependían de él (`valuar`) explotan. Al heredar, respetá las firmas y tipos del padre.
- **Herencia donde iba composición**: `Portafolio(Bono)` es un error conceptual — un portafolio no *es* un bono, *tiene* bonos. La pregunta "¿es un… o tiene un…?" decide entre heredar y componer.

4.7. Referencias

5. Capítulo 5: NumPy y Cálculo Vectorizado

El motor numérico de las finanzas cuantitativas en Python

NumPy es la librería sobre la que se construye todo el stack científico de Python (Pandas, SciPy, scikit-learn). En este capítulo entendemos por qué el `ndarray` reemplaza a las listas para trabajar con números, aprendemos a vectorizar cálculos para eliminar bucles lentos, dominamos el broadcasting y damos nuestros primeros pasos en álgebra lineal, que será la columna vertebral de la optimización de portafolios del Módulo 2.

Introducción

Hasta ahora venimos representando datos financieros con **listas** y **diccionarios** de Python. Nos sirvieron para aprender a programar, pero tienen un problema serio: son lentas y verbosas cuando hay que hacer cuentas sobre miles de números. Y en finanzas, *siempre* hay miles de números: años de cotizaciones diarias, paneles de cientos de activos, millones de trayectorias en una simulación.

Acá entra **NumPy** (de *Numerical Python*). Es la librería que le da a Python la capacidad de hacer cálculo numérico a la velocidad de un lenguaje compilado como C, pero con la comodidad de escribir Python. No es una librería más: es la base sobre la que están construidos Pandas, SciPy, scikit-learn y prácticamente todo lo que vas a usar de acá en adelante. Cuando en el Capítulo 6 hagas `pd.DataFrame`, por debajo hay arrays de NumPy; cuando en el Capítulo 8 optimices un portafolio de Markowitz, vas a estar multiplicando matrices de NumPy.

> **La idea de fondo**
>
> Programar con NumPy es cambiar el chip: en vez de pensar *"recorré la lista elemento por elemento y hacé la cuenta"*, pensás *"aplicá esta operación a todo el vector de una"*. Eso se llama **vectorización**, y es la habilidad central de este capítulo.

```
import numpy as np
```

Por convención universal, NumPy se importa como `np`. Vas a ver ese `np.`

en todo el libro (y en todo el ecosistema de Python científico), así que conviene acostumbrarse desde ya.

5.1. El problema: las listas no fueron hechas para calcular

Imaginá que tenés los precios de cierre de una acción durante cinco días y querés calcular el retorno simple diario, $R_t = P_t / P_{t-1} - 1$. Con listas puras, tenés que escribir un bucle:

```
precios = [100.0, 102.5, 101.0, 103.5, 105.0]

retornos = []
for i in range(1, len(precios)):
    retornos.append(precios[i] / precios[i-1] - 1)

print(retornos)
```

```
[0.02499999999999991, -0.014634146341463428, 0.024752475247524774, 0.014492753623188
```

Funciona, pero fijate todo lo que tuviste que hacer: crear una lista vacía, manejar índices, acordarte de arrancar en `1`. Y esto es para una sola acción y cinco días. Con NumPy, el mismo cálculo es una línea que se lee casi como la fórmula matemática:

```
precios = np.array([100.0, 102.5, 101.0, 103.5, 105.0])

retornos = precios[1:] / precios[:-1] - 1

print(retornos)
```

```
[ 0.025       -0.01463415  0.02475248  0.01449275]
```

`precios[1:]` es "todos los precios desde el segundo" (P_t) y `precios[:-1]` es "todos menos el último" (P_{t-1}). NumPy divide los dos vectores **elemento a elemento** automáticamente. Sin bucles, sin índices, sin listas vacías. Esto no es solo más corto: sobre datos reales es **decenas o cientos de veces más rápido**, porque la operación corre en código C optimizado en lugar de un bucle interpretado de Python.

La Figura 5.1 muestra el truco: los dos recortes son el mismo array **desfasado una posición**, así que al ponerlos uno sobre otro cada precio queda alineado con el del día anterior.

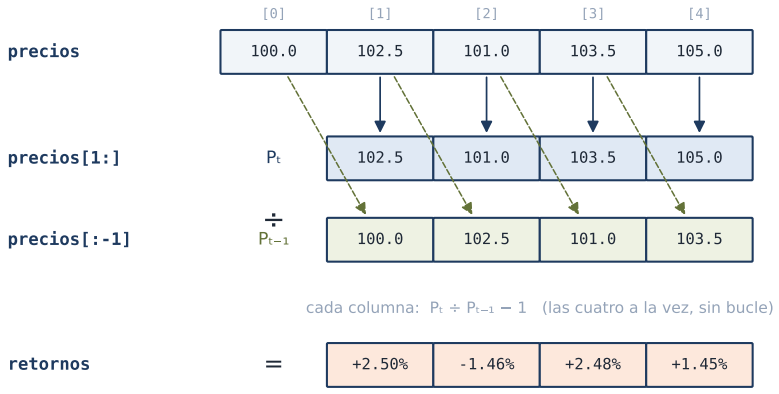


Figura 5.1.: Retornos sin bucles: `precios[1:]` (P) y `precios[:-1]` (P) son el mismo array corrido una posición. Divididos elemento a elemento, cada columna produce el retorno de un día.

5.2. El `ndarray`: qué es y por qué es distinto

El objeto central de NumPy es el **array** (`ndarray`, de *N-dimensional array*). A primera vista se parece a una lista, pero tiene tres diferencias que lo cambian todo:

1. **Es homogéneo**: todos los elementos son del mismo tipo (todos `float64`, todos `int`, etc.). Una lista de Python puede mezclar números, strings y objetos; un array no. Esa restricción es justamente lo que lo hace rápido.
2. **Es contiguo en memoria**: los datos se guardan en un bloque compacto y ordenado de RAM, no desperdigados con punteros como en una lista. El procesador puede recorrerlos de corrido.
3. **Opera vectorizado**: las operaciones aritméticas se aplican a todo el array de una sola vez, apoyándose en instrucciones de bajo nivel del procesador.

La Figura 5.2 muestra la diferencia de fondo entre los puntos 1 y 2: una lista guarda *referencias* a objetos sueltos; un array guarda los números mismos, uno al lado del otro.

```
a = np.array([10, 20, 30, 40])

print("Array:   ", a)
print("Tipo:    ", type(a))
print("dtype:   ", a.dtype)     # tipo de los elementos
print("shape:   ", a.shape)     # forma (dimensiones)
print("ndim:    ", a.ndim)      # cantidad de dimensiones
print("size:    ", a.size)      # cantidad total de elementos
```

```
Array:    [10 20 30 40]
Tipo:     <class 'numpy.ndarray'>
dtype:    int64
```

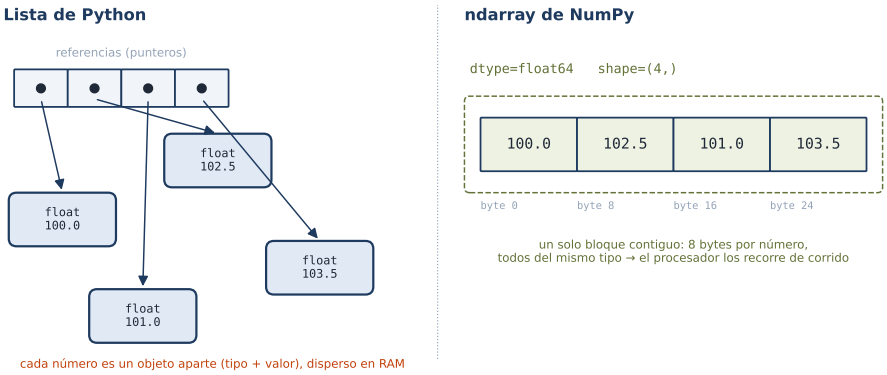


Figura 5.2.: Una lista de Python guarda referencias a objetos `float` independientes, cada uno en su lugar de la memoria. Un `ndarray` guarda los valores crudos en un bloque contiguo de 8 bytes por número, todos del mismo tipo.

```
shape:    (4,)
ndim:     1
size:     4
```

El **`dtype`** (data type) es un concepto clave: le dice a NumPy exactamente cuántos bytes ocupa cada número y cómo interpretarlos. En finanzas trabajamos casi siempre con `float64` (números reales de doble precisión), que es el default para decimales.

> **shape: la forma manda**
>
> El atributo `.shape` es una tupla que describe las dimensiones. Un vector de 4 elementos tiene shape `(4,)`; una matriz de 3 filas por 2 columnas tiene shape `(3, 2)`. Cuando algo falla en NumPy, el 90 % de las veces es un problema de shapes que no encajan. Acostumbrate a mirar `.shape` como primer paso de debugging.

5.2.1. Crear arrays

Rara vez vas a escribir los números a mano. NumPy trae funciones para generar los arrays más comunes:

```
# Desde una lista
np.array([1.0, 2.0, 3.0])

# Rango de valores (como range, pero array): inicio, fin (excluido), paso
plazos = np.arange(0, 12, 3)          # [0, 3, 6, 9] meses
print("arange:  ", plazos)

# N valores equiespaciados entre dos extremos (útil para grillas)
grilla = np.linspace(0, 1, 5)         # 5 puntos entre 0 y 1
print("linspace:", grilla)

# Arrays de ceros y unos (útiles para inicializar acumuladores)
print("zeros:   ", np.zeros(4))
print("ones:    ", np.ones(3))

# Matriz identidad (aparecerá en álgebra de portafolios)
print("identidad:\n", np.eye(3))
```

```
arange:   [0 3 6 9]
linspace: [0.   0.25 0.5  0.75 1.  ]
zeros:    [0. 0. 0. 0.]
ones:     [1. 1. 1.]
identidad:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

5.2.2. Arrays de varias dimensiones

Un vector es 1D; una **matriz** es 2D. En finanzas la matriz más importante es el **panel de retornos**: filas = fechas, columnas = activos.

```
# Retornos mensuales de 3 activos durante 4 meses (4 filas x 3 columnas)
R = np.array([
    [0.02, -0.01,  0.03],
    [0.01,  0.02, -0.02],
    [-0.03, 0.01,  0.04],
    [0.02,  0.00,  0.01],
])

print("Panel de retornos:\n", R)
print("\nShape:", R.shape, "→ 4 fechas, 3 activos")
```

```
Panel de retornos:
 [[ 0.02 -0.01  0.03]
 [ 0.01  0.02 -0.02]
 [-0.03  0.01  0.04]
 [ 0.02  0.    0.01]]

Shape: (4, 3) → 4 fechas, 3 activos
```

5.3. Números aleatorios: el generador moderno

Buena parte de las finanzas computacionales consiste en simular escenarios, y para eso necesitamos números aleatorios. NumPy recomienda hoy usar un **generador** explícito, creado con `np.random.default_rng()`. Le pasamos

una **semilla** (`seed`) para que los resultados sean reproducibles: la misma semilla da siempre los mismos "azares", lo cual es fundamental para que tus ejemplos y los de tus compañeros coincidan.

```
rng = np.random.default_rng(seed=42)

# 5 retornos diarios simulados: media 0.05%, desvío 1.5%
retornos_sim = rng.normal(loc=0.0005, scale=0.015, size=5)
print("Retornos simulados:", retornos_sim)

# Una matriz 3x3 de números uniformes entre 0 y 1
print("\nUniformes:\n", rng.random((3, 3)))
```

```
Retornos simulados: [ 0.00507076 -0.01509976  0.01175677  0.01460847 -
0.02876553]

Uniformes:
 [[0.97562235 0.7611397  0.78606431]
 [0.12811363 0.45038594 0.37079802]
 [0.92676499 0.64386512 0.82276161]]
```

> ⚠ Vas a ver `np.random.seed()` en código viejo
>
> Mucho código que vas a encontrar en internet usa el estilo antiguo `np.random.seed(42)` y `np.random.normal(...)`. Funciona, pero está considerado *legacy*. El estilo moderno con `default_rng()` es más robusto y es el que vas a encontrar en la documentación actual. Usá `rng.` de acá en adelante.

5.4. Indexado, slicing y máscaras booleanas

Acceder a partes de un array es donde NumPy se pone realmente potente.

5.4.1. Slicing básico

Funciona como en las listas, con `[inicio:fin:paso]`, pero se extiende a varias dimensiones separando con comas: `matriz[filas, columnas]`.

```
# Usamos el panel R de antes
print("Primera fila (retornos de la fecha 0):", R[0])
print("Primera columna (activo 0 en el tiempo):", R[:, 0])
print("Submatriz (2 primeras fechas, 2 primeros activos):\n", R[:2, :2])
print("Último retorno del último activo:", R[-1, -1])
```

```
Primera fila (retornos de la fecha 0): [ 0.02 -0.01  0.03]
Primera columna (activo 0 en el tiempo): [ 0.02  0.01 -
0.03  0.02]
Submatriz (2 primeras fechas, 2 primeros activos):
 [[ 0.02 -0.01]
 [ 0.01  0.02]]
Último retorno del último activo: 0.01
```

La coma es la clave: `R[:, 0]` significa "todas las filas, columna 0". Poder pedir una columna entera de un panel sin bucles es exactamente lo que necesitamos para analizar un activo a lo largo del tiempo.

5.4.2. Máscaras booleanas: filtrar por condición

Esta es una de las herramientas más usadas en análisis financiero. Una comparación sobre un array devuelve un array de `True`/`False`, y podemos usar ese array como "filtro":

```python
retornos_diarios = np.array([0.012, -0.008, 0.003, -0.021, 0.015, -0.004])

# La comparación genera una máscara booleana
mascara_perdidas = retornos_diarios < 0
print("Máscara (días con pérdida):", mascara_perdidas)

# Usamos la máscara para quedarnos solo con esos elementos
perdidas = retornos_diarios[mascara_perdidas]
print("Retornos negativos:        ", perdidas)

print(f"\nDías con pérdida: {mascara_perdidas.sum()} de {retornos_diarios.size}")
print(f"Pérdida promedio: {perdidas.mean():.2%}")
```

```
Máscara (días con pérdida): [False  True False  True False  True]
Retornos negativos:         [-0.008 -0.021 -0.004]

Días con pérdida: 3 de 6
Pérdida promedio: -1.10%
```

Fijate el truco de `mascara_perdidas.sum()`: como `True` vale 1 y `False` vale 0, sumar una máscara booleana **cuenta cuántas condiciones se cumplen**. Es un patrón que vas a repetir mil veces (¿cuántos días superaron el VaR?, ¿cuántos activos rindieron más que el mercado?, etc.).

5.5. Vectorización: pensar en vectores, no en bucles

Ya vimos la idea con los retornos. Generalicémosla. Las operaciones aritméticas (`+`, `-`, `*`, `/`, `**`) y las funciones de NumPy (`np.log`, `np.exp`, `np.sqrt`…) se aplican **elemento a elemento** sobre todo el array.

```
precios = np.array([100.0, 102.5, 101.0, 103.5, 105.0])

# Retorno logarítmico: r_t = ln(P_t / P_{t-1}), en una línea
ret_log = np.log(precios[1:] / precios[:-1])
print("Retornos log:", ret_log)

# Convertir un vector entero de tasas anuales a mensuales, todo de una
tasas_anuales = np.array([0.40, 0.35, 0.50, 0.45])
tasas_mensuales = (1 + tasas_anuales) ** (1/12) - 1
print("Tasas mensuales equivalentes:", np.round(tasas_mensuales, 4))
```

```
Retornos log: [ 0.02469261 -0.01474228  0.0244511   0.01438874]
Tasas mensuales equivalentes: [0.0284 0.0253 0.0344 0.0314]
```

> **La regla de oro**
>
> Si te encontrás escribiendo un `for` que recorre un array para hacer una cuenta en cada elemento, casi siempre hay una versión vectorizada más corta y mucho más rápida. Antes de escribir el bucle, preguntate: *"¿puedo aplicarle esto al array entero?"*

5.5.1. ¿Cuánto más rápido es, en serio?

No es una diferencia cosmética. Comparemos sumar el cuadrado de un millón de números con un bucle de Python contra NumPy:

```
import time

datos = rng.normal(size=1_000_000)

# Versión con bucle Python
```

```python
t0 = time.perf_counter()
suma = 0.0
for x in datos:
    suma += x**2
t_bucle = time.perf_counter() - t0

# Versión vectorizada
t0 = time.perf_counter()
suma_np = np.sum(datos**2)
t_numpy = time.perf_counter() - t0

print(f"Bucle Python:  {t_bucle*1000:7.1f} ms")
print(f"NumPy:         {t_numpy*1000:7.1f} ms")
print(f"NumPy es ~{t_bucle/t_numpy:.0f}x más rápido")
```

```
Bucle Python:    193.7 ms
NumPy:             1.9 ms
NumPy es ~102x más rápido
```

Sobre un millón de datos la diferencia es de uno a dos órdenes de magnitud. En una simulación de Monte Carlo con millones de trayectorias (Capítulo 13), esto es la diferencia entre que tu código termine en segundos o en horas.

¿De dónde sale esa diferencia? Del **trabajo repetido que hace el intérprete**. En el bucle de Python, *cada una* del millón de vueltas repite la misma burocracia: mirar qué tipo tiene `x` (¿es int? ¿float? ¿string?), desempaquetar el valor del objeto que lo envuelve, decidir qué operación "elevar al cuadrado" corresponde a ese tipo, ejecutarla, y volver a empaquetar el resultado en un objeto nuevo. Un millón de elementos, un millón de veces la misma verificación. NumPy hace esa burocracia **una sola vez**: como el array es homogéneo (todos `float64`, gracias al `dtype`), sabe de antemano qué operación aplicar a todos, y delega el recorrido a un bucle

compilado en C que barre la memoria contigua a toda velocidad — incluso procesando varios elementos por instrucción del procesador (SIMD). No es que Python "sea lento": es que le pedimos que vuelva a deducir lo obvio un millón de veces. La vectorización es, en el fondo, **mover la decisión fuera del bucle**.

5.6. Reducciones: resumir un array

Una **reducción** toma muchos números y devuelve pocos: la suma, el promedio, el máximo, el desvío. Son la base de toda la estadística descriptiva.

```
ret = np.array([0.012, -0.008, 0.003, -0.021, 0.015, -0.004])

print(f"Suma (retorno acumulado aprox): {ret.sum():.4f}")
print(f"Media:                          {ret.mean():.4f}")
print(f"Desvío estándar (volatilidad):  {ret.std():.4f}")
print(f"Mínimo (peor día):              {ret.min():.4f}")
print(f"Máximo (mejor día):             {ret.max():.4f}")
```

```
Suma (retorno acumulado aprox): -0.0030
Media:                          -0.0005
Desvío estándar (volatilidad):  0.0122
Mínimo (peor día):              -0.0210
Máximo (mejor día):             0.0150
```

5.6.1. El parámetro `axis`: el concepto que más confunde (y más importa)

Cuando reducimos una **matriz**, tenemos que decidir *en qué dirección* resumimos. Ahí entra `axis`:

- `axis=0` colapsa las **filas** → un resultado **por columna** (por activo).
- `axis=1` colapsa las **columnas** → un resultado **por fila** (por fecha).

Con un panel de retornos (filas = fechas, columnas = activos), esto es exactamente lo que necesitamos:

```
# Panel: 4 fechas x 3 activos
R = np.array([
    [0.02, -0.01,  0.03],
    [0.01,  0.02, -0.02],
    [-0.03, 0.01,  0.04],
    [0.02,  0.00,  0.01],
])

# Retorno medio DE CADA ACTIVO → colapsamos las fechas (axis=0)
print("Media por activo (axis=0):", R.mean(axis=0))

# Volatilidad DE CADA ACTIVO
print("Vol por activo   (axis=0):", R.std(axis=0))

# Retorno DE CADA FECHA (un portafolio equiponderado, por ejemplo) → axis=1
print("Media por fecha  (axis=1):", R.mean(axis=1))
```

```
Media por activo (axis=0): [0.005 0.005 0.015]
Vol por activo   (axis=0): [0.02061553 0.01118034 0.02291288]
Media por fecha  (axis=1): [0.01333333 0.00333333 0.00666667 0.01      ]
```

La Figura 5.3 lo muestra sobre el panel `R`: la dirección de la flecha es la dirección en la que se **colapsa** el array.

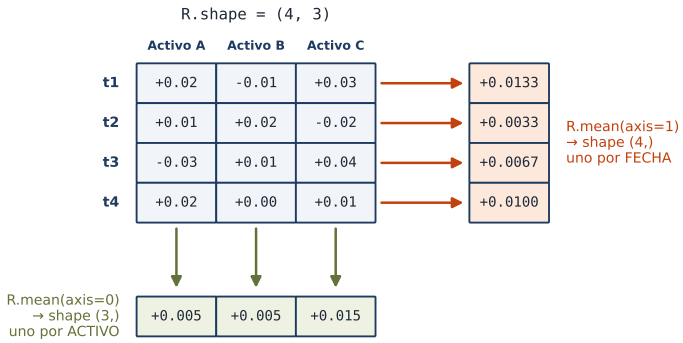


Figura 5.3.: El parámetro `axis` sobre un panel de 4 fechas × 3 activos. `axis=0` colapsa las filas (hacia abajo) y deja un valor por activo; `axis=1` colapsa las columnas (hacia la derecha) y deja un valor por fecha.

i Truco para no confundirte con axis

`axis=0` "apunta hacia abajo" (recorre las filas y deja una fila-resumen), así que el resultado tiene tantos elementos como **columnas**. Regla mnemotécnica: el eje que ponés en `axis` es el que **desaparece** del shape. Si `R.shape` es `(4, 3)`, entonces `R.mean(axis=0).shape` es `(3,)`.

5.7. Broadcasting: operar arrays de distinta forma

¿Qué pasa si querés sumarle un número a todo un array, o restarle a cada columna su propia media? NumPy tiene una regla elegante llamada **broad-**

casting: cuando las formas no coinciden, NumPy "estira" automáticamente el array más chico para que encajen, sin copiar datos de verdad.

El caso más simple ya lo usaste sin darte cuenta:

```
precios = np.array([100.0, 102.5, 101.0])

# Sumar un escalar a un vector: el escalar se "estira" a todos los elementos
print("Con impuesto del 1%:", precios * 1.01)
```

```
Con impuesto del 1%: [101.    103.525 102.01 ]
```

El caso poderoso es **normalizar un panel**: restarle a cada activo su media y dividir por su desvío (estandarización), una operación constante en modelos de factores y machine learning financiero.

```
R = np.array([
    [0.02, -0.01,  0.03],
    [0.01,  0.02, -0.02],
    [-0.03, 0.01,  0.04],
    [0.02,  0.00,  0.01],
])

medias = R.mean(axis=0)   # shape (3,) → una media por activo
desvios = R.std(axis=0)   # shape (3,) → un desvío por activo

# R tiene shape (4,3) y medias shape (3,): broadcasting alinea por columna
R_estandarizado = (R - medias) / desvios

print("Medias por activo:", np.round(medias, 4))
print("\nPanel estandarizado (media 0, desvío 1 por columna):\n",
      np.round(R_estandarizado, 3))
print("\nVerificación - media de cada columna:", np.round(R_estandarizado.mean(axis=
```

```
Medias por activo: [0.005 0.005 0.015]

Panel estandarizado (media 0, desvío 1 por columna):
 [[ 0.728 -1.342  0.655]
 [ 0.243  1.342 -1.528]
 [-1.698  0.447  1.091]
 [ 0.728 -0.447 -0.218]]

Verificación - media de cada columna: [ 0.  0. -0.]
```

La Figura 5.4 muestra lo que NumPy "imagina" para hacer la resta: repite el vector de medias en cada fila (las copias fantasma), pero sin copiar memoria de verdad.

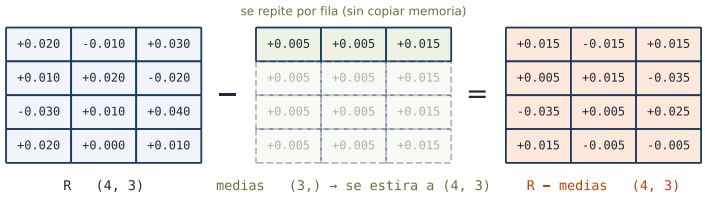


Figura 5.4.: Broadcasting en `R - medias`: el vector de forma (3,) se estira virtualmente a (4, 3), repitiéndose en cada fila, y la resta se hace elemento a elemento.

NumPy vio que `R` es `(4, 3)` y `medias` es `(3,)`, y entendió que tenía que restar **`medias` a cada fila**. No tuviste que escribir ningún bucle sobre las 4 fechas.

⚠ Cuándo el broadcasting falla

El broadcasting solo funciona si las dimensiones son compatibles: o son iguales, o una de ellas es 1. Si intentás operar un `(4, 3)` con un `(4,)` vas a recibir un `ValueError: operands could not be broadcast together`. La solución suele ser reformar el vector a `(4, 1)` con `vector.reshape(-1, 1)` para que se alinee por fila en lugar de por columna. Cuando veas ese error, mirá los shapes.

5.8. Álgebra lineal: la antesala de Markowitz

Acá NumPy muestra para qué fue diseñado. La optimización de portafolios del Módulo 2 se escribe entera en lenguaje matricial, y NumPy lo implementa directo.

El **producto matricial** se hace con el operador `@` (no con `*`, que es elemento a elemento). Veamos el cálculo más importante de toda la teoría de portafolios: el **retorno esperado de un portafolio**, que es el producto entre el vector de pesos **w** y el vector de retornos esperados μ.

```
# Pesos del portafolio (40% activo 1, 35% activo 2, 25% activo 3)
w = np.array([0.40, 0.35, 0.25])

# Retornos esperados anuales de cada activo
mu = np.array([0.12, 0.18, 0.08])

# Retorno esperado del portafolio: w · mu (producto punto)
ret_portafolio = w @ mu
print(f"Retorno esperado del portafolio: {ret_portafolio:.2%}")
```

```
Retorno esperado del portafolio: 13.10%
```

Y ahora la estrella: la **matriz de covarianzas** Σ, que resume cómo se mueven juntos los activos y es el corazón de la medición de riesgo. NumPy la calcula con `np.cov`. Con ella, la **varianza del portafolio** es la forma cuadrática $\mathbf{w}^T \Sigma \mathbf{w}$:

```python
# Panel de retornos: activos en FILAS, observaciones en COLUMNAS (rowvar=True, el de
rng = np.random.default_rng(7)
R_hist = rng.normal(loc=[0.001, 0.0015, 0.0008],
                    scale=[0.02, 0.03, 0.015],
                    size=(250, 3)).T   # .T → 3 activos x 250 días

# Matriz de covarianzas (3x3)
Sigma = np.cov(R_hist)
print("Matriz de covarianzas (Σ):\n", np.round(Sigma, 6))

# Varianza del portafolio: w  Σ w
var_portafolio = w @ Sigma @ w
vol_portafolio = np.sqrt(var_portafolio)
print(f"\nVolatilidad diaria del portafolio: {vol_portafolio:.4f}")
print(f"Volatilidad anualizada:            {vol_portafolio * np.sqrt(252):.2%}")
```

```
Matriz de covarianzas (Σ):
 [[ 3.88e-04  1.30e-05  1.20e-05]
 [ 1.30e-05  7.42e-04 -3.90e-05]
 [ 1.20e-05 -3.90e-05  1.97e-04]]

Volatilidad diaria del portafolio: 0.0128
Volatilidad anualizada:            20.36%
```

Esas dos líneas —`w @ mu` y `w @ Sigma @ w`— son, literalmente, la Teoría Moderna de Portafolios. En el Capítulo 8 le vamos a pedir a la computadora

que busque los pesos `w` que minimizan ese riesgo. Todo el andamiaje matemático ya lo tenés en NumPy.

El submódulo `np.linalg` completa el kit de álgebra lineal (inversas, determinantes, autovalores, resolución de sistemas):

```
M = np.array([[4.0, 1.0],
              [1.0, 3.0]])

print("Inversa:\n", np.linalg.inv(M))
print("\nDeterminante:", np.linalg.det(M))
print("Autovalores: ", np.linalg.eigvals(M))
```

```
Inversa:
 [[ 0.27272727 -0.09090909]
 [-0.09090909  0.36363636]]

Determinante: 11.000000000000002
Autovalores:  [4.61803399 2.38196601]
```

> **Guardá esto para más adelante**
>
> Black-Litterman (Capítulo 10) y la regresión por mínimos cuadrados (Capítulo 15) se resuelven invirtiendo matrices con `np.linalg.inv` y `np.linalg.solve`. El código que acabás de ver es la base de la mitad del libro.

5.9. De NumPy a Pandas

NumPy es potentísimo, pero tiene una limitación para el trabajo diario: los arrays no tienen **etiquetas**. En `R[:, 0]` vos sabés que la columna 0 es "el activo A" porque te lo acordás, no porque el array lo diga. Cuando manejás decenas de activos con fechas de por medio, eso se vuelve un problema.

Ahí entra **Pandas** (Capítulo 6): pone etiquetas (nombres de columnas y un índice de fechas) *encima* de los arrays de NumPy. Un `DataFrame` es, en el fondo, un array de NumPy con nombres. Por eso todo lo que aprendiste acá —vectorización, broadcasting, `axis`, máscaras booleanas— se traslada casi idéntico a Pandas. No estás aprendiendo dos cosas: estás aprendiendo NumPy, y Pandas es NumPy con etiquetas.

5.10. Errores comunes

⚠ Trampas frecuentes con NumPy

- **Confundir `*` con `@`**: `A * B` multiplica elemento a elemento; `A @ B` es el producto matricial. Para portafolios casi siempre querés `@`.
- **Copias vs. vistas**: un slice como `b = a[:2]` **no copia** los datos, es una *vista*. Si modificás `b`, modificás `a`. Si necesitás una copia independiente, usá `a[:2].copy()`.
- **Olvidarse del `axis`**: `R.mean()` sin `axis` promedia *todos* los elementos y te da un solo número, no uno por activo. Casi siempre querés especificar `axis`.
- **Enteros donde querías decimales**: `np.array([1, 2, 3])` crea un array de enteros, y asignarle un decimal lo trunca sin avisar (`a[0] = 2.7` guarda `2`). Escribí `1.0` o pasá `dtype=float`

para asegurarte de trabajar con `float64`.

5.11. Referencias

6. Capítulo 6: Series de Tiempo y Análisis de Datos con Pandas

Estructuras de Datos, Manipulación y Cálculos Financieros

Introducción a pandas como herramienta fundamental para la carga, limpieza y análisis de datos en finanzas. Estudiamos Series y DataFrames, manipulación de series temporales, cálculos vectorizados de retornos, ventanas móviles y visualización técnica.

Introducción

En el capítulo anterior descubriste el poder de NumPy, pero también su limitación: los arrays no tienen **etiquetas**. `R[:, 0]` es "la columna 0", y vos tenés que acordarte de que esa columna es el GD30 y de que la fila 40 es el 3 de marzo. Con dos activos se tolera; con doscientos activos y diez años de fechas, es inmanejable.

Pandas resuelve exactamente eso: pone nombres de columnas y un índice (típicamente de fechas) *encima* de los arrays de NumPy. No es una librería nueva que aprender desde cero — es NumPy con etiquetas: la vectorización, el broadcasting y las máscaras booleanas que ya dominás funcionan igual. Lo que se agrega es la maquinaria para el trabajo diario con datos financieros reales: leer archivos, alinear fechas, rellenar feriados, calcular retornos y ventanas móviles, agrupar por sector.

Este capítulo recorre esa maquinaria y cierra integrando todo en un caso real de renta fija: construir la **curva de rendimientos** a partir de precios de bonos.

6.1. Las estructuras: Series y DataFrames

Pandas se integra perfectamente con el ecosistema que venimos usando (NumPy y Matplotlib). Sus dos estructuras centrales son la Serie (1D) y el DataFrame (2D).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

6.1.1. Series (1D)

Una **Serie** es un array unidimensional con un índice de etiquetas. Compará con el array de NumPy: ahí el "lunes" era la posición 0 y tenías que recordarlo; acá el dato **lleva puesta su etiqueta**.

```
precios_serie = pd.Series([110, 112, 108, 115, 120],
                          index=["Lunes", "Martes", "Miércoles", "Jueves", "Viernes"
                          name="Precios")

print(precios_serie)
print(f"\nMedia: {precios_serie.mean():.2f}")
print(f"Desvío estándar: {precios_serie.std():.2f}")
```

```
Lunes        110
Martes       112
Miércoles    108
Jueves       115
Viernes      120
Name: Precios, dtype: int64

Media: 113.00
Desvío estándar: 4.69
```

Fijate la salida: cada precio aparece junto a su día, y los métodos estadísticos (`mean`, `std`) vienen incorporados — no hace falta importar nada más. Ojo con la lectura del desvío (~4.6 sobre una media de 113): mide cuánto se

dispersan los *niveles* de precio alrededor de su promedio, **no** cuánto se mueve el precio por día. La volatilidad —el riesgo de verdad— se mide sobre los *retornos*, y la calculamos más adelante en este capítulo.

6.1.2. DataFrames (2D)

Un **DataFrame** es la estructura tabular (filas y columnas) que vas a usar el 90 % del tiempo. Técnicamente, cada columna es una Serie, y todas comparten el mismo índice. Armemos una tabla de bonos soberanos argentinos:

```
datos = {
    "Ticker": ["GD30", "GD35", "GD38", "GD41", "GD46"],
    "Precio": [92.5, 85.3, 78.1, 72.4, 65.8],
    "Cupón": [0.08, 0.07, 0.065, 0.06, 0.055]
}

df = pd.DataFrame(datos)
print(df)
```

```
  Ticker  Precio  Cupón
0   GD30    92.5  0.080
1   GD35    85.3  0.070
2   GD38    78.1  0.065
3   GD41    72.4  0.060
4   GD46    65.8  0.055
```

Un diccionario de listas se convierte directamente en tabla: las claves son las columnas. Notá el patrón financiero en los datos: a mayor plazo del bono (GD30 → GD46), menor precio — el tiempo descuenta.

La Figura 6.1 separa las piezas de un DataFrame de series de tiempo, el formato que vas a usar en el resto del libro: un índice que etiqueta las filas, nombres que etiquetan las columnas y, en el medio, un array de NumPy.

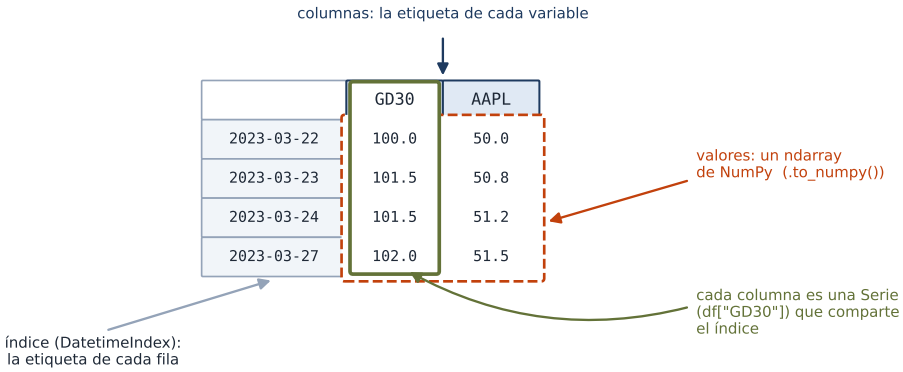


Figura 6.1.: Anatomía de un DataFrame. El índice (aquí, fechas) etiqueta las filas; las columnas etiquetan las variables; los valores son un `ndarray` de NumPy. Cada columna, por separado, es una Serie que comparte el índice.

6.1.3. Exploración Inicial

Regla de oro del análisis de datos: **antes de calcular nada, mirá qué tenés**. Pandas trae cuatro métodos de inspección que deberían ser tu reflejo automático al cargar cualquier dataset:

```
print(f"Dimensiones: {df.shape}")
print(f"Columnas: {df.columns.tolist()}\n")

# head(): las primeras filas - ¿los datos tienen la pinta esperada?
print(df.head(3))
```

```
# describe(): estadísticas rápidas - ¿hay valores absurdos? ¿escalas raras?
print()
print(df.describe().round(2))
```

```
Dimensiones: (5, 3)
Columnas: ['Ticker', 'Precio', 'Cupón']

  Ticker  Precio  Cupón
0   GD30    92.5  0.080
1   GD35    85.3  0.070
2   GD38    78.1  0.065

       Precio  Cupón
count    5.00   5.00
mean    78.82   0.07
std     10.49   0.01
min     65.80   0.06
25%     72.40   0.06
50%     78.10   0.06
75%     85.30   0.07
max     92.50   0.08
```

`head()` responde "¿cargué lo que creía?" y `describe()` responde "¿hay algo raro?" (un precio negativo, un máximo ridículo, menos filas que las esperadas delatan problemas de carga). Completan el kit `df.tail()` (últimas filas — útil para ver si la serie llega hasta la fecha esperada) y `df.info()` (tipos de datos y memoria — clave para detectar números que se cargaron como texto).

6.2. Lectura, Escritura y Alineación de Datos

6.2.1. I/O de Archivos

En la práctica, los datos no se tipean a mano: llegan en archivos (CSV, Excel) exportados de un sistema o descargados de un proveedor. Pandas lee y escribe todos los formatos usuales (`read_csv`, `read_excel`, `to_csv`, `to_excel`). Hagamos el ciclo completo — guardar nuestra tabla de bonos y volver a leerla — que es además la forma de verificar que entendés el formato:

```
import tempfile, os

# Escribimos el DataFrame a un CSV (separador ; como es usual en la región)
ruta = os.path.join(tempfile.gettempdir(), "bonos.csv")
df.to_csv(ruta, sep=";", index=False)

# Y lo volvemos a leer
df_leido = pd.read_csv(ruta, sep=";")
print(df_leido)
print(f"\n¿La ida y vuelta preservó los datos? {df.equals(df_leido)}")
```

```
  Ticker  Precio  Cupón
0   GD30    92.5  0.080
1   GD35    85.3  0.070
2   GD38    78.1  0.065
3   GD41    72.4  0.060
4   GD46    65.8  0.055

¿La ida y vuelta preservó los datos? True
```

Dos parámetros de `read_csv` que vas a usar constantemente con series de tiempo: `parse_dates=["Fecha"]` (convierte la columna a fechas de verdad

en lugar de texto) e `index_col="Fecha"` (la establece como índice del DataFrame). Sin `parse_dates`, las fechas quedan como strings y ningún método temporal funciona — es la causa #1 de "pandas no me ordena por fecha".

> **i** ¿Y de dónde saco los datos?
>
> Para descargar datos reales —precios con yfinance, series macro de FRED, o datos institucionales de Bloomberg— el apéndice Apéndice B tiene la guía completa, incluido el patrón profesional: *descargar una vez, guardar a CSV, y analizar siempre desde el archivo local* con las herramientas de esta sección.

6.2.2. Alineación de Índices e Integración de Datos

Acá viene un problema **real** que las listas de Python no pueden resolver bien. Supongamos que seguís un bono que cotiza en Buenos Aires y una acción que cotiza en Nueva York. El 24 de marzo es feriado en Argentina (no hay precio local) pero NY opera; el 4 de julio es al revés. Si guardaras los precios en dos listas, **los índices dejarían de corresponderse** a partir del primer feriado — y cualquier cálculo conjunto (un spread, una correlación) compararía días distintos sin avisarte.

Pandas resuelve esto **alineando por etiqueta**: al combinar las dos series, empareja las fechas iguales y marca explícitamente los huecos con `NaN`. Veámoslo:

```
# BYMA no opera el 24/03 (feriado argentino); NYSE no opera el 04/07
precios_byma = pd.Series(
    [100.0, 101.5, 102.0, 103.2],
    index=pd.to_datetime(["2023-03-22", "2023-03-23", "2023-03-27", "2023-07-04"]),
    name="GD30")
```

```python
precios_nyse = pd.Series(
    [50.0, 50.8, 51.2, 51.5],
    index=pd.to_datetime(["2023-03-22", "2023-03-23", "2023-03-24", "2023-03-27"]),
    name="AAPL")

# concat alinea automáticamente por fecha (outer: conserva todas las fechas)
panel = pd.concat([precios_byma, precios_nyse], axis=1)
print(panel)
```

```
             GD30  AAPL
2023-03-22  100.0  50.0
2023-03-23  101.5  50.8
2023-03-24    NaN  51.2
2023-03-27  102.0  51.5
2023-07-04  103.2   NaN
```

Leé la tabla resultante: el 24/03 tiene precio de AAPL pero `NaN` en GD30 (feriado argentino), y el 04/07 al revés. Pandas **no inventó datos ni desalineó nada**: te mostró exactamente dónde falta información, para que decidas qué hacer (rellenar, eliminar — la sección de Limpieza). Con listas, este problema se resolvía con contabilidad manual de índices; con Pandas, desaparece.

Para combinar datasets, el kit completo:

- `merge()`: unión por columnas (como un JOIN de SQL) — el caso típico: pegar la TIR y la paridad de cada ticker.
- `join()` / `concat()`: unión por índice — el caso típico con `DatetimeIndex`, como recién.
- Tipos de unión: `inner` (solo claves comunes), `outer` (todas las claves, con NaN donde falte), `left`/`right`.

```python
df_a = pd.DataFrame({"Ticker": ["GD30", "AL30"], "Tir": [0.45, 0.52]})
df_b = pd.DataFrame({"Ticker": ["GD30", "AL30"], "Paridad": [0.35, 0.32]})

df_unido = pd.merge(df_a, df_b, on="Ticker", how="inner")
print(df_unido)
```

```
  Ticker   Tir  Paridad
0   GD30  0.45     0.35
1   AL30  0.52     0.32
```

6.3. Selección y Filtrado

6.3.1. Indexación: .loc vs .iloc

- `.loc`: Busca por **etiquetas** de índice y nombres de columnas.
- `.iloc`: Busca por **posición numérica** (0-indexed).

```python
# Seleccionar fila 0 y columna 'Precio' por posición
print(f"Precio GD30 (.iloc): {df.iloc[0, 1]}")

# Seleccionar por etiqueta (si el ticker fuera el índice)
df_idx = df.set_index("Ticker")
print(f"Precio GD30 (.loc):\n{df_idx.loc['GD30', 'Precio']}")
```

```
Precio GD30 (.iloc): 92.5
Precio GD30 (.loc):
92.5
```

6.3.2. Filtrado Booleano y Máscaras

Permite filtrar filas según condiciones lógicas o pertenencia a listas mediante `.isin()`.

```
# Bonos con precio mayor a 80
mascara = df["Precio"] > 80
print(df[mascara])

# Filtrar por múltiples valores
tickers_interes = ["GD30", "GD46"]
print(df[df["Ticker"].isin(tickers_interes)])
```

```
  Ticker  Precio  Cupón
0   GD30    92.5   0.08
1   GD35    85.3   0.07
  Ticker  Precio  Cupón
0   GD30    92.5  0.080
4   GD46    65.8  0.055
```

6.4. Manejo de Fechas y Series Temporales

Pandas es excepcionalmente potente para trabajar con frecuencias temporales. Primero convertimos datos a tipo fecha con `pd.to_datetime()` y establecemos un `DatetimeIndex`.

```
fechas = ["2023-01-01", "2023-01-02", "2023-01-05"] # 02 es lunes, 01 es domingo
df_time = pd.DataFrame({"Precio": [100, 101, 102.5]}, index=pd.to_datetime(fechas))

print(df_time.index)
```

```
DatetimeIndex(['2023-01-01', '2023-01-02', '2023-01-05'], dtype='datetime64[ns]', fr
```

6.4.1. Resample: Cambio de Frecuencias

El método `resample()` permite agrupar datos por períodos (diario `"D"`, semanal `"W"`, fin de mes `"ME"`) y aplicar funciones de agregación.

```python
# Simulamos una serie diaria de precios (semilla fija para reproducibilidad)
rng = np.random.default_rng(42)
df_diario = pd.DataFrame({
    "Precio": 100 + rng.normal(0.1, 1, 100).cumsum()
}, index=pd.date_range("2023-01-01", periods=100, freq="B"))

# Cambiar de diario a mensual tomando el último valor de cada mes
df_mensual = df_diario.resample("ME").last()  # "ME" = Month End
print(f"Serie original: {len(df_diario)} observaciones diarias")
print(f"Tras resample:  {len(df_mensual)} observaciones mensuales\n")
print(df_mensual.head(3))
```

```
Serie original: 100 observaciones diarias
Tras resample:  5 observaciones mensuales

               Precio
2023-01-31  100.675566
2023-02-28  107.017009
2023-03-31  109.859575
```

De 100 días hábiles quedaron 5 puntas de mes. Elegimos `.last()` porque para precios lo que importa es el **cierre** del período; con otras variables la agregación correcta cambia (`sum()` para volúmenes operados, `mean()` para tasas). El método de agregación es una decisión económica, no técnica.

6.5. Limpieza y Transformación de Datos

Los datos financieros reales vienen con "agujeros" (`NaN`): feriados, activos que no cotizaron, errores del proveedor. Ya vimos cómo la alineación los hace visibles; ahora, qué hacer con ellos. El kit: `isna()` para detectarlos, `dropna()` para eliminar las filas afectadas, y `fillna()` para rellenarlos — en particular con `ffill` (*forward fill*: arrastrar el último valor válido).

Trabajemos sobre el panel desalineado de la sección anterior, que tenía exactamente este problema:

```
print("¿Dónde hay huecos?")
print(panel.isna().sum(), "\n")

# Opción 1: eliminar las filas incompletas (perdemos información)
print("dropna(): quedan", len(panel.dropna()), "filas de", len(panel))

# Opción 2: forward fill - cada hueco toma el último precio conocido
panel_limpio = panel.ffill()
print("\nPanel con ffill():")
print(panel_limpio)
```

```
¿Dónde hay huecos?
GD30    1
AAPL    1
dtype: int64

dropna(): quedan 3 filas de 5

Panel con ffill():
             GD30  AAPL
2023-03-22  100.0  50.0
2023-03-23  101.5  50.8
```

```
2023-03-24  101.5  51.2
2023-03-27  102.0  51.5
2023-07-04  103.2  51.5
```

La Figura 6.2 resume el recorrido completo, desde las dos series con calendarios distintos hasta el panel limpio:

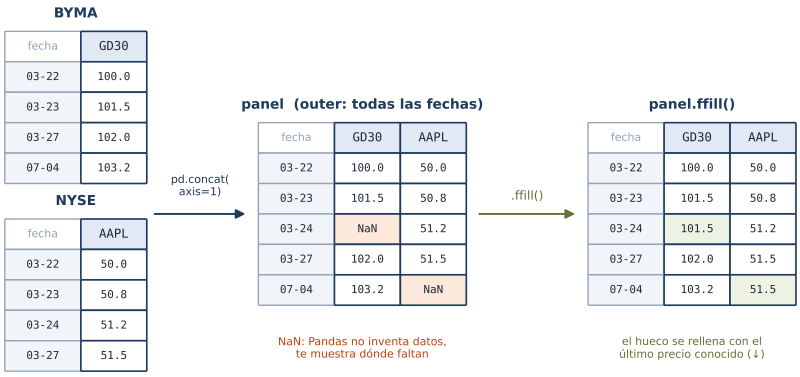


Figura 6.2.: De dos calendarios a un panel. `pd.concat` alinea por fecha y deja `NaN` donde un mercado no operó; `ffill()` rellena cada hueco con el último precio conocido de esa columna.

Compará las dos opciones sobre el caso concreto: `dropna()` tira los días de feriado enteros (perdés también el precio del mercado que **sí** operó), mientras que `ffill()` conserva todo, asumiendo que el precio del activo sin cotización "sigue siendo el del último cierre". Para precios, esa suposición es económicamente razonable — por eso `ffill` es el default en finanzas — pero no es gratis: el precio arrastrado es *viejo*, y en un día de crisis puede estar muy lejos del valor real.

Forward Fill en Finanzas

`ffill` es ideal para feriados y huecos cortos: asume que el precio vigente es el del último cierre válido. La alternativa `bfill` (rellenar con el valor *siguiente*) casi nunca corresponde en finanzas: usa información del futuro — un *look-ahead bias* disfrazado de limpieza.

6.6. Operaciones Vectorizadas y Cálculos Financieros

Pandas optimiza operaciones sobre columnas enteras sin necesidad de bucles `for`.

6.6.1. Movimientos y Variaciones

- `.shift(n)`: Desplaza los datos n posiciones (útil para comparar P_t vs P_{t-1}).
- `.pct_change()`: Calcula el retorno simple: $(P_t/P_{t-1}) - 1$.
- `.diff()`: Calcula la diferencia absoluta: $P_t - P_{t-1}$.

6.6.2. Tipos de Retornos

1. **Retorno Simple**: $R = (P_t/P_{t-1}) - 1$. Es aditivo entre activos en el mismo momento (el retorno de una cartera es el promedio ponderado de los retornos simples).
2. **Retorno Logarítmico**: $r = \ln(P_t/P_{t-1})$. Es aditivo a lo largo del tiempo (el retorno de un año es la suma de los diarios) y simétrico.

```python
df_diario["Ret_Simple"] = df_diario["Precio"].pct_change()
df_diario["Ret_Log"] = np.log(df_diario["Precio"] / df_diario["Precio"].shift(1))

print(df_diario[["Precio", "Ret_Simple", "Ret_Log"]].head(4).round(4))
print(f"\nRetorno medio diario:  {df_diario['Ret_Simple'].mean():.4%}")
print(f"Volatilidad diaria:    {df_diario['Ret_Simple'].std():.4%}")
```

```
              Precio  Ret_Simple  Ret_Log
2023-01-02  100.4047         NaN      NaN
2023-01-03   99.4647     -0.0094  -0.0094
2023-01-04  100.3152      0.0086   0.0085
2023-01-05  101.3557      0.0104   0.0103

Retorno medio diario:  0.0477%
Volatilidad diaria:    0.7393%
```

Dos lecturas de la salida. Primero, la **primera fila tiene `NaN`**: el primer día no tiene "día anterior" con quien compararse — es el mismo desfasaje que manejaste a mano con `range(len - 1)` en el Capítulo 2, ahora automático. Segundo, el retorno simple y el logarítmico son casi idénticos: para movimientos diarios chicos ($\pm 1\,\%$) la diferencia es despreciable; se separa en movimientos grandes.

6.6.3. Ventanas Móviles (Rolling)

Una pregunta constante en finanzas: *¿cómo viene el activo **últimamente**?* No en toda su historia — en los últimos 20 días, 60 días. Las **ventanas móviles** (`rolling`) calculan una estadística sobre una ventana que se desliza por la serie: la base de las medias móviles y de la volatilidad histórica.

```python
# Media móvil de 20 días: el precio "suavizado"
df_diario["SMA_20"] = df_diario["Precio"].rolling(window=20).mean()

# Volatilidad móvil de 20 días, anualizada con √252
df_diario["Vol_20"] = df_diario["Ret_Simple"].rolling(window=20).std() * np.sqrt(252

print(df_diario[["Precio", "SMA_20", "Vol_20"]].iloc[18:23].round(3))
print(f"\nVolatilidad anualizada al final de la muestra: {df_diario['Vol_20'].iloc[-
```

```
             Precio   SMA_20  Vol_20
2023-01-26  101.291      NaN     NaN
2023-01-27  101.341   99.860     NaN
2023-01-30  101.256   99.903   0.137
2023-01-31  100.676   99.963   0.135
2023-02-01  101.998  100.047   0.139

Volatilidad anualizada al final de la muestra: 10.6%
```

Mirá las filas impresas: las primeras 19 observaciones de `SMA_20` son `NaN` — la ventana de 20 días recién se "llena" en la observación 20. Y la volatilidad final (~15 %) es la lectura estándar del riesgo del activo: se anualizó multiplicando por $\sqrt{252}$, la regla que formalizamos en el Capítulo 7. Cuando en la práctica leas "vol histórica 20 ruedas", es exactamente esta cuenta.

6.7. Agrupación (Groupby)

Sigue el mecanismo **Split-Apply-Combine**: dividimos el dataset por una categoría (Ticker, Sector), aplicamos una función y combinamos los resultados.

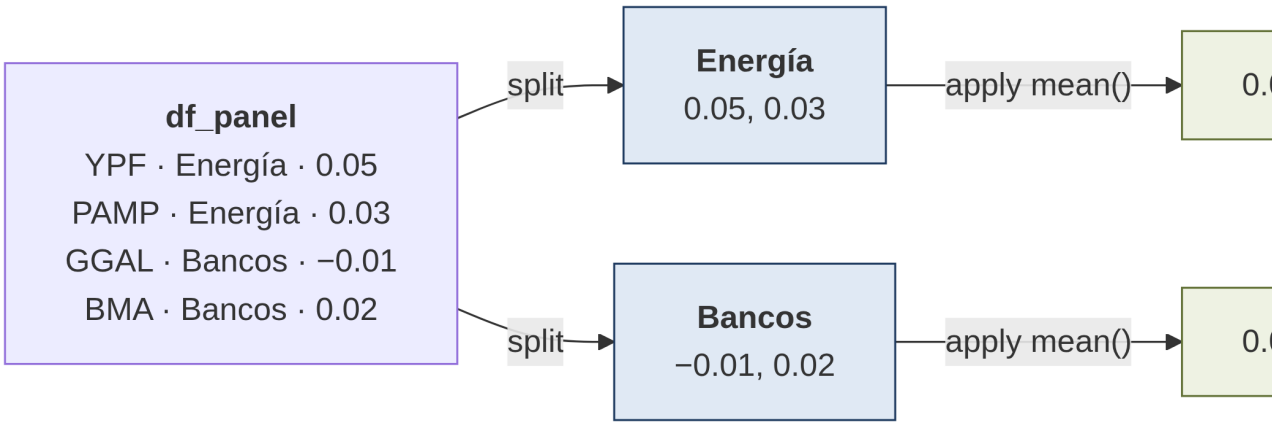


Figura 6.3.: Split-Apply-Combine con `groupby`: el DataFrame se parte por sector, a cada grupo se le aplica la misma función y los resultados se combinan en una tabla nueva, una fila por grupo.

```
df_panel = pd.DataFrame({
    "Sector": ["Energía", "Energía", "Bancos", "Bancos"],
    "Ticker": ["YPF", "PAMP", "GGAL", "BMA"],
    "Retorno": [0.05, 0.03, -0.01, 0.02]
})

# Agregación por sector
resumen_sector = df_panel.groupby("Sector")["Retorno"].agg(["mean", "std", "count"])
print(resumen_sector)
```

```
          mean       std  count
Sector
Bancos   0.005  0.021213      2
Energía  0.040  0.014142      2
```

6.8. Visualización: cómo graficar (mini-tutorial de Matplotlib)

Llegó el momento de aprender a graficar en serio, porque **de acá al final del libro no hay capítulo sin gráficos**: fronteras eficientes, distribuciones de retornos, trayectorias simuladas, sonrisas de volatilidad. Un analista que no grafica sus datos está analizando a ciegas — el histograma que revela una cola pesada o el gráfico que delata un hueco en la serie valen más que diez tablas.

6.8.1. La anatomía: Figure y Axes

Matplotlib organiza todo gráfico en dos piezas, y entender la diferencia elimina el 90 % de la confusión inicial:

- La **Figure** (`fig`) es el **lienzo completo**: la hoja donde se dibuja. Controla el tamaño total y contiene uno o más gráficos.
- Los **Axes** (`ax`) son **cada gráfico individual** dentro del lienzo: sus ejes, sus curvas, su título, su leyenda. (El nombre confunde: un objeto Axes es *el gráfico*, no "los ejes".)

La Figura 6.4 pone nombre a cada parte de un gráfico y a la llamada que la produce:

De ahí el idioma que vas a ver en todo el libro — y en todo el código profesional:

```
fig, ax = plt.subplots(figsize=(10, 5))   # un lienzo con un gráfico adentro
```

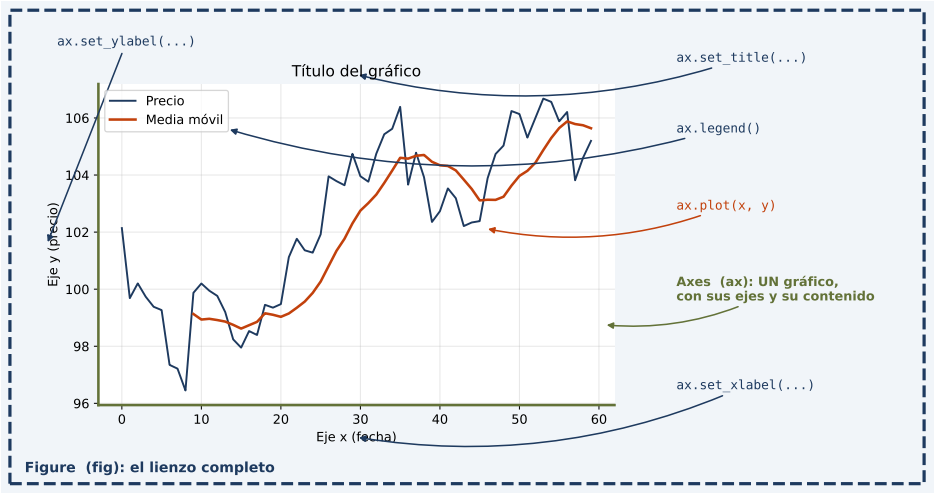


Figura 6.4.: Anatomía de un gráfico de Matplotlib. La Figure (borde punteado) es el lienzo completo; el Axes (borde verde) es el gráfico, con sus ejes y su contenido. Cada elemento se agrega con un método del `ax`.

`plt.subplots()` crea las dos piezas juntas, y a partir de ahí **todo se le pide al `ax`**: dibujar (`ax.plot`), etiquetar (`ax.set_xlabel`), etc. Construyamos un gráfico completo paso a paso, con la receta de cinco pasos que repite cada figura del libro (el resultado es la Figura 6.5):

```
# 1. Crear lienzo y gráfico
fig, ax = plt.subplots(figsize=(10, 4.5))

# 2. Dibujar los datos
ax.plot(df_diario.index, df_diario["Precio"], linewidth=1.5, label="Precio de cierre

# 3. Etiquetar TODO: quien mira el gráfico no leyó tu código
ax.set_xlabel("Fecha")
ax.set_ylabel("Precio ($)")
ax.set_title("Evolución del precio - la plantilla de todo gráfico del libro")

# 4. Leyenda y grilla (la grilla suave ayuda a leer valores)
ax.legend()
ax.grid(True, alpha=0.3)

# 5. Ajustar márgenes y mostrar
plt.tight_layout()
plt.show()
```

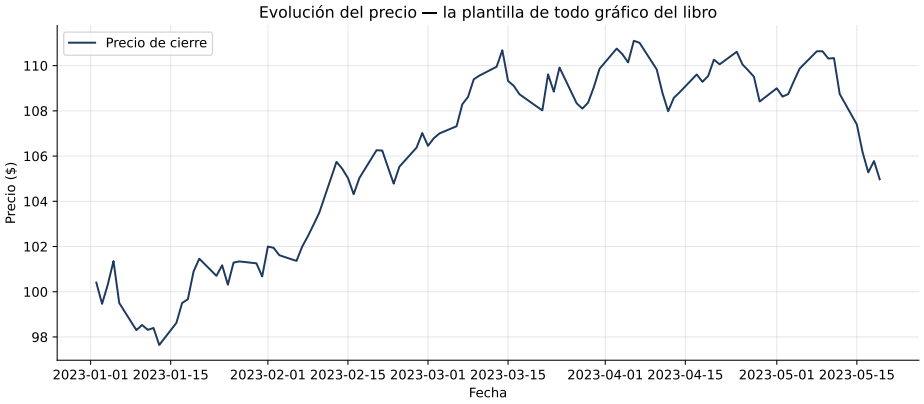


Figura 6.5.: El gráfico que produce la plantilla de cinco pasos: precio simulado de 100 días hábiles, con ejes, título, leyenda y grilla.

Esos cinco pasos —crear, dibujar, etiquetar, leyenda/grilla, mostrar— son **la plantilla**. Todo lo demás son variaciones: qué se dibuja en el paso 2 y cuántos gráficos conviven en el lienzo.

6.8.2. Los tres tipos de gráfico del libro

Con la plantilla fija, cambiar de tipo de gráfico es cambiar una línea. Los tres que cubren el 95 % del análisis financiero:

- **Línea** (`ax.plot`): evolución en el tiempo — precios, curvas, trayectorias.
- **Histograma** (`ax.hist`): la *distribución* de una variable — la herramienta central para mirar retornos.
- **Dispersión** (`ax.scatter`): la *relación* entre dos variables — ¿se mueven juntas?

```
retornos =df_diario["Ret_Simple"].dropna()

fig, axes = plt.subplots(1, 2, figsize=(11, 4))

# Histograma: ¿cómo se distribuyen los retornos diarios?
ax = axes[0]
ax.hist(retornos * 100, bins=30, edgecolor="white", alpha=0.8)
ax.set_xlabel("Retorno diario (%)")
ax.set_ylabel("Frecuencia (días)")
ax.set_title("Histograma: la distribución de retornos")
ax.grid(True, alpha=0.3)

# Dispersión: ¿el retorno de ayer dice algo sobre el de hoy?
ax = axes[1]
ax.scatter(retornos.shift(1) * 100, retornos * 100, s=15, alpha=0.6)
ax.set_xlabel("Retorno de ayer (%)")
ax.set_ylabel("Retorno de hoy (%)")
ax.set_title("Dispersión: retorno vs. retorno anterior")
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

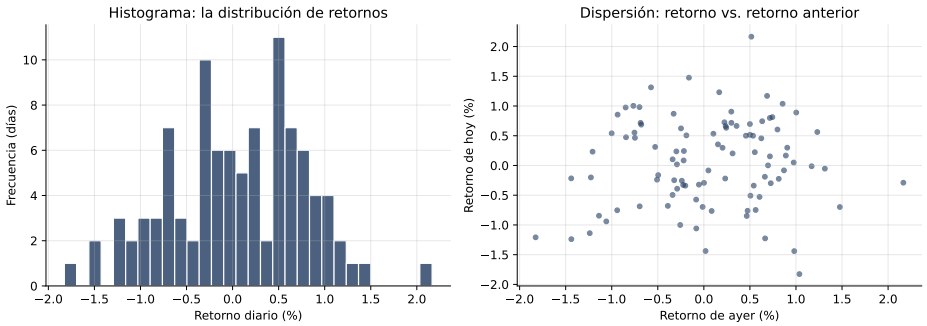


Figura 6.6.: Dos de los tres tipos de gráfico del libro sobre los mismos retornos diarios: el histograma muestra cómo se distribuyen; la dispersión, si el retorno de ayer anticipa el de hoy.

Dos novedades en este bloque. Primero, `plt.subplots(1, 2)` crea **dos gráficos en el mismo lienzo** (1 fila, 2 columnas) y devuelve un array de Axes — dibujás en cada uno por separado. Segundo, leé lo que los gráficos de la Figura 6.6 *dicen*: el histograma muestra retornos concentrados alrededor de cero con forma de campana, y la dispersión es una **nube sin patrón** — el retorno de ayer no predice el de hoy, tu primer vistazo a la eficiencia del mercado que formalizaremos en el Capítulo 16.

6.8.3. El atajo de pandas: `.plot()`

Pandas trae un envoltorio de matplotlib incorporado: toda Serie y DataFrame tiene un método `.plot()` que dibuja usando **el índice como eje x** automáticamente — con fechas, esto es oro. Acepta `ax=` para dirigir el dibujo a un Axes que ya creaste, así que se combina perfecto con la plantilla:

```
fig, ax = plt.subplots(2, 1, figsize=(10, 7), sharex=True)
```

```
# Panel 1: precio y su media móvil (dos series sobre el MISMO ax)
df_diario["Precio"].plot(ax=ax[0], label="Precio", alpha=0.7)
df_diario["SMA_20"].plot(ax=ax[0], label="SMA 20", linewidth=2)
ax[0].set_ylabel("Precio ($)")
ax[0].set_title("Precio y media móvil de 20 días")
ax[0].legend()
ax[0].grid(True, alpha=0.3)

# Panel 2: la volatilidad móvil, compartiendo el eje de fechas (sharex)
(df_diario["Vol_20"] * 100).plot(ax=ax[1], color="#c2410c", label="Volatilidad 20d")
ax[1].set_ylabel("Vol. anualizada (%)")
ax[1].set_xlabel("Fecha")
ax[1].set_title("Volatilidad móvil anualizada")
ax[1].legend()
ax[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

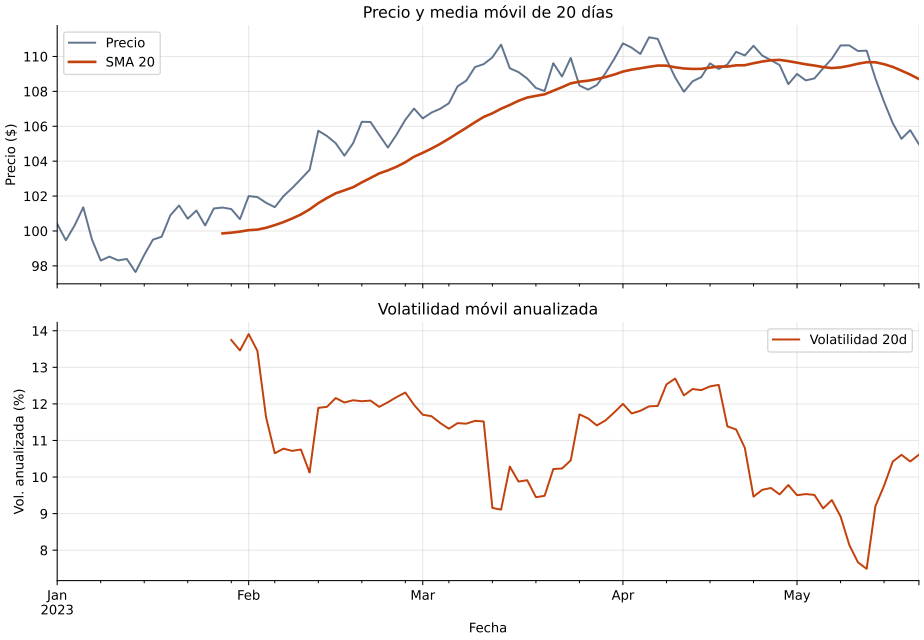


Figura 6.7.: Panel doble con eje de fechas compartido: arriba, el precio y su media móvil de 20 días; abajo, la volatilidad móvil anualizada.

El panel doble de la Figura 6.7 es el formato clásico del análisis técnico: precio arriba, indicador abajo, **compartiendo el eje de fechas** (`sharex=True`) para poder leerlos en vertical: fijate cómo los períodos de precio "nervioso" en el panel superior coinciden con los picos de volatilidad en el inferior.

> **El checklist de todo gráfico**
>
> Antes de dar por terminada una figura, cuatro preguntas: ¿los **ejes** tienen etiqueta con unidades? ¿el **título** dice qué estoy mirando? ¿hay **leyenda** si conviven varias series? ¿alguien que no vio el código

entiende el gráfico solo? Un gráfico sin etiquetas es un dibujo; con etiquetas, es un argumento.

6.9. Con datos reales: el dataset del libro

Hasta acá trabajamos con una serie simulada de 100 días. Es hora de aplicar todo el capítulo a datos de mercado de verdad: el dataset del libro (`datos/precios.csv`, ver Apéndice B), con cinco años de precios diarios ajustados de 9 acciones y del ETF SPY (el S&P 500). Es el mismo archivo que van a usar los capítulos siguientes cada vez que veas una sección "Con datos reales".

El primer paso es el de la sección de I/O: leer con la columna de fechas como índice. Y enseguida, el reflejo de la Exploración Inicial: mirar qué cargamos.

```
precios = pd.read_csv("../datos/precios.csv", index_col="Date", parse_dates=True)

print(f"Dimensiones: {precios.shape}  (ruedas × activos)")
print(f"Desde {precios.index.min():%Y-%m-%d} hasta {precios.index.max():%Y-%m-%d}"
print(f"Tipo del índice: {precios.index.dtype}   |   Huecos (NaN): {precios.isna().s
print(precios.iloc[:3, :5])
```

```
Dimensiones: (1253, 10)  (ruedas × activos)
Desde 2021-07-01 hasta 2026-06-29
Tipo del índice: datetime64[ns]   |   Huecos (NaN): 0

                AAPL      MSFT      NVDA       JPM       XOM
Date
```

```
2021-07-01  133.8042  260.6536  20.1423  137.8143  52.6366
2021-07-02  136.4263  266.4598  20.4164  137.6819  52.5617
2021-07-06  138.4342  266.4694  20.6272  135.3699  51.0640
```

Tres chequeos aprobados: el índice es de fechas de verdad (`datetime64`, gracias a `parse_dates`), el rango es el esperado y no hay huecos. Con eso, cada herramienta del capítulo se aplica en una línea:

```python
# Retornos diarios: pct_change sobre las 10 columnas a la vez
retornos_d = precios.pct_change().dropna()

# Retorno anual compuesto y volatilidad anualizada de cada activo
resumen = pd.DataFrame({
    "Retorno anual": (1 + retornos_d).prod() ** (252 / len(retornos_d)) - 1,
    "Vol. anual": retornos_d.std() * np.sqrt(252),
})
print(resumen.sort_values("Vol. anual").round(3))

# resample: de diario a mensual, y el mejor y el peor mes del S&P 500
spy_mensual = precios["SPY"].resample("ME").last().pct_change().dropna()
print(f"\nSPY - mejor mes: {spy_mensual.idxmax():%Y-%m} ({spy_mensual.max():+.1%}), "
      f"peor mes: {spy_mensual.idxmin():%Y-%m} ({spy_mensual.min():+.1%})")
```

```
      Retorno anual  Vol. anual
KO            0.123       0.162
JNJ           0.125       0.171
SPY           0.131       0.172
WMT           0.215       0.217
JPM           0.191       0.245
XOM           0.211       0.267
MSFT          0.072       0.270
AAPL          0.162       0.277
DIS          -0.107       0.294
```

```
NVDA          0.579       0.518

SPY - mejor mes: 2026-04 (+10.5%), peor mes: 2022-09 (-9.2%)
```

Leé la tabla de arriba abajo: las acciones defensivas (KO, JNJ) y el propio índice tienen la menor volatilidad, cerca del 17 % anual; NVDA es otro planeta, con más del 50 %. Y fijate que más volatilidad no garantiza más retorno: MSFT y DIS están en la mitad de arriba de la tabla de riesgo y rindieron menos que el índice (DIS, de hecho, perdió plata en los cinco años). La relación entre riesgo y retorno esperado es el tema del Módulo 2.

La Figura 6.8 junta las dos herramientas de gráfico del capítulo: precios llevados a base 100 (para comparar activos de precios muy distintos en una misma escala) y la volatilidad móvil del índice.

```
seleccion = ["NVDA", "WMT", "SPY", "KO", "DIS"]
base_100 = precios[seleccion] / precios[seleccion].iloc[0] * 100
vol_spy = retornos_d["SPY"].rolling(60).std() * np.sqrt(252) * 100

fig, ax = plt.subplots(2, 1, figsize=(10, 7), sharex=True,
                       gridspec_kw={"height_ratios": [2, 1]})
base_100.plot(ax=ax[0], logy=True)        # escala log: subas iguales en % se ven igu
ax[0].axhline(100, color="#94a3b8", linewidth=1)
ax[0].set_ylabel("Base 100 (escala log)")
ax[0].set_title("Cinco años de mercado, en una misma escala")
ax[0].grid(True, alpha=0.3)

vol_spy.plot(ax=ax[1], color="#c2410c", label="Vol. 60 ruedas del SPY")
ax[1].set_ylabel("Vol. anualizada (%)")
ax[1].set_xlabel("Fecha")
ax[1].legend()
ax[1].grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```

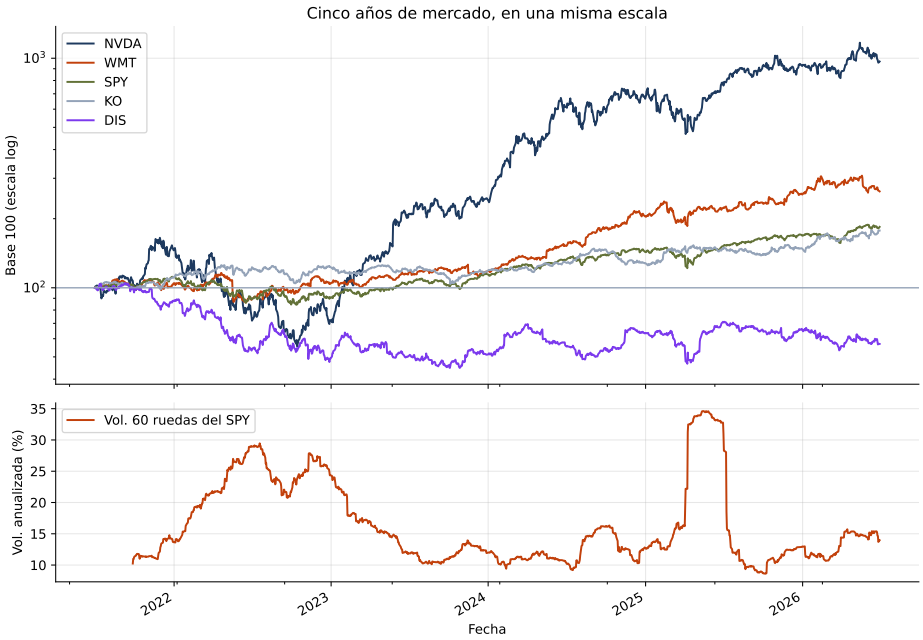


Figura 6.8.: Arriba: cinco activos del dataset en base 100 (julio de 2021 = 100). Abajo: volatilidad móvil de 60 ruedas del S&P 500, anualizada; los picos coinciden con las caídas del panel de arriba.

Dos detalles de código que vale la pena copiar. La división `precios / precios.iloc[0]` funciona columna por columna gracias a la alineación por etiquetas: cada activo se divide por *su* primer precio. Y `logy=True` usa escala logarítmica, la correcta para precios en el largo plazo: en ella, una suba del 100 % mide lo mismo arranque desde 50 o desde 500, así que NVDA no aplasta al resto del gráfico.

6.10. Caso de Uso: Construcción de la Curva de Rendimientos

Este caso integra casi todo el capítulo —DataFrames, iteración, operaciones vectorizadas— para resolver un problema central de la renta fija: **construir la curva de tasas** a partir de precios de bonos que se cotizan en el mercado.

6.10.1. ¿Por qué necesitamos "construir" una curva?

En el mercado no observamos directamente las tasas *spot* (la tasa de un instrumento que paga todo al vencimiento, sin cupones intermedios). Lo que observamos son **precios de bonos con cupón**, y cada uno mezcla tasas de muchos plazos a la vez. La técnica del **bootstrapping** desarma esa mezcla plazo por plazo: usa el bono a 1 año para despejar la tasa a 1 año, y con esa tasa ya conocida despeja la de 2 años del bono a 2 años, y así sucesivamente. Es como armar la curva "tirando del hilo".

La Figura 6.9 muestra la cadena: cada tasa que se despeja se convierte en un insumo para el paso siguiente.

Para un bono a n años con cupón anual C y valor nominal VN, su precio es la suma de todos los flujos descontados a la tasa spot correspondiente a cada plazo:

$$P = \sum_{t=1}^{n-1} \frac{C}{(1+z_t)^t} + \frac{C+VN}{(1+z_n)^n}$$

Si ya conocemos las tasas $z_1, \ldots, z_{n-1}$ (de los pasos anteriores), la única incógnita es z_n, y la despejamos.

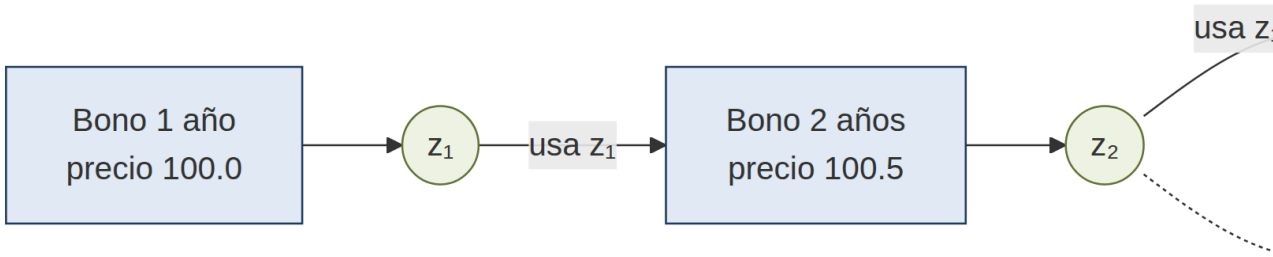


Figura 6.9.: Bootstrapping de la curva spot. Cada bono, junto con las tasas ya despejadas, deja una sola incógnita: la tasa spot de su propio plazo. Con dos spots consecutivas se obtiene la forward implícita entre ellas.

```python
bonos_mkt = pd.DataFrame([
    {"maturity": 1, "cupon": 0.05, "precio": 100.0},
    {"maturity": 2, "cupon": 0.06, "precio": 100.5},
    {"maturity": 3, "cupon": 0.065, "precio": 99.8},
    {"maturity": 4, "cupon": 0.07, "precio": 100.2},
    {"maturity": 5, "cupon": 0.075, "precio": 99.5},
]).set_index("maturity")

tasas_spot = pd.Series(index=bonos_mkt.index, dtype=float)
VN = 100

for n, bono in bonos_mkt.iterrows():
    C = VN * bono["cupon"]
    P = bono["precio"]

    # Valor presente de los cupones anteriores al vencimiento,
    # descontados con las tasas spot ya calculadas en pasos previos
    vp_cupones = sum(C / (1 + tasas_spot[t]) ** t for t in range(1, int(n)))

    # Despejamos la tasa spot del plazo n del flujo final (cupón + nominal)
    z_n = ((C + VN) / (P - vp_cupones)) ** (1 / n) - 1
    tasas_spot[n] = z_n

print("Curva Spot (tasas cero-cupón):")
print((tasas_spot * 100).round(3).astype(str) + "%")
```

```
Curva Spot (tasas cero-cupón):
maturity
1      5.0 %
2     5.75 %
3    6.649 %
4    7.042 %
5    7.826 %
```

```
dtype: object
```

6.10.2. De la curva spot a las tasas forward

La curva spot responde *"¿a qué tasa presto hoy por 3 años?"*. Pero muchas veces queremos saber otra cosa: *"¿qué tasa está descontando el mercado para el año que va del 2 al 3?"*. Esa es la **tasa forward implícita**, y sale de un argumento de no-arbitraje: invertir 3 años a la spot z_3 debe rendir lo mismo que invertir 2 años a z_2 y luego reinvertir 1 año a la forward $f_{2,3}$:

$$(1+z_n)^n = (1+z_{n-1})^{n-1}\cdot(1+f_{n-1,n}) \quad \Rightarrow \quad f_{n-1,n} = \frac{(1+z_n)^n}{(1+z_{n-1})^{n-1}} - 1$$

```python
# Factor de capitalización acumulado hasta cada plazo: (1 + z_n)^n
factor_acum = (1 + tasas_spot) ** tasas_spot.index

# Forward de un año: cociente de factores acumulados consecutivos
forwards = factor_acum / factor_acum.shift(1) - 1
forwards.iloc[0] = tasas_spot.iloc[0]  # el forward del primer tramo es la spot a 1

curva = pd.DataFrame({"spot": tasas_spot, "forward_1y": forwards})
print((curva * 100).round(3))
```

```
            spot  forward_1y
maturity
1          5.000       5.000
2          5.750       6.506
3          6.649       8.469
4          7.042       8.228
5          7.826      11.022
```

Cuando las forward están **por encima** de las spot, el mercado espera que las tasas suban (curva creciente): cada tramo nuevo "empuja" el promedio hacia arriba.

6.10.3. Interpolar la curva para plazos intermedios

Nuestra curva tiene puntos solo en 1, 2, 3, 4 y 5 años, pero en la práctica necesitamos descontar flujos que caen en cualquier fecha (por ejemplo, a 2,5 años). Para eso **interpolamos**. Pandas trae `interpolate()`, que rellena los huecos de un índice: reindexamos la curva a una grilla más fina y dejamos que complete linealmente.

La Figura 6.10 compara los cinco puntos que salieron del bootstrapping con la curva interpolada cada seis meses.

```
# Grilla de plazos cada medio año, de 1 a 5
grilla = np.arange(1.0, 5.5, 0.5)

curva_interpolada = (
    tasas_spot
    .reindex(tasas_spot.index.union(grilla))  # agregamos los plazos intermedios (qu
    .interpolate(method="index")              # interpolamos usando el valor del índ
    .reindex(grilla)                           # nos quedamos con la grilla fina
)

fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(tasas_spot.index, tasas_spot * 100, "o", markersize=9,
        label="Tasas spot observadas (bootstrapping)")
ax.plot(curva_interpolada.index, curva_interpolada * 100, ".-", alpha=0.7,
        label="Curva interpolada (cada 6 meses)")
ax.set_xlabel("Plazo (años)")
ax.set_ylabel("Tasa spot (%)")
ax.set_title("Curva de Rendimientos: puntos observados vs. interpolación")
```

```
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

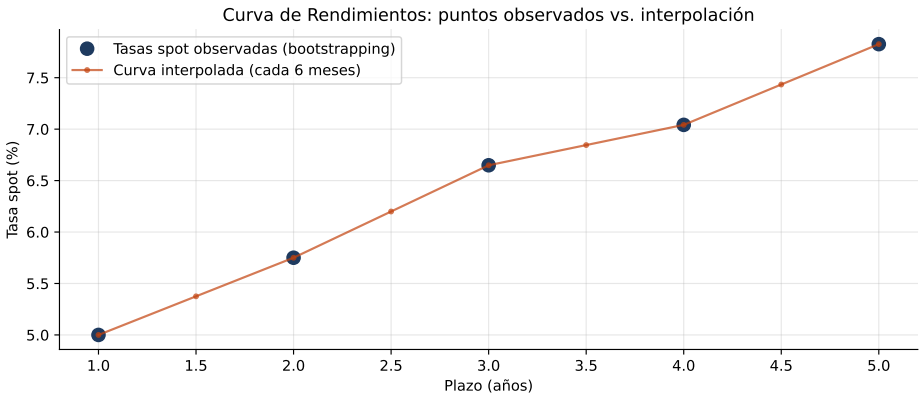


Figura 6.10.: Curva de rendimientos: las tasas spot obtenidas por bootstrapping (puntos grandes) y la interpolación lineal cada medio año entre ellas.

La interpolación lineal es un punto de partida, no la verdad

Interpolar linealmente es simple y transparente, pero los operadores de renta fija suelen usar métodos más suaves (splines cúbicos, Nelson-Siegel) para que la curva forward no tenga "quiebres" antieconómicos. Para aprender el mecanismo, lo lineal alcanza; para valuar en producción, conviene ir más fino.

6.11. Errores comunes

⚠ Trampas frecuentes

- **Fechas como texto**: si olvidás `parse_dates` (o `pd.to_datetime`), las fechas quedan como strings: se ordenan alfabéticamente, `resample` falla y los gráficos salen desordenados. Verificá con `df.index.dtype`.
- **El `NaN` inicial de `pct_change()`**: la primera observación de cualquier serie de retornos es `NaN`. Si después calculás `mean()` no pasa nada (pandas lo ignora), pero si convertís a NumPy o lo pasás a otra librería, ese `NaN` propaga y arruina el cálculo. `dropna()` primero.
- **`ffill` con huecos largos**: arrastrar un precio un día es razonable; arrastrarlo tres meses (un bono que dejó de cotizar) fabrica una serie plana con volatilidad cero — un activo "sin riesgo" que no existe. Poné un límite (`ffill(limit=5)`) o revisá los huecos antes.
- **Look-ahead con `shift` al revés**: `shift(-1)` trae el dato de *mañana* a la fila de hoy. Útil para armar targets de predicción, letal si se te cuela en una señal de trading: estarías operando con información del futuro (el tema central del Capítulo 17).
- **`SettingWithCopyWarning`**: asignar sobre un "recorte" de un DataFrame (`df[df.x > 0]["y"] = 1`) puede modificar una copia y no el original. La forma segura es una sola indexación con `.loc`: `df.loc[df.x > 0, "y"] = 1`.

6.12. Referencias

Parte II.

Módulo 2: Teoría de Portafolios y Optimización

Módulo 2: Teoría de Portafolios y Optimización

De describir activos a decidir entre ellos

En el Módulo 1 aprendiste a analizar instrumentos **de a uno**: valuar un bono, calcular el retorno de una acción, graficar su historia. Pero ningún inversor real tiene un solo activo, y la pregunta que de verdad importa es otra: **¿cómo reparto mi capital entre todos los activos disponibles?** Este módulo construye la respuesta cuantitativa a esa pregunta — y también sus contraejemplos.

El recorrido tiene forma de tesis, antítesis y síntesis. Primero armamos las herramientas: **estadística de retornos** (¿cómo se comporta cada activo?) y **optimización numérica** (¿cómo le pido a la computadora que encuentre "lo mejor"?). Con eso construimos el edificio clásico: la **frontera eficiente de Markowitz**, la idea ganadora del Nobel de que la diversificación se puede medir y optimizar. Hasta ahí la teoría es hermosa.

Después viene la parte que los libros suelen esquivar: **en la práctica, Markowitz falla**. Es hipersensible a estimaciones que son puro ruido, y produce carteras extremas que ningún gestor firmaría. Los últimos dos capítulos presentan las dos respuestas modernas más importantes a ese problema, que atacan por flancos opuestos: **HRP** (López de Prado, 2016) esquiva la matemática frágil usando clustering jerárquico, y **Black-Litterman** (Goldman Sachs, 1992) disciplina las estimaciones partiendo del equilibrio del mercado y mezclándolo con tus opiniones según tu confianza. Al final del módulo no solo vas a saber optimizar un portafolio: vas a saber **cuándo**

desconfiar del optimizador — que es la habilidad que separa al que aprendió la fórmula del que entiende el problema.

La Figura 6.1 resume ese recorrido.

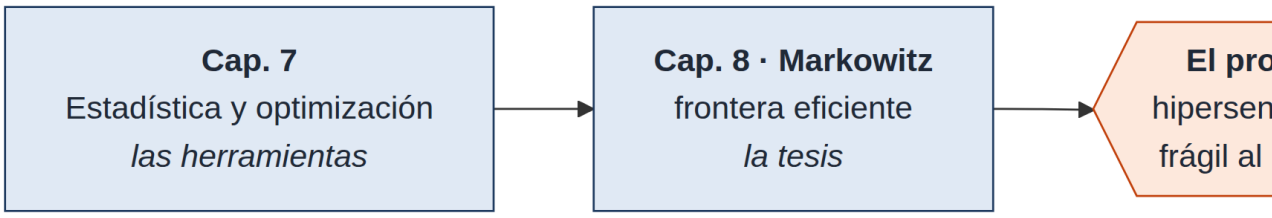


Figura 6.1.: El arco del módulo: herramientas, la tesis de Markowitz, su problema práctico y las dos respuestas modernas, que atacan flancos opuestos.

> **Lo que este módulo asume**
>
> Usamos intensivamente el álgebra matricial de NumPy (Capítulo 5) — en particular `@`, `np.linalg` y la matriz de covarianzas — y la estadística descriptiva del Capítulo 6. Si esos temas no te quedaron firmes, vale la pena repasarlos antes de entrar a Markowitz.

Objetivos de Aprendizaje

Al completar este módulo, vas a ser capaz de:

1. **Analizar distribuciones** de retornos con estadística descriptiva y tests de normalidad.
2. **Construir la frontera eficiente** de Markowitz y encontrar el portafolio tangente (máximo Sharpe).
3. **Elegir la métrica correcta** (Sharpe, Sortino, mínima varianza) según el objetivo del inversor.
4. **Aplicar Hierarchical Risk Parity** para sortear la inestabilidad de la optimización clásica.
5. **Incorporar tus views** al equilibrio del mercado con el modelo de Black-Litterman.

Recorrido Capítulo a Capítulo

Capítulo	Tema	Python	Finanzas
7	Estadística y Optimización	`scipy.stats`, `scipy.optimize`, histogramas	Media, varianza, curtosis, normalidad
8	Markowitz y Frontera Eficiente	`numpy` matricial, `minimize()`	Σ, Sharpe, Sortino, CML, dividendos, renta fija
9	Hierarchical Risk Parity	`scipy.cluster.hierarchy`	Clustering, riesgo, robustez fuera de muestra

Capítulo	Tema	Python	Finanzas
10	Black-Litterman	Álgebra matricial, `np.linalg`	Equilibrio de mercado, views, fórmula maestra

---

Herramientas y Librerías

NumPy

Álgebra matricial

SciPy

Optimización, estadística y clustering

Matplotlib

Frontera eficiente y dendrogramas

Concepto Clave: Media-Varianza

! El framework de Markowitz plantea que un inversor racional elige portafolios que **maximizan retorno** para un nivel dado de riesgo, o equivalentemente, **minimizan riesgo** para un retorno objetivo. La solución al problema de optimización genera la **frontera eficiente**.

$$\min_{\mathbf{w}} \quad \mathbf{w}^T \Sigma \mathbf{w} \quad \text{sujeto a} \quad \mathbf{w}^T \mu = \mu^*, \quad \mathbf{w}^T \mathbf{1} = 1$$

El resto del módulo puede leerse como una conversación con esta fórmula: cómo resolverla, qué esconde y cómo arreglarla.

Lecturas Recomendadas

- Markowitz, H. (1952). *Portfolio Selection*. The Journal of Finance.
- Elton, E. et al. (2014). *Modern Portfolio Theory and Investment Analysis*. Wiley.
- Black, F., & Litterman, R. (1992). *Global Portfolio Optimization*. Financial Analysts Journal.
- López de Prado, M. (2016). *Building Diversified Portfolios that Outperform Out of Sample*. The Journal of Portfolio Management.

7. Capítulo 7: Estadística Descriptiva y Optimización Matemática

Distribuciones de retornos, scipy.stats y scipy.optimize

Antes de armar portafolios necesitamos dos herramientas: saber describir estadísticamente cómo se comportan los retornos (media, volatilidad, asimetría, curtosis, y el test de si son "normales") y saber pedirle a la computadora que encuentre el mejor valor de algo (optimización numérica con scipy). Este capítulo construye ambas, que se combinan en el capítulo siguiente con Markowitz.

Introducción

Este módulo trata de una pregunta: *¿cómo repartir el capital entre activos?* Para responderla con rigor necesitamos dos habilidades previas.

La primera es **describir el comportamiento de los retornos**. "La acción rinde 10 % anual" es una descripción pobre: ¿con cuánta variabilidad? ¿las sorpresas son simétricas o hay derrumbes ocasionales? ¿los extremos son raros o frecuentes? La estadística descriptiva pone números a esas preguntas.

La segunda es **optimizar**: dado un objetivo ("minimizar el riesgo", "maximizar la utilidad") y restricciones ("los pesos suman 1", "sin ventas en corto"), encontrar la mejor solución. Si venís del mundo Excel, esto es lo que hacía el *Solver* — pero programable, más rápido y reproducible.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(42)
```

7.1. Estadística descriptiva de retornos

7.1.1. Los cuatro momentos

Una distribución de retornos se resume, en primera aproximación, con cuatro números — los **momentos**:

1. **Media** (μ): el retorno "típico". El centro de gravedad de la distribución.
2. **Varianza / volatilidad** (σ): cuánto se dispersan los retornos alrededor de la media. Es la medida clásica de riesgo.
3. **Asimetría** (*skewness*): ¿la distribución se estira más hacia un lado? Skewness **negativa** = derrumbes ocasionales más grandes que las subas (típico de acciones); **positiva** = sorpresas buenas más grandes (típico de estrategias tipo lotería).
4. **Curtosis** (*kurtosis*): ¿cuánta masa hay en las **colas**? Curtosis en exceso positiva significa que los eventos extremos son **más frecuentes** que bajo una distribución normal. En finanzas, casi siempre lo son.

La Figura 7.1 muestra qué miden los dos últimos momentos, que son los menos intuitivos: la asimetría deforma la campana hacia un lado, y la curtosis la vuelve más puntiaguda en el centro y más pesada en las colas.

```
# Simulamos un año de retornos diarios de una acción
n_dias = 252
retornos = rng.normal(0.0004, 0.018, n_dias)

print("Estadísticas descriptivas de los retornos:")
print(f"  Media diaria:       {np.mean(retornos):.6f}")
print(f"  Media anualizada:   {np.mean(retornos) * 252:.2%}")
print(f"  Volatilidad diaria: {np.std(retornos):.6f}")
print(f"  Vol. anualizada:    {np.std(retornos) * np.sqrt(252):.2%}")
print(f"  Asimetría:          {stats.skew(retornos):.4f}")
print(f"  Curtosis (exceso):  {stats.kurtosis(retornos):.4f}")
```

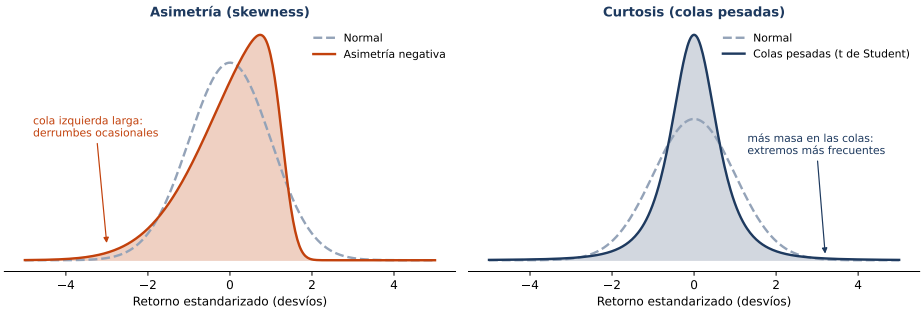


Figura 7.1.: Asimetría y curtosis frente a una normal con la misma media y el mismo desvío. Izquierda: asimetría negativa, con una cola izquierda larga (derrumbes ocasionales). Derecha: colas pesadas, con más masa en los extremos y en el centro, y menos en los hombros.

```
Estadísticas descriptivas de los retornos:
  Media diaria:       -0.000471
  Media anualizada:   -11.86%
  Volatilidad diaria: 0.016844
  Vol. anualizada:    26.74%
  Asimetría:          0.3848
  Curtosis (exceso):  0.1543
```

i ¿Por qué $\times 252$ y $\times\sqrt{252}$?

Hay ~252 ruedas bursátiles por año. Si los retornos diarios son independientes entre sí, tanto la **media** como la **varianza** de un año son 252 veces las diarias. Pero el desvío es la *raíz* de la varianza, así que la volatilidad escala con la raíz del tiempo: $\sigma_{anual} = \sigma_{diaria}\sqrt{252}$. Confundir estas dos reglas es el error de anualización más común. (El supuesto de independencia en la práctica se cumple solo aproximadamente, pero es el estándar de la industria.)

`np.std` y pandas `.std()` no dan lo mismo

`np.std` divide por n (desvío *poblacional*, `ddof=0`), mientras que pandas `.std()` y `np.cov` dividen por $n-1$ (desvío *muestral*, `ddof=1`). Con 252 datos la diferencia es de apenas 0.2 %, pero si comparás resultados de las dos librerías y no coinciden, casi siempre es esto. Para alinearlos, usá `np.std(x, ddof=1)`.

Como simulamos con `rng.normal`, la asimetría y la curtosis dan cerca de cero. Con retornos reales de acciones el panorama es otro: curtosis bien positiva y, con frecuencia, asimetría negativa (lo vamos a comprobar con datos de mercado al final de esta sección). Veámoslo primero generando retornos más realistas, con "sustos" ocasionales:

```
# Mezcla: 95% días normales + 5% días de crisis (más volátiles y sesgados a la baja)
normales = rng.normal(0.0008, 0.012, n_dias)
crisis = rng.normal(-0.02, 0.04, n_dias)
es_crisis = rng.random(n_dias) < 0.05
ret_real = np.where(es_crisis, crisis, normales)

print(f"{'':22}{'Simulado normal':>16}{'Con crisis':>12}")
print(f"{'Asimetría':<22}{stats.skew(retornos):>16.3f}{stats.skew(ret_real):>12.3f}"
print(f"{'Curtosis (exceso)':<22}{stats.kurtosis(retornos):>16.3f}{stats.kurtosis(re
```

```
                      Simulado normal  Con crisis
Asimetría                       0.385      -0.894
Curtosis (exceso)               0.154       3.804
```

7.1.2. Visualizar la distribución

El histograma con la normal teórica superpuesta es el gráfico de diagnóstico básico: muestra de un vistazo si las colas y la asimetría se apartan del modelo normal.

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4), sharey=True)

for ax, datos, titulo in [(axes[0], retornos, "Retornos ~normales"),
                          (axes[1], ret_real, "Retornos con días de crisis")]:
    ax.hist(datos, bins=40, density=True, alpha=0.7,
            color="#94a3b8", edgecolor="white")
    x = np.linspace(datos.min(), datos.max(), 200)
    ax.plot(x, stats.norm.pdf(x, datos.mean(), datos.std()),
            color="#c2410c", linewidth=2, label="Normal teórica")
    ax.set_title(titulo)
    ax.set_xlabel("Retorno diario")
    ax.legend()
    ax.grid(True, alpha=0.3)

axes[0].set_ylabel("Densidad")
plt.tight_layout()
plt.show()
```

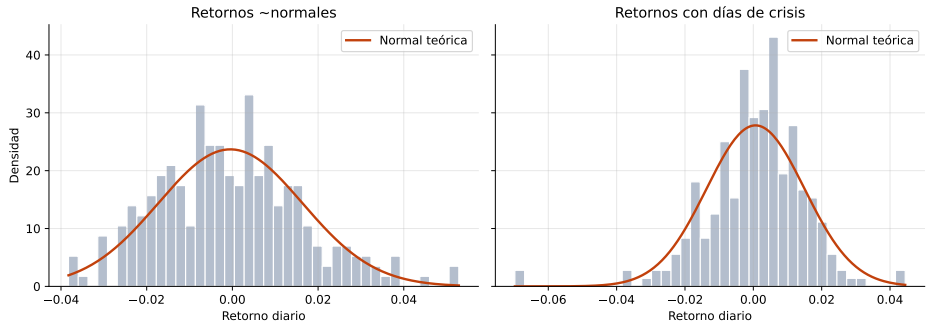


Figura 7.2.: Histogramas de los dos conjuntos de retornos simulados con la normal ajustada encima. Con días de crisis (derecha), aparece una cola izquierda que la campana no cubre.

En el panel derecho de la Figura 7.2 se ve la firma de los retornos financieros reales: una **cola izquierda pesada** que la campana normal no alcanza a cubrir. Los días de crisis existen, y la normal los declara casi imposibles.

7.1.3. Test de normalidad: Jarque-Bera

El ojo engaña; mejor un test formal. El test de **Jarque-Bera** pregunta exactamente lo que nos importa: *¿la asimetría y la curtosis de estos datos son compatibles con una distribución normal?* Su hipótesis nula (H_0) es "los datos son normales".

La mecánica de decisión es la de cualquier test de hipótesis: si el **p-value** es menor que el nivel de significancia (típicamente $\alpha = 0.05$), rechazamos H_0 — los datos serían muy improbables si la normal fuera cierta.

```
for nombre, datos in [("Simulados normales", retornos),
                      ("Con días de crisis", ret_real)]:
    stat_jb, p_value = stats.jarque_bera(datos)
    veredicto = "NO son normales" if p_value < 0.05 else "compatibles con normalidad
    print(f"{nombre:<22} JB={stat_jb:8.2f}  p-value={p_value:.4f} → {veredicto}")
```

```
Simulados normales     JB=    6.47  p-value=0.0394 → NO son normales
Con días de crisis     JB=  185.49  p-value=0.0000 → NO son normales
```

7.1.4. Con datos reales: los hechos estilizados

Los "días de crisis" de arriba los fabricamos nosotros. ¿Aparecen solos en datos de mercado? Apliquemos las mismas medidas a los diez activos del dataset del libro (`datos/precios.csv`, ver Apéndice B): cinco años de retornos diarios de 9 acciones y del S&P 500.

```python
precios = pd.read_csv("../datos/precios.csv", index_col="Date", parse_dates=True)
r_mercado = precios.pct_change().dropna()

momentos = pd.DataFrame({
    "Asimetría": r_mercado.apply(stats.skew),
    "Curtosis (exceso)": r_mercado.apply(stats.kurtosis),
    "p-value JB": r_mercado.apply(lambda col: stats.jarque_bera(col)[1]),
})
print(momentos.sort_values("Curtosis (exceso)").round(3))
```

	Asimetría	Curtosis (exceso)	p-value JB
XOM	-0.231	1.336	0.0
KO	-0.136	3.464	0.0
MSFT	0.063	3.566	0.0
JNJ	0.136	4.625	0.0
NVDA	0.547	4.647	0.0
JPM	0.116	4.974	0.0
AAPL	0.426	6.772	0.0
DIS	-0.058	7.315	0.0
SPY	0.312	8.965	0.0
WMT	-0.468	10.282	0.0

La tabla separa lo universal de lo que depende de la muestra. **La curtosis es universal**: los diez activos tienen exceso de curtosis positivo, desde 1.3 (XOM) hasta más de 10 (WMT), y el test de Jarque-Bera rechaza la normalidad en todos, con p-values que se redondean a cero. Las colas pesadas no son una rareza de un activo: son la regla. **La asimetría, en cambio, no tiene un signo fijo**: en estos cinco años, mayormente alcistas, sale negativa en algunos activos y positiva en otros, incluido el propio S&P 500. Y ahí hay una lección: la asimetría positiva del índice la explica **un solo día**, el rebote de $+10.5\,\%$ del 9 de abril de 2025; si se saca ese día de la muestra, la asimetría pasa de $+0.31$ a -0.43. Un estimador

basado en el cubo de los desvíos queda a merced de un puñado de días extremos, y esa fragilidad es otra cara de las colas pesadas. La asimetría negativa es un rasgo frecuente en muestras largas, pero conviene medirla con desconfianza.

> ⚠ ¿Y entonces por qué usamos la normal?
>
> Si los retornos reales no son normales, ¿por qué media-varianza, Black-Scholes y media industria asumen normalidad? Porque es **tratable** y suele ser una aproximación razonable *en el centro* de la distribución. El problema aparece **en las colas** — justamente donde vive el riesgo extremo. Por eso el Módulo 3 dedica capítulos enteros (VaR, Expected Shortfall, procesos con saltos) a medir lo que la normal esconde. Usá la normal, pero sabiendo qué te está simplificando.

7.2. Funciones lambda: el pegamento de la optimización

Antes de optimizar, un mini-desvío de Python. Los optimizadores reciben *funciones* como argumentos, y muchas veces son funciones de una sola línea que no ameritan un `def` con nombre. Para eso existen las **lambdas**: funciones anónimas de una expresión.

```
# Una función objetivo de juguete: una parábola con mínimo en x=3.5
funcion_error = lambda x: (x - 3.5) ** 2 + 2

for x in [0, 2, 3.5, 5]:
    print(f"f({x}) = {funcion_error(x):.2f}")
```

```
f(0) = 14.25
f(2) = 4.25
f(3.5) = 2.00
f(5) = 4.25
```

Las vas a ver constantemente en las **restricciones** de los optimizadores: `{"type": "eq", "fun": lambda w: w.sum() - 1}` se lee como "la función `w.sum() - 1` debe valer cero", es decir, los pesos deben sumar 1.

⚠ Lambdas dentro de un bucle: el truco `r=r_obj`

Una lambda no guarda el *valor* de las variables que usa: guarda la variable misma, y la lee recién cuando se ejecuta. Si creás lambdas dentro de un bucle, todas terminan viendo el **último** valor:

```
fs = [lambda x: x + k for k in range(3)]
print([f(0) for f in fs])          # [2, 2, 2]  ← todas usan k = 2
fs = [lambda x, k=k: x + k for k in range(3)]
print([f(0) for f in fs])          # [0, 1, 2]  ← el default congela cada valor
```

El argumento por defecto se evalúa al *crear* la lambda, así que congela el valor de esa vuelta. Vas a ver este truco en el próximo capítulo (`lambda w, r=r_obj: w @ mu - r`), al armar una restricción distinta para cada punto de la frontera.

7.3. Optimización con `scipy.optimize`

7.3.1. La idea: descenso guiado

`minimize` recibe una función objetivo y un punto de partida, y se mueve iterativamente "cuesta abajo" hasta encontrar un mínimo. No necesitás decirle cómo: los algoritmos (usaremos **SLSQP**, que soporta restricciones) estiman la pendiente y deciden el próximo paso.

La Figura 7.3 muestra la idea con un "valle" de dos variables: desde el punto de partida, cada paso baja en la dirección de mayor pendiente. Los pasos son largos donde el terreno es empinado y se acortan cerca del fondo. SLSQP es bastante más astuto que este descenso básico (en la parábola del código de abajo llega en dos pasos), pero la lógica de fondo es la misma.

```python
from scipy.optimize import minimize

# Función de dos variables con mínimo en (3, 5)
def funcion_objetivo(x):
    return (x[0] - 3) ** 2 + (x[1] - 5) ** 2

resultado = minimize(funcion_objetivo, x0=[0, 0], method="SLSQP")

print(f"Mínimo encontrado en: x={resultado.x[0]:.4f}, y={resultado.x[1]:.4f}")
print(f"Valor mínimo: {resultado.fun:.6f}")
print(f"Convergió: {resultado.success}")
```

```
Mínimo encontrado en: x=3.0000, y=5.0000
Valor mínimo: 0.000000
Convergió: True
```

La Figura 7.4 resume las piezas de una llamada a `minimize`, que vas a repetir en todo el módulo:

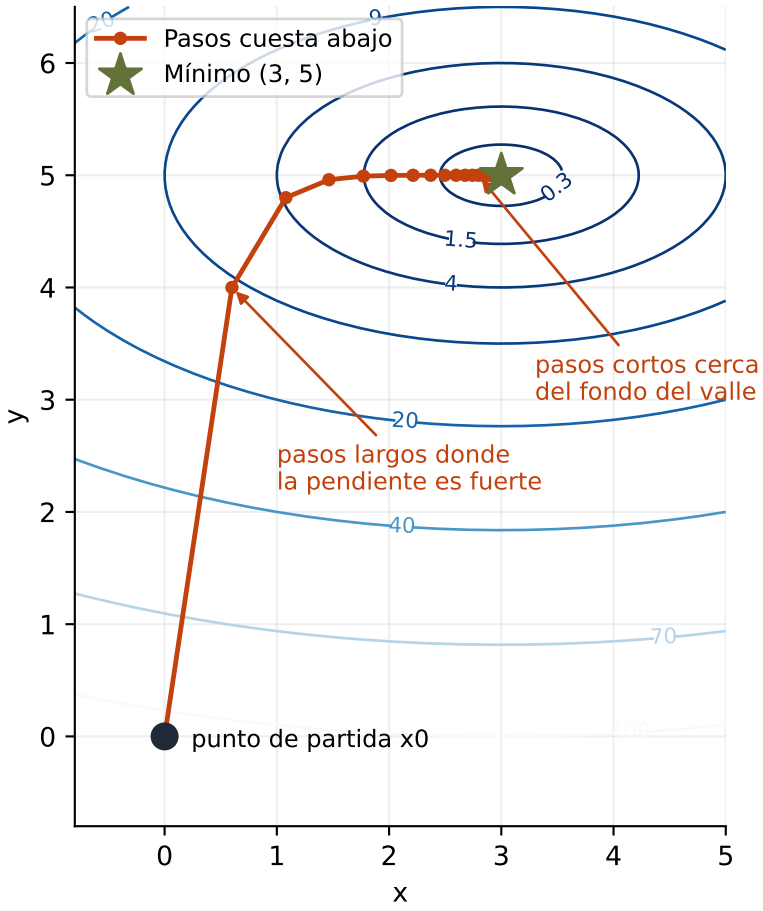


Figura 7.3.: Descenso por gradiente sobre $f(x, y) = (x - 3)^2 + 4(y - 5)^2$. Las curvas de nivel unen puntos con el mismo valor de la función; el camino baja desde (0, 0) hasta el mínimo en (3, 5).

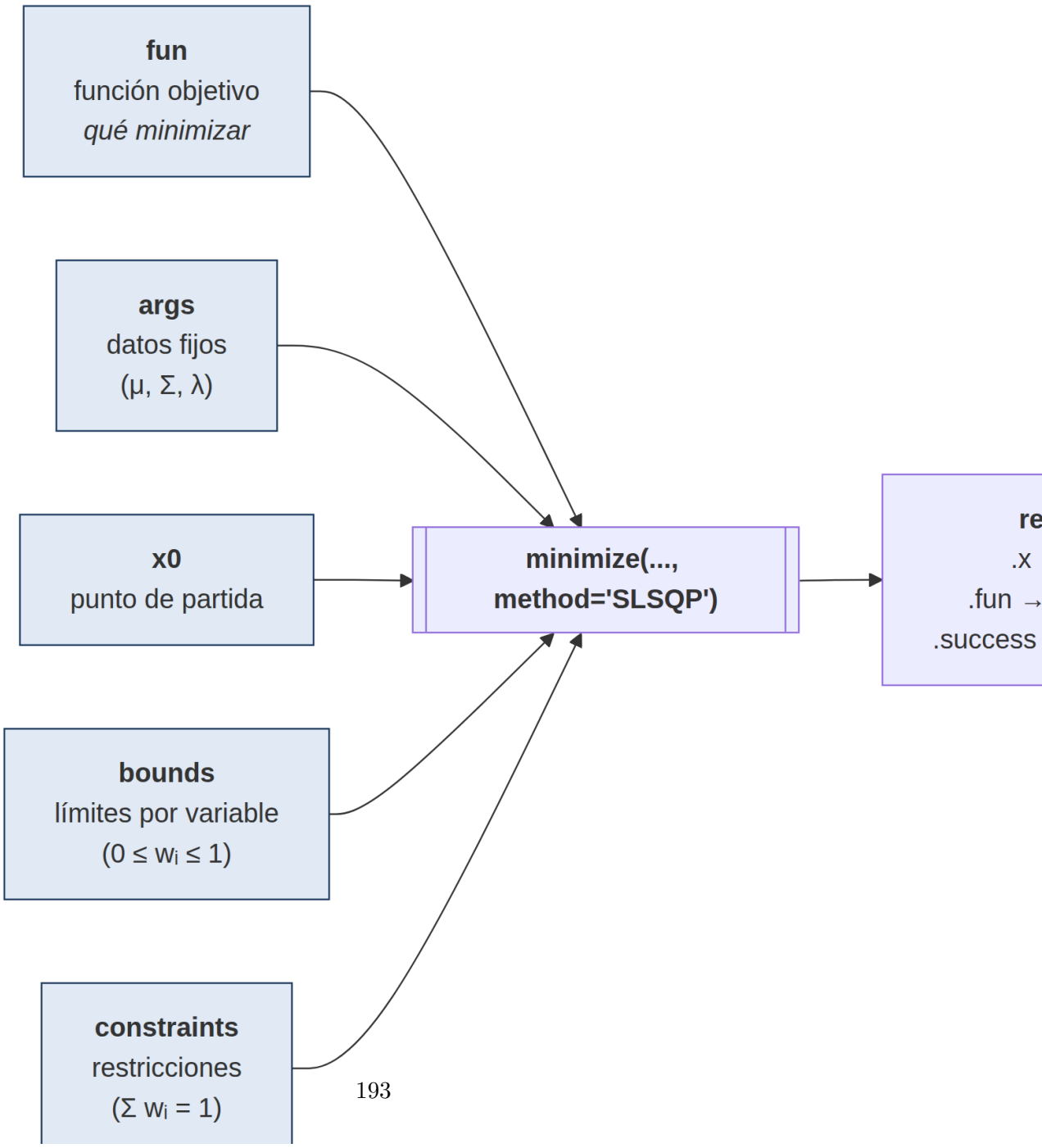


Figura 7.4.: Anatomía de `scipy.optimize.minimize`: qué le pasamos (objetivo, datos fijos, punto de partida, límites y restricciones) y qué hay que revisar en lo que devuelve antes de usarlo.

Tres cosas para mirar siempre en el resultado: `x` (dónde está el óptimo), `fun` (cuánto vale la función ahí) y `success` (si el algoritmo convergió — **nunca uses un resultado sin chequearlo**).

> **Maximizar = minimizar el negativo**
>
> `scipy` solo minimiza. Cuando quieras **maximizar** algo (utilidad, Sharpe), minimizá su negativo: $\max f(x) \quad \min -f(x)$. Por eso vas a ver funciones llamadas `utilidad_negativa` o `neg_sharpe` en todo el libro.

7.3.2. Optimización con restricciones: primer portafolio óptimo

Ahora un problema financiero de verdad. Un inversor con **aversión al riesgo** λ valora un portafolio según su utilidad media-varianza:

$$U(\mathbf{w}) = \mathbf{w}^T \mu - \frac{\lambda}{2} \mathbf{w}^T \Sigma \mathbf{w}$$

Es un balance explícito: el retorno suma, la varianza resta, y λ regula cuánto pesa el miedo. Busquemos los pesos óptimos para dos activos —uno agresivo y uno defensivo— con las restricciones de siempre: pesos que suman 1, sin cortos.

```
def utilidad_negativa(w, mu, Sigma, aversion):
    """Utilidad media-varianza, negada para minimizar."""
    ret_port = w @ mu
    var_port = w @ Sigma @ w
    return -(ret_port - 0.5 * aversion * var_port)

# Dos activos: uno agresivo (10%, vol 20%) y uno defensivo (6%, vol 10%)
mu = np.array([0.10, 0.06])
Sigma = np.array([[0.04, 0.005],
```

```
                  [0.005, 0.01]])

restricciones = [{"type": "eq", "fun": lambda w: w.sum() - 1}]
bounds = [(0, 1)] * 2

for aversion in [1, 3, 10]:
    res = minimize(utilidad_negativa, x0=[0.5, 0.5],
                   args=(mu, Sigma, aversion),
                   method="SLSQP", bounds=bounds, constraints=restricciones)
    w = res.x
    print(f" ={aversion:>2}: agresivo {w[0]:6.1%} | defensivo {w[1]:6.1%}"
          f" | E[r]={w @ mu:.2%} | vol={np.sqrt(w @ Sigma @ w):.2%}")
```

```
 = 1: agresivo 100.0% | defensivo   0.0% | E[r]=10.00% | vol=20.00%
 = 3: agresivo  45.8% | defensivo  54.2% | E[r]=7.83% | vol=11.76%
 =10: agresivo  22.5% | defensivo  77.5% | E[r]=6.90% | vol=9.89%
```

El resultado cuenta una historia coherente: a medida que sube la aversión al riesgo, el óptimo **migra del activo agresivo al defensivo**, sacrificando retorno esperado a cambio de menos volatilidad. Tres inversores con la misma información y distinta tolerancia obtienen tres carteras distintas —y las tres son "óptimas" para quien las tiene.

La Figura 7.5 extiende la tabla a un barrido continuo de λ: con aversión muy baja, todo va al activo agresivo; a medida que λ crece, el defensivo gana terreno rápido al principio y cada vez más lento después.

Este mecanismo —función objetivo + restricciones + `minimize`— es exactamente el que escala a la **frontera eficiente completa** del próximo capítulo. Ya tenés todas las piezas.

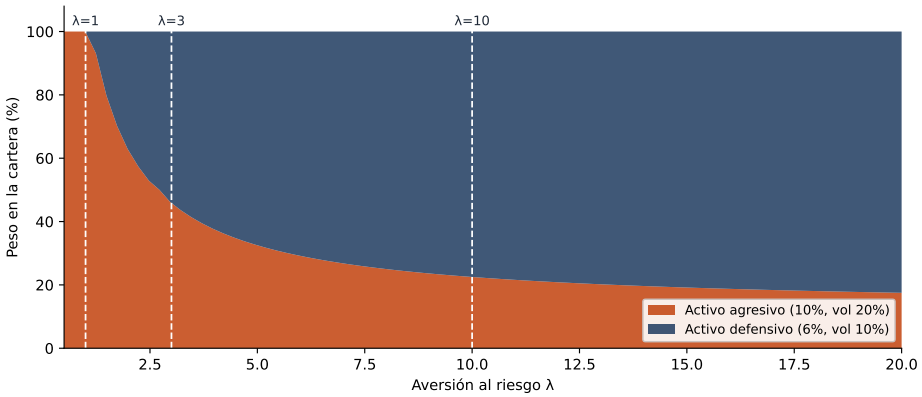


Figura 7.5.: Pesos óptimos de media-varianza según la aversión al riesgo . Las líneas punteadas marcan los tres valores de la tabla anterior.

7.4. Errores comunes

Trampas frecuentes

- **No chequear `resultado.success`**: si el optimizador no convergió, `resultado.x` puede ser basura con aspecto respetable.
- **Anualizar la volatilidad con $\times 252$** en lugar de $\times\sqrt{252}$.
- **Comparar `np.std` con pandas `.std()`**: el primero usa `ddof=0` y el segundo `ddof=1`; con pocos datos la diferencia se nota.
- **Confundir curtosis con "exceso de curtosis"**: `scipy.stats.kurtosis` devuelve el *exceso* (normal = 0). Otras librerías devuelven la curtosis cruda (normal = 3).
- **Interpretar un p-value alto como prueba de normalidad**: "no rechazar H_0" no es "aceptar H_0"; con pocas observaciones, el test simplemente no tiene potencia para detectar la anormalidad.
- **Punto de partida absurdo**: `minimize` es un buscador local.

> Con restricciones complejas, un `x0` razonable (p. ej., pesos equiponderados) evita óptimos locales o fallas de convergencia.

7.5. Practicá en el navegador

Podés experimentar con el código del capítulo acá mismo: la celda ejecuta Python **en tu navegador**, sin instalar nada (la primera ejecución tarda unos segundos mientras carga el entorno; los paquetes que importes se instalan solos).

```
#| label: practica-navegador
import numpy as np
from scipy.optimize import minimize
from scipy import stats

# Tu código acá...
```

7.6. Referencias

8. Capítulo 8: Teoría Moderna de Portafolios (Markowitz)

Frontera eficiente, métricas de desempeño y decisiones de diseño

El modelo de Markowitz formaliza la pregunta más importante de la inversión: ¿cómo repartir el capital entre activos riesgosos? Construimos la frontera eficiente con álgebra matricial y optimización numérica, encontramos el portafolio de mínima varianza y el tangente, y discutimos las decisiones que el paper de 1952 no responde: qué métrica optimizar (Sharpe, Sortino, varianza), cómo tratar los dividendos, qué rol juega la renta fija, cuándo trabajar con retornos reales y cuándo con excesos de retorno.

Introducción

Hasta ahora analizamos activos de a uno: su retorno, su volatilidad, su distribución. Pero ningún inversor serio tiene un solo activo. La pregunta real es otra: **¿cómo combinar activos para obtener el mejor balance entre retorno y riesgo?**

En 1952, Harry Markowitz —con 25 años— respondió esa pregunta con una idea que le valdría el Nobel: el riesgo de una cartera **no es el promedio de los riesgos individuales**, porque los activos no se mueven en sincronía perfecta. Cuando uno cae, otro puede subir. Esa compensación —la **covarianza**— es la materia prima de la diversificación, y puede medirse y optimizarse.

En este capítulo implementamos el modelo completo con NumPy y SciPy, y después nos metemos en las decisiones prácticas que todo gestor enfrenta y que el modelo básico deja abiertas.

> **Lo que ya sabés**
>
> Todo el andamiaje matemático de este capítulo —producto matricial `@`, matriz de covarianzas, la forma cuadrática $\mathbf{w}^T \Sigma \mathbf{w}$— lo construimos en el Capítulo 5 (NumPy). Y la herramienta de optimización, `scipy.optimize.minimize`, la vimos en el Capítulo 7. Acá las juntamos.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize
```

```
from scipy import stats

np.set_printoptions(precision=4, suppress=True)
rng = np.random.default_rng(42)
```

8.1. El problema en lenguaje matricial

Un portafolio de n activos queda definido por su vector de **pesos** $\mathbf{w} = (w_1, \dots, w_n)$, con $\sum w_i = 1$. Dadas las estimaciones de retornos esperados μ y la matriz de covarianzas Σ:

- **Retorno esperado del portafolio**: $E[R_p] = \mathbf{w}^T \mu$
- **Varianza del portafolio**: $\sigma_p^2 = \mathbf{w}^T \Sigma \mathbf{w}$

La segunda fórmula es la clave: incluye no solo las varianzas individuales (la diagonal de Σ) sino **todas las covarianzas cruzadas**. Ahí vive la diversificación.

```
# Universo de ejemplo: 4 activos con perfiles bien distintos
activos = ["Acción Growth", "Acción Value", "Acción Defensiva", "Bono Corporativo"]

mu = np.array([0.14, 0.10, 0.07, 0.05])      # retornos esperados anuales

vols = np.array([0.28, 0.20, 0.14, 0.06])    # volatilidades anuales
corr = np.array([
    [1.00, 0.65, 0.45, 0.10],
    [0.65, 1.00, 0.50, 0.15],
    [0.45, 0.50, 1.00, 0.20],
    [0.10, 0.15, 0.20, 1.00],
])
```

```python
Sigma = np.outer(vols, vols) * corr           # Σ = D   D

# Un portafolio cualquiera para arrancar
w = np.array([0.25, 0.25, 0.25, 0.25])

ret_p = w @ mu
vol_p = np.sqrt(w @ Sigma @ w)

print(f"Portafolio equiponderado:")
print(f"  Retorno esperado: {ret_p:.2%}")
print(f"  Volatilidad:      {vol_p:.2%}")
print(f"  Promedio simple de vols individuales: {vols.mean():.2%}")
```

```
Portafolio equiponderado:
  Retorno esperado: 9.00%
  Volatilidad:      13.42%
  Promedio simple de vols individuales: 17.00%
```

Fijate el resultado: la volatilidad del portafolio es **menor que el promedio de las volatilidades individuales**. No elegimos nada inteligente todavía —son pesos iguales— y la diversificación ya nos regaló varios puntos de riesgo. Eso es lo que Markowitz llamó *the only free lunch in finance*.

La Figura 8.1 muestra de dónde sale ese regalo con el caso más simple: dos activos combinados en todas las proporciones posibles. Si se movieran en perfecta sincronía ($\rho = 1$), la volatilidad de la mezcla sería el promedio ponderado de las dos y la combinación caería sobre una recta. Cuanto menor es la correlación, más se curva la línea hacia la izquierda: **menos riesgo por el mismo retorno**.

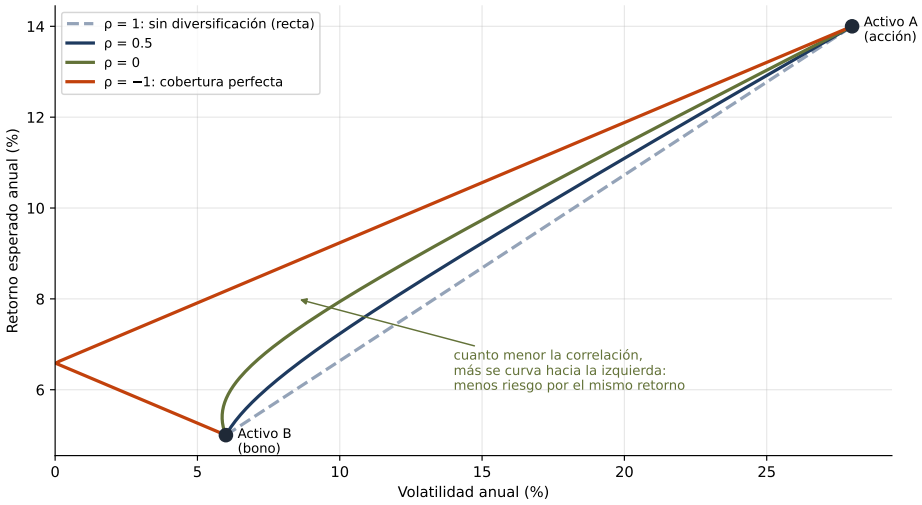


Figura 8.1.: Combinaciones de una acción (14 %, vol 28 %) y un bono (5 %, vol 6 %) para distintas correlaciones. Con $= 1$ no hay diversificación; con $= -1$ existe una mezcla de volatilidad cero.

> **Simulador interactivo**
>
> En la versión web de este capítulo hay un simulador donde podés mover la correlación y los pesos y ver cómo se curva esta línea.

8.2. La frontera eficiente

De todos los portafolios posibles, ¿cuáles valen la pena? Markowitz lo formuló como un problema de optimización: para cada nivel de retorno objetivo μ^*, buscar los pesos que **minimizan la varianza**:

$$\min_{\mathbf{w}} \ \mathbf{w}^T \Sigma \mathbf{w} \quad \text{sujeto a} \quad \mathbf{w}^T \mu = \mu^*, \quad \mathbf{w}^T \mathbf{1} = 1$$

El conjunto de soluciones dibuja una curva en el plano riesgo-retorno: la **frontera eficiente**. Todo portafolio por debajo de ella es mejorable (misma volatilidad, más retorno); por encima no hay nada alcanzable.

Un detalle: este problema genera la frontera de *mínima varianza* completa. Su tramo inferior —por debajo del portafolio de mínima varianza global que vemos enseguida— **no es eficiente**: para la misma volatilidad, el tramo superior ofrece más retorno. Por eso en el gráfico lo dibujamos punteado. En este universo ese tramo es muy corto (va del bono al GMV y casi queda tapado por su marcador), pero con otros activos puede ser largo, y conviene no confundirlo con la frontera.

```
def vol_portafolio(w, Sigma):
    return np.sqrt(w @ Sigma @ w)

n = len(mu)
```

```
w0 = np.ones(n) / n                        # punto de partida: equiponderado
bounds = [(0, 1)] * n                      # sin ventas en corto

retornos_objetivo = np.linspace(mu.min(), mu.max(), 60)
frontera_vol = []
frontera_pesos = []

for r_obj in retornos_objetivo:
    restricciones = [
        {"type": "eq", "fun": lambda w: w.sum() - 1},
        {"type": "eq", "fun": lambda w, r=r_obj: w @ mu - r},
    ]
    res = minimize(vol_portafolio, w0, args=(Sigma,),
                   method="SLSQP", bounds=bounds, constraints=restricciones)
    frontera_vol.append(res.fun if res.success else np.nan)
    frontera_pesos.append(res.x)

frontera_vol = np.array(frontera_vol)
```

El argumento `r=r_obj` dentro de la lambda no es decorativo: congela el retorno objetivo de cada vuelta del bucle (ver el recuadro sobre lambdas en bucles del Capítulo 7). Sin él, todas las restricciones usarían el último `r_obj` y la frontera colapsaría en un solo punto.

8.2.1. El portafolio de mínima varianza global

Un punto especial de la frontera es su extremo izquierdo: el portafolio de **mínima varianza global (GMV)**, el más conservador posible con estos activos. No necesita estimar retornos esperados —solo Σ—, lo que lo hace más robusto que el resto de la frontera (recordá: μ es lo más ruidoso).

```python
res_gmv = minimize(vol_portafolio, w0, args=(Sigma,),
                   method="SLSQP", bounds=bounds,
                   constraints=[{"type": "eq", "fun": lambda w: w.sum() - 1}])
w_gmv = res_gmv.x

print("Portafolio de Mínima Varianza Global:")
for a, wi in zip(activos, w_gmv):
    print(f"  {a:<18} {wi:6.1%}")
print(f"  Retorno: {w_gmv @ mu:.2%} | Volatilidad: {res_gmv.fun:.2%}")
```

```
Portafolio de Mínima Varianza Global:
  Acción Growth        0.0%
  Acción Value         1.3%
  Acción Defensiva     8.8%
  Bono Corporativo    89.9%
  Retorno: 5.24% | Volatilidad: 5.84%
```

Como era de esperar, el GMV se carga hacia el bono (el activo menos volátil) pero **no pone todo ahí**: mantiene algo de acciones porque su baja correlación con el bono sigue restando riesgo total. La diversificación manda incluso cuando solo minimizás varianza.

8.2.2. El portafolio tangente y la Capital Market Line

Ahora agreguemos un ingrediente: un **activo libre de riesgo** con tasa r_f (un plazo fijo, una letra del tesoro). Poder combinar la frontera con ese activo cambia la geometría del problema: las combinaciones entre r_f y cualquier portafolio riesgoso son **rectas** en el plano riesgo-retorno.

¿Cuál es la mejor recta? La que sale de r_f y **apenas toca** la frontera: la **Capital Market Line (CML)**. El punto de tangencia es el **portafolio tangente**, que maximiza el **ratio de Sharpe**:

$$\text{Sharpe} = \frac{E[R_p] - r_f}{\sigma_p}$$

```
rf = 0.03

def neg_sharpe(w, mu, Sigma, rf):
    return -(w @ mu - rf) / np.sqrt(w @ Sigma @ w)

res_tan = minimize(neg_sharpe, w0, args=(mu, Sigma, rf),
                   method="SLSQP", bounds=bounds,
                   constraints=[{"type": "eq", "fun": lambda w: w.sum() - 1}])
w_tan = res_tan.x
ret_tan, vol_tan = w_tan @ mu, np.sqrt(w_tan @ Sigma @ w_tan)
sharpe_tan = (ret_tan - rf) / vol_tan

print("Portafolio Tangente (máximo Sharpe):")
for a, wi in zip(activos, w_tan):
    print(f"  {a:<18} {wi:6.1%}")
print(f"  Retorno: {ret_tan:.2%} | Vol: {vol_tan:.2%} | Sharpe: {sharpe_tan:.3f}")
```

```
Portafolio Tangente (máximo Sharpe):
  Acción Growth       14.9%
  Acción Value         7.9%
  Acción Defensiva     6.1%
  Bono Corporativo    71.1%
  Retorno: 6.85% | Vol: 7.67% | Sharpe: 0.502
```

```
fig, ax = plt.subplots(figsize=(10, 6))

# Frontera: por encima del GMV es eficiente; por debajo, ineficiente (punteada)
corte = np.searchsorted(retornos_objetivo, w_gmv @ mu)
ax.plot(frontera_vol[corte:] * 100, retornos_objetivo[corte:] * 100, color="#1e3a5f"
```

```
        linewidth=2, label="Frontera eficiente")
ax.plot(frontera_vol[:corte + 1] * 100, retornos_objetivo[:corte + 1] * 100, ":",
        color="#1e3a5f", linewidth=1.5, label="Tramo ineficiente")

# Capital Market Line: recta desde (0, rf) que pasa por el tangente
x_cml = np.linspace(0, vol_tan * 1.4, 50)
ax.plot(x_cml * 100, (rf + sharpe_tan * x_cml) * 100, "--", color="#637238", linewid
        label="Capital Market Line")

# Puntos notables
ax.scatter(vols * 100, mu * 100, s=90, c="#c2410c", zorder=5, label="Activos individ
for a, v, m in zip(activos, vols, mu):
    ax.annotate(a, (v * 100, m * 100), textcoords="offset points",
                xytext=(8, -4), fontsize=8)
ax.scatter(res_gmv.fun * 100, (w_gmv @ mu) * 100, s=160, c="#b45309", marker="D",
           zorder=6, label="Mínima varianza (GMV)")
ax.scatter(vol_tan * 100, ret_tan * 100, s=180, c="#637238", marker="*",
           zorder=6, label="Portafolio tangente")
ax.scatter(0, rf * 100, s=90, c="black", marker="s", zorder=6,
           label=f"Libre de riesgo ({rf:.0%})")

ax.set_xlabel("Volatilidad anual (%)")
ax.set_ylabel("Retorno esperado anual (%)")
ax.set_title("Frontera Eficiente, GMV, Tangente y CML")
ax.legend(loc="lower right")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

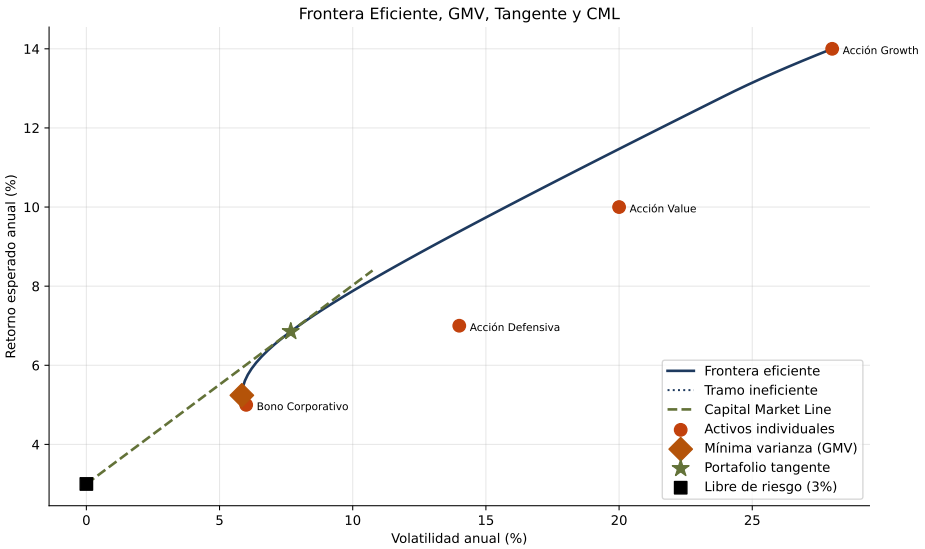


Figura 8.2.: La frontera eficiente (línea llena) y su tramo ineficiente (punteado), los cuatro activos, el portafolio de mínima varianza, el tangente y la Capital Market Line que une la tasa libre de riesgo con el tangente.

La lectura de la Figura 8.2 es una de las ideas más profundas de las finanzas: **con un activo libre de riesgo disponible, la frontera curva deja de ser el menú óptimo**. Todo inversor racional debería pararse sobre la CML: elegir el portafolio tangente para su plata riesgosa, y regular su nivel de riesgo total moviendo la proporción entre el tangente y el activo libre de riesgo. El conservador pone 30 % en el tangente y 70 % a tasa; el agresivo, todo en el tangente (o incluso se apalanca). La *composición* de la parte riesgosa es la misma para ambos.

¿Y qué hay *adentro* de cada punto de la frontera? La Figura 8.3 lo muestra, de izquierda (GMV) a derecha (máximo retorno): el bono domina el extremo conservador, y a medida que se le pide más retorno la cartera va cambiando

bono por acciones, hasta terminar 100 % en la acción growth. Fijate que la acción defensiva nunca pesa mucho y sale de la cartera a mitad de camino: su combinación de retorno y correlación no compite con la de sus vecinas.

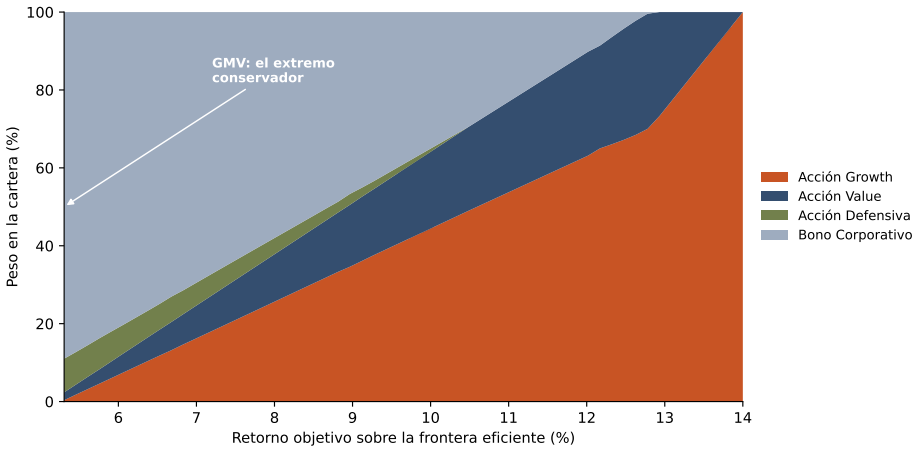


Figura 8.3.: Composición de la cartera a lo largo de la frontera eficiente. Cada corte vertical es un portafolio de la frontera; las bandas muestran el peso de cada activo.

> **Simulador interactivo**
>
> En la versión web de este capítulo hay un simulador de la frontera eficiente donde podés cambiar los retornos esperados, la tasa libre de riesgo y la aversión al riesgo, y ver cómo se recompone el tangente.

8.3. ¿Qué métrica optimizar? Sharpe, Sortino y compañía

Hasta acá maximizamos Sharpe como si fuera la única opción. No lo es, y elegir la función objetivo es una **decisión de diseño** con consecuencias. Repasemos el menú:

Objetivo	Qué premia / castiga	Cuándo tiene sentido
Mínima varianza	Castiga toda dispersión, ignora el retorno	Perfil ultraconservador; cuando desconfiás de tu estimación de μ
Máximo retorno	Ignora el riesgo por completo	Casi nunca solo: lleva a concentrar todo en el activo más prometedor
Máximo Sharpe	Retorno en exceso por unidad de volatilidad *total*	El estándar de la industria para comparar estrategias
Máximo Sortino	Como Sharpe, pero solo castiga la volatilidad *a la baja*	Cuando la distribución es asimétrica y no querés castigar las subas

El punto fino está entre Sharpe y Sortino. La volatilidad castiga **ambas colas**: para el Sharpe, un activo que da sorpresas positivas enormes es tan "riesgoso" como uno que se derrumba. El **Sortino** corrige eso usando en el denominador solo la **desviación a la baja** (*downside deviation*), calculada con los retornos por debajo de un umbral (típicamente r_f o cero):

$$\text{Sortino} = \frac{E[R_p] - r_f}{\sigma_{down}}, \qquad \sigma_{down} = \sqrt{\frac{1}{n}\sum_{r_i<\tau}(r_i - \tau)^2}$$

```python
# Simulamos la historia de dos estrategias con igual media y varianza...
n_dias = 2500
r_A = rng.normal(0.0005, 0.012, n_dias)                    # simétrica

# ...pero B tiene asimetría positiva: pierde poco seguido, gana mucho de a ratos
r_B = rng.exponential(0.009, n_dias) - 0.0085
r_B = (r_B - r_B.mean()) / r_B.std() * 0.012 + 0.0005      # igual media y desvío qu

rf_diario = 0.03 / 252

def sharpe_anual(r):
    return (r.mean() - rf_diario) / r.std() * np.sqrt(252)

def sortino_anual(r, umbral=rf_diario):
    downside = np.sqrt(np.mean(np.minimum(r - umbral, 0) ** 2))
    return (r.mean() - umbral) / downside * np.sqrt(252)

print(f"{'':12}{'Sharpe':>8}{'Sortino':>9}{'Asimetría':>11}")
for nombre, r in [("Estrategia A", r_A), ("Estrategia B", r_B)]:
    print(f"{nombre:<12}{sharpe_anual(r):>8.2f}{sortino_anual(r):>9.2f}"
          f"{stats.skew(r):>11.2f}")
```

```
              Sharpe  Sortino  Asimetría
Estrategia A   -0.20    -0.28      -0.04
Estrategia B    0.50     1.02       1.96
```

Las dos estrategias tienen el mismo Sharpe (misma media, misma varianza), pero el Sortino revela que **B pierde de manera más contenida**: su riesgo "malo" es menor. Si tu inversor sufre las caídas más de lo que disfruta las subas —o sea, si es humano—, el Sortino describe mejor su experiencia.

¿Cuál usar?

No hay respuesta universal, pero sí una guía: **Sharpe para comparar y comunicar** (todo el mundo lo entiende y lo calcula igual), **Sortino cuando la asimetría importa** (estrategias con opciones, crédito, o cualquier cosa con colas raras), **mínima varianza cuando no confiás en tus retornos esperados**, y "máximo retorno" solo como provocación intelectual. Y siempre mirá más de una métrica: cada una ilumina un ángulo distinto.

8.4. Los inputs importan más que el optimizador

El 90 % del esfuerzo práctico en Markowitz no está en la optimización (eso lo hace `scipy` en milisegundos) sino en **estimar bien μ y Σ**. Tres decisiones concretas que cambian los resultados:

La Figura 8.4 ordena esas decisiones como lo que son: pasos de una tubería que termina en μ y Σ, mucho antes de llegar al optimizador.

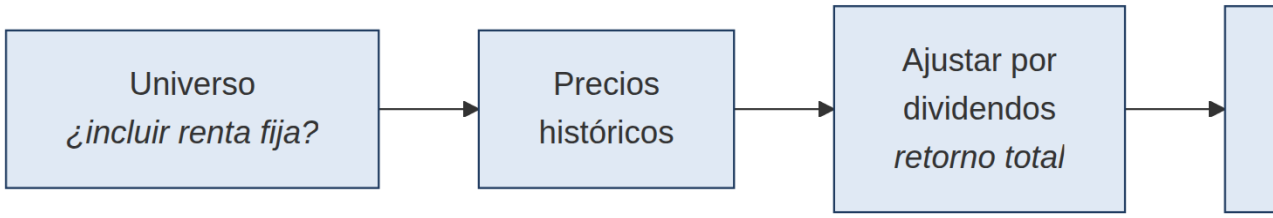


Figura 8.4.: De los precios a los pesos. El optimizador es el último paso y el más rápido; las decisiones que determinan el resultado se toman antes, al construir y Σ.

8.4.1. Dividendos: usar retorno total, no solo precio

El retorno de una acción tiene dos componentes: la variación del precio **y los dividendos**. Ignorarlos —un error común al bajar series de precios sin ajustar— subestima sistemáticamente μ:

$$R_{total} = \underbrace{\frac{P_t - P_{t-1}}{P_{t-1}}}_{\text{apreciación}} + \underbrace{\frac{D_t}{P_{t-1}}}_{\text{dividend yield}}$$

```
# Una acción "value" típica: sube poco de precio pero reparte 4% anual
apreciacion = 0.05
dividend_yield = 0.04

print(f"Retorno solo-precio: {apreciacion:.1%}")
print(f"Retorno total:       {apreciacion + dividend_yield:.1%}")

# En 20 años, la diferencia es enorme por capitalización:
print(f"\n$100 a 20 años, solo precio:   ${100 * (1 + apreciacion) ** 20:,.0f}")
print(f"$100 a 20 años, retorno total: ${100 * (1 + apreciacion + dividend_yield) **
```

```
Retorno solo-precio: 5.0%
Retorno total:       9.0%

$100 a 20 años, solo precio:   $265
$100 a 20 años, retorno total: $560
```

Para un optimizador, esto no es un detalle: una acción de alto dividendo puede parecer mediocre mirando solo el precio y ser excelente en retorno total. **Usá siempre precios ajustados por dividendos** (*adjusted close*) o sumá el yield explícitamente.

8.4.2. Renta fija en el universo: más que "el activo sin riesgo"

En el modelo de la CML, la renta fija aparece como el activo libre de riesgo de volatilidad cero. Pero un **bono real** (como los que valuamos en el Módulo 1) no es eso: tiene riesgo de tasa (duración), riesgo de crédito y volatilidad de precio. La forma correcta de tratarlo es **incluirlo como un activo riesgoso más** dentro del universo, con su μ, su σ y —lo más valioso— su **baja correlación** con las acciones.

Ya lo hicimos sin decirlo: nuestro cuarto activo es un bono corporativo. Mirá cuánto aporta:

```
# Frontera SIN el bono (solo los 3 primeros activos)
mu3, Sigma3 = mu[:3], Sigma[:3, :3]
w03 = np.ones(3) / 3

vols_sin_bono = []
rets_grid = np.linspace(mu3.min(), mu3.max(), 40)
for r_obj in rets_grid:
    res = minimize(vol_portafolio, w03, args=(Sigma3,), method="SLSQP",
                   bounds=[(0, 1)] * 3,
                   constraints=[{"type": "eq", "fun": lambda w: w.sum() - 1},
                                {"type": "eq", "fun": lambda w, r=r_obj: w @ mu3 - r
    vols_sin_bono.append(res.fun)

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(np.array(vols_sin_bono) * 100, rets_grid * 100, "--", color="#c2410c", linew
        label="Frontera sin bonos (solo acciones)")
ax.plot(frontera_vol * 100, retornos_objetivo * 100, color="#1e3a5f", linewidth=2,
        label="Frontera con bonos")
ax.set_xlabel("Volatilidad anual (%)")
ax.set_ylabel("Retorno esperado anual (%)")
ax.set_title("El efecto de agregar renta fija al universo")
ax.legend()
```

```
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

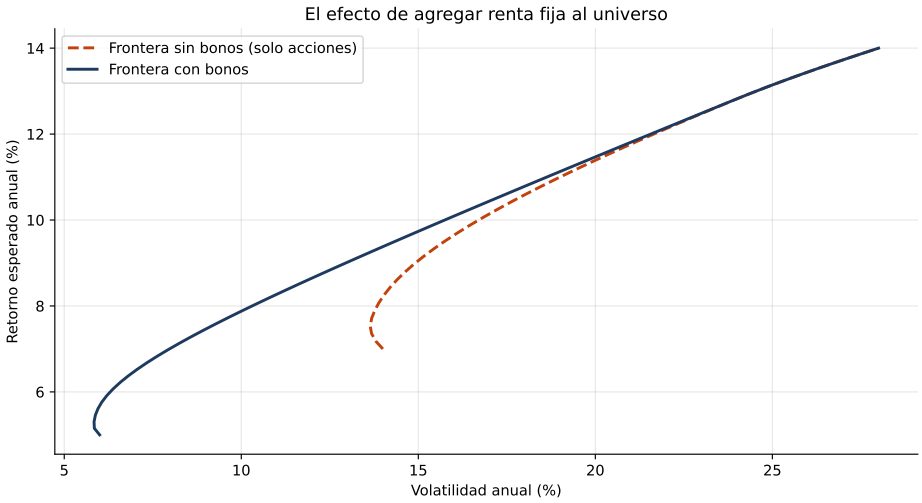


Figura 8.5.: Frontera eficiente con y sin el bono en el universo. Con el bono, la frontera se corre a la izquierda en la zona de riesgo bajo y medio: mismo retorno, menos volatilidad.

En la Figura 8.5, la frontera con bonos **domina** a la de solo-acciones en toda la zona de riesgo bajo y medio: para los mismos retornos objetivo, menos volatilidad. El bono no está ahí por su retorno (es el más bajo del universo) sino por su **correlación baja**: es el mejor diversificador de la mesa. Esta es la base cuantitativa del clásico portafolio 60/40.

8.4.3. Inflación y horizontes largos: deflactar o engañarse

Cuando el análisis abarca períodos largos —décadas de historia, o economías de alta inflación—, trabajar con retornos **nominales** distorsiona todo. Un retorno nominal del 40 % anual con inflación del 45 % es una **pérdida** de poder adquisitivo, aunque el número se vea espectacular. La conversión correcta es:

$$1 + r_{real} = \frac{1 + r_{nominal}}{1 + \pi} \quad \Rightarrow \quad r_{real} = \frac{r_{nominal} - \pi}{1 + \pi}$$

```
casos = [
    ("EEUU, año típico", 0.10, 0.03),
    ("Argentina, año de alta inflación", 0.40, 0.45),
    ("Argentina, buen año real", 0.80, 0.50),
]

print(f"{'Caso':<34}{'Nominal':>9}{'Inflación':>11}{'Real':>8}")
for nombre, r_nom, pi in casos:
    r_real = (r_nom - pi) / (1 + pi)
    print(f"{nombre:<34}{r_nom:>8.0%}{pi:>10.0%}{r_real:>8.1%}")
```

```
Caso                               Nominal  Inflación    Real
EEUU, año típico                       10%         3%    6.8%
Argentina, año de alta inflación       40%        45%   -3.4%
Argentina, buen año real               80%        50%   20.0%
```

¿Por qué importa para Markowitz? Porque en series largas o inflacionarias, la inflación **infla artificialmente las medias y las correlaciones**: todos los activos "suben" juntos con el nivel de precios, y el optimizador cree ver retornos altos y co-movimientos que no existen en términos reales. La regla práctica:

- **Horizontes cortos y baja inflación** (retornos diarios/mensuales en economías estables): trabajar en nominal es aceptable; la inflación es un ruido de fondo casi constante.
- **Horizontes largos o alta inflación**: deflactá las series (o trabajá en una moneda dura de referencia) *antes* de estimar μ y Σ. Lo que le importa al inversor es el poder de compra, no los pesos nominales.

8.4.4. ¿Retornos directos o excesos de retorno?

Última decisión de diseño: ¿optimizamos sobre los retornos "crudos" de cada activo, o sobre su **exceso** respecto de una referencia ($r_i - r_f$, o $r_i - r_{mkt}$)? La respuesta depende de qué pregunta estés haciendo:

- **Retornos directos (o reales)** — cuando la pregunta es de **asignación de capital total**: "tengo X, ¿cómo lo reparto?". El nivel absoluto de cada retorno importa, porque la alternativa de no invertir (o de quedarse en tasa) es parte del problema. La frontera y la CML de este capítulo trabajan así.
- **Excesos sobre la tasa libre de riesgo** ($r_i - r_f$) — cuando la tasa varía en el tiempo y querés que tu estimación de μ no mezcle "premio por riesgo" con "nivel de tasas". El ratio de Sharpe ya está definido sobre excesos; estimar directamente en excesos hace todo más consistente, especialmente con datos que cruzan regímenes de tasas muy distintos.
- **Excesos sobre el mercado** ($r_i - r_{mkt}$) — cuando el problema no es asignar capital sino **construir una cartera activa** que le gane a un benchmark. Ahí lo que optimizás es el *tracking error* y el exceso de retorno relativo; el riesgo "de mercado" es irrelevante porque el benchmark ya lo carga. Es el mundo de los fondos con mandato relativo.

⚠ El error clásico

Mezclar marcos: estimar μ como promedio de retornos nominales de una década de tasas altas, y usarlo para optimizar hoy con tasas bajas. El "premio" que el optimizador ve es en gran parte nivel de tasas que ya no existe. Trabajar en excesos sobre r_f inmuniza contra esto.

8.5. Las limitaciones del modelo (y adónde vamos)

Markowitz es el punto de partida de toda la teoría de portafolios, pero conviene salir del capítulo con sus límites claros:

1. **Hipersensibilidad a los inputs**: cambios chicos en μ producen carteras completamente distintas, y μ es casi imposible de estimar bien. Es el problema que atacan **HRP** (capítulo siguiente, esquivando la inversión de Σ) y **Black-Litterman** (Capítulo 10, disciplinando la estimación de μ).
2. **Supone que la varianza captura el riesgo**: ignora asimetrías y colas pesadas. Las métricas de cola (VaR, Expected Shortfall) llegan en el Módulo 3.
3. **Un solo período**: el modelo decide una vez y se queda quieto; no contempla rebalanceo, costos de transacción ni impuestos.
4. **Las correlaciones no son constantes**: en las crisis, justo cuando más importa, las correlaciones suben y la diversificación se evapora. Lo veremos con crudeza en el capítulo de riesgo y regulación.

Nada de esto "refuta" a Markowitz: define la agenda de todo lo que sigue en el libro.

8.6. Con datos reales

Todo el capítulo trabajó con un universo inventado de 4 activos, donde nosotros elegimos μ y Σ. Cerremos con el ejercicio completo sobre **datos de mercado**: el libro incluye en `datos/precios.csv` cinco años de precios diarios ajustados (jul-2021 a jun-2026) de 9 acciones de sectores distintos más el ETF SPY (ver el Apéndice B para los detalles del dataset).

```
import pandas as pd

precios = pd.read_csv("../datos/precios.csv", index_col="Date", parse_dates=True)
acciones = [c for c in precios.columns if c != "SPY"]   # SPY lo reservamos como "me

# Retornos mensuales (más estables que los diarios para estimar )
r_mensual = precios[acciones].resample("ME").last().pct_change().dropna()

mu_real = r_mensual.mean().values * 12                  # anualizado
Sigma_real = r_mensual.cov().values * 12

print(f"Observaciones mensuales: {len(r_mensual)}")
print(f"{'Ticker':<8}{' anual':>9}{' anual':>9}")
for t, m, s in zip(acciones, mu_real, np.sqrt(np.diag(Sigma_real))):
    print(f"{t:<8}{m:>8.1%}{s:>9.1%}")
```

```
Observaciones mensuales: 59
Ticker      anual    anual
AAPL       17.1%    25.3%
MSFT        9.3%    25.6%
NVDA       60.3%    51.0%
JPM        21.1%    24.0%
XOM        24.8%    27.0%
KO         11.9%    16.5%
JNJ        12.8%    18.4%
```

```
WMT         21.5%    20.7%
DIS         -6.0%    33.2%
```

Con μ y Σ estimados, el resto es exactamente el código que ya escribimos:

```python
n_r = len(acciones)
w0_r = np.ones(n_r) / n_r
bounds_r = [(0, 1)] * n_r

# GMV
res_gmv_r = minimize(vol_portafolio, w0_r, args=(Sigma_real,),
                     method="SLSQP", bounds=bounds_r,
                     constraints=[{"type": "eq", "fun": lambda w: w.sum() - 1}])

# Tangente
res_tan_r = minimize(neg_sharpe, w0_r, args=(mu_real, Sigma_real, rf),
                     method="SLSQP", bounds=bounds_r,
                     constraints=[{"type": "eq", "fun": lambda w: w.sum() - 1}])

print(f"{'Ticker':<8}{'GMV':>8}{'Tangente':>10}")
for t, wg, wt in zip(acciones, res_gmv_r.x, res_tan_r.x):
    print(f"{t:<8}{wg:>8.1%}{wt:>10.1%}")

ret_tan_r = res_tan_r.x @ mu_real
vol_tan_r = np.sqrt(res_tan_r.x @ Sigma_real @ res_tan_r.x)
print(f"\nTangente: retorno {ret_tan_r:.1%}, vol {vol_tan_r:.1%}, "
      f"Sharpe {(ret_tan_r - rf) / vol_tan_r:.2f}")
```

```
Ticker       GMV  Tangente
AAPL         1.6%      0.0%
MSFT        23.1%      0.0%
NVDA         0.0%     23.3%
JPM          4.8%      0.0%
```

```
XOM          13.4%      29.2%
KO           19.3%       5.4%
JNJ          27.9%      17.9%
WMT           9.9%      24.2%
DIS           0.0%       0.0%

Tangente: retorno 29.4%, vol 16.9%, Sharpe 1.57
```

La Figura 8.6 pone las dos carteras lado a lado:

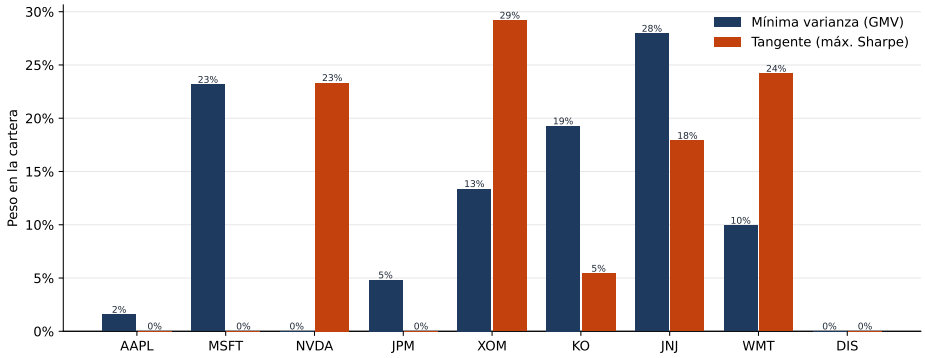


Figura 8.6.: Pesos de la cartera de mínima varianza y de la tangente estimadas con 5 años de datos reales mensuales. La tangente concentra el capital en pocos tickers; el GMV reparte más.

El resultado ilustra en carne propia las advertencias del capítulo. Mirá primero la tabla de μ estimados: NVDA con un retorno anualizado altísimo (la ola de la IA cayó entera dentro de la ventana), DIS en terreno negativo. El portafolio tangente **se toma esos números al pie de la letra**: asigna cero a más de la mitad del universo y concentra el capital en los cuatro o cinco tickers que mejor combinaron retorno y volatilidad *en esa muestra*. ¿De verdad creés que NVDA va a repetir ese los próximos cinco años? El optimizador sí. El GMV, que no usa μ, queda bastante más repartido y cargado en los activos defensivos. Esa fragilidad frente al μ estimado es

exactamente el problema que atacan HRP (capítulo siguiente) y Black-Litterman (Capítulo 10) — y ahora la viste con datos de verdad.

8.7. Errores comunes

Trampas frecuentes

- **Anualizar mal**: los retornos escalan por T ($\times 252$ de diario a anual) pero la volatilidad escala por $\sqrt{T}$ ($\times\sqrt{252}$). Mezclar las dos reglas rompe todos los ratios.
- **Usar precios sin ajustar por dividendos**: subestima el retorno de los activos de alto yield y sesga la optimización hacia las acciones growth.
- **Olvidar las restricciones**: sin `bounds=[(0,1)]`, el optimizador puede sugerir ventas en corto del 300 %. Verificá siempre que la solución sea implementable.
- **Confiar en un único punto de partida**: `minimize` puede quedar atrapado en óptimos locales con restricciones complejas. Ante dudas, probá varios `w0`.
- **Interpretar la frontera como predicción**: la frontera es óptima *para las estimaciones que le diste*. Si μ y Σ están mal, la frontera es una ficción precisa.

8.8. Practicá en el navegador

Podés experimentar con el código del capítulo acá mismo: la celda ejecuta Python **en tu navegador**, sin instalar nada (la primera ejecución tarda unos segundos mientras carga el entorno; los paquetes que importes se instalan solos).

```
#| label: practica-navegador
import numpy as np
from scipy.optimize import minimize

# El universo del capítulo, listo para experimentar:
mu = np.array([0.14, 0.10, 0.07, 0.05])
vols = np.array([0.28, 0.20, 0.14, 0.06])
corr = np.array([[1.00, 0.65, 0.45, 0.10],
                 [0.65, 1.00, 0.50, 0.15],
                 [0.45, 0.50, 1.00, 0.20],
                 [0.10, 0.15, 0.20, 1.00]])
Sigma = np.outer(vols, vols) * corr

# Para usar el dataset real del libro (requiere conexión):
# import pandas as pd
# from pyodide.http import open_url
# url = "https://diegoonna.github.io/libro_finanzas_computacionales/datos/precios.cs
# precios = pd.read_csv(open_url(url), index_col="Date", parse_dates=True)

# Tu código acá...
```

8.9. Referencias

9. Capítulo 9: Hierarchical Risk Parity (HRP)

Clustering jerárquico para construir portafolios robustos

La optimización de Markowitz exige invertir la matriz de covarianzas, y esa operación amplifica brutalmente los errores de estimación: carteras extremas, inestables y que rinden mal fuera de muestra. Marcos López de Prado propuso en 2016 una alternativa que no invierte ninguna matriz: agrupar los activos por similitud con clustering jerárquico y repartir el capital de arriba hacia abajo según el riesgo de cada grupo. En este capítulo implementamos HRP desde cero y lo comparamos con Markowitz.

Introducción: el talón de Aquiles de Markowitz

En el capítulo anterior celebramos la elegancia de la frontera eficiente. Ahora toca la mala noticia: en la práctica, la optimización de media-varianza tiene fama de producir carteras **inestables y extremas**. Los operadores la llaman, con sorna, *"error maximizer"*: una máquina de maximizar errores.

¿Por qué? La solución de Markowitz depende de Σ^{-1}, la **inversa de la matriz de covarianzas**. Y esa inversión tiene dos problemas serios:

1. **Amplifica el error de estimación.** Σ se estima con datos históricos, así que trae ruido. Al invertirla, el ruido no se atenúa: se multiplica. Un cambio minúsculo en una correlación puede dar vuelta la cartera entera.
2. **Se vuelve numéricamente frágil cuando los activos están correlacionados.** Si dos activos se mueven casi igual (pensá en dos ETFs del mismo sector), Σ está cerca de ser *singular* —no invertible— y la inversa "explota": aparecen pesos gigantes, positivos en uno y negativos en el otro.

Veámoslo con un experimento antes de proponer la solución.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.cluster import hierarchy
```

```
from scipy.spatial.distance import squareform

np.set_printoptions(precision=4, suppress=True)
rng = np.random.default_rng(42)
```

Experimento: la inestabilidad en acción

Simulamos retornos de 5 activos donde dos de ellos están **muy correlacionados**, y calculamos la cartera de mínima varianza de Markowitz con dos muestras distintas de la misma historia. Si el método fuera robusto, ambas muestras deberían dar carteras parecidas.

```
def simular_retornos(n_dias, seed):
    """5 activos; los activos 0 y 1 comparten un factor común (muy correlacionados).
    rng = np.random.default_rng(seed)
    factor_comun = rng.normal(0, 0.015, n_dias)
    r = np.column_stack([
        factor_comun + rng.normal(0.0004, 0.004, n_dias),   # activo 0
        factor_comun + rng.normal(0.0004, 0.004, n_dias),   # activo 1 (casi gemelo
        rng.normal(0.0005, 0.012, n_dias),                  # activo 2
        rng.normal(0.0003, 0.010, n_dias),                  # activo 3
        rng.normal(0.0004, 0.008, n_dias),                  # activo 4
    ])
    return r

def pesos_min_varianza(Sigma):
    """Cartera de mínima varianza: w   Σ ¹ 1."""
    unos = np.ones(Sigma.shape[0])
    w = np.linalg.solve(Sigma, unos)
    return w / w.sum()

# Misma "realidad", dos muestras distintas
```

```python
for seed in [1, 2]:
    r = simular_retornos(500, seed)
    w = pesos_min_varianza(np.cov(r.T))
    print(f"Muestra {seed}: pesos = {np.round(w, 3)}")
```

```
Muestra 1: pesos = [-0.012  0.154  0.159  0.266  0.433]
Muestra 2: pesos = [0.081 0.02  0.215 0.294 0.39 ]
```

Mirá los dos primeros activos en la Figura 9.1: entre una muestra y otra **intercambian su peso** —el que tenía mucho pasa a tener casi nada, y viceversa, con alguna posición corta de por medio—, mientras los otros tres activos apenas se mueven. El optimizador está apostando a diferencias entre los activos 0 y 1 que son puro ruido muestral: como son casi sustitutos, cualquier mínima ventaja estimada en uno se traduce en sacarle peso al otro. Con carteras reales de 50 o 100 activos y gemelos más correlacionados, el efecto se agrava hasta producir cortos grandes que se compensan entre sí.

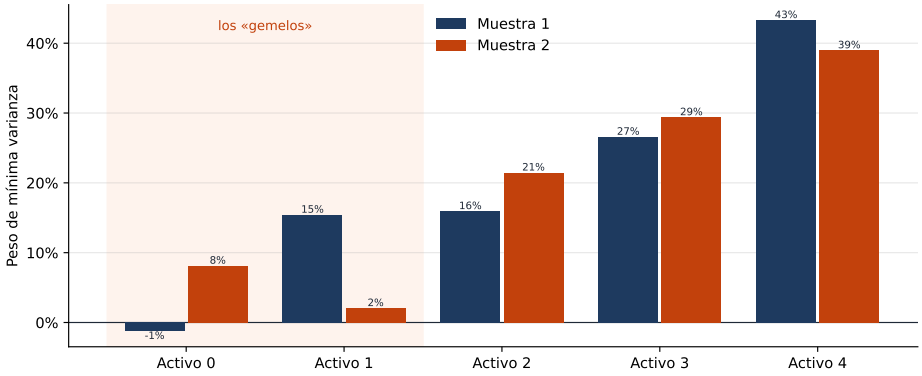


Figura 9.1.: Pesos de mínima varianza estimados con dos muestras de la misma realidad. Los activos 0 y 1 (casi gemelos) intercambian su peso; el resto queda estable.

El diagnóstico de López de Prado

Marcos López de Prado (2016) resumió el problema así: la optimización clásica trata a todos los activos como sustitutos potenciales de todos, cuando en realidad los mercados tienen **estructura jerárquica**: las acciones bancarias se parecen entre sí, los bonos entre sí, y recién después compiten los grupos. Ignorar esa jerarquía obliga a invertir una matriz enorme y frágil. Usarla permite repartir el capital **sin invertir nada**.

La propuesta, **Hierarchical Risk Parity (HRP)**, tiene tres pasos: (1) agrupar activos por similitud, (2) reordenarlos según esos grupos, y (3) repartir el capital descendiendo por el árbol de grupos. Vamos paso a paso.

La Figura 9.2 muestra la tubería completa antes de entrar en cada paso.

9.1. Paso 1: Tree clustering — agrupar activos por similitud

Primero necesitamos medir qué tan "parecidos" son dos activos. Partimos de la correlación ρ_{ij} y la convertimos en una **distancia**:

$$d_{ij} = \sqrt{\frac{1 - \rho_{ij}}{2}}$$

La fórmula tiene sentido: dos activos perfectamente correlacionados ($\rho = 1$) están a distancia 0; dos perfectamente opuestos ($\rho = -1$) están a distancia 1. Con esa matriz de distancias aplicamos **clustering jerárquico**: un

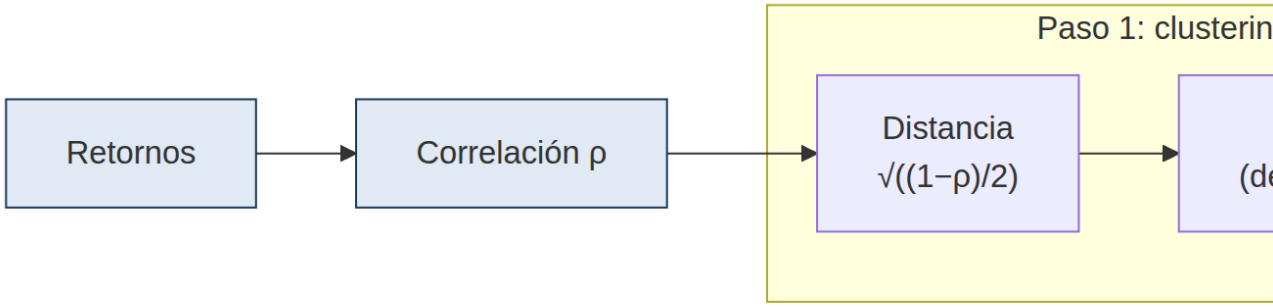


Figura 9.2.: La tubería de HRP. Los dos primeros pasos solo usan correlaciones para descubrir y ordenar la estructura; el tercero reparte el capital usando varianzas de grupos, sin invertir nunca la matriz completa.

algoritmo que va uniendo los pares más cercanos hasta armar un árbol (el *dendrograma*).

```
# Trabajamos con una muestra más larga para estimar mejor
retornos = pd.DataFrame(
    simular_retornos(1000, seed=7),
    columns=["Banco A", "Banco B", "Energía", "Consumo", "Telecom"],
)

corr = retornos.corr()
print("Matriz de correlaciones:\n", corr.round(2))

# Correlación → distancia
dist = np.sqrt((1 - corr) / 2)

# Clustering jerárquico (single linkage, como en el paper original)
enlaces = hierarchy.linkage(squareform(dist.values), method="single")
```

```
Matriz de correlaciones:
          Banco A  Banco B  Energía  Consumo  Telecom
Banco A     1.00     0.92    -0.00     0.03     0.05
Banco B     0.92     1.00     0.00     0.03     0.05
Energía    -0.00     0.00     1.00    -0.02    -0.02
Consumo     0.03     0.03    -0.02     1.00    -0.03
Telecom     0.05     0.05    -0.02    -0.03     1.00
```

```
fig, ax = plt.subplots(figsize=(8, 4))
hierarchy.dendrogram(enlaces, labels=corr.columns.tolist(), ax=ax)
ax.set_title("Dendrograma: la estructura jerárquica del universo de activos")
ax.set_ylabel("Distancia")
plt.tight_layout()
plt.show()
```

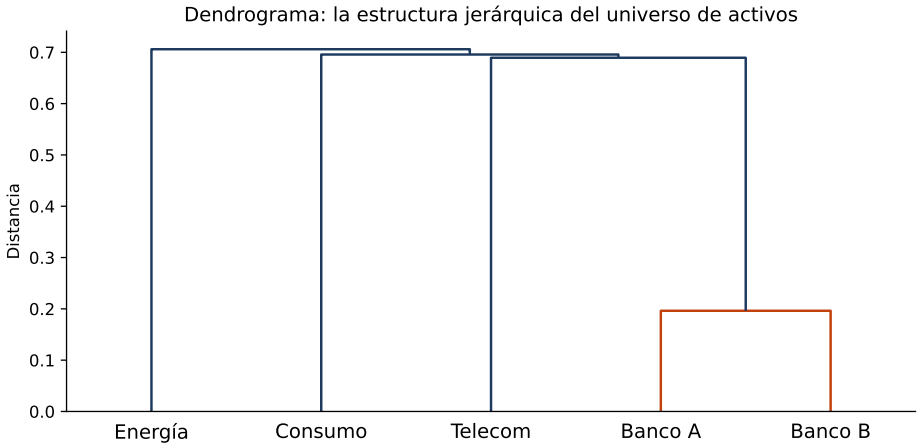


Figura 9.3.: Dendrograma de los cinco activos simulados. La altura de cada unión es la distancia a la que se juntan los grupos: los dos bancos se unen primero, muy abajo.

El dendrograma de la Figura 9.3 cuenta la historia que el optimizador de Markowitz nunca ve: los dos bancos forman un grupo compacto (distancia chica), y recién después se conectan con el resto. HRP va a usar exactamente esa estructura.

9.2. Paso 2: Quasi-diagonalización — reordenar la matriz

El segundo paso es puramente organizativo: **reordenar los activos según el orden de las hojas del árbol**, de modo que los activos similares queden contiguos. Con ese orden, la matriz de covarianzas queda "casi

diagonal por bloques": las covarianzas grandes se concentran cerca de la diagonal.

```
# El orden de las hojas del dendrograma
orden = hierarchy.leaves_list(enlaces)
tickers_ordenados = corr.columns[orden].tolist()
print("Orden jerárquico:", tickers_ordenados)

# Reordenamos la matriz de correlaciones con ese orden
corr_ordenada = corr.iloc[orden, orden]
```

```
Orden jerárquico: ['Energía', 'Consumo', 'Telecom', 'Banco A', 'Banco B']
```

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, m, titulo in [(axes[0], corr, "Original"),
                      (axes[1], corr_ordenada, "Quasi-diagonalizada")]:
    im = ax.imshow(m, cmap="coolwarm", vmin=-1, vmax=1)
    ax.set_xticks(range(len(m)), m.columns, rotation=45, ha="right")
    ax.set_yticks(range(len(m)), m.columns)
    ax.set_title(titulo)
fig.colorbar(im, ax=axes, shrink=0.8, label="Correlación")
plt.show()
```

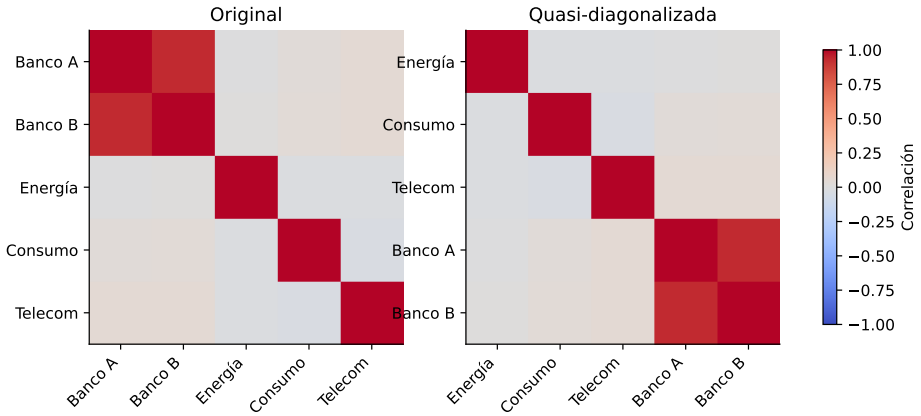


Figura 9.4.: La matriz de correlaciones antes y después de reordenarla según las hojas del dendrograma. Reordenada, los activos parecidos quedan contiguos y las correlaciones altas se agrupan cerca de la diagonal.

En este ejemplo chico el reordenamiento de la Figura 9.4 es sutil, pero con decenas de activos el efecto es dramático: los bloques de color (sectores, regiones) se vuelven visibles a simple vista. La matriz reordenada es el "mapa" sobre el que repartiremos el capital.

9.3. Paso 3: Recursive bisection — repartir el capital por el árbol

Acá está el corazón de HRP. En lugar de resolver un sistema global (que exigiría invertir Σ), repartimos el capital **de arriba hacia abajo**:

1. Arrancamos con el 100 % del capital y la lista completa (ya ordenada) de activos.
2. Partimos la lista en dos mitades y calculamos el **riesgo de cada mitad**.
3. Le damos más capital a la mitad **menos riesgosa**, en proporción inversa a su varianza:

$$\alpha_1 = 1 - \frac{V_1}{V_1 + V_2}, \qquad \alpha_2 = 1 - \alpha_1$$

4. Repetimos recursivamente dentro de cada mitad, hasta llegar a activos individuales.

Un detalle de implementación: esta versión parte la lista ordenada **por la mitad**, como el paper original de 2016, y no siguiendo exactamente las ramas del dendrograma. Gracias al reordenamiento del Paso 2, las mitades igual tienden a respetar los grupos; existen variantes que cortan siguiendo los clusters del árbol.

Para medir el riesgo de un grupo usamos la cartera *inverse-variance* dentro del grupo (cada activo pesa lo inverso de su varianza), que tiene forma cerrada y **solo usa la diagonal** —nada de inversas de matrices completas.

```
def pesos_inverse_variance(cov):
    """Pesos proporcionales a 1/varianza (solo usa la diagonal)."""
    iv = 1 / np.diag(cov.values)
    return iv / iv.sum()

def varianza_cluster(cov, items):
    """Varianza del sub-portafolio inverse-variance de un grupo de activos."""
    sub = cov.loc[items, items]
    w = pesos_inverse_variance(sub)
    return float(w @ sub.values @ w)
```

```python
def hrp(cov, orden_tickers):
    """Recursive bisection sobre la lista ordenada de activos."""
    pesos = pd.Series(1.0, index=orden_tickers)
    grupos = [orden_tickers]                      # arrancamos con un solo grupo: to

    while grupos:
        # Partimos cada grupo vigente en dos mitades
        nuevos = []
        for g in grupos:
            if len(g) <= 1:
                continue
            mitad = len(g) // 2
            izq, der = g[:mitad], g[mitad:]

            # Riesgo de cada mitad y factor de asignación
            v_izq = varianza_cluster(cov, izq)
            v_der = varianza_cluster(cov, der)
            alpha = 1 - v_izq / (v_izq + v_der)

            pesos[izq] *= alpha        # la mitad menos riesgosa recibe más capital
            pesos[der] *= (1 - alpha)

            nuevos += [izq, der]
        grupos = nuevos

    return pesos

cov = retornos.cov()
w_hrp = hrp(cov, tickers_ordenados)

print("Pesos HRP:")
print((w_hrp * 100).round(1).astype(str) + "%")
print(f"\nSuma: {w_hrp.sum():.1%}")
```

```
Pesos HRP:
Energía    20.8 %
Consumo    29.0 %
Telecom    38.4 %
Banco A     5.9 %
Banco B     5.9 %
dtype: object

Suma: 100.0%
```

La Figura 9.5 muestra el recorrido completo de la bisección para este universo: cada flecha lleva el factor que recibe esa mitad, y cada caja, el capital acumulado.

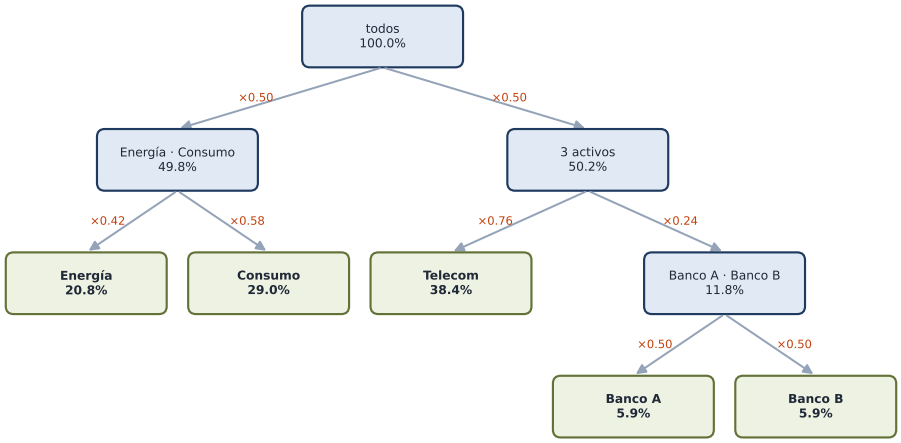


Figura 9.5.: Bisección recursiva sobre los 5 activos del ejemplo. En cada nodo el capital se reparte entre las dos mitades en proporción inversa a su varianza; el peso final de cada activo es el producto de los factores del camino.

Observá el resultado: los dos bancos —que comparten casi todo su riesgo— **se reparten entre sí la porción del capital que le toca a su grupo**, en lugar de recibir cada uno una apuesta independiente. Los activos diversificadores reciben pesos sustanciales y **no aparece ninguna posición corta ni extrema**. Y en todo el proceso no invertimos una sola matriz.

> **Por qué esto es 'risk parity' jerárquico**
>
> En cada bifurcación, el capital se reparte de manera inversamente proporcional al riesgo: las dos ramas quedan aportando riesgos comparables. Es la idea de *risk parity* (paridad de riesgo), aplicada recursivamente sobre la jerarquía del árbol en lugar de activo por activo.

9.4. HRP vs. Markowitz: la prueba fuera de muestra

La comparación justa no es "qué cartera se ve mejor" sino **cuál funciona con datos que no vio**. Repetimos muchas veces el experimento: estimamos ambas carteras con un período de entrenamiento y medimos su volatilidad en el período siguiente.

```
def experimento(n_sims=200, n_train=250, n_test=250):
    vol_mkw, vol_hrp = [], []

    for sim in range(n_sims):
        r = simular_retornos(n_train + n_test, seed=1000 + sim)
        train, test = r[:n_train], r[n_train:]

        df_train = pd.DataFrame(train, columns=corr.columns)
        cov_train = df_train.cov()
```

```
        # --- Markowitz: mínima varianza (requiere invertir Σ) ---
        w_m = pesos_min_varianza(cov_train.values)

        # --- HRP: clustering + bisección (sin inversas) ---
        d = np.sqrt((1 - df_train.corr()) / 2)
        lk = hierarchy.linkage(squareform(d.values), method="single")
        orden_t = df_train.columns[hierarchy.leaves_list(lk)].tolist()
        w_h = hrp(cov_train, orden_t).reindex(df_train.columns).values

        # Volatilidad realizada FUERA de muestra (anualizada)
        vol_mkw.append(np.std(test @ w_m) * np.sqrt(252))
        vol_hrp.append(np.std(test @ w_h) * np.sqrt(252))

    return np.array(vol_mkw), np.array(vol_hrp)

vol_mkw, vol_hrp = experimento()

print("Volatilidad anualizada fuera de muestra (promedio de 200 simulaciones):")
print(f"  Markowitz (mín. varianza): {vol_mkw.mean():.2%}  (desvío entre sims: {vol_
print(f"  HRP:                       {vol_hrp.mean():.2%}  (desvío entre sims: {vol_
```

```
Volatilidad anualizada fuera de muestra (promedio de 200 simulaciones):
  Markowitz (mín. varianza): 8.31%  (desvío entre sims: 0.38%)
  HRP:                       8.39%  (desvío entre sims: 0.42%)
```

```
fig, ax = plt.subplots(figsize=(9, 4))
ax.hist(vol_mkw * 100, bins=30, alpha=0.6, label="Markowitz (mín. varianza)")
ax.hist(vol_hrp * 100, bins=30, alpha=0.6, label="HRP")
ax.set_xlabel("Volatilidad anualizada fuera de muestra (%)")
ax.set_ylabel("Frecuencia")
ax.set_title("Riesgo realizado fuera de muestra: 200 simulaciones")
ax.legend()
```

```
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

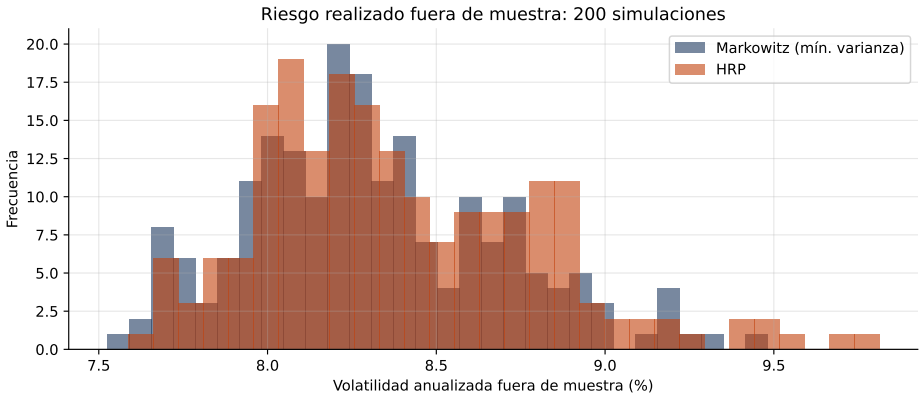


Figura 9.6.: Distribución de la volatilidad realizada fuera de muestra en 200 simulaciones, para Markowitz de mínima varianza y para HRP.

La Figura 9.6 tiene un resultado que sorprende a primera vista: las dos distribuciones casi se superponen, y Markowitz hasta queda unas décimas abajo (8.31 % contra 8.39 % en promedio). Dentro de la muestra Markowitz siempre "gana" —por definición minimiza la varianza *de esos datos*—, y en este universo chico también aguanta fuera de muestra: con 5 activos y 250 días, Σ se estima bastante bien, y entre los gemelos el error casi no cuesta riesgo, porque pasar peso de un sustituto al otro deja la cartera casi igual. La ventaja de HRP que reporta el paper original aparece cuando la estimación es pobre: muchos activos, pocas observaciones, y una inversa de Σ que amplifica mucho más el ruido. Lo que sí se ve ya acá es la diferencia en los **pesos** de la Figura 9.1: Markowitz llega a un riesgo parecido con carteras que cambian de una muestra a otra, y en la práctica esa inestabilidad se

paga en rotación, costos de transacción y posiciones cortas.

> **HRP no es magia**
>
> HRP renuncia a usar información que Markowitz sí usa (las correlaciones exactas entre grupos lejanos y los retornos esperados). Si tus estimaciones de μ y Σ fueran perfectas, Markowitz sería imbatible. HRP gana justamente porque en el mundo real **nunca lo son**. La elección depende de cuánto confiás en tus estimaciones — un tema que retomamos en el capítulo siguiente con Black-Litterman, que ataca el mismo problema desde otro ángulo: mejorar las estimaciones en lugar de esquivarlas.

> **Simulador interactivo**
>
> En la versión web de este capítulo hay un simulador que repite el experimento con cuarenta muestras, donde podés mover la correlación de los gemelos y el largo de la muestra y comparar la estabilidad y el riesgo fuera de muestra de los dos métodos.

9.5. Con datos reales

La simulación nos dejó controlar la estructura de correlaciones; ahora veamos si HRP la **descubre solo** en datos de mercado. Usamos el dataset del libro (`datos/precios.csv`, ver Apéndice B): 9 acciones de sectores distintos, cinco años de precios diarios ajustados.

```
precios = pd.read_csv("../datos/precios.csv", index_col="Date", parse_dates=True)
r_real = precios.drop(columns="SPY").pct_change().dropna()   # SPY es un índice, no
```

```
# El pipeline completo de HRP, tal cual lo construimos
dist_real = np.sqrt((1 - r_real.corr()) / 2)
enlaces_real = hierarchy.linkage(squareform(dist_real.values), method="single")

fig, ax = plt.subplots(figsize=(9, 4))
hierarchy.dendrogram(enlaces_real, labels=r_real.columns.tolist(), ax=ax)
ax.set_title("Dendrograma de 9 acciones reales (2021-2026)")
ax.set_ylabel("Distancia")
plt.tight_layout()
plt.show()
```

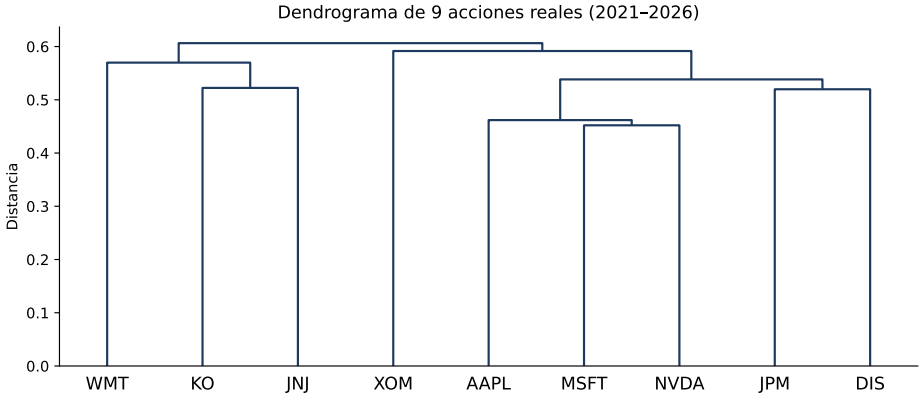


Figura 9.7.: Dendrograma de las 9 acciones del dataset del libro (retornos diarios 2021–2026), construido solo a partir de sus correlaciones.

Mirá la Figura 9.7 antes de seguir: **nadie le dijo al algoritmo qué sector es cada empresa**, y sin embargo las tecnológicas (AAPL, MSFT, NVDA) se abrazan en un cluster compacto, y las defensivas de consumo (KO, JNJ, WMT) forman el suyo. La estructura jerárquica que López de Prado postula no es una hipótesis: está en los datos.

```python
orden_real = r_real.columns[hierarchy.leaves_list(enlaces_real)].tolist()
w_hrp_real = hrp(r_real.cov(), orden_real)
w_mkw_real = pd.Series(pesos_min_varianza(r_real.cov().values), index=r_real.columns

comparacion = pd.DataFrame({"HRP": w_hrp_real, "Markowitz mín. var.": w_mkw_real})
print((comparacion * 100).round(1).astype(str) + " %")
```

```
         HRP Markowitz mín. var.
AAPL   6.3 %              -3.5 %
DIS    4.9 %               1.6 %
JNJ   25.5 %              30.4 %
JPM    7.1 %               6.0 %
KO    23.6 %              27.9 %
MSFT   6.6 %              12.6 %
NVDA   2.3 %               1.5 %
WMT   13.2 %              12.1 %
XOM   10.5 %              11.6 %
```

La Figura 9.8 pone las dos columnas lado a lado, en el orden del dendrograma:

La comparación repite lo que vimos en la simulación, ahora sin trucos. Buscá en la columna de Markowitz las tecnológicas: **shortea una para comprar otra** — la apuesta fina entre "gemelos" que en el experimento sintético identificamos como puro ruido muestral, reproducida con datos de verdad. Con 9 activos el daño es contenido; con 50 o 100, esos cortos se multiplican. HRP, en cambio, asigna a **todos** los activos pesos positivos y moderados, dándole más capital al cluster defensivo y repartiendo dentro de cada grupo. Es la cartera que podrías ejecutar mañana sin explicarle nada raro a un comité de inversiones.

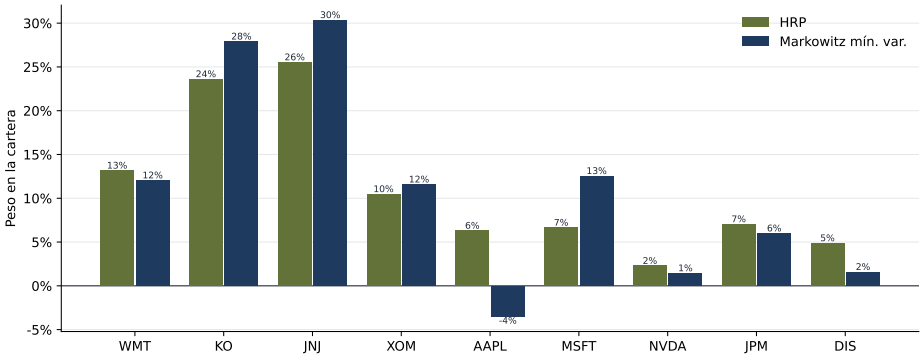


Figura 9.8.: Pesos HRP frente a mínima varianza de Markowitz con datos reales, en el orden del dendrograma. Las barras bajo cero son posiciones cortas.

9.6. Errores comunes

Trampas frecuentes

- **Pasarle a `linkage` la matriz cuadrada directamente**: `scipy` espera la forma *condensada* (vector triangular); usá `squareform(dist)` o interpretará las filas como observaciones.
- **Olvidar reordenar antes de la bisección**: la partición en mitades solo tiene sentido sobre la lista quasi-diagonalizada. Sobre el orden original, las mitades mezclan grupos y HRP pierde su lógica.
- **Usar la correlación como distancia sin transformar**: la correlación no es una métrica (activos idénticos tendrían "distancia" 1). La transformación $\sqrt{(1-\rho)/2}$ es la que convierte similitud en distancia válida.
- **Comparar métodos dentro de la muestra**: cualquier optimizador le gana a HRP con los datos con los que fue ajustado.

> La comparación honesta es siempre fuera de muestra.

9.7. Practicá en el navegador

Podés experimentar con el código del capítulo acá mismo: la celda ejecuta Python **en tu navegador**, sin instalar nada (la primera ejecución tarda unos segundos mientras carga el entorno; los paquetes que importes se instalan solos).

```
#| label: practica-navegador
import numpy as np
import pandas as pd
from scipy.cluster import hierarchy
from scipy.spatial.distance import squareform

# Para usar el dataset real del libro (requiere conexión):
# import pandas as pd
# from pyodide.http import open_url
# url = "https://diegoonna.github.io/libro_finanzas_computacionales/datos/precios.cs
# precios = pd.read_csv(open_url(url), index_col="Date", parse_dates=True)

# Tu código acá...
```

9.8. Referencias

10. Capítulo 10: El Modelo de Black-Litterman

Del equilibrio de mercado a las views del inversor (enfoque algebraico)

Markowitz tiene un problema práctico: es hipersensible a los retornos esperados que le pasamos, y esos retornos son justamente lo más difícil de estimar. Black y Litterman lo resolvieron con una idea elegante: partir del retorno que el propio mercado está implícitamente asumiendo (el equilibrio) y ajustarlo solo donde el inversor tiene una opinión fundada. En este capítulo lo derivamos por el método clásico y algebraico, paso a paso.

Introducción: por qué Markowitz solo no alcanza

En el Capítulo 8 aprendimos a construir la frontera eficiente. Pero si intentás usar Markowitz "puro" con datos reales, te chocás con un problema incómodo: **el modelo es extremadamente sensible a los retornos esperados** μ que le das de entrada. Cambiás la estimación del retorno de un activo en medio punto y la cartera óptima se da vuelta por completo, concentrándose en dos o tres posiciones extremas (a veces con ventas en corto gigantes).

El problema de fondo es que μ es lo más difícil de estimar en finanzas: el promedio histórico es un estimador malísimo del retorno futuro. Entonces alimentamos el modelo más sensible con el input más ruidoso. El resultado son carteras que ningún gestor firmaría.

Black-Litterman (1992), nacido en Goldman Sachs, ataca el problema con un cambio de perspectiva:

> En lugar de *inventar* los retornos esperados desde cero, arranquemos del retorno que el mercado ya está asumiendo hoy, y ajustémoslo solo donde tengamos una opinión concreta y en la medida de nuestra confianza en ella.

La idea en una frase

Markowitz te pregunta *"¿cuánto va a rendir cada activo?"* y se cree tu respuesta al pie de la letra. Black-Litterman te pregunta *"¿en qué*

> *diferís del mercado, y cuán seguro estás?"* — una pregunta mucho más honesta y fácil de responder.

La Figura 10.5, al final del capítulo, resume el recorrido completo en un solo diagrama; volvé a ella si en algún paso perdés el hilo.

```
import numpy as np
import matplotlib.pyplot as plt
np.set_printoptions(precision=4, suppress=True)
```

10.1. La intuición: combinar una creencia previa con nueva información

Antes de la matemática, la intuición, porque es el corazón del modelo.

Imaginate que querés estimar cuánto va a rendir una acción. Tenés dos fuentes de información:

1. Una **creencia previa**: "el mercado, con todo su peso, está descontando un 8 % para este activo". Es una referencia sensata y difícil de discutir.
2. Una **opinión propia** (nueva información): "yo estudié el sector y creo que va a rendir 12 %".

¿Con qué te quedás? Ni una ni otra a ciegas: hacés un **promedio ponderado por tu confianza**. Si tu análisis es muy sólido, tu estimación final se acerca al 12 %; si tenés dudas, se queda cerca del 8 % del mercado. Y si tu opinión coincide con el mercado, no cambia nada.

La Figura 10.1 dibuja ese razonamiento con números: cada fuente es una campana centrada en su estimación, más angosta cuanto más confiable.

La combinación cae entre ambas, más cerca de la más precisa, y es más angosta que las dos: juntar información reduce la incertidumbre.

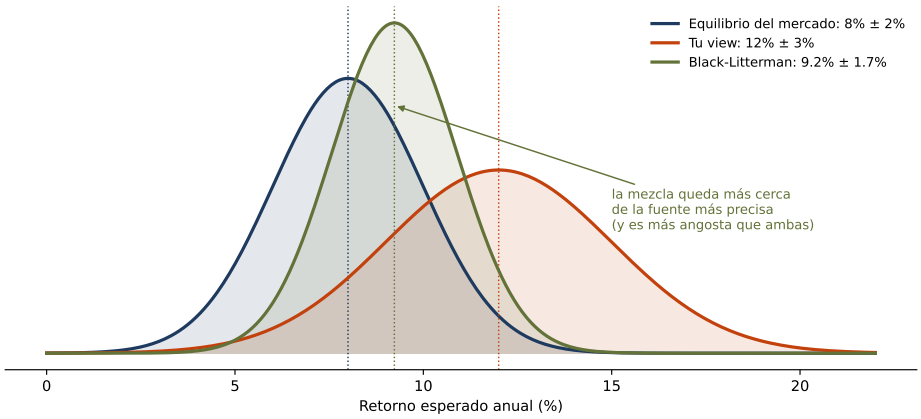


Figura 10.1.: Combinar el equilibrio (8 % ± 2 %) con una view (12 % ± 3 %): el resultado es un promedio ponderado por precisión, 9.2 % ± 1.7 %, más cerca del equilibrio porque es la fuente más precisa.

Ese mecanismo —partir de una creencia y moverla hacia la nueva evidencia en proporción a la confianza— es exactamente lo que formaliza Black-Litterman. En la literatura vas a ver que la versión completa se deriva como una actualización bayesiana (*prior* + verosimilitud → *posterior*), pero **no hace falta la maquinaria bayesiana** para entenderlo ni usarlo: se puede escribir como un conjunto de operaciones matriciales cerradas. Ese es el camino que vamos a tomar.

10.2. Paso 1: los retornos implícitos de equilibrio (optimización inversa)

La primera pieza es la creencia de partida: ¿cuáles son los retornos que el mercado está asumiendo? Acá viene el truco más lindo del modelo.

En Markowitz, dada una matriz de covarianzas Σ y unos retornos esperados μ, buscamos los pesos óptimos $\mathbf{w}$. Black-Litterman **invierte esa lógica**: si suponemos que la cartera de mercado (la capitalización de cada activo) ya es óptima —porque es el consenso agregado de todos los inversores—, entonces podemos despejar qué retornos Π la harían óptima. Es *optimización inversa*.

El resultado es la fórmula del retorno implícito de equilibrio:

$$\Pi = \delta \, \Sigma \, \mathbf{w}_{mkt}$$

donde:

- $\mathbf{w}_{mkt}$ son los pesos de la cartera de mercado (por capitalización),
- Σ es la matriz de covarianzas de los retornos,
- δ es el **coeficiente de aversión al riesgo** del mercado (típicamente entre 2 y 3).

```
# Universo de ejemplo: 3 activos (digamos, tres clases de activos)
activos = ["Acciones USA", "Acciones EU", "Bonos"]

# Pesos de mercado (por capitalización) - suman 1
w_mkt = np.array([0.55, 0.30, 0.15])

# Matriz de covarianzas anual de los retornos
Sigma = np.array([
    [0.0225, 0.0075, 0.0010],
```

```
    [0.0075, 0.0196, 0.0012],
    [0.0010, 0.0012, 0.0036],
])

# Aversión al riesgo del mercado
delta = 2.5

# Retornos implícitos de equilibrio: Π =   Σ w_mkt
Pi = delta * Sigma @ w_mkt

for a, pi in zip(activos, Pi):
    print(f"{a:<14} retorno implícito: {pi:6.2%}")
```

```
Acciones USA   retorno implícito:  3.69%
Acciones EU    retorno implícito:  2.55%
Bonos          retorno implícito:  0.36%
```

Fijate lo que acabamos de lograr: **sin estimar un solo retorno esperado a mano**, obtuvimos un vector Π razonable, coherente con el riesgo de cada activo y con lo que el mercado efectivamente tiene comprado. Los activos más riesgosos y con más peso en el mercado tienen mayor retorno implícito. Este Π es nuestra creencia de partida.

> **i** ¿De dónde sale ?
>
> El coeficiente de aversión al riesgo se puede estimar como $\delta = (\mu_{mkt} - r_f)/\sigma^2_{mkt}$: el premio por riesgo del mercado dividido por su varianza. Con un premio del ~5 % y una volatilidad del ~14 % anual, da $\delta \approx 2.5$, el valor que usamos.

10.3. Paso 2: expresar las views del inversor

Ahora incorporamos nuestras opiniones. Black-Litterman las representa con tres objetos:

- P (matriz de *picks*): qué activos involucra cada view. Cada fila es una view.
- Q (vector): el retorno que afirma cada view.
- Ω (matriz): la **incertidumbre** de cada view (cuánto desconfiamos de ella). Diagonal, un valor por view.

Las views pueden ser de dos tipos:

- **Absolutas**: "Acciones EU van a rendir 12 %". La fila de P tiene un `1` en ese activo y `0` en el resto.
- **Relativas**: "Acciones USA van a superar a Bonos por 4 %". La fila tiene un `+1` en USA y un `-1` en Bonos.

```
# View 1 (absoluta): "Acciones EU rendirán 12%"
# View 2 (relativa): "Acciones USA superarán a Bonos por 4%"
P = np.array([
    [0.0, 1.0,  0.0],   # EU
    [1.0, 0.0, -1.0],   # USA - Bonos
])

Q = np.array([0.12, 0.04])   # retornos afirmados por cada view

print("Matriz P (picks):\n", P)
print("\nVector Q (views):", Q)
```

```
Matriz P (picks):
 [[ 0.  1.  0.]
 [ 1.  0. -1.]]
```

```
Vector Q (views): [0.12 0.04]
```

Para la incertidumbre Ω, una convención estándar (He y Litterman, 1999) es hacerla proporcional a la varianza de la propia view bajo el modelo: views sobre activos más volátiles arrastran más incertidumbre. Aparece también un parámetro escalar τ (chico, ~0.025–0.05) que refleja la confianza global en el equilibrio. Una alternativa popular es la de Idzorek (2007): en lugar de fijar Ω directamente, le pide al inversor un **porcentaje de confianza** (de 0 a 100 %) para cada view y deduce Ω a partir de él.

```python
tau = 0.05   # escala de la incertidumbre del equilibrio

# Ω diagonal, proporcional a la incertidumbre de cada view: diag(P (τΣ) P')
Omega = np.diag(np.diag(P @ (tau * Sigma) @ P.T))

print("Matriz Ω (incertidumbre de las views):\n", Omega)
```

```
Matriz Ω (incertidumbre de las views):
 [[0.001  0.    ]
 [0.     0.0012]]
```

La Figura 10.2 junta las tres piezas: cada fila es una view, y se lee de izquierda a derecha.

> **El sentido de Ω**
>
> Ω es la perilla de confianza. Si ponés valores **chicos**, le estás diciendo al modelo "estoy muy seguro de estas views" y el resultado se pegará a Q. Si los ponés **grandes**, el modelo casi las ignora y se queda con el equilibrio Π. Es la traducción matemática del "cuán seguro estás" de la introducción.

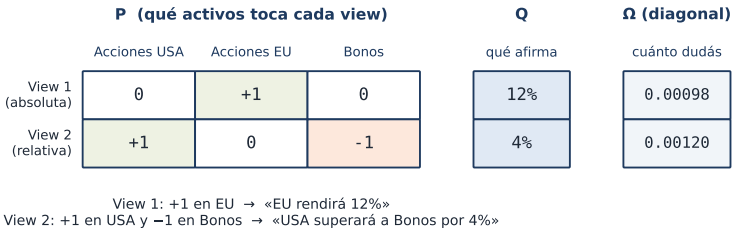


Figura 10.2.: Anatomía de las views del capítulo. Cada fila de P indica qué activos involucra la view (+1 compra, −1 vende, 0 no participa); Q dice qué retorno afirma, y Ω, cuánta incertidumbre le asignamos.

10.4. Paso 3: la fórmula maestra de Black-Litterman

Con el equilibrio Π y las views (P, Q, Ω), combinamos ambas fuentes en un único vector de retornos esperados ajustados, μ_{BL}. Esta es la célebre *fórmula maestra*:

$$\mu_{BL} = \left[(\tau\Sigma)^{-1} + P^T\Omega^{-1}P\right]^{-1}\left[(\tau\Sigma)^{-1}\Pi + P^T\Omega^{-1}Q\right]$$

Parece intimidante, pero es exactamente el **promedio ponderado por confianza** de la intuición, escrito en matrices. Cada término $(\cdot)^{-1}$ es una "precisión" (lo inverso de la incertidumbre): el equilibrio pesa según su precisión $(\tau\Sigma)^{-1}$ y las views según la suya $P^T\Omega^{-1}P$. Donde no hay view, prevalece el equilibrio; donde hay una view muy confiable, ella domina.

Para verlo, conviene mirar el caso de **un solo activo con una sola view**: la fórmula se reduce a

$$\mu_{BL} = \frac{\pi/\sigma_\pi^2 + q/\sigma_q^2}{1/\sigma_\pi^2 + 1/\sigma_q^2}$$

que es literalmente un promedio de π (equilibrio) y q (view) ponderado por las precisiones. La versión matricial hace lo mismo con todos los activos y views a la vez, respetando las correlaciones.

```
inv_tau_Sigma = np.linalg.inv(tau * Sigma)
inv_Omega = np.linalg.inv(Omega)

# Fórmula maestra
A = inv_tau_Sigma + P.T @ inv_Omega @ P
b = inv_tau_Sigma @ Pi + P.T @ inv_Omega @ Q
mu_BL = np.linalg.solve(A, b)   # resolvemos A   = b (más estable que invertir A)

print(f"{'Activo':<14}{'Equilibrio Π':>14}{'Black-Litterman':>18}")
for a, pi, m in zip(activos, Pi, mu_BL):
    print(f"{a:<14}{pi:>13.2%}{m:>17.2%}")
```

```
Activo          Equilibrio Π   Black-Litterman
Acciones USA          3.69%            5.14%
Acciones EU           2.55%            7.22%
Bonos                 0.36%            0.70%
```

Leé la tabla con calma: los retornos ajustados se **movieron desde el equilibrio hacia nuestras views**. Acciones EU subió del 2.5 % al 7.2 %: bastante, pero lejos del 12 % que afirmamos, porque el modelo pondera esa view contra el equilibrio. USA también subió, empujado por la view relativa. Ningún valor saltó hasta la view: el modelo mezcló, no reemplazó. Eso es precisamente lo que le faltaba a Markowitz.

Fijate también un efecto menos obvio: Bonos **subió** (de 0.4 % a 0.7 %), aunque la view 2 lo ponía *por debajo* de USA. La razón es que la fórmula propaga la información a través de Σ: si EU y USA rinden más, y Bonos está (poco, pero) correlacionado con ellos, su retorno esperado también se ajusta. Las views no actúan aisladas: viajan por las covarianzas.

10.5. Paso 4: del retorno ajustado a la cartera óptima

El vector μ_{BL} es un input de retornos esperados **mucho más robusto** que un promedio histórico. Ahora sí lo metemos en la optimización de Markowitz para obtener los pesos. Sin restricciones adicionales, los pesos óptimos tienen forma cerrada:

$$\mathbf{w}_{BL} = \frac{1}{\delta}\,\Sigma^{-1}\,\mu_{BL}$$

```
# Pesos óptimos con los retornos de Black-Litterman
w_BL = (1 / delta) * np.linalg.solve(Sigma, mu_BL)

print(f"{'Activo':<14}{'Cartera mercado':>16}{'BL con views':>15}")
for a, wm, wb in zip(activos, w_mkt, w_BL):
    print(f"{a:<14}{wm:>15.1%}{wb:>14.1%}")

print(f"\nSuma de pesos BL: {w_BL.sum():.1%}")
```

```
Activo         Cartera mercado   BL con views
Acciones USA             55.0%         47.8%
Acciones EU              30.0%        127.6%
Bonos                    15.0%         22.2%

Suma de pesos BL: 197.6%
```

Cuando **no hay views**, $\mathbf{w}_{BL}$ reproduce la cartera de mercado (por construcción: es el punto de partida). Al agregar opiniones, el modelo **se desvía**

del mercado en la dirección de esas views, y la magnitud del desvío depende de dos cosas: cuánto difiere cada view del equilibrio y cuánto confiás en ella. Acá la view sobre EU es muy agresiva —12 % contra un equilibrio de 2.5 %—, y la cartera sin restricciones lo refleja: Acciones EU pasa del 30 % a casi 130 %, y los pesos suman cerca de 200 %, es decir, una cartera apalancada. No es un error del modelo: es lo que implica *creer* esa view con esa confianza. Por eso en la práctica se combinan views más moderadas con las restricciones del recuadro siguiente. La Figura 10.3 muestra los dos desplazamientos juntos.

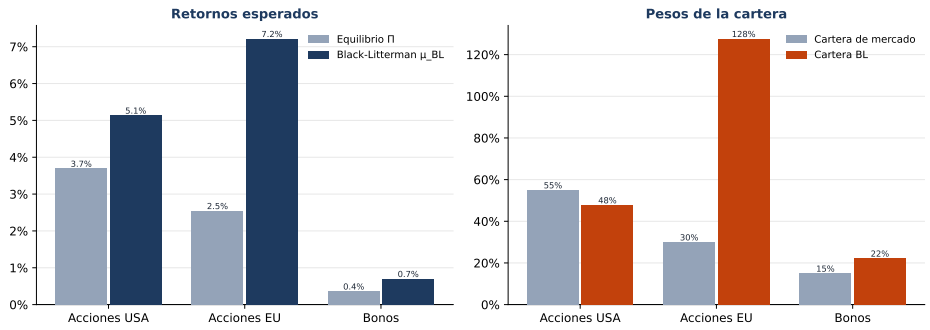


Figura 10.3.: Del equilibrio a Black-Litterman. Izquierda: los retornos esperados se desplazan desde Π hacia las views. Derecha: la cartera sin restricciones sobrepondera fuertemente Acciones EU y suma casi 200 % (apalancada).

> **Detalle práctico**
>
> Los pesos de arriba no suman 1 (acá, casi 2) y pueden incluir cortos. En la práctica se re-normalizan y se agregan restricciones (no negatividad, límites por activo) resolviendo la optimización con `scipy.optimize.minimize`, como en el Capítulo 8, pero usando μ_{BL} como retornos esperados.

10.6. Visualizando el efecto de la confianza

Para cerrar la intuición, veamos cómo la confianza en las views gobierna el resultado. Escalamos Ω con un factor: valores chicos = mucha confianza, grandes = poca.

```
factores = np.logspace(-1.5, 1.5, 40)   # de mucha a poca confianza
trayectorias = []

for f in factores:
    Omega_f = Omega * f
    A_f = inv_tau_Sigma + P.T @ np.linalg.inv(Omega_f) @ P
    b_f = inv_tau_Sigma @ Pi + P.T @ np.linalg.inv(Omega_f) @ Q
    trayectorias.append(np.linalg.solve(A_f, b_f))

trayectorias = np.array(trayectorias)

fig, ax = plt.subplots(figsize=(9, 5))
for i, a in enumerate(activos):
    linea, = ax.plot(factores, trayectorias[:, i] * 100, linewidth=2, label=a)
    ax.axhline(Pi[i] * 100, linestyle=":", alpha=0.6, color=linea.get_color())  # su

ax.set_xscale("log")
ax.set_xlabel("Factor de escala de Ω  (← más confianza en las views | menos confianz
ax.set_ylabel("Retorno esperado  _BL (%)")
ax.set_title("Los retornos BL viajan entre las views y el equilibrio según la confia
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

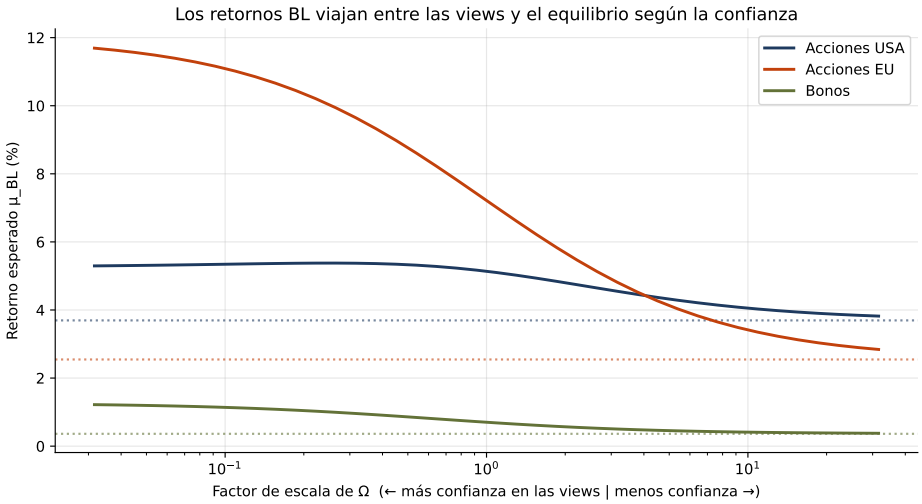


Figura 10.4.: Retornos esperados de Black-Litterman de cada activo según la confianza en las views (factor de escala de Ω, en escala log). Cada línea punteada es el retorno de equilibrio Π del activo del mismo color.

En la Figura 10.4, a la izquierda (mucha confianza) los retornos se acercan a lo que afirman las views; a la derecha (poca confianza) vuelven a las líneas punteadas del equilibrio. **El modelo nunca te deja "irte" más allá de tus propias views ni ignorar del todo al mercado**: interpola entre ambos de manera controlada.

> **Simulador interactivo**
>
> En la versión web de este capítulo hay un simulador donde podés mover cada view y su confianza, y ver cómo cambian los retornos esperados y la cartera de Black-Litterman.

10.7. Recapitulando el flujo

El modelo completo es una tubería de cuatro pasos:

1. **Equilibrio** $\rightarrow \Pi = \delta \Sigma \mathbf{w}_{mkt}$ (la creencia de partida, gratis, desde el mercado).
2. **Views** $\rightarrow$ armar P, Q, Ω (nuestras opiniones y su confianza).
3. **Fórmula maestra** $\rightarrow \mu_{BL}$ (la mezcla ponderada por precisión).
4. **Optimización** $\rightarrow \mathbf{w}_{BL}$ (la cartera final).

La gran ventaja: cada paso tiene una interpretación económica clara, y en ningún momento tuvimos que inventar retornos esperados de la nada. Por eso Black-Litterman es, todavía hoy, uno de los modelos de asignación de activos más usados en la industria.

10.8. Errores comunes

⚠ Trampas frecuentes

- **Confundir τ con Ω**: τ escala la incertidumbre *global* del equilibrio; Ω es la incertidumbre *de cada view*. Son perillas distintas.
- **Views inconsistentes con Σ**: si afirmás que dos activos altamente correlacionados se moverán en direcciones opuestas, el modelo hará contorsiones raras. Las views deberían ser coherentes con la estructura de covarianzas.
- **Olvidar re-normalizar los pesos** o permitir cortos sin querer: siempre revisá que $\mathbf{w}_{BL}$ sume 1 y respete tus restricciones.
- **Invertir matrices con `inv` para resolver sistemas**: usá `np.linalg.solve`, que es más estable numéricamente que calcular la inversa y multiplicar.

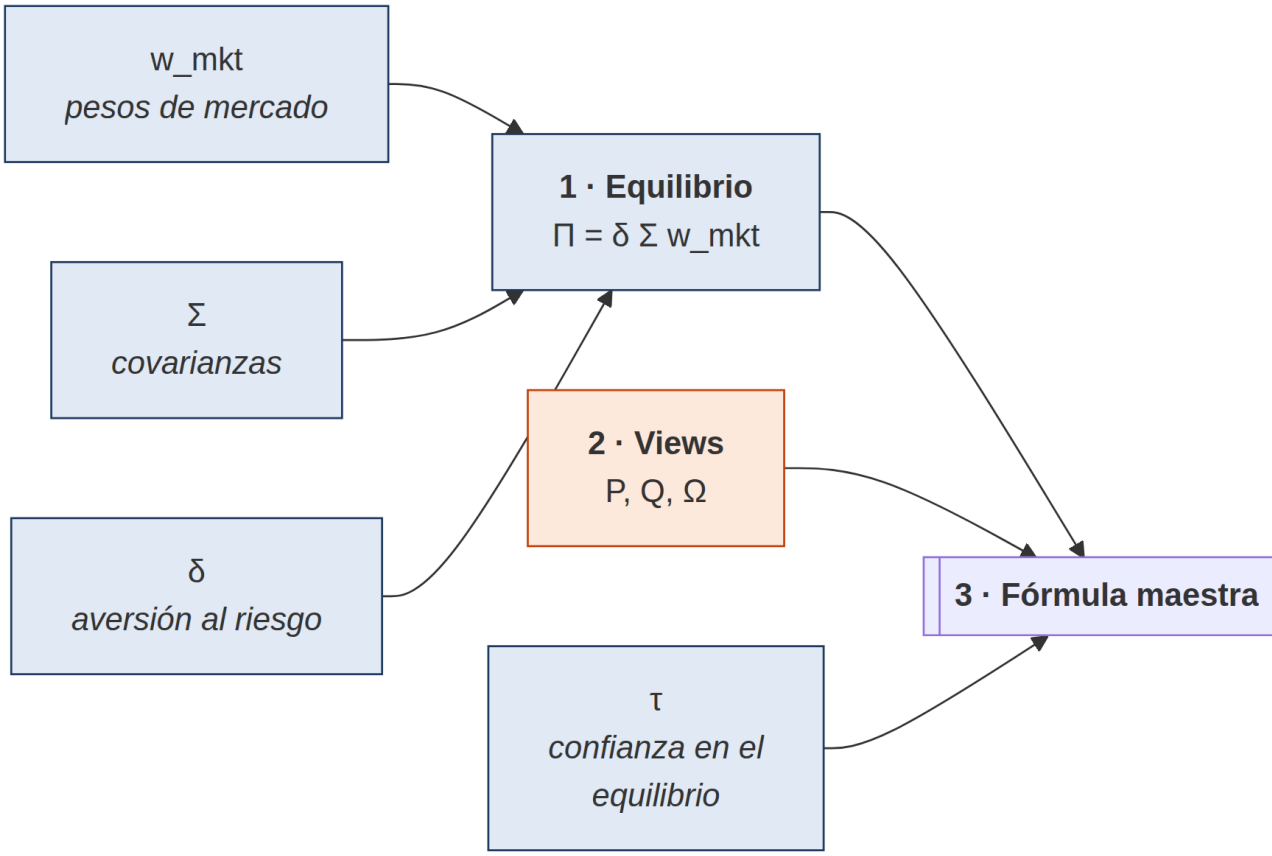


Figura 10.5.: La tubería de Black-Litterman. El equilibrio sale del mercado sin estimar retornos a mano; las views entran con su incertidumbre; la fórmula maestra las mezcla y el resultado alimenta la optimización.

10.9. Practicá en el navegador

Podés experimentar con el código del capítulo acá mismo: la celda ejecuta Python **en tu navegador**, sin instalar nada (la primera ejecución tarda unos segundos mientras carga el entorno; los paquetes que importes se instalan solos).

```
#| label: practica-navegador
import numpy as np

# Tu código acá...
```

10.10. Referencias

Parte III.

Módulo 3: Riesgo, Simulación y Econometría

Módulo 3: Riesgo, Simulación y Econometría

Medir lo que puede salir mal y validar lo que creemos saber

El Módulo 2 terminó con una advertencia: los modelos de portafolio son tan buenos como sus supuestos, y los supuestos fallan. Este módulo —el más largo y diverso del libro— se toma esa advertencia en serio y construye el oficio completo del análisis cuantitativo de riesgo: **medir la incertidumbre, simularla, modelarla y, sobre todo, desconfiar de los propios modelos con método**.

El recorrido avanza en cuatro actos. Primero, la **medición del riesgo** y su contexto institucional: el VaR y el Expected Shortfall como lenguaje estándar (Capítulo 11), y el marco regulatorio que las crisis fueron dejando —Basilea III, stress testing— junto a las dos historias que todo analista debería saber contar: LTCM y 2008 (Capítulo 12). Segundo, los **modelos del mundo**: la simulación de Monte Carlo con el Browniano geométrico (Capítulo 13) y su corrección necesaria, los procesos con saltos y colas gruesas que describen los mercados reales (Capítulo 14).

El tercer acto es la **inferencia**: cómo aprender de los datos con modelos lineales bien especificados —CAPM, multifactor, p-values leídos con honestidad— (Capítulo 15) y cómo modelar series que dependen de su propio pasado con ARIMA y VAR (Capítulo 16). Y el cierre es el acto que le da sentido a todo lo anterior: el **backtesting** (Capítulo 17), la disciplina de separar la señal de la casualidad, que convierte "mi estrategia

funcionó en el pasado" en una afirmación estadísticamente defendible… o la desarma.

La Figura 10.1 resume ese recorrido.

> **El hilo conductor**
>
> Si tuviéramos que resumir el módulo en una frase, sería esta: **la media y la varianza no cuentan la historia de las colas, y en las colas vive el riesgo**. Cada capítulo ataca una consecuencia distinta de esa frase.

Objetivos de Aprendizaje

Al completar este módulo, vas a ser capaz de:

1. **Calcular Value at Risk y Expected Shortfall** con métodos paramétrico, histórico y por Monte Carlo.
2. **Ubicar el riesgo en su marco regulatorio** (stress testing, Basilea III) y aprender de las crisis.
3. **Simular trayectorias de precios** con Browniano geométrico y modelos con saltos.
4. **Construir e interpretar modelos lineales** aplicados a finanzas (CAPM, multifactor).
5. **Modelar y pronosticar series de tiempo** con ARIMA y VAR.
6. **Validar estrategias** con backtesting out-of-sample y walk-forward.

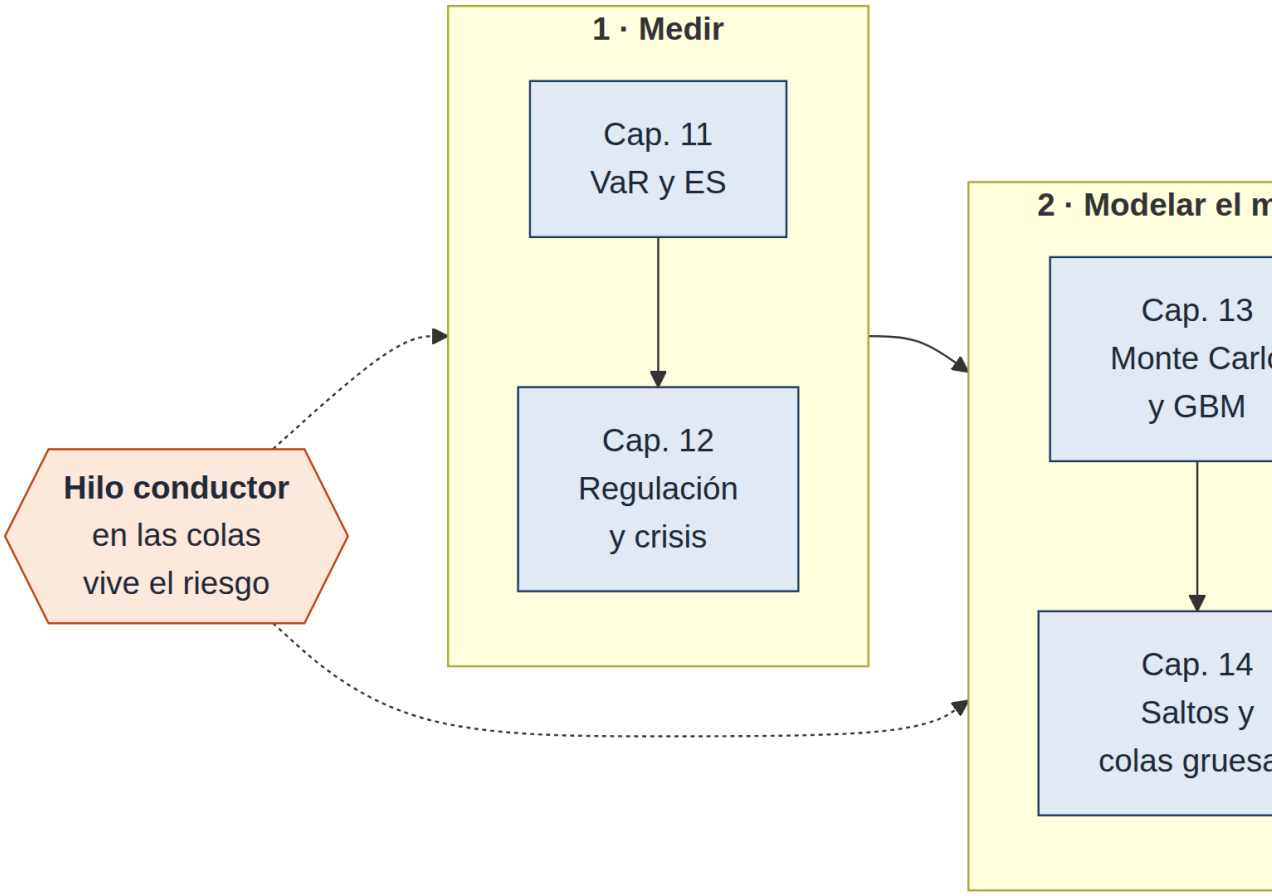


Figura 10.1.: Los cuatro actos del módulo. Medir el riesgo, modelar cómo se mueven los precios, aprender de los datos y, al final, validar con método. Las colas atraviesan los cuatro.

Recorrido Capítulo a Capítulo

Capítulo	Tema	Python	Finanzas
11	VaR y Expected Shortfall	`scipy.stats`, `np.percentile`	VaR, CVaR/ES, backtesting de riesgo
12	Riesgo y Regulación	Simulación de escenarios	Stress testing, Basilea III, LTCM, 2008
13	Monte Carlo	Simulación vectorizada, `np.random`	GBM, pricing por MC, convergencia
14	Procesos Estocásticos	Poisson, saltos	Jump-diffusion, Lévy, colas gruesas
15	Econometría y Modelos Lineales	MCO, `statsmodels`	CAPM, multifactor, p-values
16	Series Temporales	`statsmodels.tsa`	ARIMA, VAR, estacionariedad
17	Backtesting	Ventanas rodantes	In/out-of-sample, walk-forward, Sharpe OOS

Herramientas y Librerías

NumPy

Simulación vectorizada

SciPy

Distribuciones y tests

Statsmodels

Regresión y series de tiempo

Matplotlib

Distribuciones y trayectorias

Concepto Clave: VaR y Expected Shortfall

! El **Value at Risk** al nivel de confianza α responde: *¿cuál es la pérdida que solo se excede el* $(1-\alpha)\,\%$ *de las veces?*

$$VaR_\alpha = -(\mu + z_\alpha \cdot \sigma) \cdot W$$

Pero el VaR solo marca **dónde empieza** la zona de desastre. El **Expected Shortfall** mira adentro: *¿cuánto pierdo en promedio cuando la cruzo?*

$$ES_\alpha = -E\big[\, r \mid r < -VaR_\alpha \,\big]$$

La tensión entre estas dos preguntas —y por qué la regulación migró de una a la otra— es el punto de partida del módulo.

Lecturas Recomendadas

- Jorion, P. (2006). *Value at Risk*. McGraw-Hill.
- Glasserman, P. (2003). *Monte Carlo Methods in Financial Engineering*. Springer.
- Wooldridge, J. (2019). *Introductory Econometrics: A Modern Approach*. Cengage.
- López de Prado, M. (2018). *Advances in Financial Machine Learning*. Wiley.
- Lowenstein, R. (2000). *When Genius Failed*. Random House.

11. Capítulo 11: Valor en Riesgo (VaR) y Expected Shortfall

Medición de Riesgo de Mercado: VaR Paramétrico, Histórico y CVaR

Introducimos las herramientas fundamentales de medición de riesgo: VaR paramétrico e histórico, Expected Shortfall (CVaR), volatilidad EWMA, y tests de normalidad para entender las limitaciones de los modelos gaussianos.

Introducción

El **Value at Risk (VaR)** responde a la pregunta más importante en gestión de riesgo:

> *¿Cuál es la máxima pérdida esperada en un horizonte dado, con un nivel de confianza determinado?*

Ejemplo intuitivo

Si el VaR diario al 95 % de un portafolio es \$150,000, significa que **en 19 de cada 20 días**, la pérdida no superará esa cifra. Pero en 1 de cada 20 días, sí podría.

11.1. Generación de Datos

Antes de calcular el VaR, necesitamos retornos. Simulamos un año de retornos diarios:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Configuración
```

```python
rng = np.random.default_rng(42)
valor_portafolio = 10_000_000  # $10M

# Simular retornos diarios (252 días hábiles)
retornos = rng.normal(0.0004, 0.015, 252)

mu = np.mean(retornos)
sigma = np.std(retornos)

print(f"Retornos diarios simulados: {len(retornos)} observaciones")
print(f"Media diaria: {mu:.6f} ({mu * 252:.2%} anualizada)")
print(f"Volatilidad diaria: {sigma:.6f} ({sigma * np.sqrt(252):.2%} anualizada)")
```

```
Retornos diarios simulados: 252 observaciones
Media diaria: -0.000326 (-8.20% anualizada)
Volatilidad diaria: 0.014036 (22.28% anualizada)
```

11.2. VaR Paramétrico (Normal)

El primer método es el más rápido y el que más supuestos hace: asumir que los retornos siguen una **distribución normal**. Si eso es cierto, la distribución entera queda descripta por dos números que ya estimamos —media μ y desvío σ— y el cuantil que define el VaR tiene fórmula cerrada:

$$VaR_\alpha = -(\mu + z_\alpha \cdot \sigma) \cdot W$$

donde z_α es el cuantil de la normal estándar (para 95 %, $z = -1.645$: el punto que deja el 5 % de la campana a su izquierda) y W es el valor del portafolio.

```
confianza = 0.95

# Z-score para 95%
z = stats.norm.ppf(1 - confianza)

# VaR paramétrico (1 día)
var_param_95 = -(mu + z * sigma) * valor_portafolio

# También al 99%
z_99 = stats.norm.ppf(0.01)
var_param_99 = -(mu + z_99 * sigma) * valor_portafolio

print(f"VaR Paramétrico (95%, 1 día): ${var_param_95:>12,.2f}")
print(f"VaR Paramétrico (99%, 1 día): ${var_param_99:>12,.2f}")
print(f"Como% del portafolio (95%):  {var_param_95 / valor_portafolio:.4%}")
print(f"Como% del portafolio (99%):  {var_param_99 / valor_portafolio:.4%}")
```

```
VaR Paramétrico (95%, 1 día): $  234,134.38
VaR Paramétrico (99%, 1 día): $  329,791.90
Como % del portafolio (95%):  2.3413%
Como % del portafolio (99%):  3.2979%
```

Traduzcamos el número a castellano, porque es lo que te van a pedir en cualquier comité de riesgo: *"con un portafolio de $10 millones, en 19 de cada 20 días la pérdida no debería superar los ~$240.000 (un 2.4 % del portafolio)"*. Y la comparación entre niveles enseña algo general: pasar del 95 % al 99 % de confianza **encarece el VaR en casi un 50 %** — meterse más adentro de la cola cuesta cada vez más caro, porque la cola se estira.

11.3. VaR Histórico

El método paramétrico se apoya por completo en el supuesto de normalidad. El **VaR histórico** toma el camino opuesto: no asume *ninguna* distribución — deja que los datos hablen solos. La receta es de una simplicidad brutal: ordenás los retornos observados de peor a mejor y leés el percentil correspondiente. El VaR 95 % es, literalmente, "el retorno del día que está en el puesto 5 % de los peores".

```
# VaR Histórico: percentil de los retornos reales
percentil_5 = np.percentile(retornos, 5)
percentil_1 = np.percentile(retornos, 1)

var_hist_95 = -percentil_5 * valor_portafolio
var_hist_99 = -percentil_1 * valor_portafolio

print(f"VaR Histórico (95%, 1 día): ${var_hist_95:>12,.2f}")
print(f"VaR Histórico (99%, 1 día): ${var_hist_99:>12,.2f}")
```

```
VaR Histórico (95%, 1 día): $  213,744.80
VaR Histórico (99%, 1 día): $  295,941.84
```

Con nuestros datos, el histórico da parecido al paramétrico — lógico, porque los retornos *fueron simulados* de una normal. Con retornos reales (colas pesadas), el histórico al 99 % suele salir **más alto**, porque los peores días de la muestra son peores de lo que la normal admite. Esa diferencia entre métodos no es un defecto: es un **diagnóstico** — cuando el histórico y el paramétrico divergen, la distribución real se está apartando de la normal justo donde más importa.

El precio de la flexibilidad del histórico: al 99 % con un año de datos (252 observaciones), el percentil 1 se estima con **apenas 2 o 3 observaciones**.

Tu medida de riesgo cuelga de un puñado de días — por eso en la práctica se usan ventanas más largas o métodos híbridos.

La Figura 11.1 muestra la receta completa: los 252 días puestos en fila, del peor al mejor. El VaR histórico no es más que leer dónde termina la franja roja.

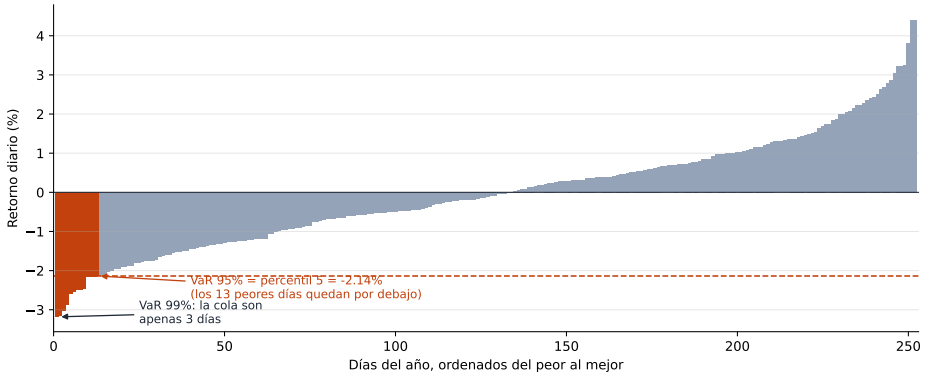


Figura 11.1.: VaR histórico: los 252 retornos del año ordenados del peor al mejor. Los días en rojo forman la cola del 5 %; el VaR 95 % es la frontera de esa franja. Al 99 %, la cola se reduce a 2 o 3 barras.

11.4. Comparación Visual: Paramétrico vs. Histórico

La Figura 11.2 pone los dos métodos sobre el mismo dibujo: el histograma de los datos, la campana que supone el método paramétrico y el corte del 5 % que marca cada uno.

```
fig, ax = plt.subplots(figsize=(10, 5))

# Histograma de retornos
ax.hist(retornos, bins=40, density=True, alpha=0.6, color="#94a3b8",
        edgecolor="white", label="Retornos observados")

# Curva normal teórica
x = np.linspace(retornos.min(), retornos.max(), 200)
ax.plot(x, stats.norm.pdf(x, mu, sigma), color="#1e3a5f", linewidth=2,
        label="Normal teórica")

# Líneas de VaR
ax.axvline(percentil_5, color="#c2410c", linewidth=2, linestyle="--",
           label=f"VaR Hist 95% ({percentil_5:.4f})")
ax.axvline(mu + z * sigma, color="#637238", linewidth=2, linestyle=":",
           label=f"VaR Param 95% ({mu + z * sigma:.4f})")

ax.set_xlabel("Retorno Diario")
ax.set_ylabel("Densidad")
ax.set_title("Distribución de Retornos con Líneas de VaR")
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

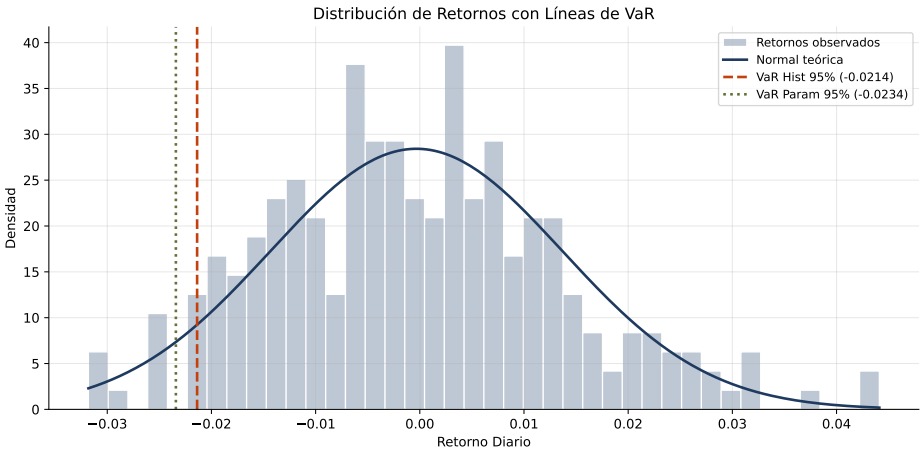


Figura 11.2.: Retornos observados, normal ajustada y las dos líneas de VaR 95 %. Con datos simulados de una normal, ambos cortes caen casi en el mismo lugar.

Diferencias entre métodos

El VaR paramétrico y el histórico **no necesariamente coinciden**. La diferencia se agranda cuando la distribución real se desvía de la normal (colas pesadas, asimetría).

11.5. Expected Shortfall (CVaR): mirar dentro de la cola

El VaR tiene un defecto conceptual serio: te dice *dónde empieza* la zona de desastre, pero **no te dice nada de lo que pasa adentro**. Dos portafolios

pueden tener el mismo VaR al 95 % y comportamientos totalmente distintos en ese 5 % restante: uno pierde apenas un poco más que el VaR, el otro se pulveriza. Para el VaR, son idénticos.

El **Expected Shortfall** (ES), también llamado **CVaR** (*Conditional Value at Risk*), corrige exactamente eso. Su pregunta es:

> *En los peores escenarios —los que superan el VaR—, ¿cuánto pierdo* ***en promedio****?*

$$ES_\alpha = -E[\, r \mid r < -VaR_\alpha \,]$$

Es una esperanza **condicional**: promediamos los retornos *dado que* caímos en la cola. Por construcción, el ES siempre es mayor o igual que el VaR — es el "qué tan feo se pone" después de cruzar la frontera.

La Figura 11.3 muestra las dos medidas sobre la misma cola. A la izquierda, el VaR es una frontera y el ES es el centro de gravedad de lo que queda del otro lado. A la derecha, dos distribuciones con **exactamente el mismo VaR 95 %**: para el VaR son idénticas, pero la de colas pesadas esconde pérdidas mucho más profundas, y el ES las ve.

11.5.1. Cálculo histórico: promediar la cola

Con el método histórico, el cálculo es de una transparencia total: filtrás los días peores que el VaR y promediás.

```
# ES histórico: media de los retornos por debajo del percentil correspondiente
retornos_cola = retornos[retornos < percentil_5]
cvar_95 = -np.mean(retornos_cola) * valor_portafolio

retornos_cola_99 = retornos[retornos < percentil_1]
cvar_99 = -np.mean(retornos_cola_99) * valor_portafolio
```

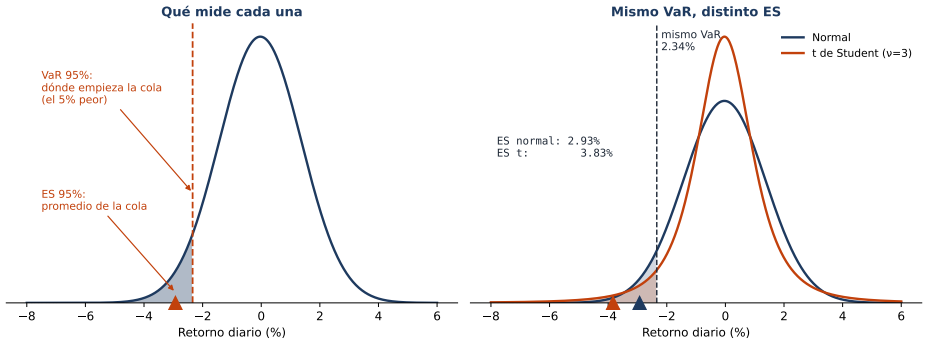


Figura 11.3.: Izquierda: el VaR marca dónde empieza la cola del 5 %; el ES es el promedio de esa cola. Derecha: una normal y una t de Student (3 grados de libertad) calibradas al mismo VaR 95 %. El VaR no las distingue; el ES de la t es bastante mayor.

```
print(f"Días en la cola del 5%: {len(retornos_cola)}")
print(f"ES/CVaR (95%, 1 día): ${cvar_95:>12,.2f}   (VaR hist: ${var_hist_95:,.2f})")
print(f"ES/CVaR (99%, 1 día): ${cvar_99:>12,.2f}   (VaR hist: ${var_hist_99:,.2f})")
print(f"\nEl ES 95% es {cvar_95/var_hist_95:.2f}x el VaR 95%")
print("→ Cuando el día es malo de verdad, la pérdida promedio supera al VaR en ese f
```

```
Días en la cola del 5%: 13
ES/CVaR (95%, 1 día): $  257,564.55   (VaR hist: $213,744.80)
ES/CVaR (99%, 1 día): $  312,475.37   (VaR hist: $295,941.84)

El ES 95% es 1.21x el VaR 95%
→ Cuando el día es malo de verdad, la pérdida promedio supera al VaR en ese factor.
```

11.5.2. Cálculo paramétrico: fórmula cerrada bajo normalidad

Si asumimos retornos normales, el ES también tiene fórmula cerrada. Para nivel de confianza α:

$$ES_\alpha = -\left(\mu - \sigma \cdot \frac{\phi(z_{1-\alpha})}{1-\alpha}\right) \cdot W$$

donde ϕ es la densidad normal estándar y $z_{1-\alpha}$ el cuantil. El cociente $\phi(z)/(1-\alpha)$ es la "media de la cola" de la normal.

```
alpha = 0.95
z_a = stats.norm.ppf(1 - alpha)

es_param_95 = -(mu - sigma * stats.norm.pdf(z_a) / (1 - alpha)) * valor_portafolio
print(f"ES paramétrico (95%): ${es_param_95:>12,.2f}")
print(f"ES histórico   (95%): ${cvar_95:>12,.2f}")
```

```
ES paramétrico (95%): $  292,786.93
ES histórico   (95%): $  257,564.55
```

Ambos métodos deberían parecerse con datos ~normales como los nuestros. Con retornos reales (colas pesadas), el histórico suele dar **más alto** — y es el que está diciendo la verdad. También se puede estimar el ES por **Monte Carlo**: simulás miles de retornos del modelo que quieras (con saltos, con t-Student) y promediás la cola simulada; lo haremos en el Capítulo 13.

> **Simulador interactivo**
>
> En la versión web de este capítulo hay un simulador donde podés cambiar la forma de la distribución de los retornos y el nivel de confianza, y comparar el VaR y el ES verdaderos con los de una normal de igual volatilidad.

11.5.3. Por qué el ES es "coherente" y el VaR no

Hay un argumento matemático de fondo, y vale la pena entenderlo porque explica por qué **Basilea cambió del VaR al ES** para riesgo de mercado.

Una medida de riesgo razonable debería premiar la diversificación: el riesgo de una cartera combinada no debería superar la suma de los riesgos individuales. Esa propiedad se llama **subaditividad**:

$$\rho(A + B) \;\leq\; \rho(A) + \rho(B)$$

El VaR **puede violarla**. El ejemplo clásico: dos bonos independientes que rara vez caen en default, pero cuando caen, pierden mucho.

```
# Dos bonos independientes: 3% de probabilidad de default con pérdida del 60%
rng_b = np.random.default_rng(7)
n_esc = 100_000

def perdida_bono(rng, n):
    default = rng.random(n) < 0.03
    return np.where(default, -0.60, 0.02)   # -60% si default, +2% si no

r_A = perdida_bono(rng_b, n_esc)
r_B = perdida_bono(rng_b, n_esc)
r_AB = 0.5 * r_A + 0.5 * r_B                # cartera diversificada 50/50

def var_h(r, conf=0.95):
    return -np.percentile(r, (1 - conf) * 100)

def es_h(r, conf=0.95):
    umbral = np.percentile(r, (1 - conf) * 100)
    return -r[r <= umbral].mean()
```

```
print(f"{'':18}{'Bono A':>9}{'Bono B':>9}{'Cartera 50/50':>15}")
print(f"{'VaR 95%':<18}{var_h(r_A):>9.1%}{var_h(r_B):>9.1%}{var_h(r_AB):>15.1%}")
print(f"{'ES  95%':<18}{es_h(r_A):>9.1%}{es_h(r_B):>9.1%}{es_h(r_AB):>15.1%}")
```

```
                    Bono A   Bono B  Cartera 50/50
VaR 95%              -2.0%    -2.0%          29.0%
ES  95%              -0.2%    -0.2%          29.5%
```

Mirá el resultado con atención, porque es contraintuitivo: cada bono individual tiene **VaR negativo o nulo** (la probabilidad de default, 3 %, es menor que el 5 % de la cola, así que el VaR "no ve" el default). Pero la cartera diversificada tiene VaR **positivo y grande**: al combinar dos bonos, la probabilidad de que *al menos uno* caiga (~6 %) ya asoma dentro de la cola del 5 %.

Según el VaR, **diversificar empeoró el riesgo** — un absurdo económico. El ES, en cambio, ve el default de cada bono individual (porque promedia toda la cola, no solo mira la frontera) y reporta correctamente que la cartera combinada es menos riesgosa que cada bono solo. Esa es la diferencia entre una medida coherente y una que no lo es.

La Figura 11.4 muestra de dónde sale la paradoja. Cada barra reparte el 100 % de la probabilidad entre los resultados posibles, del peor al mejor, y la línea marca el 5 %. El VaR es el resultado sobre el que cae esa línea.

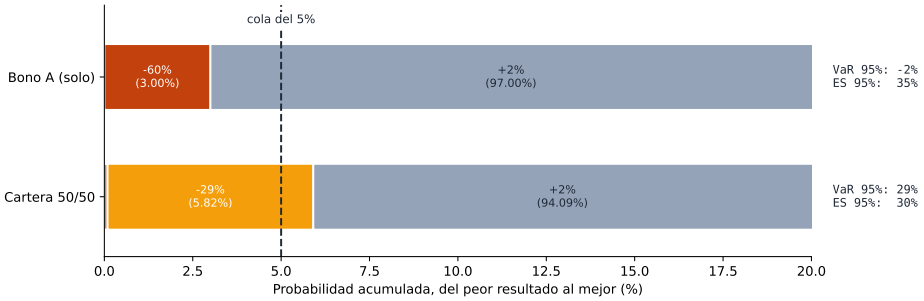


Figura 11.4.: Distribución de resultados ordenada del peor al mejor (eje recortado al 20 % de probabilidad más mala). En el bono solo, el default (3 %) queda entero dentro de la cola y la línea del 5 % cae en un escenario sin pérdida: VaR 0. En la cartera, la línea cae en el escenario «un bono en default» (−29 %): el VaR salta, aunque el riesgo bajó.

! VaR vs. Expected Shortfall, en síntesis

Métrica	Pregunta que responde	¿Subaditiva?	¿Ve dentro de la cola?
VaR	¿Dónde empieza la zona de pérdidas extremas?	No siempre	No
ES/CVaR	¿Cuánto pierdo en promedio cuando estoy en esa zona?	Sí (coherente)	Sí

Por estas propiedades, **Basilea III (FRTB)** reemplazó el VaR 99 % por el **ES 97.5 %** como métrica regulatoria de riesgo de mercado. En la práctica profesional se reportan ambos: el VaR por tradición y comparabilidad, el ES por honestidad sobre las colas.

11.6. Backtesting de VaR: ¿el modelo dice la verdad?

Calculamos un VaR 95 %. ¿Cómo sabemos si es *correcto*? Hay una forma directa de auditarlo, y es la que exigen los reguladores: contar. Si el VaR 95 % está bien calibrado, la pérdida real debería superarlo **el 5 % de los días** — ni mucho más (el modelo subestima el riesgo) ni mucho menos (lo sobreestima, y estás inmovilizando capital de más). Cada día en que la pérdida supera al VaR se llama una **excepción**.

El **test de Kupiec** (o *proportion of failures*) formaliza el conteo. Su lógica: si cada día hay una probabilidad del 5 % de excepción, y los días son independientes, el número total de excepciones en n días sigue una **distribución binomial** — igual que contar caras en tiradas de una moneda cargada al 5 %. Con eso podemos preguntar: *¿el número de excepciones que observé es compatible con esa binomial?* Si observamos demasiadas, la probabilidad de que el modelo esté bien calibrado (el p-value) se hace chica y lo rechazamos.

```
# Contar cuántos días la pérdida real superó al VaR
var_limite = mu + z * sigma  # VaR paramétrico expresado como retorno
excepciones = np.sum(retornos < var_limite)
tasa_excepciones = excepciones / len(retornos)
```

```python
print(f"Excepciones: {excepciones} de {len(retornos)} días")
print(f"Tasa de excepciones: {tasa_excepciones:.2%}")
print(f"Tasa esperada:       {1 - confianza:.2%}")
print(f"Excepciones esperadas: {len(retornos) * (1 - confianza):.1f}")

# Test de Kupiec: ¿el número de excepciones es compatible con una binomial(n, 5%)?
from scipy.stats import binom
p_value = 1 - binom.cdf(excepciones - 1, len(retornos), 1 - confianza)
print(f"\np-value (Kupiec): {p_value:.4f}")
print("Aprobado" if p_value > 0.05 else "Rechazado: demasiadas excepciones")
```

```
Excepciones: 9 de 252 días
Tasa de excepciones: 3.57%
Tasa esperada:       5.00%
Excepciones esperadas: 12.6

p-value (Kupiec): 0.8867
Aprobado
```

Interpretemos el resultado: esperábamos ~12.6 excepciones en 252 días y observamos un número cercano; el p-value alto dice que esa cantidad es perfectamente compatible con un modelo bien calibrado — **no hay evidencia contra el VaR**. Si hubiéramos observado 25 excepciones, el p-value se derrumbaría y el modelo quedaría rechazado. Este es exactamente el mecanismo del "semáforo de Basilea": los reguladores cuentan las excepciones del año y, si son demasiadas, el banco sufre un multiplicador de capital mayor.

La Figura 11.5 dibuja el test: la binomial dice cuántas excepciones son esperables si el modelo está bien, y la zona roja reúne las cantidades tan altas que serían una sorpresa (menos del 5 % de probabilidad).

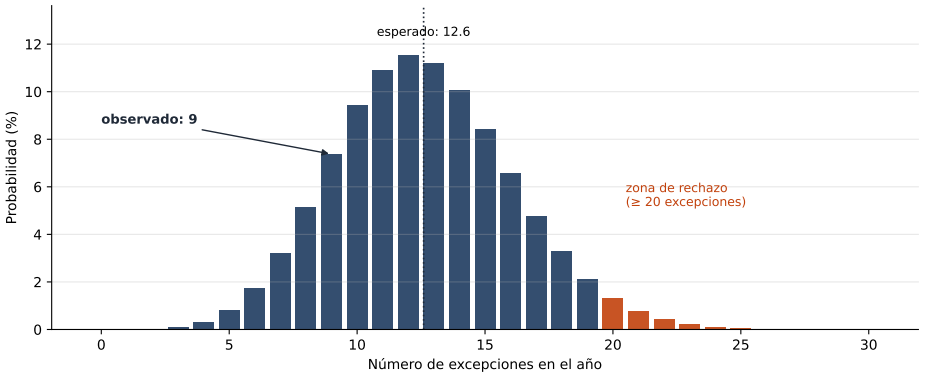


Figura 11.5.: Test de Kupiec: distribución del número de excepciones en 252 días si el VaR 95 % está bien calibrado, Binomial(252, 5 %). Las barras rojas forman la zona de rechazo; la flecha marca las excepciones observadas.

Lo que Kupiec no ve

El test cuenta *cuántas* excepciones hubo, pero no *cuándo*: diez excepciones repartidas en el año y diez excepciones en dos semanas de crisis dan el mismo veredicto, aunque el segundo caso delata un modelo que no reacciona a los cambios de régimen. Tests más finos (Christoffersen) auditan también la independencia de las excepciones. Y ese defecto nos lleva directo al problema siguiente: la volatilidad **cambia en el tiempo**.

11.7. EWMA: cuando la volatilidad tiene memoria

Todo lo anterior usó **una** volatilidad, estimada con el año entero de datos. Pero mirá cualquier serie financiera real: hay semanas planas y semanas de pánico, y los días agitados vienen **en rachas** (el *volatility clustering* que formalizaremos en el Capítulo 20). Un VaR con volatilidad constante es un promedio entre la calma y la tormenta: sobreestima el riesgo en las épocas tranquilas y —mucho peor— **lo subestima justo cuando la tormenta llega**.

El modelo **EWMA** (*Exponentially Weighted Moving Average*, popularizado por RiskMetrics de J.P. Morgan) corrige esto con una idea simple: estimar la varianza dando **más peso a lo reciente**. Cada día, la varianza nueva es una mezcla de la varianza de ayer y del shock de ayer:

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1-\lambda) r_{t-1}^2$$

El parámetro λ (RiskMetrics usa 0.94) controla la memoria: un shock grande de ayer entra con peso $(1-\lambda) = 6\,\%$ y después se va desvaneciendo geométricamente. Resultado: cuando el mercado se agita, la volatilidad estimada **sube en días, no en meses** — y el VaR con ella.

Desarrollando la recursión, la varianza EWMA resulta ser un promedio de todos los retornos al cuadrado pasados, con pesos $(1-\lambda)\lambda^k$ para el día de hace $k+1$ ruedas. La Figura 11.6 compara esos pesos con los de la volatilidad "de siempre", que le da a cada día del año el mismo peso.

```
lambd = 0.94  # Factor de decaimiento (RiskMetrics)

# Calcular varianza EWMA
varianza_ewma = np.zeros(len(retornos))
varianza_ewma[0] = retornos[0] ** 2

for t in range(1, len(retornos)):
```

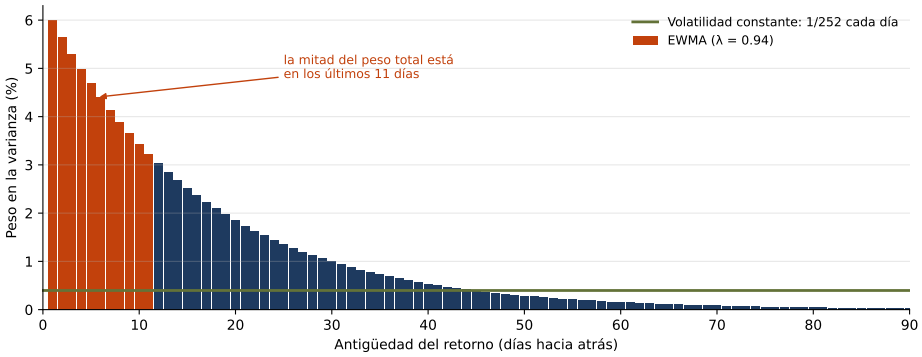


Figura 11.6.: Cuánto pesa cada día pasado en la estimación de la varianza. La volatilidad constante reparte el peso por igual entre los 252 días; EWMA (= 0.94) concentra la mitad del peso en las últimas ~11 ruedas y olvida rápido lo viejo.

```
    varianza_ewma[t] = lambd * varianza_ewma[t-1] + (1 - lambd) * retornos[t-1] ** 2

vol_ewma = np.sqrt(varianza_ewma)

# VaR dinámico con EWMA
var_ewma = -(mu + z * vol_ewma) * valor_portafolio

fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True)

# Panel superior: volatilidad
ax = axes[0]
ax.plot(vol_ewma * np.sqrt(252) * 100, linewidth=1.5, color="#1e3a5f", label="EWMA (
ax.axhline(sigma * np.sqrt(252) * 100, color="#c2410c", linestyle="--", alpha=0.7, l
ax.set_ylabel("Volatilidad Anualizada (%)")
ax.set_title("Comparación: Volatilidad Constante vs. EWMA")
ax.legend()
ax.grid(True, alpha=0.3)
```

```
# Panel inferior: VaR dinámico vs constante
ax = axes[1]
ax.fill_between(range(len(var_ewma)), var_ewma, alpha=0.3, color="#c2410c", label="V
ax.axhline(var_param_95, color="#1e3a5f", linestyle="--", label="VaR constante")
ax.set_xlabel("Día")
ax.set_ylabel("VaR ($)")
ax.set_title("VaR Dinámico (EWMA) vs. Constante")
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

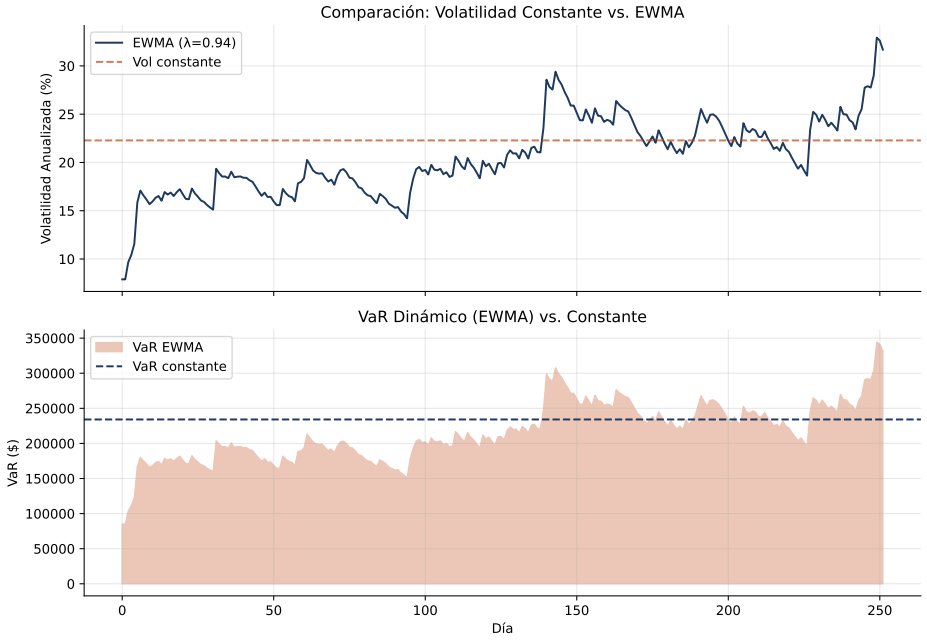


Figura 11.7.: Arriba: volatilidad EWMA contra la volatilidad constante del año. Abajo: el VaR 95 % que resulta de cada una.

Leé los dos paneles de la Figura 11.7 juntos. Arriba: la volatilidad EWMA oscila alrededor de la constante (línea roja), con episodios donde se dispara por encima y períodos donde duerme por debajo — la línea roja es el promedio de dos regímenes distintos. Abajo, la consecuencia para el riesgo: el **VaR dinámico se expande en los períodos agitados y se contrae en los tranquilos**, mientras el VaR constante ignora todo. En los picos, el VaR constante está *subestimando* el riesgo del día — y esas son exactamente las excepciones agrupadas que el test de Kupiec no sabe detectar.

> **Simulador interactivo**
>
> En la versión web de este capítulo hay un simulador que aplica el VaR constante, el VaR EWMA y el test de Kupiec a los datos reales del libro, donde podés elegir el activo, λ y la confianza.

11.8. Test de Normalidad

Cerramos auditando el supuesto sobre el que descansa todo el enfoque paramétrico: ¿los retornos son normales? El **test de Jarque-Bera** lo contrasta midiendo asimetría y curtosis:

- H_0: Los retornos son normales (skewness = 0, curtosis = 3)
- H_1: No son normales

```
from scipy.stats import jarque_bera, skew, kurtosis

jb_stat, jb_pvalue = jarque_bera(retornos)
s = skew(retornos)
k = kurtosis(retornos, fisher=True)  # Exceso de curtosis

print(f"Skewness (asimetría):    {s:>+.4f}  (normal = 0)")
print(f"Exc. Curtosis:           {k:>+.4f}  (normal = 0)")
print(f"Jarque-Bera stat:        {jb_stat:>.4f}")
print(f"p-value:                 {jb_pvalue:>.4f}")
print(f"{'-> No se rechaza normalidad' if jb_pvalue > 0.05 else '-> Se rechaza norma
```

```
Skewness (asimetría):    +0.3848  (normal = 0)
Exc. Curtosis:           +0.1543  (normal = 0)
```

```
Jarque-Bera stat:        6.4692
p-value:                 0.0394
-> Se rechaza normalidad
```

El test no rechaza — pero ojo con la lectura: nuestros retornos **fueron simulados de una normal**, así que el test está funcionando como debe (no encuentra anormalidad donde no la hay). El valor pedagógico está en el contraste con lo que pasa al aplicar exactamente este test a datos reales:

> ⚠ En datos reales…
>
> Los retornos financieros reales típicamente **rechazan** la normalidad con p-values microscópicos. Presentan:
>
> - **Curtosis excesiva** (colas pesadas → más eventos extremos que la normal)
> - **Asimetría negativa** (más crashes que rallies)
>
> Esto hace que el VaR paramétrico normal **subestime** el riesgo verdadero — especialmente al 99 %, donde manda la cola. Es el motivo de fondo por el que existen el VaR histórico, el ES, y los modelos con saltos del Capítulo 14.

11.9. Resumen de Métricas

Los tres métodos del capítulo (y el que llega en el Capítulo 13) comparten el final: un percentil de la distribución de pérdidas y el promedio de lo que queda más allá. Lo que cambia es cómo se obtiene esa distribución, y la Figura 11.8 lo resume.

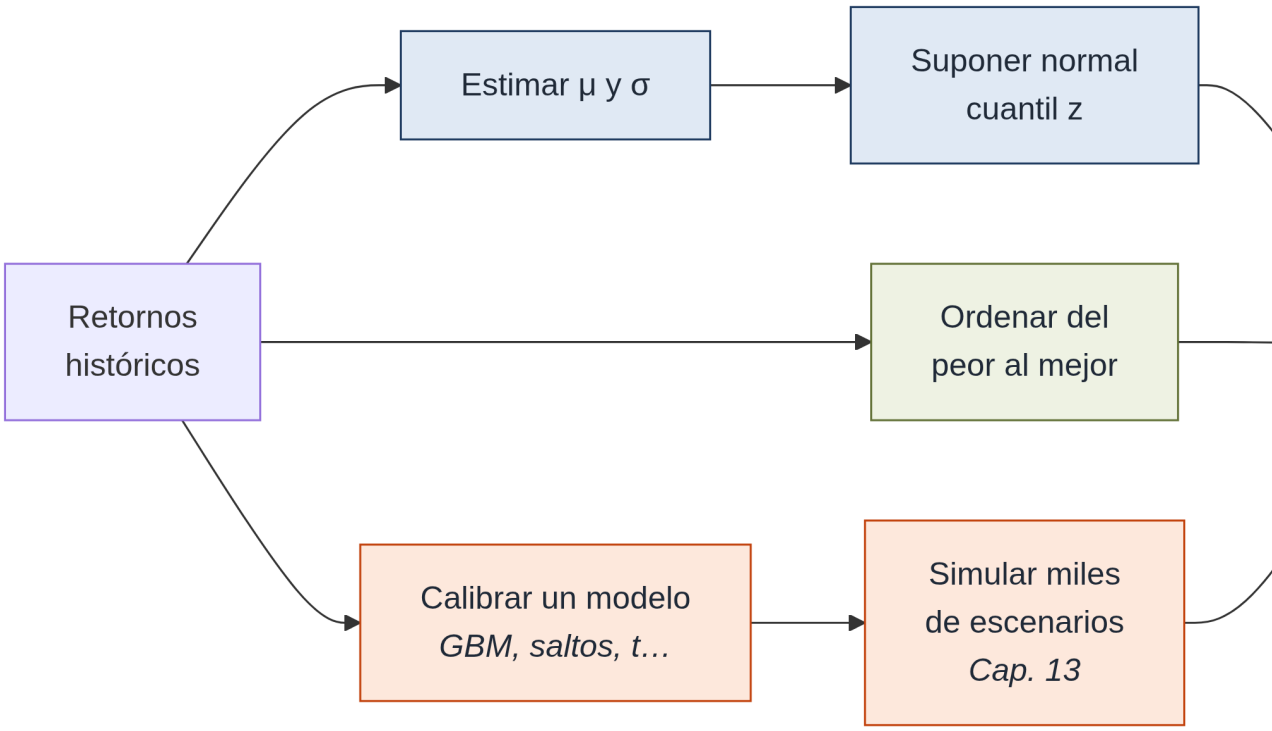


Figura 11.8.: Tres caminos hacia el VaR y el ES. Difieren en de dónde sale la distribución de pérdidas: una fórmula (normal), el pasado tal cual (histórico) o un modelo simulado (Monte Carlo).

```python
import pandas as pd

resumen = pd.DataFrame({
    "Métrica": ["VaR Paramétrico 95%", "VaR Paramétrico 99%",
                "VaR Histórico 95%", "VaR Histórico 99%",
                "CVaR 95%", "CVaR 99%"],
    "Valor ($)": [var_param_95, var_param_99, var_hist_95, var_hist_99,
                  cvar_95, cvar_99],
    "% Portafolio": [var_param_95/valor_portafolio, var_param_99/valor_portafolio,
                     var_hist_95/valor_portafolio, var_hist_99/valor_portafolio,
                     cvar_95/valor_portafolio, cvar_99/valor_portafolio],
})

resumen["Valor ($)"] = resumen["Valor ($)"].apply(lambda x: f"${x:,.2f}")
resumen["% Portafolio"] = resumen["% Portafolio"].apply(lambda x: f"{x:.4%}")

print(resumen.to_string(index=False))
```

```
            Métrica   Valor ($) % Portafolio
VaR Paramétrico 95% $234,134.38      2.3413%
VaR Paramétrico 99% $329,791.90      3.2979%
  VaR Histórico 95% $213,744.80      2.1374%
  VaR Histórico 99% $295,941.84      2.9594%
           CVaR 95% $257,564.55      2.5756%
           CVaR 99% $312,475.37      3.1248%
```

11.10. Con datos reales

El capítulo entero descansó sobre retornos simulados de una normal — ideal para aprender los métodos, tramposo para juzgarlos. El veredicto

final se lo damos a los datos: cinco años del ETF SPY (el S&P 500), del dataset del libro (`datos/precios.csv`, ver Apéndice B).

```
precios = pd.read_csv("../datos/precios.csv", index_col="Date", parse_dates=True)
r_spy = precios["SPY"].pct_change().dropna().values

mu_spy, sigma_spy = r_spy.mean(), r_spy.std()
p5, p1 = np.percentile(r_spy, 5), np.percentile(r_spy, 1)

print(f"SPY: {len(r_spy)} ruedas ({precios.index[0]:%Y-%m} a {precios.index[-1]:%Y-
print(f"{'':24}{'95%':>12}{'99%':>12}")
print(f"{'VaR paramétrico':<24}"
      f"${-(mu_spy + stats.norm.ppf(0.05) * sigma_spy) * valor_portafolio:>11,.0f}"
      f"${-(mu_spy + stats.norm.ppf(0.01) * sigma_spy) * valor_portafolio:>11,.0f}")
print(f"{'VaR histórico':<24}"
      f"${-p5 * valor_portafolio:>11,.0f}"
      f"${-p1 * valor_portafolio:>11,.0f}")
print(f"{'ES histórico':<24}"
      f"${-r_spy[r_spy < p5].mean() * valor_portafolio:>11,.0f}"
      f"${-r_spy[r_spy < p1].mean() * valor_portafolio:>11,.0f}")

jb_r, jb_p = jarque_bera(r_spy)
print(f"\nJarque-Bera: estadístico {jb_r:,.0f}, p-value {jb_p:.2e}")
print(f"Exceso de curtosis: {kurtosis(r_spy, fisher=True):+.2f} | "
      f"Asimetría: {skew(r_spy):+.2f}")
```

```
SPY: 1252 ruedas (2021-07 a 2026-06)

                                95%         99%
VaR paramétrico         $    172,498$    246,231
VaR histórico           $    166,301$    292,978
ES histórico            $    246,686$    387,806
```

```
Jarque-Bera: estadístico 4,213, p-value 0.00e+00
Exceso de curtosis: +8.97 | Asimetría: +0.31
```

Todo lo que el capítulo anticipó, junto y en un solo resultado. Al 95 %, el paramétrico y el histórico casi coinciden — a esa profundidad de la cola, la normal todavía zafa. Al **99 % se separan**: los peores días reales del S&P son peores de lo que la campana admite, y el histórico lo reporta. El ES amplifica el mensaje (la cola no solo empieza antes: es más profunda). Y el test de Jarque-Bera, que con los datos simulados no encontraba nada, acá **rechaza la normalidad sin despeinarse** — el exceso de curtosis positivo es la firma de las colas pesadas. Con datos reales, el VaR paramétrico normal es la métrica que menos deberías creerle justo donde más la necesitás.

11.11. Errores comunes

⚠ Trampas frecuentes

- **El signo del cuantil**: el percentil 5 de los retornos es un número *negativo*; el VaR se reporta *positivo* (es una pérdida). Ese signo menos perdido es el bug más común en implementaciones de VaR — verificá que tu VaR sea positivo.
- **Escalar con $\sqrt{t}$ sin pensar**: $VaR_{10d} = VaR_{1d}\sqrt{10}$ solo vale si los retornos son i.i.d. y sin autocorrelación. Con volatility clustering (o sea, siempre), la regla subestima el riesgo de horizontes largos en épocas agitadas.
- **Confundir VaR en % con VaR en $**: mezclar las dos unidades a mitad de un cálculo (multiplicar dos veces por W, o ninguna) produce errores de orden de magnitud que a veces sobreviven hasta el reporte.

- **Volatilidad sin anualizar (o anualizada dos veces)**: sé consistente con el horizonte — VaR diario con σ diaria, VaR anual con σ anual. La mezcla es otro clásico silencioso.
- **Auditar solo la cantidad de excepciones**: Kupiec aprueba un modelo cuyas excepciones vienen todas juntas en una crisis. Mirá también *cuándo* ocurren (independencia), no solo cuántas son.

11.12. Practicá en el navegador

Podés experimentar con el código del capítulo acá mismo: la celda ejecuta Python **en tu navegador**, sin instalar nada (la primera ejecución tarda unos segundos mientras carga el entorno; los paquetes que importes se instalan solos).

```
#| label: practica-navegador
import numpy as np
from scipy import stats

# Para usar el dataset real del libro (requiere conexión):
# import pandas as pd
# from pyodide.http import open_url
# url = "https://diegoonna.github.io/libro_finanzas_computacionales/datos/precios.cs
# precios = pd.read_csv(open_url(url), index_col="Date", parse_dates=True)

# Tu código acá...
```

11.13. Referencias

12. Capítulo 12: Riesgo y Regulación

Stress testing, Basilea III y las lecciones de las crisis

Las métricas del capítulo anterior no viven en el vacío: existen dentro de un marco regulatorio construido, crisis tras crisis, sobre las ruinas de modelos que fallaron. Recorremos el stress testing, la arquitectura de Basilea III, el PPNR y la taxonomía de riesgos, y estudiamos en detalle dos casos que todo analista cuantitativo debería conocer de memoria: el colapso de LTCM (1998) y la crisis de 2008, con su correlación falsa en el corazón.

Introducción

En el capítulo anterior aprendimos a medir el riesgo con VaR y Expected Shortfall. Este capítulo responde una pregunta distinta: **¿qué pasa cuando los modelos fallan?** Porque fallan. Y cuando fallan a escala de sistema financiero, las consecuencias las paga toda la economía.

La regulación bancaria moderna es, en gran medida, la **historia acumulada de esos fracasos**: cada crisis dejó una regla nueva. Entender ese marco no es un tema "legal" ajeno al análisis cuantitativo — define qué modelos se usan, cuánto capital inmovilizan los bancos y qué límites tiene la toma de riesgo. Y las dos historias que cierran el capítulo (LTCM y 2008) son las mejores clases de humildad cuantitativa jamás dictadas.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(42)
```

12.1. Stress testing: preguntar "¿y si...?"

El VaR mira la distribución *estimada con datos recientes*. El **stress testing** hace la pregunta complementaria: *¿cuánto perdería la cartera si se repitiera*

un escenario extremo concreto? No pregunta por probabilidades — pregunta por **consecuencias**.

Hay dos familias de escenarios:

- **Históricos**: replicar episodios reales (crash de 1987, crisis de 2008, COVID de marzo 2020) sobre la cartera actual.
- **Hipotéticos**: construir escenarios que no ocurrieron pero podrían ("el petróleo cae 50 % y la tasa sube 300 puntos básicos").

```
# Una cartera simple: acciones, bonos y commodities
cartera = pd.Series({"Acciones": 6_000_000, "Bonos": 3_000_000, "Commodities": 1_000

# Escenarios de estrés: shock (%) aplicado a cada clase de activo
escenarios = pd.DataFrame({
    "Crash 1987":        {"Acciones": -0.23, "Bonos": +0.02, "Commodities": -0.05},
    "Crisis 2008 (oct)": {"Acciones": -0.17, "Bonos": -0.05, "Commodities": -0.20},
    "COVID mar-2020":    {"Acciones": -0.12, "Bonos": +0.01, "Commodities": -0.25},
    "Suba de tasas":     {"Acciones": -0.08, "Bonos": -0.12, "Commodities": +0.03},
}).T

perdidas = escenarios @ cartera

print(f"Cartera total: ${cartera.sum():,.0f}\n")
print("Pérdida por escenario de estrés:")
for esc, p in perdidas.items():
    print(f"  {esc:<18} ${p:>12,.0f}  ({p / cartera.sum():>6.1%})")
```

```
Cartera total: $10,000,000

Pérdida por escenario de estrés:
  Crash 1987         $  -1,370,000  (-13.7%)
  Crisis 2008 (oct)  $  -1,370,000  (-13.7%)
  COVID mar-2020     $    -940,000  ( -9.4%)
```

```
Suba de tasas      $    -810,000  ( -8.1%)
```

La tabla de arriba es, en miniatura, lo que los reguladores exigen a los bancos a escala gigante: los **CCAR/DFAST** en EEUU y los stress tests del BCE someten los balances completos a escenarios macro adversos de varios años. Un banco que "no aprueba" enfrenta restricciones para pagar dividendos o recomprar acciones.

> **VaR y stress testing se complementan**
>
> El VaR dice *"esto es lo que puede pasar en un día normal-malo, según los datos recientes"*. El stress test dice *"esto es lo que pasaría si vuelve 2008, tengan los datos recientes la pinta que tengan"*. El primero es probabilístico y miope; el segundo es determinístico y de memoria larga. Un buen tablero de riesgo muestra ambos.

12.2. Basilea III: la arquitectura regulatoria

Los **Acuerdos de Basilea** son los estándares globales de regulación bancaria, emitidos por el Comité de Basilea (BIS). La versión III, respuesta directa a 2008, se apoya en tres pilares conceptuales: capital de mejor calidad, límites al apalancamiento y —por primera vez— reglas de liquidez.

La Figura 12.1 es el mapa de esta sección: qué riesgos mide el regulador, con qué herramienta, y en qué ratio termina cada uno.

12.2.1. Capital: el colchón contra pérdidas

La idea central: los activos de un banco pueden perder valor; el **capital propio** es el colchón que absorbe esas pérdidas antes de que las paguen

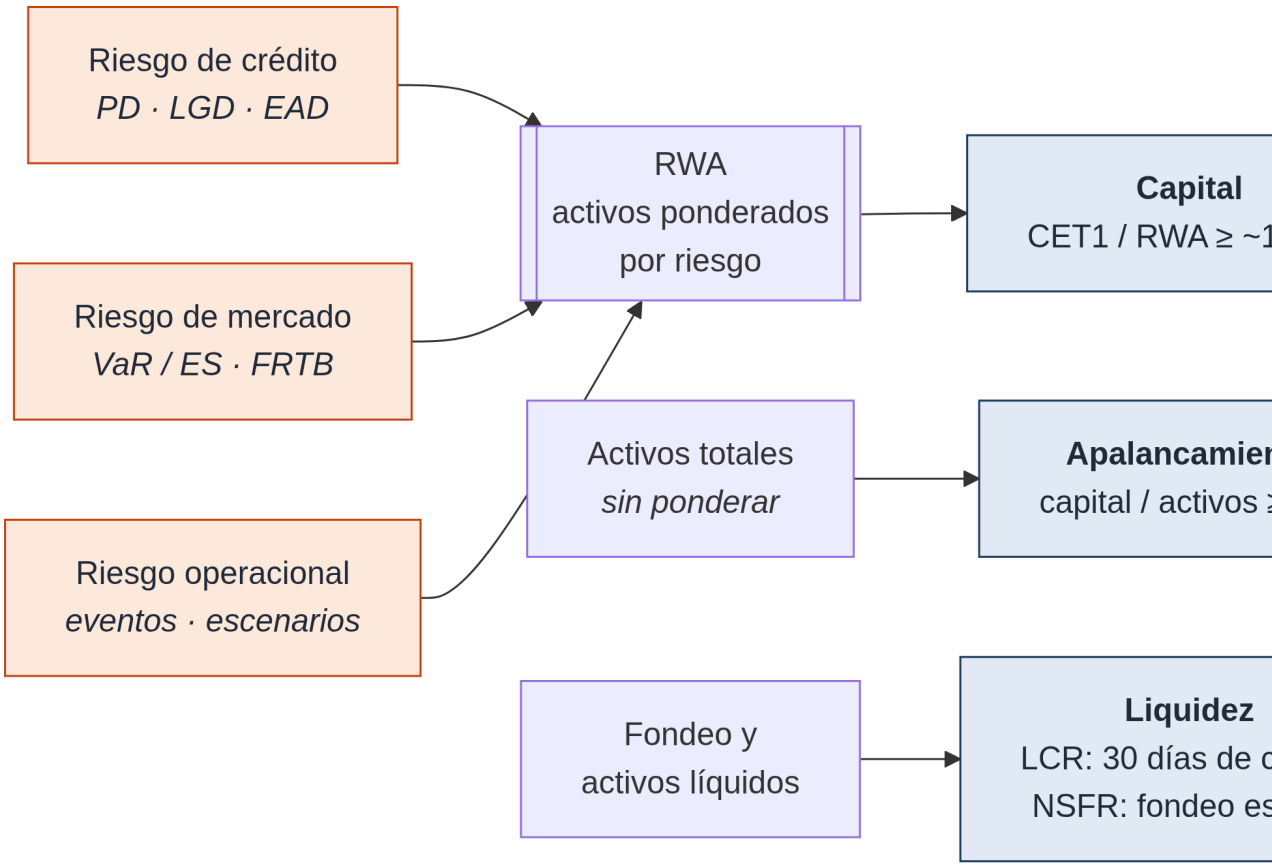


Figura 12.1.: La arquitectura de Basilea III en una página. Los tres tipos de riesgo determinan los activos ponderados por riesgo (RWA); el capital se mide contra ellos y contra los activos totales; la liquidez tiene sus propios ratios; el stress test proyecta todo bajo un escenario adverso.

los depositantes. Basilea exige un mínimo de capital **en proporción al riesgo de los activos**:

$$\text{Ratio de capital} = \frac{\text{Capital}}{\text{RWA}} \geq \text{mínimos regulatorios}$$

donde **RWA** (*Risk-Weighted Assets*) son los activos ponderados por riesgo: un bono del tesoro pondera cerca de 0 %, una hipoteca ~35-50 %, un préstamo corporativo riesgoso 100 % o más. Mil millones en letras del tesoro exigen mucho menos capital que mil millones en préstamos a startups — razonable.

```
# Balance simplificado de un banco
activos = pd.DataFrame({
    "monto":       [400, 300, 200, 100],     # en millones
    "ponderacion": [0.00, 0.35, 0.75, 1.00], # peso por riesgo (Basilea, aprox.)
}, index=["Bonos soberanos", "Hipotecas", "Préstamos PyME", "Préstamos corporativos"

activos["RWA"] = activos["monto"] * activos["ponderacion"]

capital = 60  # millones de capital CET1

print(activos)
print(f"\nActivos totales:  ${activos['monto'].sum():,.0f}M")
print(f"RWA totales:      ${activos['RWA'].sum():,.0f}M")
print(f"Ratio CET1:       {capital / activos['RWA'].sum():.1%}  (mínimo con colchone
print(f"Ratio de apalancamiento: {capital / activos['monto'].sum():.1%}  (mínimo: 3%
```

```
                       monto  ponderacion    RWA
Bonos soberanos          400         0.00    0.0
Hipotecas                300         0.35  105.0
Préstamos PyME           200         0.75  150.0
Préstamos corporativos   100         1.00  100.0
```

```
Activos totales:  $1,000M
RWA totales:      $355M
Ratio CET1:       16.9%  (mínimo con colchones: ~10.5%)
Ratio de apalancamiento: 6.0%  (mínimo: 3%)
```

Fijate el contraste entre los dos ratios finales: el de capital usa RWA (sensible al riesgo), el de **apalancamiento** usa activos totales sin ponderar. Basilea III agregó el segundo justamente porque en 2008 varios bancos lucían ratios de capital sólidos… calculados sobre RWA que subestimaban el riesgo real.

La Figura 12.2 muestra la transformación: cada bloque del balance se achica según su ponderación, y el mismo capital se mide contra dos denominadores muy distintos.

12.2.2. Liquidez: sobrevivir a la corrida

2008 demostró que un banco puede ser *solvente* y morir igual por *iliquidez*: si el fondeo de corto plazo desaparece de un día para otro, no hay capital que alcance. Basilea III introdujo dos ratios:

- **LCR** (*Liquidity Coverage Ratio*): tener activos líquidos de alta calidad suficientes para sobrevivir **30 días** de fuga de depósitos y fondeo.
- **NSFR** (*Net Stable Funding Ratio*): que los activos de largo plazo estén fondeados con pasivos estables, no con deuda que vence mañana.

12.2.3. PPNR: la otra mitad del stress test

En los stress tests regulatorios, la pérdida de capital de un banco no depende solo de cuánto pierden sus activos, sino de cuánto **sigue ganando** su negocio durante la crisis. Esa capacidad de generar resultados se proyecta

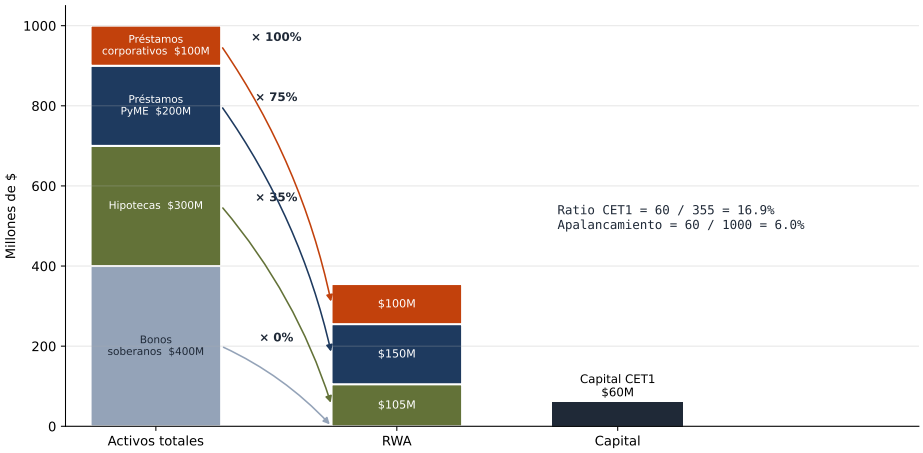


Figura 12.2.: Del balance a los RWA. Cada tipo de activo se multiplica por su ponderación de riesgo: los bonos soberanos desaparecen, los préstamos corporativos pesan completos. El mismo capital da un ratio CET1 alto (contra RWA) y uno de apalancamiento mucho menor (contra activos totales).

con el **PPNR** (*Pre-Provision Net Revenue*): ingresos netos por intereses + comisiones − gastos operativos, **antes** de provisiones por incobrables.

$$\Delta\text{Capital} \approx \text{PPNR} - \text{Pérdidas crediticias} - \text{Pérdidas de trading} - \text{Impuestos y dividendos}$$

```
# Proyección estilizada a 9 trimestres (formato CCAR) bajo escenario adverso
trimestres = np.arange(1, 10)

ppnr = 2.0 - 0.15 * trimestres            # el negocio se deteriora gradualmente
perdidas_credito = 1.0 + 1.5 * np.exp(-0.5 * (trimestres - 4) ** 2 / 4)  # pico de m
capital_inicial = 60

capital = capital_inicial + np.cumsum(ppnr - perdidas_credito)

fig, ax = plt.subplots(figsize=(9, 4.5))
ax.plot(trimestres, capital, "o-", linewidth=2, label="Capital proyectado")
ax.axhline(45, color="#c2410c", linestyle="--", label="Mínimo regulatorio (ilustrati
ax.fill_between(trimestres, 0, ppnr * 10, alpha=0.15, color="#637238",
                label="PPNR (escala ×10)")
ax.set_xlabel("Trimestre del escenario adverso")
ax.set_ylabel("Capital ($M)")
ax.set_title("Anatomía de un stress test: el PPNR amortigua las pérdidas")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

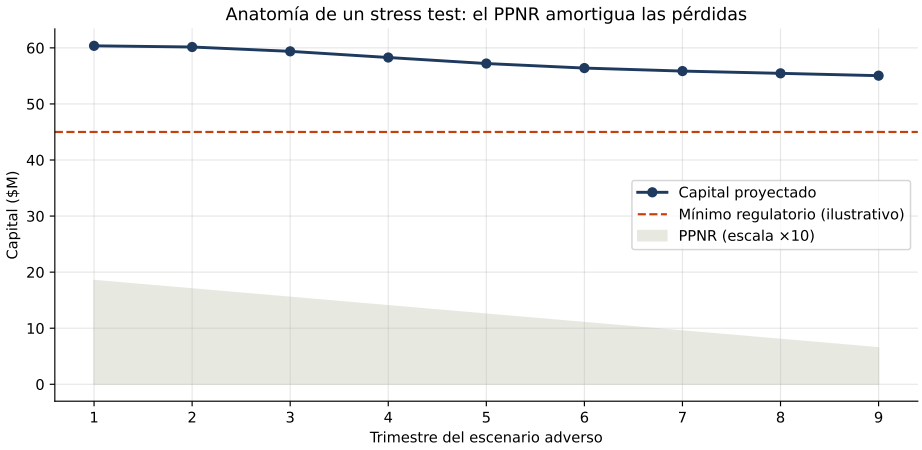


Figura 12.3.: Stress test estilizado a 9 trimestres: el capital cae mientras las pérdidas crediticias superan al PPNR, y el PPNR (área verde) es el amortiguador que lo mantiene por encima del mínimo.

La lección de la Figura 12.3: un banco con PPNR robusto puede absorber una ola de pérdidas crediticias sin acercarse al mínimo; uno con PPNR débil llega a la misma crisis sin amortiguador. Por eso los equipos de PPNR de los bancos grandes emplean pequeños ejércitos de modeladores.

12.2.4. Riesgo de mercado vs. riesgo operacional

Basilea clasifica los riesgos en tres grandes categorías, cada una con su carga de capital:

Riesgo	Qué es	Ejemplo	Herramienta típica
De crédito	La contraparte no paga	Mora hipotecaria	PD/LGD/EAD, provisiones
De mercado	Los precios se mueven en contra	Caída del portafolio de trading	VaR, ES (FRTB)
Operacional	Fallas de procesos, sistemas, personas	Fraude, cyberataque, *fat finger*	Bases de eventos, escenarios

El riesgo **operacional** es el más resbaladizo: no tiene "posición" que cerrar ni distribución de mercado que estimar. Casos célebres: Barings (1995, un solo trader ocultando pérdidas quebró un banco de 233 años), Société Générale (2008, €4.9bn), Knight Capital (2012, un deploy de software defectuoso perdió $440M en 45 minutos). Su gestión es más de procesos y controles que de matemática — pero su carga de capital es real.

12.3. Caso 1 — LTCM: cuando los genios fallan

Long-Term Capital Management era, en 1997, el hedge fund más prestigioso del planeta: fundado por el legendario trader John Meriwether, con **Myron Scholes y Robert Merton** (Nobel 1997, los del modelo de opciones) en el directorio. Su estrategia: *convergence trades* — apostar a que diferenciales de precio entre bonos casi idénticos (por ejemplo, un bono del tesoro recién emitido contra uno viejo) se achicarían. Diferenciales minúsculos, así que para volverlos rentables usaban un **apalancamiento de ~25:1**: con $4.7bn de capital controlaban más de $125bn en posiciones.

Los modelos decían que esas posiciones eran seguras: los diferenciales *siempre* convergían, y las posiciones estaban diversificadas entre mercados supuestamente independientes.

En agosto de 1998, **Rusia defaulteó su deuda**. Y ocurrió lo que los modelos no contemplaban: en la corrida global hacia la calidad, *todos* los diferenciales se ampliaron *a la vez*. Posiciones "independientes" en Japón, Europa y EEUU perdieron juntas: la correlación, que en los datos históricos era baja, **saltó hacia 1 exactamente cuando importaba**. Con 25:1 de apalancamiento, no hay margen para esperar la convergencia: los margin calls obligan a vender en el peor momento, ampliando aún más los diferenciales — una espiral. En pocas semanas LTCM perdió $4.4bn, y la Fed de Nueva York tuvo que coordinar un rescate privado para evitar que la liquidación forzada arrastrara al sistema.

Lo que mató a LTCM no fue una pérdida, sino un circuito que se alimenta solo. La Figura 12.4 lo dibuja: una vez que arranca, cada vuelta agranda la siguiente.

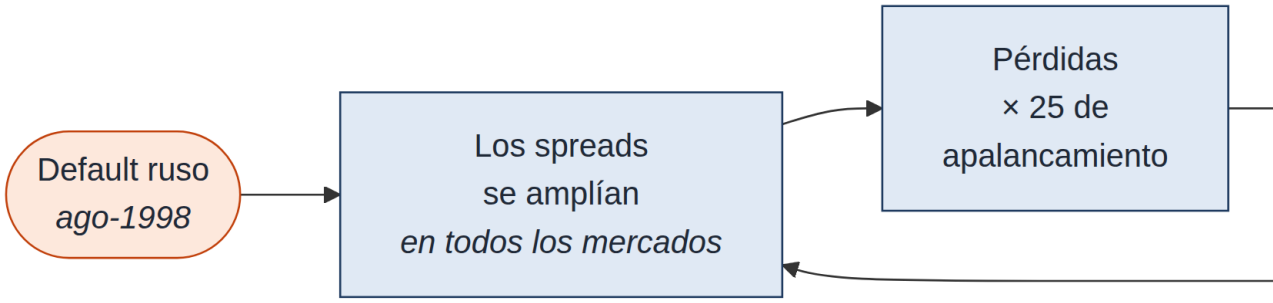


Figura 12.4.: La espiral de LTCM. Con apalancamiento 25:1, una ampliación de spreads se convierte en llamadas de margen; vender para cubrirlas amplía todavía más los spreads, porque todos los que tienen posiciones parecidas venden a la vez.

```
# La mecánica de LTCM en miniatura: apalancamiento + correlaciones que saltan
n_dias = 250
n_estrategias = 10          # "apuestas independientes" de convergencia
apalancamiento = 25

# Días normales: estrategias casi independientes (correlación 0.1), retorno positivo
# Crisis (día 150 en adelante): correlación 0.9 y drift negativo
def simular_pnl(rho, mu_d, n_dias, n_est, rng):
    L = np.linalg.cholesky(np.full((n_est, n_est), rho) + (1 - rho) * np.eye(n_est))
    shocks = rng.standard_normal((n_dias, n_est)) @ L.T
    return mu_d + 0.0015 * shocks

pnl_normal = simular_pnl(0.10, +0.0004, 150, n_estrategias, rng)
pnl_crisis = simular_pnl(0.90, -0.0025, 100, n_estrategias, rng)
pnl_total = np.vstack([pnl_normal, pnl_crisis]).mean(axis=1)  # cartera equiponderad

equity = 100 * np.cumprod(1 + apalancamiento * pnl_total)

fig, ax = plt.subplots(figsize=(9, 4.5))
ax.plot(equity, linewidth=2)
ax.axvline(150, color="#c2410c", linestyle="--", label="Default ruso: las correlacio
ax.set_xlabel("Día")
ax.set_ylabel("Capital (base 100)")
ax.set_title(f"LTCM estilizado: {n_estrategias} apuestas 'independientes' con apalan
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

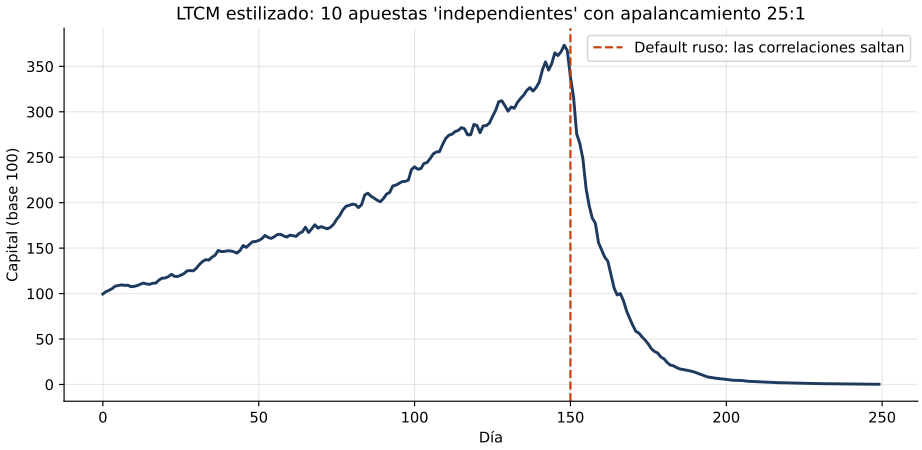


Figura 12.5.: LTCM estilizado: diez apuestas de convergencia casi independientes, apalancadas 25:1. Mientras la correlación es baja, el capital sube parejo; cuando salta a 0.9, las diez se vuelven una sola y el apalancamiento multiplica la caída.

```
print(f"Capital final: {equity[-1]:.0f} (arrancó en 100)")
```

```
Capital final: 0 (arrancó en 100)
```

Recorré la Figura 12.5 de izquierda a derecha: meses de retornos suaves y consistentes (la diversificación "funciona", el apalancamiento multiplica una ganancia estable) y luego el precipicio, cuando las diez apuestas se vuelven una sola y el 25:1 multiplica la pérdida. **Las lecciones de LTCM**, que la industria recita desde entonces:

1. **Las correlaciones históricas no son un contrato**: en las crisis convergen a 1, exactamente cuando la diversificación más se necesita.

2. **El apalancamiento convierte problemas de precio en problemas de supervivencia**: sin margen para esperar, tenés que vender en el peor momento.
3. **La liquidez es un riesgo en sí mismo**: LTCM tenía razón "a largo plazo" en muchas posiciones — pero como dijo Keynes, el mercado puede permanecer irracional más tiempo del que vos podés permanecer solvente.
4. **El prestigio de los modelos (y de los Nobel) no es gestión de riesgo.**

12.4. Caso 2 — 2008: la correlación falsa

La crisis de 2008 es demasiado grande para un capítulo; acá nos enfocamos en su **corazón cuantitativo**: cómo un supuesto de correlación mal calibrado, industrializado a escala de billones de dólares, hizo estallar el sistema.

12.4.1. La maquinaria: securitización y CDOs

La cadena, simplificada: los bancos originaban **hipotecas** (crecientemente *subprime*, de baja calidad), las empaquetaban en bonos (**MBS**), y luego empaquetaban esos bonos en **CDOs** (*Collateralized Debt Obligations*) divididos en **tramos**: el tramo *senior* cobra primero (y tenía calificación AAA), el *equity* absorbe las primeras pérdidas.

¿Cómo puede un paquete de hipotecas mediocres producir un tramo AAA? Por la **diversificación**: si los defaults son suficientemente *independientes* entre sí, la probabilidad de que fallen *muchas a la vez* —que es lo único que daña al tramo senior— es minúscula. Todo el edificio descansa

en una pregunta: **¿cuán correlacionados están los defaults de las hipotecas?**

La Figura 12.6 muestra la cascada de pérdidas: lo que pierde el pool se va apilando desde abajo, y cada tramo solo empieza a perder cuando el de abajo ya se perdió entero.

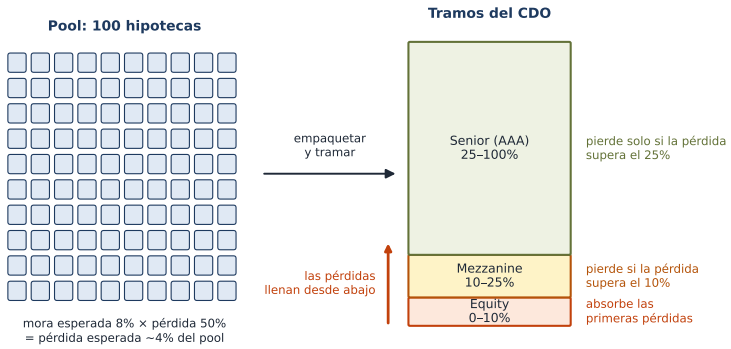


Figura 12.6.: La cascada de un CDO. Las pérdidas del pool de hipotecas llenan los tramos desde abajo: el equity absorbe las primeras, el mezzanine las siguientes, y el tramo senior (AAA) solo pierde si la pérdida total supera el 25 %. Con una pérdida esperada del 4 %, el senior parece intocable; la pregunta es cuán seguido aparece una pérdida mucho mayor que la esperada.

12.4.2. La fórmula: la cópula gaussiana

Para responderla, la industria adoptó masivamente la **cópula gaussiana** de David Li (2000): un modelo elegante que describe la dependencia entre defaults con un único parámetro de correlación ρ, calibrado con datos históricos… de años en que la vivienda en EEUU nunca había caído a escala

nacional. La calibración típica asumía correlaciones **bajas**: los defaults eran eventos idiosincráticos (alguien pierde su empleo), no sistémicos.

El error fatal: cuando los precios de la vivienda caen a nivel nacional, **todos los deudores subprime entran en problemas por la misma causa al mismo tiempo**. La correlación real en el escenario relevante no era 0.2: era cercana a 1.

```
# Un CDO de juguete: 100 hipotecas, tramo senior absorbe pérdidas > 25%
n_hipotecas, n_escenarios = 100, 50_000
prob_default, perdida_por_default = 0.08, 0.5   # 8% de mora esperada, pierde 50%

def perdida_cartera(rho, rng):
    """Modelo de un factor gaussiano: factor común (economía) + shock individual."""
    factor_comun = rng.standard_normal((n_escenarios, 1))
    idiosincratico = rng.standard_normal((n_escenarios, n_hipotecas))
    salud = np.sqrt(rho) * factor_comun + np.sqrt(1 - rho) * idiosincratico
    umbral = stats.norm.ppf(prob_default)
    defaults = salud < umbral
    return defaults.mean(axis=1) * perdida_por_default

print(f"{' ':>5}{'Pérdida media':>15}{'P(pérdida > 25%)':>20}   ¿El tramo senior (AA
for rho in [0.05, 0.20, 0.60]:
    perdidas = perdida_cartera(rho, np.random.default_rng(1))
    p_senior = (perdidas > 0.25).mean()
    print(f"{rho:>5.2f}{perdidas.mean():>14.1%}{p_senior:>19.2%}   "
          f"{'casi siempre' if p_senior < 0.005 else 'NO: riesgo real'}")
```

```
      Pérdida media    P(pérdida > 25%)   ¿El tramo senior (AAA) paga?
 0.05          4.0%              0.00%   casi siempre
 0.20          4.0%              0.09%   casi siempre
 0.60          4.0%              3.39%   NO: riesgo real
```

Mirá la tabla: la **pérdida promedio es la misma** en las tres filas (la correlación no cambia cuántas hipotecas caen *en promedio*), pero la probabilidad de que la pérdida atraviese el tramo senior **se multiplica por órdenes de magnitud** al subir ρ. Con ρ bajo, las pérdidas siempre rondan el promedio y el AAA es intocable; con ρ alto, la distribución se vuelve bimodal — o no pasa casi nada, **o caen todas juntas**. El mismo instrumento, la misma mora esperada… y calificaciones que pasaron de "prácticamente libre de riesgo" a basura en cuestión de meses de 2007-2008.

> **Simulador interactivo**
>
> En la versión web de este capítulo hay un simulador donde podés mover la correlación de los defaults y los tramos del CDO, y ver la distribución exacta de las pérdidas.

El resto de la historia se sigue de ahí: los tramos AAA estaban en todos lados (bancos, fondos, aseguradoras como AIG que los aseguraban vía CDS), nadie sabía quién tenía cuánto, el interbancario se congeló, Lehman cayó, y el mundo descubrió la diferencia entre *riesgo* (lo que el modelo mide) e *incertidumbre* (lo que el modelo ni siquiera contempla).

Las lecciones de 2008 para el modelador:

1. **Un parámetro calibrado en tiempos de calma no describe la crisis**: la correlación es *condicional al régimen*, y el régimen que importa es el peor.
2. **La elegancia matemática no valida los supuestos**: la cópula gaussiana era tratable y estándar; su supuesto central era falso.
3. **El riesgo de modelo se industrializa**: un modelo mediocre usado por una mesa es un problema local; usado por toda la industria para calificar billones, es sistémico.
4. **Las colas y la dependencia extrema importan más que el centro**: dos distribuciones con la misma media y varianza pueden

tener riesgos de cola radicalmente distintos — el puente perfecto hacia el próximo capítulo.

12.5. Errores comunes

⚠ Trampas conceptuales de este capítulo

- **Tratar la correlación como constante**: es la trampa que une LTCM y 2008. Estimá correlaciones condicionales a regímenes de estrés, no promedios históricos.
- **Confundir solvencia con liquidez**: podés tener razón en la valuación y quebrar igual si el fondeo desaparece antes de que el precio converja.
- **Leer los stress tests como pronósticos**: un escenario no es una predicción; es una linterna apuntada a una vulnerabilidad.
- **Creer que la regulación elimina el riesgo**: lo redistribuye y lo acota. Cada marco regulatorio genera, además, sus propios incentivos y puntos ciegos (el arbitraje regulatorio de los RWA fue parte del problema en 2008).

12.6. Practicá en el navegador

Podés experimentar con el código del capítulo acá mismo: la celda ejecuta Python **en tu navegador**, sin instalar nada (la primera ejecución tarda unos segundos mientras carga el entorno; los paquetes que importes se instalan solos).

```
#| label: practica-navegador
import numpy as np

rng = np.random.default_rng(42)

# Tu código acá...
```

12.7. Referencias

13. Capítulo 13: Simulación de Monte Carlo

Movimiento Browniano Geométrico y generación de escenarios

Cuando un problema no tiene fórmula cerrada, la simulación de Monte Carlo lo resuelve por fuerza bruta inteligente: generar miles de futuros posibles y estudiar su distribución. Presentamos el Movimiento Browniano Geométrico como modelo canónico del precio de un activo, simulamos trayectorias de forma vectorizada, calculamos VaR y Expected Shortfall por simulación, estudiamos la convergencia del método y lo aceleramos con variables antitéticas. Cerramos valuando una opción, el puente hacia el Módulo 4.

Introducción

Hay preguntas financieras que no tienen fórmula: *¿cuál es la distribución del valor de mi cartera en 5 años si rebalanceo cada mes? ¿cuánto vale una opción cuyo pago depende de la trayectoria del precio y no solo del valor final?* Para ese tipo de problemas existe el método de **Monte Carlo**, llamado así por el casino: en lugar de resolver el problema analíticamente, **tiramos los dados miles de veces** —simulamos miles de futuros posibles— y estudiamos la distribución de resultados.

La receta general tiene tres pasos:

1. **Modelar** la dinámica de la variable incierta (para nosotros: el precio de un activo).
2. **Simular** muchas trayectorias independientes de ese modelo.
3. **Agregar**: estimar lo que nos interese (media, percentiles, probabilidad de un evento) sobre las trayectorias.

El paso 1 exige elegir un modelo del precio. Empezamos por el más importante de todos: el **Movimiento Browniano Geométrico**.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(42)
```

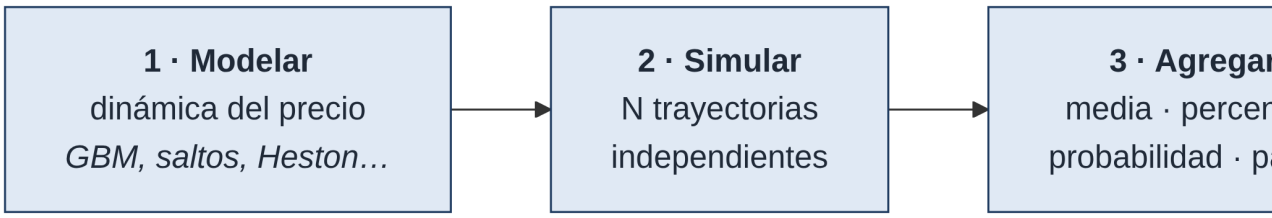


Figura 13.1.: La receta de Monte Carlo. El modelo es la única parte que cambia de un problema a otro; simular y agregar son siempre iguales. El resultado es una estimación, y por eso viene con su error estándar.

13.1. El Movimiento Browniano Geométrico (GBM)

13.1.1. La idea

¿Cómo modelar el precio de una acción? Dos observaciones empíricas nos guían:

- Los precios tienen una **tendencia** de largo plazo (el mercado sube "en promedio").
- Alrededor de esa tendencia hay **ruido** permanente e impredecible.

El GBM formaliza exactamente eso. En su forma de ecuación diferencial estocástica:

$$\frac{dS}{S} = \mu\, dt + \sigma\, dW$$

Leela por partes: el retorno instantáneo del activo (dS/S) tiene un componente determinístico $\mu\, dt$ (el **drift**, la tendencia) y un componente aleatorio $\sigma\, dW$ (la **difusión**: ruido gaussiano escalado por la volatilidad). Es literalmente "tendencia + ruido".

Para simular necesitamos la versión discreta. Integrando la ecuación (los detalles requieren cálculo estocástico, pero el resultado es lo que importa):

$$S_{t+\Delta t} = S_t \cdot \exp\left[(\mu - \tfrac{\sigma^2}{2})\Delta t + \sigma\sqrt{\Delta t}\, Z\right], \qquad Z \sim N(0,1)$$

> **¿De dónde sale el $-\,^2/2$?**
>
> Ese término sorprende la primera vez. La intuición: la exponencial es convexa, así que el ruido simétrico en el exponente produce resultados *asimétricos* en el precio (una suba del 10 % seguida de una baja del 10 % no te devuelve al punto de partida: $1.1 \times 0.9 = 0.99$). El término $-\sigma^2/2$ corrige exactamente ese efecto para que el retorno *esperado* del precio sea μ. Sin la corrección, la simulación tendría un sesgo alcista espurio.

La Figura 13.2 muestra el efecto con el ejemplo más chico posible: dos días, cada uno con +10 % o −10 % con igual probabilidad. El precio *medio* no se mueve, pero el resultado *típico* (la mediana) pierde. Esa brecha entre media y mediana es exactamente lo que el término $-\sigma^2/2$ lleva en cuenta.

Dos propiedades del GBM que conviene tener presentes:

- El precio **nunca se hace negativo** (es una exponencial) — realista para acciones.
- Los retornos logarítmicos son normales, así que el precio final es **lognormal**: asimétrico, con una cola larga hacia arriba.

13.1.2. Simulación vectorizada

Simulamos **todas las trayectorias a la vez** con una matriz de shocks: filas = pasos de tiempo, columnas = simulaciones. El único bucle recorre el tiempo (cada precio depende del anterior); las 10.000 simulaciones avanzan en paralelo gracias a la vectorización del Capítulo 5.

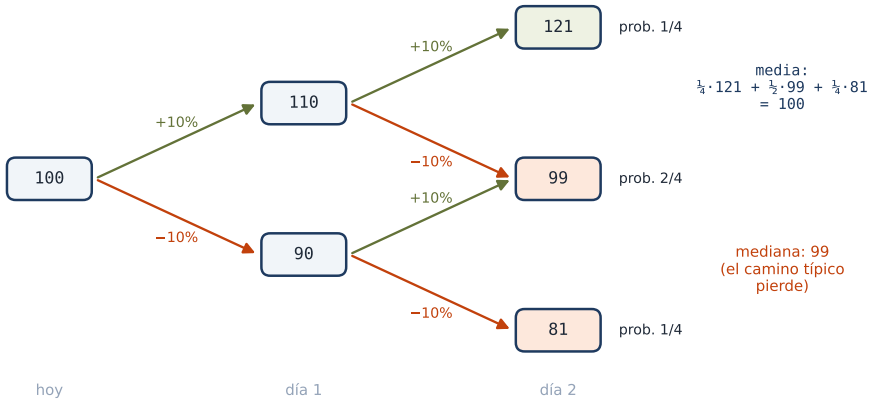


Figura 13.2.: Dos días de $\pm 10\,\%$ con probabilidad $1/2$ cada uno. La media de los cuatro caminos sigue en 100, pero tres de cuatro terminan en 99 o menos: ruido simétrico en los retornos produce un sesgo hacia abajo en el resultado típico.

La Figura 13.3 muestra la forma de los datos antes de escribir el código: una fila por día y una columna por futuro posible.

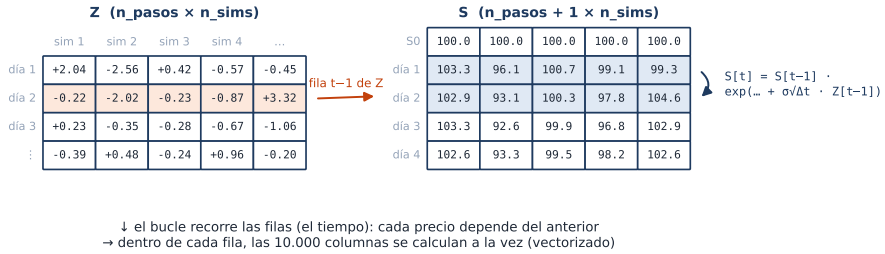


Figura 13.3.: Organización de la simulación. Z tiene un shock N(0,1) por día (fila) y por trayectoria (columna); S tiene una fila más, la del precio inicial. El bucle avanza fila por fila: cada fila de S se calcula con una sola operación vectorizada sobre la fila anterior y la fila de Z.

```
# Parámetros del activo y de la simulación
S0 = 100          # precio inicial
mu = 0.10         # drift anual (10%)
sigma = 0.25      # volatilidad anual (25%)
T = 1.0           # horizonte: 1 año
n_pasos = 252     # pasos diarios
n_sims = 10_000   # trayectorias

dt = T / n_pasos

# Matriz de shocks: un N(0,1) por paso y por simulación
Z = rng.standard_normal((n_pasos, n_sims))

# Simulación: el bucle recorre el tiempo; las columnas avanzan en paralelo
```

```
S = np.empty((n_pasos + 1, n_sims))
S[0] = S0
for t in range(1, n_pasos + 1):
    S[t] = S[t-1] * np.exp((mu - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z[t-1]

print(f"Precio inicial:       ${S0:.2f}")
print(f"Precio medio final:   ${S[-1].mean():.2f}   (teórico: ${S0 * np.exp(mu * T):
print(f"Precio mediano final: ${np.median(S[-1]):.2f}")
print(f"Rango simulado:       ${S[-1].min():.2f} - ${S[-1].max():.2f}")
```

```
Precio inicial:       $100.00
Precio medio final:   $110.78   (teórico: $110.52)
Precio mediano final: $107.23
Rango simulado:       $43.61 - $265.43
```

Verificación de cordura: la media simulada coincide con la teórica $E[S_T] = S_0 e^{\mu T}$. Notá también que la **mediana está por debajo de la media**: es la asimetría lognormal en acción (unas pocas trayectorias excelentes tiran el promedio hacia arriba).

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

# Panel 1: una muestra de trayectorias
ax = axes[0]
tiempo = np.arange(n_pasos + 1) / 252
ax.plot(tiempo, S[:, :100], alpha=0.25, linewidth=0.6)
ax.plot(tiempo, S0 * np.exp(mu * tiempo), "k--", linewidth=2, label="Tendencia $S_0
ax.set_title(f"100 trayectorias (de {n_sims:,})")
ax.set_xlabel("Años")
ax.set_ylabel("Precio ($)")
ax.legend()
ax.grid(True, alpha=0.3)
```

```
# Panel 2: distribución de precios finales
ax = axes[1]
ax.hist(S[-1], bins=80, density=True, alpha=0.7, color="#94a3b8", edgecolor="white")
ax.axvline(S[-1].mean(), color="#637238", linestyle="--",
           label=f"Media: ${S[-1].mean():.0f}")
ax.axvline(np.median(S[-1]), color="#c2410c", linestyle=":",
           label=f"Mediana: ${np.median(S[-1]):.0f}")
ax.set_title("Distribución de $S_T$ (lognormal)")
ax.set_xlabel("Precio final ($)")
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

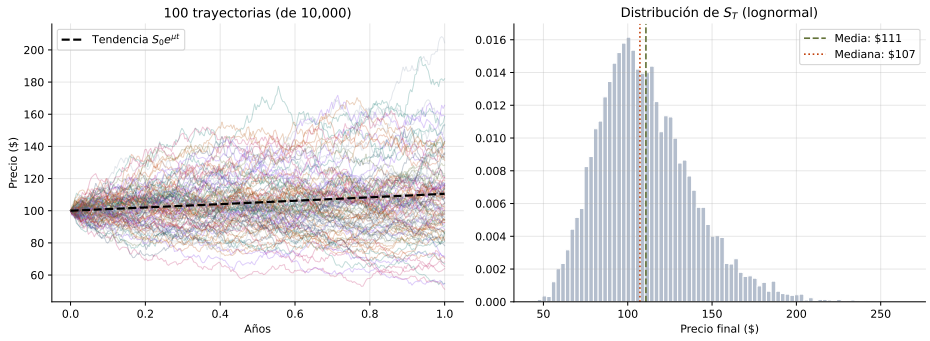


Figura 13.4.: Izquierda: 100 de las 10.000 trayectorias simuladas y la tendencia teórica. Derecha: la distribución de los precios a un año, lognormal, con la media por encima de la mediana.

El panel izquierdo de la Figura 13.4 es el "abanico de futuros": todas las trayectorias arrancan del mismo punto y se dispersan con el tiempo. El

derecho resume dónde terminan: la forma asimétrica con cola derecha larga es la distribución lognormal.

13.2. VaR y Expected Shortfall por Monte Carlo

En el Capítulo 11 calculamos VaR y ES con dos métodos: paramétrico (asume normal) e histórico (usa el pasado). Monte Carlo es el **tercer método**, y el más flexible: la distribución de pérdidas sale de las trayectorias simuladas, con el modelo que queramos. Hoy el modelo es GBM; en el próximo capítulo será uno con saltos — y el código de abajo no cambia.

```
valor_portafolio = 10_000_000
retornos_sim = S[-1] / S0 - 1     # retorno de cada trayectoria a 1 año

var_95 = -np.percentile(retornos_sim, 5) * valor_portafolio
umbral = np.percentile(retornos_sim, 5)
es_95 = -retornos_sim[retornos_sim <= umbral].mean() * valor_portafolio

print(f"Sobre un portafolio de ${valor_portafolio/1e6:.0f}M, horizonte 1 año:")
print(f"  VaR 95% (Monte Carlo): ${var_95:>12,.0f}")
print(f"  ES  95% (Monte Carlo): ${es_95:>12,.0f}")
```

```
Sobre un portafolio de $10M, horizonte 1 año:
  VaR 95% (Monte Carlo): $   2,921,024
  ES  95% (Monte Carlo): $   3,594,308
```

13.3. ¿Cuántas simulaciones son suficientes? Convergencia

Monte Carlo da una **estimación**, no un valor exacto, y su error decrece con la raíz del número de simulaciones: error $\propto 1/\sqrt{N}$. Esa raíz es una mala noticia práctica: para ganar **un dígito** de precisión hay que multiplicar las simulaciones **por 100**.

```
ns =np.unique(np.logspace(2, 4, 25).astype(int))
medias = [S[-1, :n].mean() for n in ns]
teorico = S0 * np.exp(mu * T)

# Banda de error teórica: ±2 desvíos del estimador de la media
sd_ST = S[-1].std()
banda = 2 * sd_ST / np.sqrt(ns)

fig, ax = plt.subplots(figsize=(9, 4.5))
ax.plot(ns, medias, "o-", color="#1e3a5f", markersize=4, label="Media estimada")
ax.axhline(teorico, color="#c2410c", linestyle="--", label=f"Valor teórico (${teoric
ax.fill_between(ns, teorico - banda, teorico + banda, alpha=0.15, color="#c2410c",
                label="Banda ±2 errores estándar")
ax.set_xscale("log")
ax.set_xlabel("Número de simulaciones (escala log)")
ax.set_ylabel("Precio medio final ($)")
ax.set_title("Convergencia de Monte Carlo: el error cae como 1/√N")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

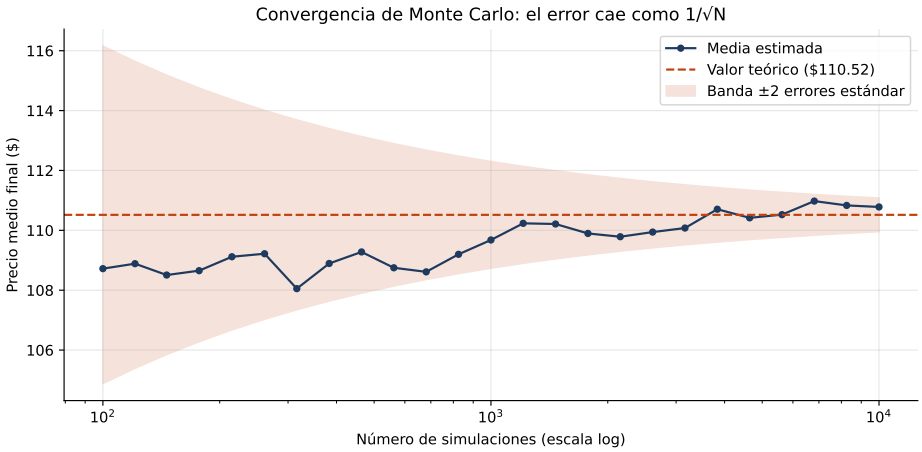


Figura 13.5.: Estimación de E[S_T] con cada vez más simulaciones. La estimación oscila dentro de la banda de ±2 errores estándar, que se angosta como $1/\sqrt{N}$.

En la Figura 13.5, la estimación "baila" dentro de la banda teórica y se aquieta a medida que N crece. Regla práctica: reportá siempre el **error estándar** de tu estimación Monte Carlo ($s/\sqrt{N}$), no solo el número.

13.3.1. Reducción de varianza: variables antitéticas

¿Se puede lograr más precisión sin más simulaciones? Sí — hay toda una familia de técnicas de **reducción de varianza**. La más simple y elegante: las **variables antitéticas**. La idea: por cada shock Z que uses, usá también $-Z$. Cada trayectoria "optimista" queda emparejada con su gemela "pesimista", los errores de ambas se compensan, y el promedio del par es mucho más estable que dos trayectorias independientes.

La Figura 13.6 muestra tres de esos pares: cada trayectoria tiene una gemela que recibe exactamente los shocks opuestos y queda reflejada del

otro lado de la tendencia.

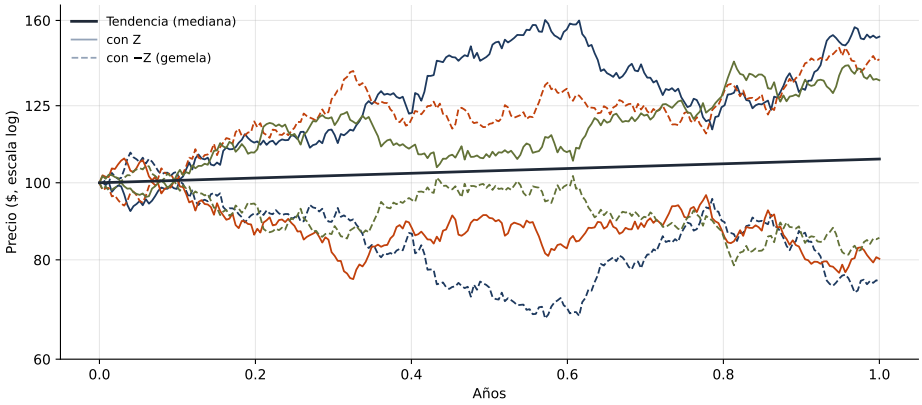


Figura 13.6.: Tres pares antitéticos: la línea llena usa los shocks Z y la punteada usa −Z. Cada par queda en espejo alrededor de la tendencia (en escala logarítmica), así que los desvíos de uno compensan los del otro.

```python
def precio_final_gbm(Z):
    """Precio final S_T dado un vector de shocks acumulados por trayectoria."""
    log_ST = (np.log(S0) + (mu - 0.5 * sigma**2) * T
              + sigma * np.sqrt(dt) * Z.sum(axis=0))
    return np.exp(log_ST)

n_pares = n_sims // 2
Z_half = rng.standard_normal((n_pasos, n_pares))

# Método estándar: n_sims trayectorias independientes
ST_std = precio_final_gbm(rng.standard_normal((n_pasos, n_sims)))

# Método antitético: n_pares trayectorias + sus espejos (mismo costo total)
ST_pos = precio_final_gbm(Z_half)
```

```python
ST_neg = precio_final_gbm(-Z_half)
ST_anti = 0.5 * (ST_pos + ST_neg)      # promedio por par

print(f"{'':24}{'Estimación':>12}{'Error estándar':>16}")
print(f"{'Estándar':<24}{ST_std.mean():>12.3f}{ST_std.std()/np.sqrt(n_sims):>16.4f}"
print(f"{'Antitético':<24}{ST_anti.mean():>12.3f}{ST_anti.std()/np.sqrt(n_pares):>16
print(f"\nValor teórico: {teorico:.3f}")
```

```
                         Estimación  Error estándar
Estándar                    110.682          0.2817
Antitético                  110.449          0.0671

Valor teórico: 110.517
```

Con el **mismo costo computacional**, el error estándar del método antitético es visiblemente menor. En simulaciones industriales (pricing de exóticos, XVA), estas técnicas marcan la diferencia entre horas y minutos.

> **i** Simulador interactivo
>
> En la versión web de este capítulo hay un laboratorio donde podés cambiar μ, σ, el horizonte y el número de simulaciones, y activar las variables antitéticas.

13.4. Puente al Módulo 4: valuar una opción por Monte Carlo

Cerremos con el uso más famoso de Monte Carlo en finanzas. Una **call europea** da derecho a comprar el activo a precio K en la fecha T; su pago

final es máx$(S_T - K, 0)$. Bajo valuación neutral al riesgo (lo desarrollamos en el Capítulo 18), su precio es el pago esperado descontado, **simulando con drift igual a la tasa libre de riesgo** r:

$$C = e^{-rT} \cdot E\big[\,\text{máx}(S_T - K, 0)\big]$$

```
r = 0.05      # tasa libre de riesgo
K = 105       # strike

# Simulamos S_T con drift r (mundo neutral al riesgo)
Z_rn = rng.standard_normal(200_000)
ST_rn = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * Z_rn)

payoff = np.maximum(ST_rn - K, 0)
precio_mc = np.exp(-r * T) * payoff.mean()
error_mc = np.exp(-r * T) * payoff.std() / np.sqrt(len(payoff))

# Comparación con la fórmula cerrada de Black-Scholes
d1 = (np.log(S0/K) + (r + 0.5*sigma**2)*T) / (sigma*np.sqrt(T))
d2 = d1 - sigma*np.sqrt(T)
precio_bs = S0 * stats.norm.cdf(d1) - K * np.exp(-r*T) * stats.norm.cdf(d2)

print(f"Precio call por Monte Carlo: ${precio_mc:.3f} ± {2*error_mc:.3f}")
print(f"Precio call Black-Scholes:   ${precio_bs:.3f}")
```

```
Precio call por Monte Carlo: $9.954 ± 0.076
Precio call Black-Scholes:   $10.002
```

La Figura 13.7 muestra qué está promediando la simulación: la mayoría de las trayectorias terminan por debajo del strike y pagan cero; el precio sale de las que terminan arriba.

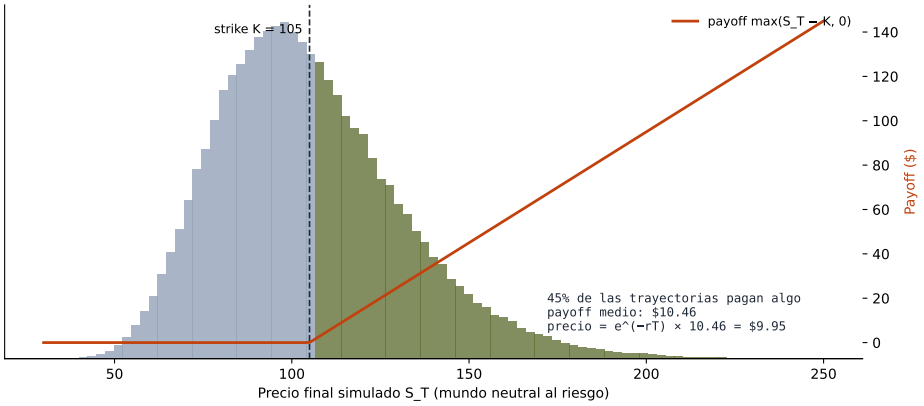


Figura 13.7.: Valuación de la call por Monte Carlo. Histograma de los precios finales simulados con drift r; solo las trayectorias a la derecha del strike (zona sombreada) pagan algo. El precio es el promedio del payoff de todas las trayectorias (incluidos los ceros), descontado.

La simulación reproduce la fórmula cerrada — y esa es justamente la gracia: para una call europea Black-Scholes ya existe, pero para opciones **path-dependent** (asiáticas, barrera) o con modelos más ricos que el GBM, la fórmula no existe y **Monte Carlo es el camino**. Volveremos sobre esto en el Módulo 4.

13.5. Errores comunes

⚠ Trampas frecuentes

- **Olvidar el término** $-\sigma^2/2$ en el exponente: infla sistemáticamente los precios simulados.
- **Confundir mundo real y mundo neutral al riesgo**: para *proyectar* escenarios usá el drift real μ; para *valuar* derivados usá r. Mezclarlos da precios de opciones sin sentido.
- **Reportar la estimación sin su error estándar**: dos corridas con semillas distintas pueden diferir; el error estándar te dice cuánto es esperable.
- **Reusar los mismos shocks para todo**: si comparás dos estrategias, simulalas sobre los mismos shocks (para aislar la diferencia) — pero si estimás la incertidumbre, necesitás muestras independientes. Saber cuál corresponde a cada caso es parte del oficio.
- **Escalar mal el tiempo**: σ anual con pasos diarios exige $\sqrt{\Delta t} = \sqrt{1/252}$ en el ruido. Un Δt inconsistente distorsiona toda la dinámica.

13.6. Practicá en el navegador

Podés experimentar con el código del capítulo acá mismo: la celda ejecuta Python **en tu navegador**, sin instalar nada (la primera ejecución tarda unos segundos mientras carga el entorno; los paquetes que importes se instalan solos).

```
#| label: practica-navegador
import numpy as np

rng = np.random.default_rng(42)

# Tu código acá...
```

13.7. Referencias

14. Capítulo 14: Procesos Estocásticos del Precio de un Activo

Más allá del Browniano: saltos, Lévy y colas gruesas

El Movimiento Browniano Geométrico es elegante pero miente en lo importante: sus retornos son normales, sin saltos y con colas finas, mientras que los mercados reales se desploman de a saltos y tienen colas pesadas. Extendemos el modelo con el jump-diffusion de Merton, presentamos la familia de procesos de Lévy, y comparamos las distribuciones de retornos simuladas contra el GBM y contra la firma estadística de los datos reales.

Introducción

En el capítulo anterior simulamos precios con el Movimiento Browniano Geométrico y todo funcionó de maravilla. Ahora toca la pregunta incómoda: **¿el GBM describe bien los precios reales?**

La respuesta corta es *no en las colas*. Los retornos reales exhiben tres características —los *hechos estilizados* que ya asomaron en el Capítulo 7— que el GBM no puede producir:

1. **Colas pesadas**: los movimientos extremos ocurren muchísimo más seguido de lo que predice la normal. El crash de 1987 ($-20\,\%$ en un día) tiene, bajo normalidad, una probabilidad tan pequeña que no debería haber ocurrido ni una vez en la edad del universo. Ocurrió un martes.
2. **Saltos**: los precios no se mueven solo en pasitos continuos; a veces **saltan** — un resultado electoral, un default, un anuncio de la Fed. El GBM, por construcción, tiene trayectorias continuas.
3. **Asimetría**: las caídas violentas son más frecuentes y más grandes que las subas violentas.

Este capítulo construye modelos que sí capturan esto, quedándonos en el terreno de **modelar el precio del activo** (la volatilidad estocástica, que ataca otro hecho estilizado, llega en el Capítulo 20 con Heston y SABR).

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
```

```
rng = np.random.default_rng(42)
```

14.1. El diagnóstico: qué le falta al GBM

Antes de proponer modelos nuevos, midamos la falla del viejo. Comparemos los retornos diarios de un GBM contra una serie con "días de shock" ocasionales (nuestra maqueta de los datos reales):

```
n_dias = 2500  # ~10 años

# GBM: retornos log normales puros
ret_gbm = rng.normal(0.0004, 0.012, n_dias)

# "Mercado real" estilizado: normal + shocks ocasionales grandes y sesgados a la baj
shocks = rng.random(n_dias) < 0.02                      # ~5 shocks por año
ret_real = np.where(shocks,
                    rng.normal(-0.02, 0.035, n_dias),   # días de salto
                    rng.normal(0.0006, 0.009, n_dias))  # días normales

print(f"{'':22}{'GBM':>10}{'Mercado real':>14}")
print(f"{'Volatilidad diaria':<22}{ret_gbm.std():>10.4f}{ret_real.std():>14.4f}")
print(f"{'Asimetría':<22}{stats.skew(ret_gbm):>10.2f}{stats.skew(ret_real):>14.2f}")
print(f"{'Curtosis (exceso)':<22}{stats.kurtosis(ret_gbm):>10.2f}{stats.kurtosis(ret
print(f"{'Peor día':<22}{ret_gbm.min():>10.2%}{ret_real.min():>14.2%}")
```

```
                            GBM  Mercado real
Volatilidad diaria       0.0121        0.0107
Asimetría                 -0.04         -1.52
```

Curtosis (exceso)	0.04	11.96
Peor día	-4.34%	-10.68%

Misma volatilidad total, pero el "mercado real" tiene curtosis alta, asimetría negativa y días mucho peores. Un gestor de riesgo que use el GBM para estimar el VaR de esta serie va a **subestimar sistemáticamente las colas** — el pecado capital del Capítulo 11.

Las tablas de momentos resumen, pero la herramienta visual clásica para diagnosticar colas es el **gráfico Q-Q**: enfrenta cada cuantil observado con el que tendría una normal. Si los datos son normales, los puntos caen sobre la recta; si tienen colas pesadas, los extremos se despegan. La Figura 14.1 lo aplica a las dos series.

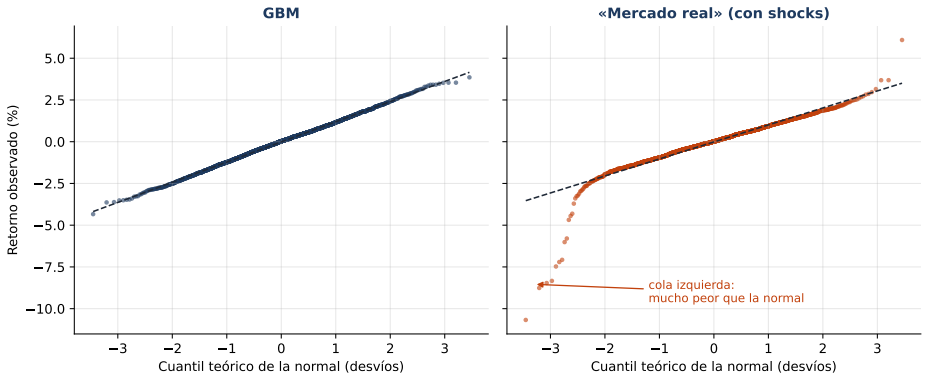


Figura 14.1.: Gráficos Q-Q contra la normal. Los retornos del GBM caen sobre la recta en toda su extensión. Los del «mercado real» se despegan en los extremos, sobre todo en la cola izquierda: los peores días son mucho peores de lo que la normal admite.

14.2. Jump-diffusion de Merton: difusión + saltos de Poisson

En 1976, Robert Merton (sí, el mismo de Black-Scholes y de LTCM — la historia tiene ironías) propuso la extensión natural: superponer al Browniano un **proceso de saltos**. El precio evoluciona como un GBM *casi todo el tiempo*, pero en momentos aleatorios llega un shock discreto:

$$\frac{dS}{S} = \mu\,dt + \sigma\,dW + (J-1)\,dN$$

Los tres ingredientes:

- $\mu\,dt + \sigma\,dW$: la difusión de siempre (tendencia + ruido fino).
- dN: un **proceso de Poisson** con intensidad λ — el "timbre" que suena en promedio λ veces por año, avisando que llegó un salto.
- J: el **tamaño del salto**, aleatorio; Merton lo modela lognormal: $\ln J \sim N(\mu_J, \sigma_J^2)$. Con $\mu_J < 0$, los saltos son mayormente caídas.

> **La intuición del Poisson**
>
> El proceso de Poisson es el modelo matemático de "eventos raros que llegan de sorpresa": no sabés *cuándo* llega el próximo salto, pero sí *cada cuánto* llegan en promedio. Con $\lambda = 5$, esperás unos 5 saltos por año, quizás 2 este año y 8 el próximo. Es el mismo objeto que modela llegadas de clientes o fallas de máquinas — acá modela pánicos y sorpresas.

La Figura 14.2 desarma una trayectoria en sus piezas: la difusión de siempre, el "timbre" de Poisson que cuenta los saltos, y la suma de ambas.

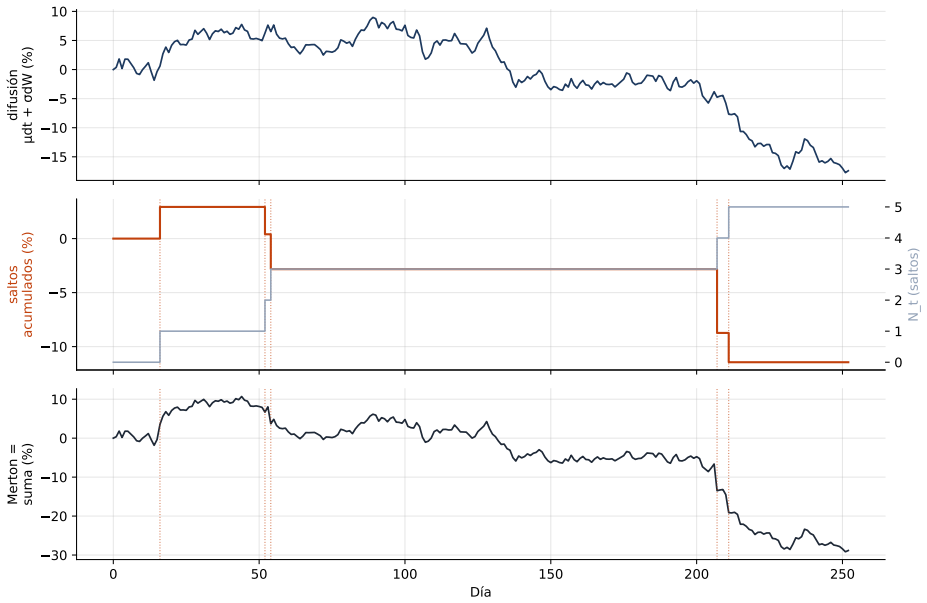


Figura 14.2.: Anatomía de una trayectoria de Merton (log-precio, un año). Arriba: la difusión continua del GBM. Al medio: el proceso de saltos; el contador N_t sube de a uno cada vez que suena el timbre de Poisson, y cada salto tiene un tamaño aleatorio. Abajo: la suma, que avanza suave y de pronto escalona.

14.2.1. Simulación

Discretizar es directo: en cada paso, la difusión avanza como siempre, y además sorteamos cuántos saltos ocurrieron en el intervalo (casi siempre 0, a veces 1):

```
def simular_merton(S0, mu, sigma, lam, mu_J, sigma_J, T, n_pasos, n_sims, rng):
    """Jump-diffusion de Merton. Devuelve la matriz de precios (pasos+1, sims)."""
    dt = T / n_pasos

    # Compensador: resta el efecto medio de los saltos para que el drift total sea m
    k = np.exp(mu_J + 0.5 * sigma_J**2) - 1

    S = np.empty((n_pasos + 1, n_sims))
    S[0] = S0
    for t in range(1, n_pasos + 1):
        Z = rng.standard_normal(n_sims)
        # ¿Cuántos saltos en este intervalo? (Poisson con media lambda*dt)
        n_saltos = rng.poisson(lam * dt, n_sims)
        # Suma de log-saltos (si hubo más de uno, se acumulan)
        log_J = np.where(n_saltos > 0,
                         rng.normal(mu_J * n_saltos, sigma_J * np.sqrt(np.maximum(n_
                         0.0)
        S[t] = S[t-1] * np.exp((mu - lam * k - 0.5 * sigma**2) * dt
                               + sigma * np.sqrt(dt) * Z + log_J)
    return S

# Parámetros: difusión tranquila + ~4 saltos por año, mayormente caídas del ~3%
S0, mu, sigma = 100, 0.10, 0.15
lam, mu_J, sigma_J = 4, -0.03, 0.05
T, n_pasos, n_sims = 1.0, 252, 5000

S_merton = simular_merton(S0, mu, sigma, lam, mu_J, sigma_J, T, n_pasos, n_sims, rng
```

```
# GBM de referencia con la MISMA volatilidad total, para comparar peras con peras
var_saltos = lam * (mu_J**2 + sigma_J**2)
sigma_total = np.sqrt(sigma**2 + var_saltos)

Z = rng.standard_normal((n_pasos, n_sims))
S_gbm = np.empty((n_pasos + 1, n_sims))
S_gbm[0] = S0
dt = T / n_pasos
for t in range(1, n_pasos + 1):
    S_gbm[t] = S_gbm[t-1] * np.exp((mu - 0.5 * sigma_total**2) * dt
                                   + sigma_total * np.sqrt(dt) * Z[t-1])

print(f"Vol. anual de la difusión:      {sigma:.0%}")
print(f"Vol. anual aportada por saltos: {np.sqrt(var_saltos):.1%}")
print(f"Vol. total (ambos modelos):     {sigma_total:.1%}")
```

```
Vol. anual de la difusión:      15%
Vol. anual aportada por saltos: 11.7%
Vol. total (ambos modelos):     19.0%
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5), sharey=True)

for ax, S, titulo in [(axes[0], S_gbm, "GBM (continuo)"),
                      (axes[1], S_merton, "Merton (difusión + saltos)")]:
    ax.plot(S[:, :8], alpha=0.8, linewidth=1)
    ax.set_title(titulo)
    ax.set_xlabel("Día")
    ax.grid(True, alpha=0.3)
axes[0].set_ylabel("Precio ($)")

plt.tight_layout()
plt.show()
```

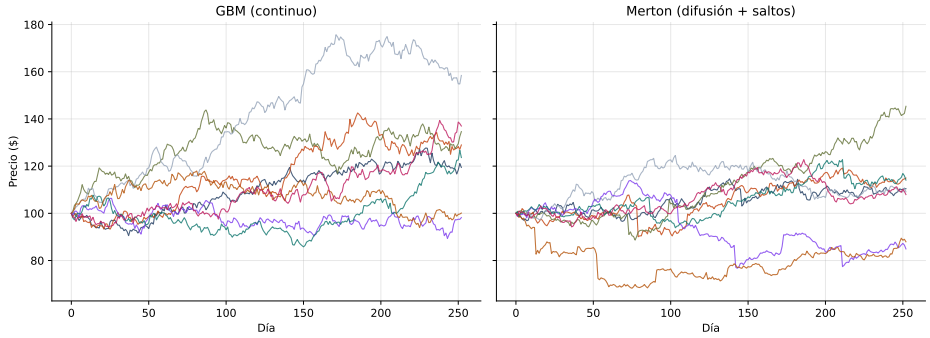


Figura 14.3.: Ocho trayectorias de cada modelo, con la misma volatilidad total. Las del GBM se mueven de a pasitos; las de Merton tienen escalones.

Se ve a simple vista en la Figura 14.3: las trayectorias del GBM son "peludas pero suaves"; las de Merton muestran **escalones** — el precio de pronto está en otro lado, sin pasar por el medio. Así se ve un evento de riesgo real.

14.2.2. La firma en las distribuciones

La diferencia decisiva está en la distribución de los retornos:

```
ret_m = np.log(S_merton[-1] / S0)
ret_g = np.log(S_gbm[-1] / S0)

fig, ax = plt.subplots(figsize=(9, 5))
bins = np.linspace(min(ret_m.min(), ret_g.min()), max(ret_m.max(), ret_g.max()), 80)
ax.hist(ret_g, bins=bins, density=True, alpha=0.55, label="GBM", color="#1e3a5f")
ax.hist(ret_m, bins=bins, density=True, alpha=0.55, label="Merton", color="#c2410c")
ax.set_yscale("log")   # escala log para VER las colas
ax.set_xlabel("Retorno logarítmico a 1 año")
```

```
ax.set_ylabel("Densidad (escala log)")
ax.set_title("Misma volatilidad total, colas muy distintas")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

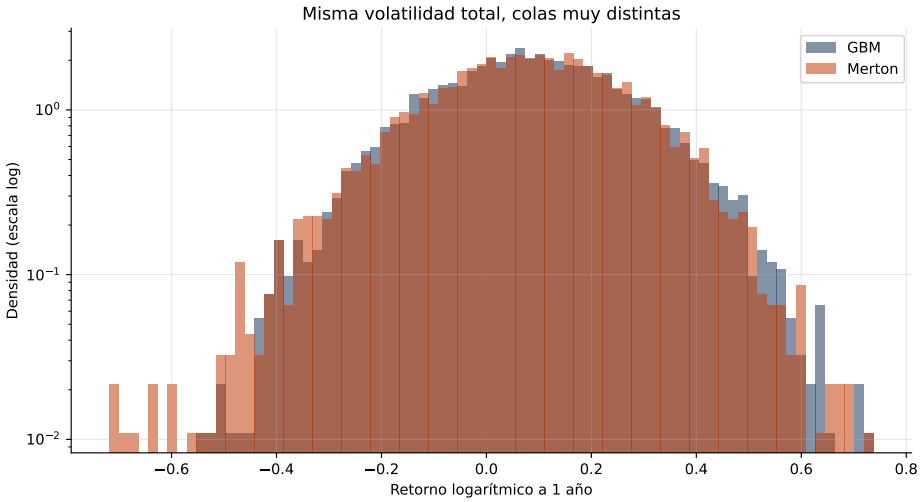


Figura 14.4.: Distribución de los retornos a un año, con el eje vertical en escala logarítmica para que se vean las colas. Misma varianza, pero la cola izquierda de Merton llega mucho más lejos.

```
print(f"{'':24}{'GBM':>10}{'Merton':>10}")
print(f"{'Asimetría':<24}{stats.skew(ret_g):>10.2f}{stats.skew(ret_m):>10.2f}")
print(f"{'Curtosis (exceso)':<24}{stats.kurtosis(ret_g):>10.2f}{stats.kurtosis(ret_m
print(f"{'Percentil 1%':<24}{np.percentile(ret_g, 1):>10.2%}{np.percentile(ret_m, 1
```

```
                        GBM     Merton
```

Asimetría	0.01	-0.19
Curtosis (exceso)	-0.09	0.22
Percentil 1%	-36.24%	-39.69%

En la Figura 14.4, la escala logarítmica del eje y revela lo que el histograma común esconde: la **cola izquierda de Merton se extiende mucho más lejos** con probabilidad no despreciable. Ambos modelos tienen la misma varianza — un optimizador de Markowitz los vería idénticos — pero el VaR y el ES del 1 % son muy distintos. Esta es la materialización de la lección de 2008 del capítulo anterior: **media y varianza no cuentan la historia de las colas**.

14.3. Procesos de Lévy: la familia general

El Browniano y el Poisson comparten una propiedad estructural: **incrementos independientes y estacionarios** (lo que pasa hoy no depende de ayer, y las reglas del juego no cambian con el tiempo). La familia completa de procesos con esa propiedad son los **procesos de Lévy**, y hay un teorema hermoso (Lévy-Itô) que dice que *todo* proceso de Lévy se descompone en exactamente tres partes:

$$X_t = \underbrace{bt}_{\text{tendencia}} + \underbrace{\sigma W_t}_{\text{difusión}} + \underbrace{\text{saltos}}_{\text{grandes y pequeños}}$$

Es decir: **tendencia + Browniano + saltos es todo lo que puede haber** (bajo incrementos independientes). El GBM usa solo las dos primeras piezas; Merton agrega saltos "raros y grandes"; y otros miembros de la familia usados en la industria empujan la idea más lejos:

- **Variance Gamma (VG)**: infinitos saltos pequeños y ninguna difusión — el precio se mueve *solo* por saltitos. Se construye elegantemente como un Browniano evaluado en un "reloj aleatorio" (subordinador Gamma): el tiempo del mercado corre más rápido en días agitados.
- **Normal Inverse Gaussian (NIG)**, **CGMY**: variantes con más control fino sobre el peso de cada cola.

```
# Variance Gamma por subordinación: Browniano en "tiempo de mercado" aleatorio
def simular_vg(theta, sigma_vg, nu, T, n_pasos, n_sims, rng):
    """Retornos log acumulados de un proceso Variance Gamma."""
    dt = T / n_pasos
    # El reloj aleatorio: incrementos Gamma con media dt y varianza nu*dt
    dG = rng.gamma(shape=dt / nu, scale=nu, size=(n_pasos, n_sims))
    # Browniano con drift evaluado en ese reloj
    dX = theta * dG + sigma_vg * np.sqrt(dG) * rng.standard_normal((n_pasos, n_sims)
    return dX.sum(axis=0)

ret_vg = simular_vg(theta=-0.10, sigma_vg=0.18, nu=0.4, T=1.0,
                    n_pasos=252, n_sims=5000, rng=rng)

print("Retornos a 1 año - Variance Gamma vs. GBM:")
print(f"{'':24}{'GBM':>10}{'VG':>10}")
print(f"{'Asimetría':<24}{stats.skew(ret_g):>10.2f}{stats.skew(ret_vg):>10.2f}")
print(f"{'Curtosis (exceso)':<24}{stats.kurtosis(ret_g):>10.2f}{stats.kurtosis(ret_v
```

```
Retornos a 1 año - Variance Gamma vs. GBM:
                              GBM        VG
Asimetría                    0.01     -0.58
Curtosis (exceso)           -0.09      1.43
```

La idea del "reloj aleatorio" es más fácil de ver que de leer. En la Figura 14.5, el reloj de mercado avanza en promedio igual que el calendario, pero a

tirones: a veces casi se detiene y a veces corre. El precio solo se mueve cuando corre ese reloj.

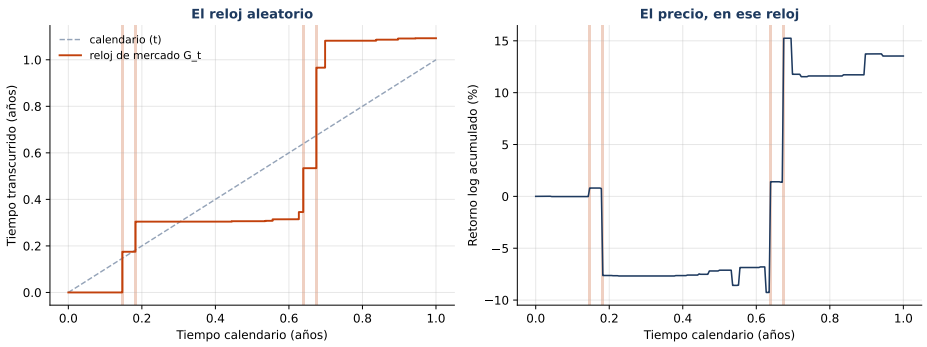


Figura 14.5.: Variance Gamma por subordinación. Izquierda: el «tiempo de mercado» G_t (incrementos Gamma) contra el tiempo calendario; en promedio coinciden, pero G_t avanza a saltos. Derecha: un Browniano evaluado en ese reloj. Los tramos donde el reloj corre son los días agitados; donde se frena, el precio casi no se mueve.

i ¿Por qué molestarse con estos modelos?

Porque el uso profesional los exige en dos frentes. **Riesgo**: un VaR/ES calculado con colas realistas (Capítulo 11 + este) es la diferencia entre estar preparado o no. **Derivados**: las opciones fuera del dinero cotizan "caras" respecto de Black-Scholes precisamente porque el mercado *sí* les pone precio a los saltos; los modelos de Lévy reproducen ese patrón (la *sonrisa* de volatilidad, que formalizamos en el Módulo 4).

14.4. ¿Qué modelo usar? Un mapa

La descomposición de Lévy-Itô funciona como un menú: cada modelo es una elección de qué ingredientes poner en la olla. La Figura 14.6 muestra qué toma cada uno.

Modelo	Colas	Saltos	Costo	Cuándo usarlo
GBM	Finas (normal)	No	Trivial	Primer boceto; horizontes largos donde el promedio manda
Merton	Pesadas	Sí, raros y grandes	Bajo	Riesgo de eventos; activos expuestos a anuncios/default
Variance Gamma / NIG	Pesadas, control fino	Infinitos, pequeños	Medio	Calibración fina a opciones; alta frecuencia
Heston / SABR (Cap. 20)	Pesadas vía vol estocástica	Opcional	Medio-alto	Superficies de volatilidad; pricing de exóticos

La regla general del oficio: **el modelo correcto depende de la pregunta**.

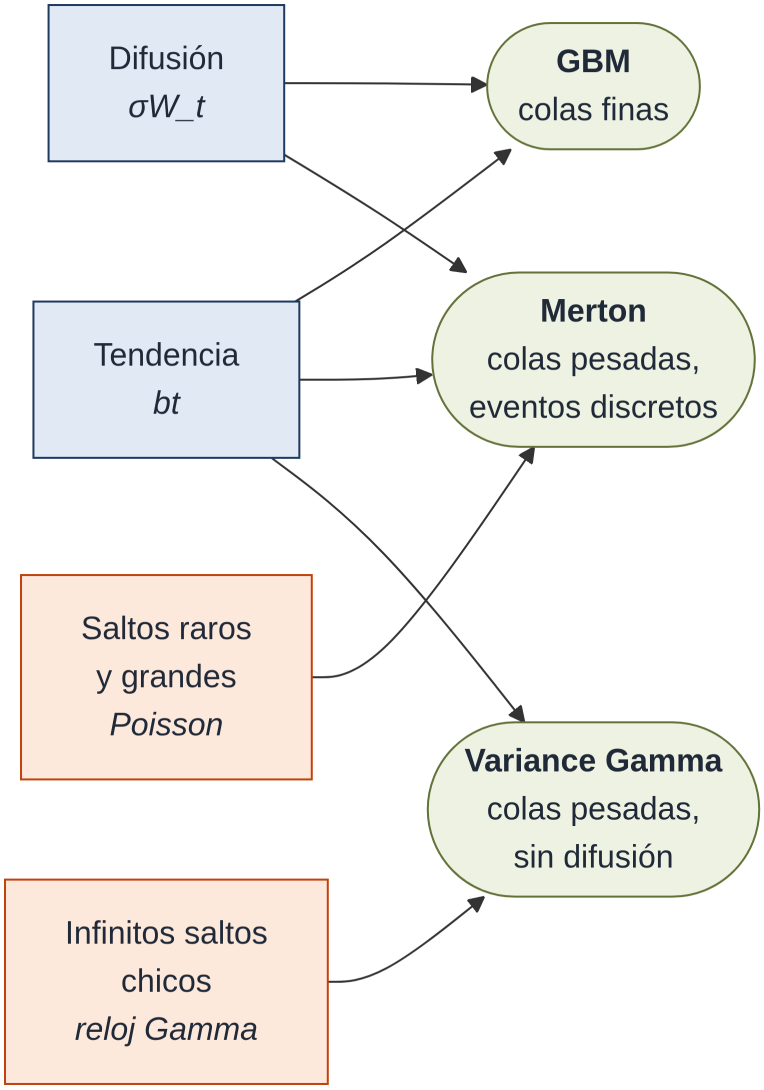


Figura 14.6.: Los modelos del capítulo como combinaciones de los tres ingredientes de Lévy-Itô. El GBM usa solo tendencia y difusión; Merton agrega saltos raros y grandes; Variance Gamma reemplaza la difusión por infinitos saltos chicos.

Para proyectar la jubilación a 30 años, el GBM alcanza. Para calcular el ES al 99 % de un book de opciones vendidas, usar GBM es negligencia.

14.5. Errores comunes

⚠ Trampas frecuentes

- **Olvidar el compensador** ($-\lambda k\,dt$) en Merton: sin él, los saltos negativos sesgan el drift y la media simulada no coincide con la teórica.
- **Calibrar los saltos con datos tranquilos**: si la muestra histórica no contiene crisis, el estimador de λ será ~0 y el modelo degenera en GBM. Los parámetros de salto suelen calibrarse con opciones (forward-looking), no solo con historia.
- **Comparar modelos con distinta varianza total**: para aislar el efecto de *forma* (colas, asimetría), igualá la varianza como hicimos acá; si no, confundís "más riesgo" con "riesgo distinto".
- **Confundir curtosis muestral con poblacional**: la curtosis estimada con pocas observaciones de un proceso con saltos es tremendamente ruidosa (¡depende de cuántos saltos cayeron en tu muestra!).

14.6. Practicá en el navegador

Podés experimentar con el código del capítulo acá mismo: la celda ejecuta Python **en tu navegador**, sin instalar nada (la primera ejecución tarda

unos segundos mientras carga el entorno; los paquetes que importes se instalan solos).

```
#| label: practica-navegador
import numpy as np

rng = np.random.default_rng(42)

# Tu código acá...
```

14.7. Referencias

15. Capítulo 15: Econometría y Modelos Lineales en Finanzas

De la regresión simple a los modelos multifactor

La regresión lineal es la herramienta más usada de las finanzas cuantitativas: con ella se estiman betas, se construyen coberturas, se descomponen retornos en factores y se testean estrategias. Este capítulo la desarrolla en serio: cómo especificar un modelo, qué supuestos lo sostienen, cómo leer coeficientes, errores estándar y p-values, y cómo aplicarla al CAPM, a los modelos multifactor y al cálculo de ratios de cobertura. Implementamos MCO desde cero y con statsmodels.

Introducción

Si tuvieras que quedarte con una sola herramienta estadística para trabajar en finanzas, sería la **regresión lineal**. Con ella se responde una familia enorme de preguntas que comparten la misma estructura: *¿cómo se mueve y cuando se mueve x?*

- ¿Cuánto cae mi acción cuando cae el mercado? (beta del CAPM)
- ¿Cuántos futuros necesito vender para cubrir mi cartera? (ratio de cobertura)
- ¿Mi fondo genera valor o solo cobra caro por exposición al mercado? (alfa)
- ¿Qué parte del retorno se explica por tamaño, valor, momentum? (modelos de factores)

Este capítulo construye la herramienta con el rigor necesario para usarla bien: no alcanza con llamar a una librería; hay que saber **qué supone el modelo, cuándo mienten los p-values y cómo se leen los resultados**.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.api as sm

rng = np.random.default_rng(42)
```

15.1. El modelo lineal: qué afirma exactamente

El modelo de regresión lineal postula que la variable a explicar y es una combinación lineal de las explicativas más un término de error:

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \cdots + \beta_k x_{ki} + \varepsilon_i$$

Tres aclaraciones que evitan malentendidos comunes:

1. **"Lineal" es en los coeficientes, no en las variables.** $y = \beta_0 + \beta_1 x^2$ es un modelo lineal perfectamente válido; podés transformar las variables como quieras.
2. **El error ε no es un defecto**: representa todo lo que afecta a y y no está en el modelo. La pregunta clave será qué propiedades tiene.
3. **Los β se interpretan "todo lo demás constante"**: β_1 es el efecto de x_1 sobre y *manteniendo fijas las otras variables*. Esta lectura condicional es lo que hace potente (y delicada) a la regresión múltiple.

15.1.1. Estimación por Mínimos Cuadrados Ordinarios (MCO)

¿Cómo elegir los β? El criterio de **MCO** (*OLS* en inglés): minimizar la suma de los errores al cuadrado. Para el caso simple ($y = \alpha + \beta x + \varepsilon$) la solución tiene forma cerrada:

$$\hat{\beta} = \frac{\mathrm{Cov}(x, y)}{\mathrm{Var}(x)}, \qquad \hat{\alpha} = \bar{y} - \hat{\beta}\bar{x}$$

La primera fórmula es reveladora: **el beta es una covarianza normalizada**. Ya la conocés de la teoría de portafolios — no es casualidad, es la misma matemática.

La Figura 15.1 muestra qué es exactamente lo que se minimiza: para cada punto, la distancia *vertical* hasta la recta (el residuo). MCO eleva cada residuo al cuadrado, los suma, y elige la recta que hace más chica esa suma.

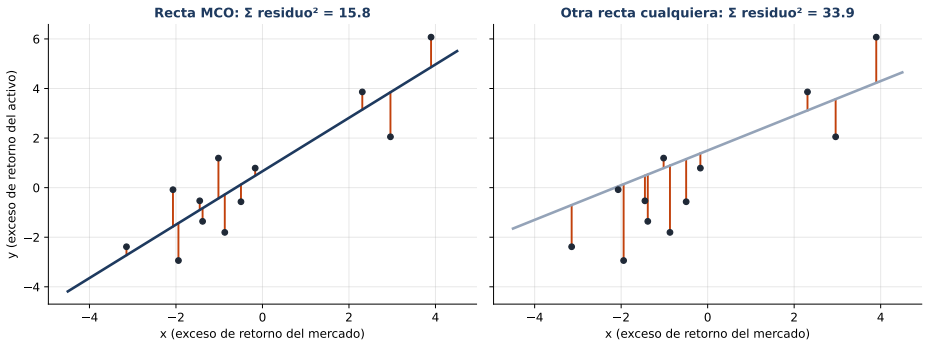


Figura 15.1.: Lo que minimiza MCO. Cada segmento vertical es un residuo (dato menos recta). La recta MCO (izquierda) es la que hace mínima la suma de los residuos al cuadrado; cualquier otra recta, como la de la derecha, deja una suma mayor.

Implementémosla desde cero para desmitificarla:

```
class RegresionLineal:
    """Regresión lineal simple por MCO, implementada desde cero."""

    def fit(self, x, y):
        x, y = np.asarray(x), np.asarray(y)
        self.n = len(x)
        x_mean, y_mean = x.mean(), y.mean()

        # Estimadores MCO
        self.beta = np.sum((x - x_mean) * (y - y_mean)) / np.sum((x - x_mean) ** 2)
        self.alpha = y_mean - self.beta * x_mean
```

```
        # Bondad de ajuste y errores estándar
        y_hat = self.alpha + self.beta * x
        residuos = y - y_hat
        ss_res = np.sum(residuos ** 2)
        ss_tot = np.sum((y - y_mean) ** 2)
        self.r2 = 1 - ss_res / ss_tot

        # Error estándar del beta y test t (H0: beta = 0)
        s2 = ss_res / (self.n - 2)                    # varianza residual
        self.se_beta = np.sqrt(s2 / np.sum((x - x_mean) ** 2))
        self.t_beta = self.beta / self.se_beta
        self.p_beta = 2 * (1 - stats.t.cdf(abs(self.t_beta), df=self.n - 2))
        return self

    def resumen(self):
        print(f"alpha = {self.alpha:.6f}")
        print(f"beta  = {self.beta:.4f}  (se: {self.se_beta:.4f}, "
              f"t: {self.t_beta:.2f}, p-value: {self.p_beta:.4f})")
        print(f"R²    = {self.r2:.4f}")
```

15.2. Aplicación 1: el CAPM y la estimación del beta

El **CAPM** (Capital Asset Pricing Model) afirma que el exceso de retorno esperado de un activo es proporcional al exceso de retorno del mercado:

$$E[R_i] - r_f = \beta_i \left(E[R_m] - r_f \right)$$

Su versión empírica es una regresión lineal sobre datos históricos:

$$\underbrace{R_{i,t} - r_f}_{y} = \alpha + \beta \underbrace{(R_{m,t} - r_f)}_{x} + \varepsilon_t$$

- β mide la **sensibilidad al mercado**: $=1.3$ significa "cuando el mercado sube 1 %, este activo sube 1.3 % en promedio".
- α es el retorno **no explicado por el mercado**: si es positivo y significativo, el activo rindió más de lo que su riesgo justifica. Todo gestor activo afirma tener alfa; la regresión permite verificarlo.

```
# Simulamos 10 años de datos mensuales con parámetros conocidos
n = 120
beta_real, alpha_real = 1.3, 0.002

exceso_mercado = rng.normal(0.005, 0.04, n)
exceso_activo = alpha_real + beta_real * exceso_mercado + rng.normal(0, 0.03, n)

modelo = RegresionLineal().fit(exceso_mercado, exceso_activo)
print("=== CAPM estimado (desde cero) ===")
modelo.resumen()
print(f"\nValores reales: alpha={alpha_real}, beta={beta_real}")
```

```
=== CAPM estimado (desde cero) ===
alpha = 0.001308
beta  = 1.1774  (se: 0.0878, t: 13.41, p-value: 0.0000)
R²    = 0.6038

Valores reales: alpha=0.002, beta=1.3
```

El beta se recupera con precisión; el alfa, no tanto (mirá su p-value en la salida de `statsmodels` más abajo). Esa asimetría no es un accidente de esta muestra, es una lección general: **detectar alfa exige muchísimos**

más datos que estimar beta, porque el alfa es una constante pequeña escondida en mucho ruido.

La Figura 15.2 muestra la regresión que acabamos de estimar: cada punto es un mes, y la pendiente de la recta es el beta.

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(exceso_mercado * 100, exceso_activo * 100, alpha=0.5, s=25)
x_line = np.linspace(exceso_mercado.min(), exceso_mercado.max(), 50)
ax.plot(x_line * 100, (modelo.alpha + modelo.beta * x_line) * 100, color="#c2410c",
        linewidth=2, label=f"Recta MCO:   = {modelo.beta:.2f}")
ax.set_xlabel("Exceso de retorno del mercado (%)")
ax.set_ylabel("Exceso de retorno del activo (%)")
ax.set_title(f"Security Characteristic Line - R² = {modelo.r2:.2f}")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

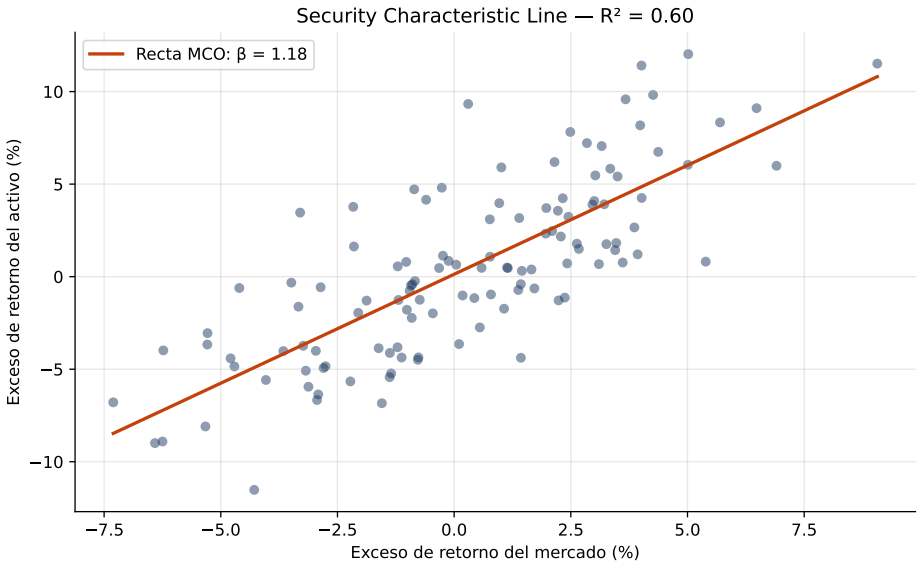


Figura 15.2.: Security Characteristic Line: 120 meses de excesos de retorno del activo contra los del mercado. La pendiente de la recta MCO es el beta; su ordenada en el origen, el alfa.

15.3. Leer una regresión: errores estándar, t y p-values

Los coeficientes estimados son **variables aleatorias**: con otra muestra habrían dado distinto. La inferencia estadística cuantifica esa incertidumbre, y saber leerla es lo que separa un análisis serio de uno decorativo.

- **Error estándar (se)**: el "desvío típico" del estimador. Un beta de 1.30 con se de 0.07 está bien clavado; con se de 0.60 no sabés casi

nada.

- **Estadístico t**: cuántos errores estándar dista el coeficiente de cero: $t = \hat{\beta}/se(\hat{\beta})$. Regla mental: $|t| > 2$ significativo al 5 %.
- **p-value**: la probabilidad de observar un coeficiente tan grande (en valor absoluto) **si el verdadero fuera cero**. Un p-value de 0.03 dice: "si el activo no tuviera relación con el mercado, ver este beta sería raro (3 %)".

Para ver que los coeficientes son variables aleatorias, alcanza con repetir el experimento. La Figura 15.3 simula 2000 muestras de 120 meses con los mismos α y β verdaderos del ejemplo y estima la regresión en cada una. El beta sale casi siempre cerca de 1.3; el alfa, en cambio, cae a ambos lados de cero muy seguido.

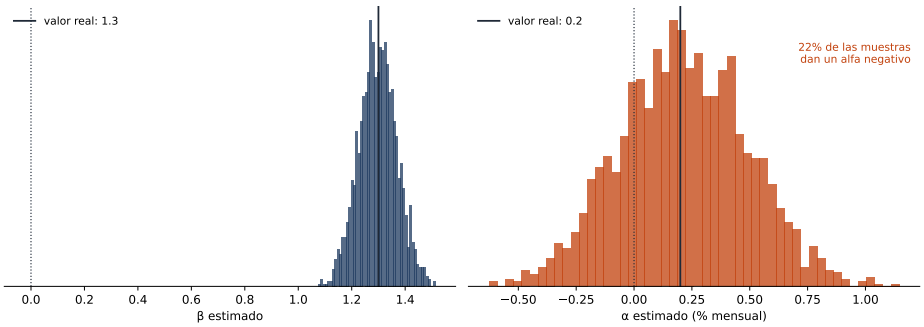


Figura 15.3.: Distribución de los estimadores en 2000 muestras simuladas de 120 meses (= 0.2 % mensual, = 1.3). El beta queda bien concentrado lejos de cero; el alfa verdadero es positivo, pero una parte grande de las estimaciones sale negativa: con 10 años de datos, el alfa se confunde con el ruido.

La Figura 15.4 traduce esa incertidumbre a un p-value, con el alfa de nuestra muestra. Si el alfa verdadero fuera cero, el estadístico t seguiría la campana de la figura; el p-value es el área de las dos colas más allá del t que observamos.

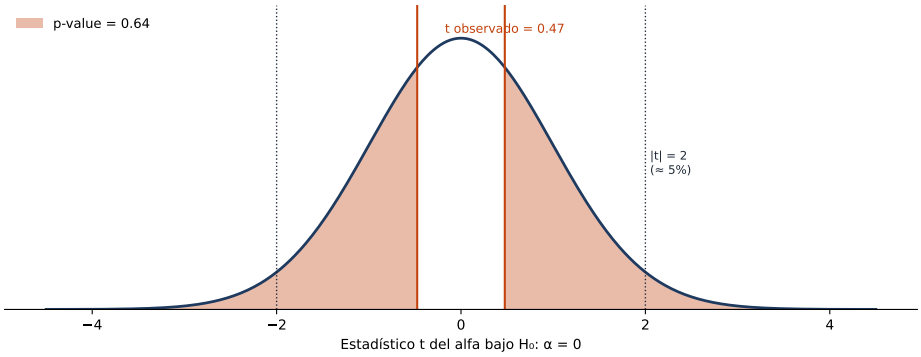


Figura 15.4.: El p-value del alfa estimado. La campana es la distribución t bajo la hipótesis = 0; las áreas sombreadas, más allá de ±t observado, suman el p-value. Las líneas en ±2 marcan la regla mental de significancia al 5 %.

Lo que el p-value NO dice

El p-value **no** es "la probabilidad de que el coeficiente sea cero", ni "la probabilidad de que el modelo esté mal". Es una afirmación sobre los *datos bajo una hipótesis*, no sobre la hipótesis. Y con miles de datos, coeficientes económicamente irrelevantes se vuelven "significativos": **significancia estadística relevancia económica**. Mirá siempre la magnitud del coeficiente además de su p-value. Esta distinción reaparecerá con fuerza en el Capítulo 17 (backtesting), donde probar muchas estrategias infla los falsos positivos.

Simulador interactivo

En la versión web de este capítulo hay un simulador donde podés cambiar los años de datos, el alfa verdadero y el ruido, y ver cuántas veces la regresión detecta el alfa.

15.3.1. La versión profesional: statsmodels

Nuestra clase enseña la mecánica; para trabajar en serio usamos `statsmodels`, que reporta toda la inferencia de una vez:

```
X = sm.add_constant(exceso_mercado)     # agrega la columna del intercepto
ols = sm.OLS(exceso_activo, X).fit()
print(ols.summary(xname=["alpha", "beta_mkt"]))
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.604
Model:                            OLS   Adj. R-squared:                  0.600
Method:                 Least Squares   F-statistic:                     179.8
Date:                Thu, 29 Oct 2026   Prob (F-statistic):           1.77e-
25
Time:                        02:29:14   Log-Likelihood:                251.24
No. Observations:                 120   AIC:                             -
498.5
Df Residuals:                     118   BIC:                             -
492.9
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
alpha          0.0013      0.003      0.475      0.636      -
0.004       0.007
beta_mkt       1.1774      0.088     13.410      0.000       1.004       1.351
==============================================================================
Omnibus:                        3.957   Durbin-Watson:                   1.575
Prob(Omnibus):                  0.138   Jarque-Bera (JB):                3.816
Skew:                           0.436   Prob(JB):                        0.148
```

```
Kurtosis:                       2.946   Cond. No.                         32.0
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly spe
```

De esta tabla, lo que hay que saber mirar: los **coeficientes** con sus intervalos de confianza (¿el intervalo del beta contiene 1? ¿el del alfa contiene 0?), el $\mathbf{R^2}$ (proporción de la varianza explicada), y los p-values. El resto (Durbin-Watson, Jarque-Bera de los residuos) son diagnósticos de supuestos — el tema siguiente.

15.4. Los supuestos: cuándo confiar en MCO

Los estimadores MCO son insesgados y de mínima varianza (**teorema de Gauss-Markov**) cuando se cumplen ciertos supuestos. En finanzas, algunos se violan sistemáticamente, y conviene saber cuáles y qué hacer:

Supuesto	Qué significa	¿Se cumple en finanzas?	Remedio usual
Linealidad	La relación es lineal en los	Aproximada-mente	Transformar variables
Exogeneidad	El error no está correlacionado con las x	El más delicado	Diseño / variables instrumentales
Homocedastici-dad	Varianza del error constante	**No** (la volatilidad se agrupa)	Errores robustos (HC/HAC)

Supuesto	Qué significa	¿Se cumple en finanzas?	Remedio usual
No autocorrelación	Errores independientes en el tiempo	**A veces no**	Errores Newey-West (HAC)
Normalidad	Errores normales (para inferencia exacta)	No en colas	Muestras grandes ayudan (TCL)

Las violaciones de homocedasticidad y autocorrelación **no sesgan los coeficientes**, pero **mienten los errores estándar** — y con ellos los p-values. La solución estándar en finanzas es usar errores robustos:

```
# Mismos coeficientes, errores estándar robustos a heterocedasticidad y autocorrelac
ols_robusto = sm.OLS(exceso_activo, X).fit(cov_type="HAC", cov_kwds={"maxlags": 6})

print("Comparación de errores estándar del beta:")
print(f"  Clásico (asume homocedasticidad): {ols.bse[1]:.4f}")
print(f"  Robusto (Newey-West, HAC):        {ols_robusto.bse[1]:.4f}")
```

```
Comparación de errores estándar del beta:
  Clásico (asume homocedasticidad): 0.0878
  Robusto (Newey-West, HAC):        0.0794
```

> **Regla práctica**
>
> En regresiones con series de tiempo financieras, reportá errores **Newey-West (HAC)** por defecto. Si los datos fueran ideales, no perdés nada; si no lo son (lo usual), tus p-values dejan de estar inflados.

La Figura 15.5 ordena todo lo anterior como una rutina de trabajo: la estimación es un paso rápido en el medio; lo que decide si el resultado es creíble está antes y después.

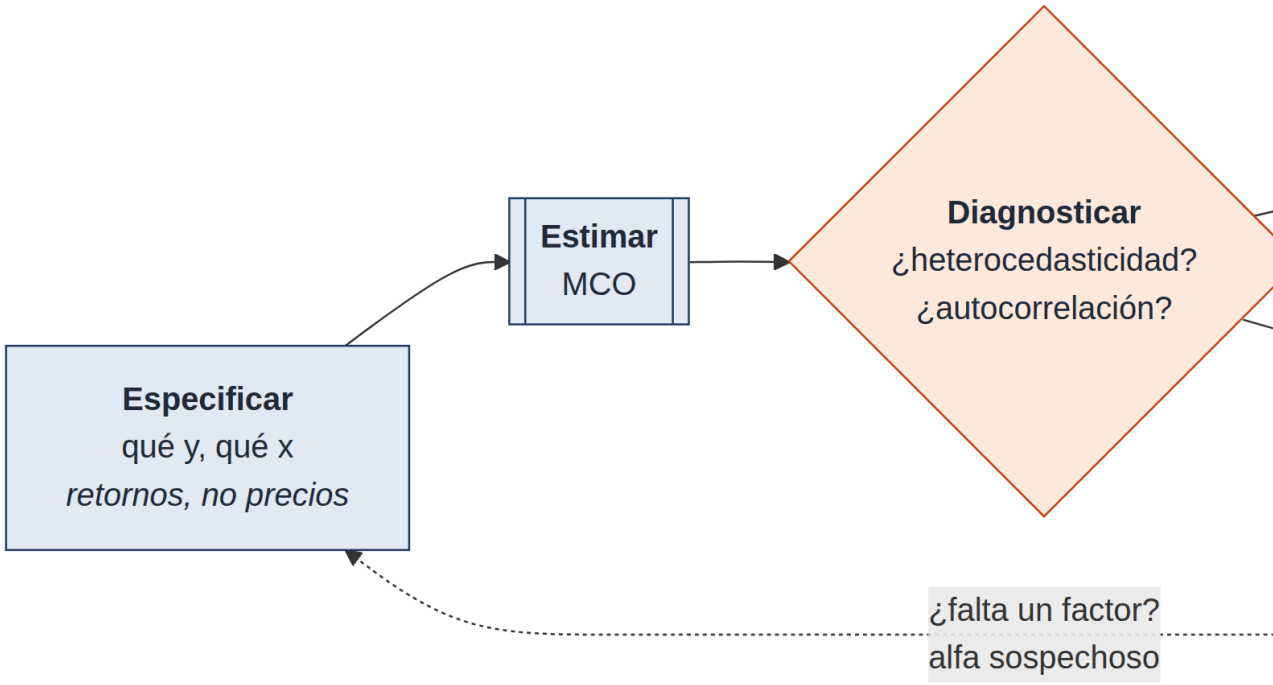


Figura 15.5.: Rutina de una regresión financiera. Especificar antes de mirar los resultados, diagnosticar los supuestos después de estimar, y leer magnitud e incertidumbre juntas.

15.5. Regresión múltiple: más de una explicativa

La extensión a varias variables es directa en la práctica (`statsmodels` recibe una matriz en lugar de un vector) pero cambia la **interpretación**: cada coeficiente mide el efecto de su variable *controlando por las demás*. Esto permite responder preguntas imposibles para la regresión simple: *¿este fondo tiene alfa, o su "alfa" es en realidad exposición a las acciones pequeñas?*

15.5.1. Aplicación 2: modelos multifactor (Fama-French)

El CAPM explica los retornos con un solo factor (el mercado). La evidencia empírica encontró sistemáticamente dos anomalías: las empresas **chicas** rinden más que las grandes, y las **baratas** (alto valor libro / valor de mercado) más que las caras. Fama y French (1993) propusieron agregarlas como factores:

$$R_i - r_f = \alpha + \beta_m (R_m - r_f) + \beta_s \cdot SMB + \beta_v \cdot HML + \varepsilon$$

donde **SMB** (*Small Minus Big*) es el retorno de las chicas menos las grandes, y **HML** (*High Minus Low*) el de las baratas menos las caras.

```
# Simulamos los tres factores y un fondo con exposiciones conocidas
n = 240   # 20 años de datos mensuales
mkt = rng.normal(0.005, 0.040, n)
smb = rng.normal(0.001, 0.025, n)
hml = rng.normal(0.002, 0.025, n)

# Un fondo "value con sesgo small": expuesto a los tres factores, sin alfa real
ret_fondo = 0.0 + 1.1 * mkt + 0.5 * smb + 0.7 * hml + rng.normal(0, 0.02, n)

# Regresión CAPM (solo mercado) vs. modelo de 3 factores
```

```python
X1 = sm.add_constant(mkt)
X3 = sm.add_constant(np.column_stack([mkt, smb, hml]))

capm_fit = sm.OLS(ret_fondo, X1).fit()
ff_fit = sm.OLS(ret_fondo, X3).fit()

print(f"{'':22}{'CAPM':>10}{'3 factores':>12}")
print(f"{'alpha (mensual)':<22}{capm_fit.params[0]:>10.4f}{ff_fit.params[0]:>12.4f}"
print(f"{'p-value del alpha':<22}{capm_fit.pvalues[0]:>10.3f}{ff_fit.pvalues[0]:>12.
print(f"{'R²':<22}{capm_fit.rsquared:>10.3f}{ff_fit.rsquared:>12.3f}")
print(f"\nExposiciones estimadas (3F): mkt={ff_fit.params[1]:.2f}, "
      f"smb={ff_fit.params[2]:.2f}, hml={ff_fit.params[3]:.2f}")
```

```
                          CAPM  3 factores
alpha (mensual)         0.0010      0.0003
p-value del alpha        0.618       0.838
R²                       0.696       0.874

Exposiciones estimadas (3F): mkt=1.13, smb=0.46, hml=0.70
```

La Figura 15.6 pone lado a lado lo que dice cada modelo. A la izquierda, el alfa estimado con su intervalo de confianza del 95 %; a la derecha, las exposiciones que estima el modelo de tres factores contra las verdaderas con las que simulamos el fondo.

La lección del ejemplo: bajo el CAPM el fondo puede *parecer* tener alfa (el modelo de un factor le atribuye al gestor lo que en realidad es exposición a small y value). El modelo de tres factores **desenmascara** ese alfa: era beta disfrazado. Este análisis —"¿tu alfa sobrevive a los factores?"— es hoy el estándar para evaluar gestores.

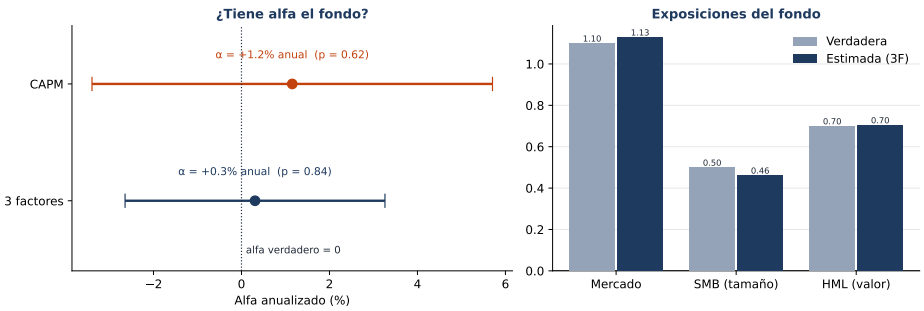


Figura 15.6.: Izquierda: alfa anualizado estimado por cada modelo, con su intervalo del 95 %. El CAPM estima un alfa positivo (con un intervalo muy ancho: en esta muestra ni siquiera es significativo); con tres factores, la estimación cae casi a cero, el valor verdadero. Derecha: el modelo de tres factores recupera las exposiciones con las que se construyó el fondo; esa exposición a tamaño y valor es lo que el CAPM confundía con alfa.

15.5.2. Aplicación 3: ratio de cobertura

Otra pregunta con forma de regresión: tenés una cartera y querés **cubrirla** vendiendo futuros de un índice. ¿Cuántos? El ratio de cobertura de mínima varianza es, precisamente, el beta de tu cartera contra el instrumento de cobertura:

$$h^* = \frac{\text{Cov}(R_{cartera}, R_{futuro})}{\text{Var}(R_{futuro})} = \beta$$

```
# Cartera correlacionada con el índice, pero no idéntica
ret_indice = rng.normal(0.0004, 0.012, 500)
ret_cartera = 0.85 * ret_indice + rng.normal(0.0002, 0.006, 500)

h = sm.OLS(ret_cartera, sm.add_constant(ret_indice)).fit().params[1]

var_sin = np.var(ret_cartera)
var_con = np.var(ret_cartera - h * ret_indice)

print(f"Ratio de cobertura óptimo: h* = {h:.3f}")
print(f"Varianza sin cobertura: {var_sin:.6f}")
print(f"Varianza cubierta:      {var_con:.6f}  "
      f"(reducción del {1 - var_con/var_sin:.0%})")
```

```
Ratio de cobertura óptimo: h* = 0.818
Varianza sin cobertura: 0.000139
Varianza cubierta:      0.000034  (reducción del 76%)
```

Por cada peso de cartera, vendés h^* pesos de futuro, y la varianza cae en R^2 — el mismo R^2 de la regresión. Todas las piezas encajan.

15.6. Con datos reales: los betas del dataset

Todo el capítulo usó datos simulados con parámetros conocidos, para poder verificar que la regresión los recupera. Ahora el caso real, donde nadie conoce la respuesta: estimamos el CAPM de las 9 acciones del dataset del libro (`datos/precios.csv`, ver Apéndice B) contra el S&P 500, con cinco años de retornos diarios y errores Newey-West, como recomienda la regla práctica de arriba.

Una simplificación: el dataset no trae la tasa libre de riesgo, así que regresamos retornos simples en lugar de excesos. Con datos diarios el beta casi no cambia; el alfa queda corrido en $r_f(1-\beta)$, un ajuste chico frente a la incertidumbre que vamos a ver.

```
precios = pd.read_csv("../datos/precios.csv", index_col="Date", parse_dates=True)
r = precios.pct_change().dropna()
X_mkt = sm.add_constant(r["SPY"])

filas = []
for ticker in r.columns.drop("SPY"):
    ajuste = sm.OLS(r[ticker], X_mkt).fit(cov_type="HAC", cov_kwds={"maxlags": 5})
    filas.append({"Ticker": ticker,
                  "Beta": ajuste.params["SPY"],
                  "se(beta)": ajuste.bse["SPY"],
                  "Alfa anual": ajuste.params["const"] * 252,
                  "p-value alfa": ajuste.pvalues["const"],
                  "R²": ajuste.rsquared})

capm_real = pd.DataFrame(filas).set_index("Ticker").sort_values("Beta")
print(capm_real.round(3))
```

```
         Beta  se(beta)  Alfa anual  p-value alfa     R²
Ticker
```

JNJ	0.179	0.029	0.108	0.156	0.032
KO	0.268	0.039	0.092	0.192	0.081
WMT	0.419	0.053	0.161	0.080	0.110
XOM	0.438	0.067	0.167	0.158	0.080
JPM	0.884	0.036	0.083	0.336	0.384
DIS	1.027	0.043	-0.211	0.044	0.360
MSFT	1.115	0.046	-0.047	0.567	0.503
AAPL	1.195	0.044	0.024	0.786	0.549
NVDA	2.128	0.084	0.297	0.063	0.497

Tres lecturas, cada una con una idea del capítulo:

- **Los betas tienen sentido económico, y están bien estimados.** Ordenados de menor a mayor, aparecen primero las defensivas (JNJ, KO, WMT: betas por debajo de 0.5) y al final la tecnología (MSFT y AAPL entre 1.1 y 1.2; NVDA por encima de 2). Con más de 1.200 datos diarios, los errores estándar son de pocas centésimas: el beta se estima bien.
- **El R^2 dice cuánto del riesgo es "de mercado".** Para AAPL o MSFT, el mercado explica alrededor de la mitad de la varianza diaria; para JNJ, apenas un 3 %. El resto es riesgo propio de cada empresa, el que la diversificación elimina.
- **El alfa casi nunca es significativo.** NVDA subió más de 50 % anual en el período y su alfa estimado es enorme (cerca de 30 % anual), pero su p-value no baja de 0.05: con cinco años de datos, ni siquiera eso alcanza para descartar la suerte. El único alfa "significativo" es el negativo de DIS (p 0.04). Pero probamos nueve acciones, y con nueve tests al 5 % es esperable que alguno pase el umbral por azar. Es la trampa de las pruebas múltiples, el tema central del Capítulo 17.

15.7. Errores comunes

⚠ Trampas frecuentes

- **Regresar precios en lugar de retornos**: dos precios que suben con el tiempo siempre parecen "correlacionados" (regresión espuria). Trabajá con retornos; la excepción (cointegración) llega en el capítulo siguiente.
- **Leer causalidad en una regresión**: MCO mide asociación condicional. "El beta es 1.3" no dice que el mercado *cause* el retorno del activo, solo que se mueven juntos en esa proporción.
- **Omitir variables relevantes**: si al fondo value le regresás solo el mercado, el alfa absorbe lo omitido. El sesgo de variable omitida es el error más costoso en evaluación de estrategias.
- **Ignorar la inestabilidad temporal del beta**: el beta de una empresa cambia con su negocio y su apalancamiento. Estimalo por ventanas móviles antes de asumir que es constante.
- **Cazar p-values**: correr veinte especificaciones y reportar la significativa es garantía de falsos positivos. Definí el modelo antes de mirar los resultados (más de esto en el Capítulo 17).

15.8. Practicá en el navegador

Podés experimentar con el código del capítulo acá mismo: la celda ejecuta Python **en tu navegador**, sin instalar nada (la primera ejecución tarda unos segundos mientras carga el entorno; los paquetes que importes se instalan solos).

```
#| label: practica-navegador
import numpy as np
import statsmodels.api as sm

# Tu código acá...
```

15.9. Referencias

16. Capítulo 16: Series Temporales — ARIMA y VAR

Modelar y pronosticar variables macroeconómicas y financieras

Los modelos lineales del capítulo anterior explican una variable con otras; los modelos de series de tiempo la explican con su propio pasado. Desarrollamos el concepto de estacionariedad y cómo diagnosticarla (test ADF), la identificación de modelos con ACF/PACF, el ajuste y pronóstico con ARIMA, y los modelos VAR para sistemas de varias series macro con sus funciones de impulso-respuesta.

Introducción

En el Capítulo 15 explicamos el retorno de un activo con el retorno del mercado: una variable explicada por *otras*. Los modelos de **series de tiempo** cambian el ángulo: explican una variable con **su propio pasado**. La pregunta ya no es "¿cómo se mueve y con x?" sino *"¿qué dice la historia de esta serie sobre su próximo valor?"*

En finanzas y macro, esta caja de herramientas es la estándar para pronosticar inflación, actividad, tasas de interés o volatilidad, y para responder preguntas de política: *si sube la tasa, ¿qué le pasa a la inflación en los próximos trimestres?*

Este capítulo retoma la manipulación de series del Capítulo 6 (fechas, resampleo, retornos con Pandas) y le agrega los modelos: **ARIMA** para una serie, **VAR** para varias a la vez.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.api import VAR

rng = np.random.default_rng(42)
```

16.1. Estacionariedad: la condición de entrada

16.1.1. Por qué importa

Todo pronóstico basado en el pasado descansa en un supuesto: que **las reglas del juego no cambian**. Formalmente, una serie es **estacionaria** (en sentido débil) si su media, su varianza y sus autocovarianzas no dependen del tiempo. Si la serie tiene tendencia, o su varianza explota, el pasado ya no es una guía del futuro y los modelos que veremos dan resultados sin sentido.

El peligro concreto se llama **regresión espuria**: dos series con tendencia, sin ninguna relación real, muestran correlaciones altísimas y p-values espectaculares. Veámoslo — es el error más caro de la econometría aplicada:

```
import statsmodels.api as sm

# Dos paseos aleatorios TOTALMENTE independientes
n = 500
x = np.cumsum(rng.normal(0, 1, n))
y = np.cumsum(rng.normal(0, 1, n))

res_niveles = sm.OLS(y, sm.add_constant(x)).fit()
res_difs = sm.OLS(np.diff(y), sm.add_constant(np.diff(x))).fit()

print("Regresión de y sobre x (dos paseos aleatorios independientes):")
print(f"  En NIVELES:      R² = {res_niveles.rsquared:.3f}, "
      f"p-value = {res_niveles.pvalues[1]:.2e}  ← ¡relación 'significativa' FALSA!")
print(f"  En DIFERENCIAS:  R² = {res_difs.rsquared:.3f}, "
      f"p-value = {res_difs.pvalues[1]:.3f}  ← la verdad: no hay relación")
```

```
Regresión de y sobre x (dos paseos aleatorios independientes):
  En NIVELES:      R² = 0.200, p-value = 5.42e-26  ← ¡relación 'significativa' FALSA
  En DIFERENCIAS:  R² = 0.000, p-value = 0.829  ← la verdad: no hay relación
```

Las series en **niveles** (los paseos) no son estacionarias y la regresión "encuentra" una relación fuerte que no existe. Las mismas series en **diferencias** (sus incrementos, que sí son estacionarios) revelan la verdad. Moraleja operativa que ya usamos en todo el libro sin justificarla: **por eso trabajamos con retornos y no con precios**.

La Figura 16.1 muestra el engaño. Dos paseos aleatorios independientes, por pura casualidad, derivan en direcciones parecidas durante tramos largos; en niveles, la nube de puntos se inclina y la regresión la declara "significativa" con un p-value minúsculo. En diferencias, la ilusión desaparece.

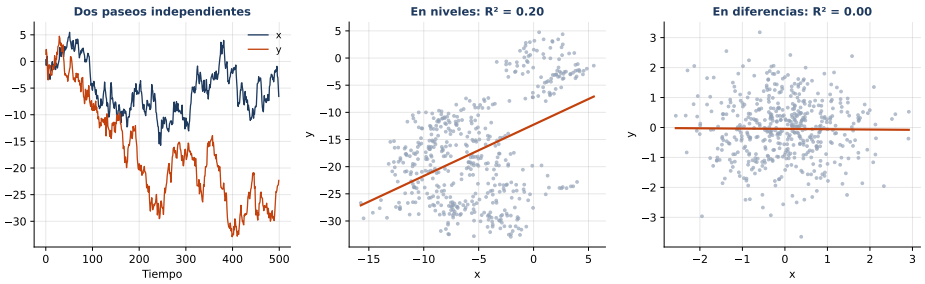


Figura 16.1.: Regresión espuria. Izquierda: dos paseos aleatorios generados de forma independiente. Centro: en niveles, la regresión encuentra una pendiente «significativa» que no existe; los puntos no son independientes entre sí, y el test lo ignora. Derecha: los mismos datos en diferencias (los incrementos) no muestran ninguna relación.

> **Simulador interactivo**
>
> En la versión web de este capítulo hay un simulador donde podés cambiar la persistencia y el largo de las series, y ver en cuántos de 1000 pares independientes la regresión encuentra una relación que no existe.

16.1.2. Diagnóstico formal: el test ADF

El test de **Dickey-Fuller aumentado (ADF)** formaliza el diagnóstico. Su hipótesis nula es que la serie tiene una **raíz unitaria** (no es estacionaria, tipo paseo aleatorio); un p-value bajo permite rechazarla.

```
series_prueba = {
    "Paseo aleatorio (precio)": np.cumsum(rng.normal(0, 1, 500)),
    "Diferencias (retorno)": np.diff(np.cumsum(rng.normal(0, 1, 500))),
    "AR(1) estacionario": None,  # lo generamos abajo
}

# Un AR(1) con persistencia alta pero < 1: estacionario
ar1 = np.zeros(500)
for t in range(1, 500):
    ar1[t] = 0.7 * ar1[t-1] + rng.normal()
series_prueba["AR(1) estacionario"] = ar1

print(f"{'Serie':<28}{'ADF stat':>10}{'p-value':>10}   Veredicto")
for nombre, serie in series_prueba.items():
    stat, pval = adfuller(serie)[:2]
    veredicto = "estacionaria" if pval < 0.05 else "NO estacionaria"
    print(f"{nombre:<28}{stat:>10.2f}{pval:>10.3f}   {veredicto}")
```

```
Serie                        ADF stat   p-value   Veredicto
Paseo aleatorio (precio)        -2.27     0.183   NO estacionaria
Diferencias (retorno)          -24.10     0.000   estacionaria
AR(1) estacionario              -5.76     0.000   estacionaria
```

Regla de trabajo: **testeá antes de modelar**. Si la serie no es estacionaria, diferenciala (una vez suele bastar: precios → retornos, PIB → crecimiento) y volvé a testear. El número de diferenciaciones necesarias es la "d" del ARIMA que viene ahora.

La Figura 16.2 muestra las tres series del test. La pregunta que contesta el ADF se puede mirar a ojo: ¿la media y la dispersión de la serie se quedan quietas, o se desplazan con el tiempo?

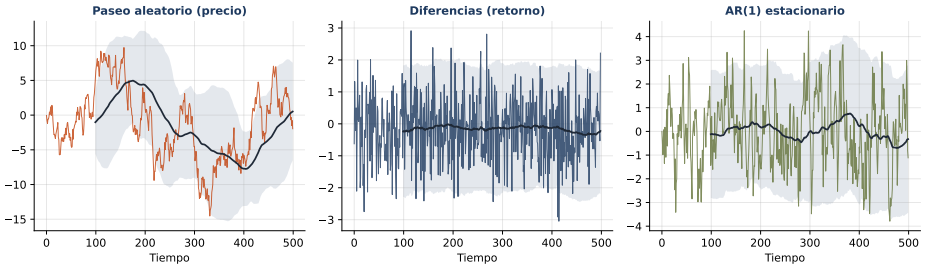


Figura 16.2.: Las tres series del test ADF, con su media móvil (100 períodos) y una banda de ±2 desvíos móviles. El paseo aleatorio deambula sin volver: su media cambia con el tiempo. Las diferencias y el AR(1) oscilan alrededor de un nivel fijo, con una dispersión estable.

16.2. Modelos ARMA: la gramática de una serie estacionaria

Una serie estacionaria se modela combinando dos mecanismos:

- **AR(p)** — *autorregresivo*: el valor de hoy depende de los últimos p valores. "La inflación de este mes se parece a la de los meses anteriores" (inercia, persistencia).

$$y_t = c + \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \varepsilon_t$$

- **MA(q)** — *media móvil*: el valor de hoy depende de los últimos q **shocks**. "El error de ayer todavía repercute hoy" (efectos transitorios de las sorpresas).

$$y_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}$$

ARMA(p, q) combina ambos, y **ARIMA(p, d, q)** agrega la "I" de *integrated*: diferenciar la serie d veces antes de ajustar el ARMA. Así, un ARIMA(1,1,0) sobre un precio es un AR(1) sobre sus retornos.

16.2.1. Identificación: leer la ACF y la PACF

¿Cómo elegir p y q? Las dos herramientas clásicas:

- **ACF** (autocorrelación): correlación de la serie con sus propios rezagos.
- **PACF** (autocorrelación *parcial*): correlación con el rezago k **descontando** el efecto de los rezagos intermedios.

Los patrones de manual: un **AR(p)** tiene PACF que se corta en el rezago p y ACF que decae gradualmente; un **MA(q)** es el espejo (ACF se corta en q, PACF decae).

La Figura 16.3 muestra esos patrones en su versión teórica, sin ruido de muestreo: es la tabla de referencia contra la que se compara la ACF y la PACF de una serie real.

Ahora el mismo diagnóstico sobre una serie simulada, con el ruido de muestreo que trae cualquier serie real. La Figura 16.4 debería parecerse a la fila superior de la figura anterior.

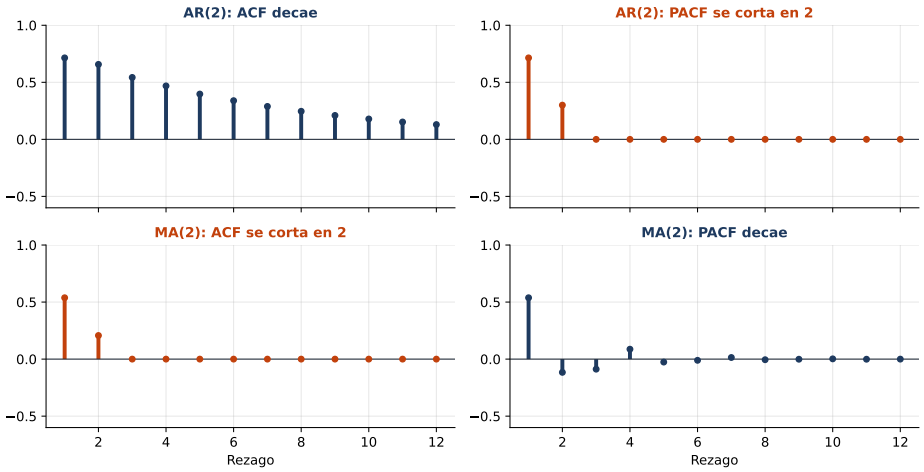


Figura 16.3.: Firmas teóricas para identificar el modelo. Fila superior, AR(2) (= 0.5, = 0.3): la ACF decae de a poco y la PACF se corta después del rezago 2. Fila inferior, MA(2) (= 0.6, = 0.3): al revés, la ACF se corta en 2 y la PACF decae.

```python
# Generamos un AR(2) conocido y verificamos que las herramientas lo identifican
n = 800
serie_ar2 = np.zeros(n)
for t in range(2, n):
    serie_ar2[t] = 0.5 * serie_ar2[t-1] + 0.3 * serie_ar2[t-2] + rng.normal()

fig, axes = plt.subplots(1, 2, figsize=(11, 3.5))
plot_acf(serie_ar2, lags=20, ax=axes[0], title="ACF: decae gradualmente")
plot_pacf(serie_ar2, lags=20, ax=axes[1], title="PACF: se corta en el rezago 2 → AR(
plt.tight_layout()
plt.show()
```

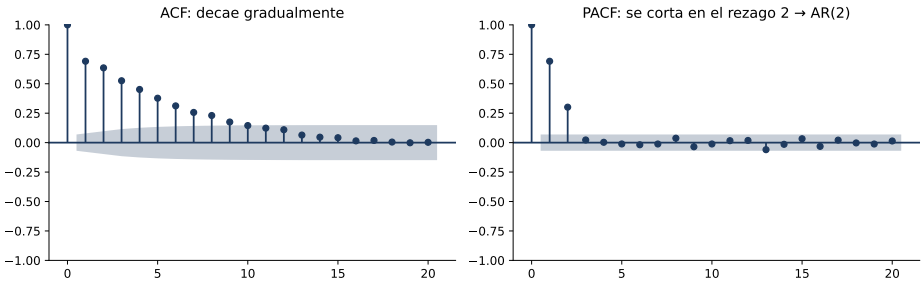


Figura 16.4.: ACF y PACF muestrales de un AR(2) simulado de 800 observaciones. La banda sombreada es el intervalo del 95 % bajo la hipótesis de correlación cero: las barras que la superan son significativas.

La PACF muestra dos barras significativas y luego nada: la firma de un AR(2). En la práctica, la identificación visual se complementa con criterios de información (AIC/BIC): ajustar varios órdenes candidatos y quedarse con el que mejor balancea ajuste y parsimonia.

16.3. ARIMA en acción: modelar y pronosticar la inflación

Armemos el flujo completo con una serie macro simulada con la persistencia típica de la inflación mensual. El orden de los pasos es la metodología de **Box-Jenkins**, que resume la Figura 16.5.

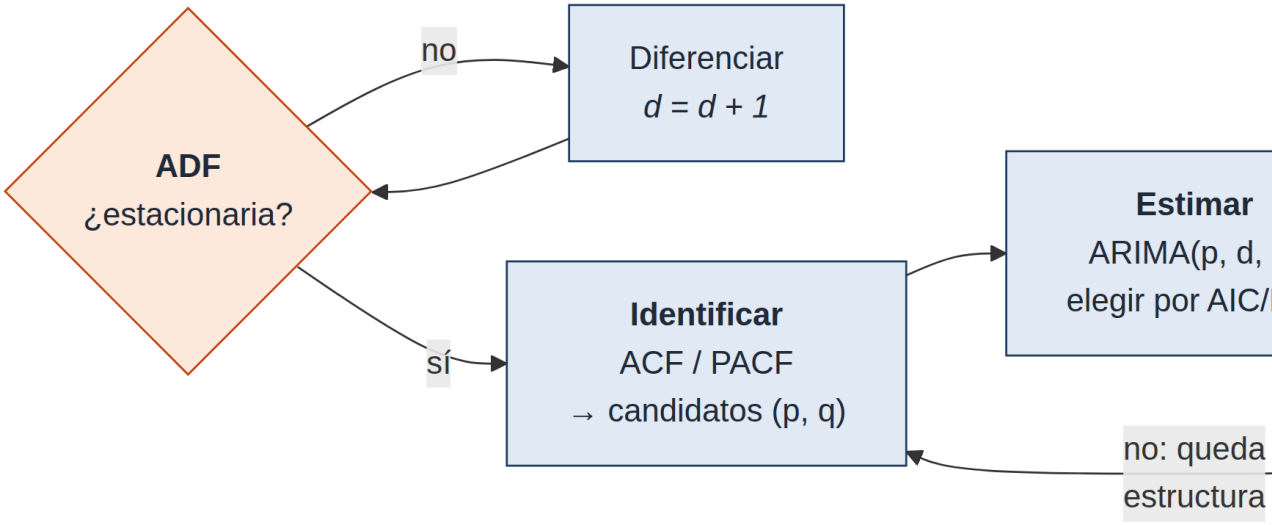


Figura 16.5.: El ciclo de Box-Jenkins. Primero se decide d (cuántas veces diferenciar), después p y q, y el modelo solo se usa para pronosticar si sus residuos ya no tienen estructura; si la tienen, se vuelve a identificar.

```
# Serie mensual de "inflación" con inercia AR(1) fuerte (phi=0.75)
n = 240   # 20 años
inflacion = np.zeros(n)
mu_inf = 2.0 / 12  # ~2% anual en mensual
```

```
for t in range(1, n):
    inflacion[t] = mu_inf * 0.25 + 0.75 * inflacion[t-1] + rng.normal(0, 0.15)

fechas = pd.date_range("2006-01-01", periods=n, freq="MS")
serie = pd.Series(inflacion, index=fechas, name="inflacion_mensual")

# 1) ¿Estacionaria?
stat, pval = adfuller(serie)[:2]
print(f"ADF: stat={stat:.2f}, p-value={pval:.4f} → "
      f"{'estacionaria, no hace falta diferenciar (d=0)' if pval < 0.05 else 'difere

# 2) Selección de orden por AIC entre algunos candidatos
candidatos = [(1,0,0), (2,0,0), (1,0,1), (0,0,2)]
aics = {orden: ARIMA(serie, order=orden).fit().aic for orden in candidatos}
mejor = min(aics, key=aics.get)
print("\nAIC por modelo candidato:")
for orden, aic in aics.items():
    marca = "  ← elegido" if orden == mejor else ""
    print(f"  ARIMA{orden}: {aic:8.1f}{marca}")

# 3) Ajuste final
modelo = ARIMA(serie, order=mejor).fit()
print(f"\nCoeficientes del ARIMA{mejor}:")
print(modelo.params.round(3))
```

```
ADF: stat=-5.25, p-value=0.0000 → estacionaria, no hace falta diferenciar (d=0)

AIC por modelo candidato:
  ARIMA(1, 0, 0):   -236.9
  ARIMA(2, 0, 0):   -237.8
  ARIMA(1, 0, 1):   -238.1  ← elegido
  ARIMA(0, 0, 2):   -196.3
```

```
Coeficientes del ARIMA(1, 0, 1):
const     0.165
ar.L1     0.762
ma.L1     0.145
sigma2    0.021
dtype: float64
```

El AIC elige el AR(1) —el proceso generador verdadero— y el coeficiente estimado ronda el 0.75 real. Ahora, el pronóstico:

```
horizonte = 24
pron = modelo.get_forecast(steps=horizonte)
media = pron.predicted_mean
banda = pron.conf_int(alpha=0.05)

fig, ax = plt.subplots(figsize=(10, 4.5))
ax.plot(serie.iloc[-72:], label="Inflación observada")
ax.plot(media, color="#c2410c", linewidth=2, label="Pronóstico ARIMA")
ax.fill_between(banda.index, banda.iloc[:, 0], banda.iloc[:, 1],
                alpha=0.2, color="#c2410c", label="IC 95%")
ax.set_ylabel("Inflación mensual (%)")
ax.set_title("Pronóstico a 24 meses con intervalos de confianza")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Figura 16.6.: Últimos seis años de la serie y pronóstico a 24 meses del ARIMA elegido. El pronóstico converge a la media de largo plazo y la banda del 95 % se abre con el horizonte.

Dos rasgos para leer en la Figura 16.6, típicos de todo pronóstico ARIMA: la predicción **converge a la media de largo plazo** de la serie (la inercia se agota), y las bandas **se abren con el horizonte** hasta estabilizarse — el modelo es honesto sobre lo poco que sabe del futuro lejano.

> ⚠ ¿Y pronosticar retornos de acciones?
>
> Tentador, pero el ARIMA sobre retornos accionarios suele encontrar… nada ($p = q = 0$): los retornos son casi impredecibles por su propio pasado (eficiencia de mercado en su forma débil). Donde los modelos de series de tiempo brillan es en variables **con inercia estructural**: inflación, tasas, actividad, y —muy especialmente— la *volatilidad* (la familia GARCH, prima hermana de ARMA aplicada a la varianza).

16.4. VAR: varias series que se influyen mutuamente

La macro rara vez es univariada: la inflación depende de la tasa de interés, la tasa responde a la inflación, y la actividad interactúa con ambas. El modelo **VAR** (*Vector Autoregression*) generaliza el AR a un **sistema**: cada variable se regresa sobre los rezagos de **todas** las variables.

Para dos variables y un rezago:

$$y_{1,t} = c_1 + \phi_{11}\, y_{1,t-1} + \phi_{12}\, y_{2,t-1} + \varepsilon_{1,t}$$
$$y_{2,t} = c_2 + \phi_{21}\, y_{1,t-1} + \phi_{22}\, y_{2,t-1} + \varepsilon_{2,t}$$

Los coeficientes cruzados (ϕ_{12}, ϕ_{21}) son la novedad: capturan cómo una variable **anticipa** a la otra.

La Figura 16.7 dibuja el sistema que vamos a simular: cada flecha es un coeficiente, y cada variable recibe una flecha de su propio pasado y otra del pasado de la otra.

```
# Sistema simulado con estructura conocida:
#   - la tasa sube cuando la inflación pasada fue alta (regla de política)
#   - la inflación baja cuando la tasa pasada fue alta (efecto de la política)
n = 300
inf = np.zeros(n)
tasa = np.zeros(n)
for t in range(1, n):
    inf[t] = 0.6 * inf[t-1] - 0.20 * tasa[t-1] + rng.normal(0, 0.3)
    tasa[t] = 0.7 * tasa[t-1] + 0.25 * inf[t-1] + rng.normal(0, 0.2)

datos = pd.DataFrame({"inflacion": inf, "tasa": tasa},
                     index=pd.date_range("2001-01-01", periods=n, freq="MS"))

# Ajuste del VAR con selección automática del número de rezagos
var_modelo = VAR(datos)
```

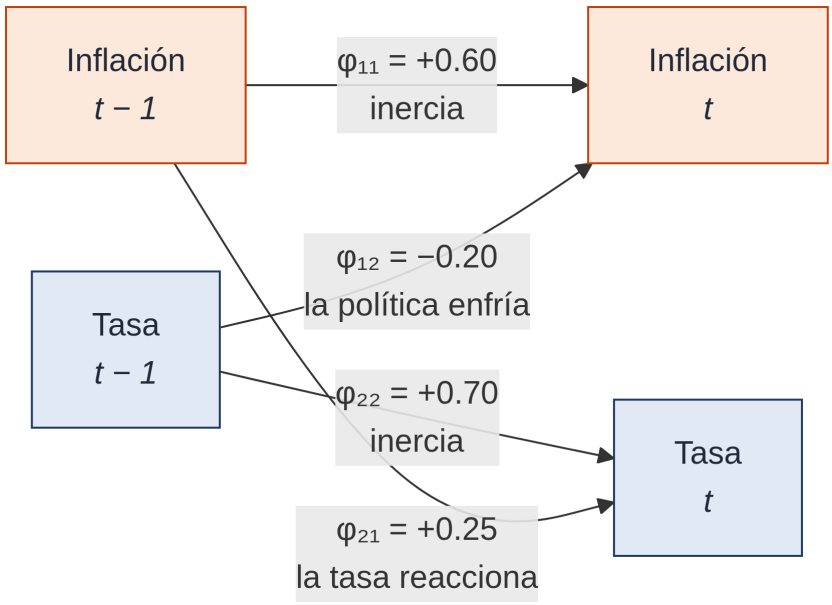


Figura 16.7.: El VAR(1) simulado como diagrama. Las flechas horizontales son la inercia de cada variable (,); las cruzadas son la política monetaria: la tasa reacciona a la inflación pasada (= +0.25) y la inflación cae cuando la tasa pasada fue alta (= −0.20).

```
seleccion = var_modelo.select_order(maxlags=8)
p_optimo = seleccion.aic
resultado = var_modelo.fit(p_optimo)

print(f"Rezagos elegidos por AIC: {p_optimo}\n")
print("Coeficientes de la ecuación de inflación:")
print(resultado.params["inflacion"].round(3))
```

```
Rezagos elegidos por AIC: 1

Coeficientes de la ecuación de inflación:
const          -0.018
L1.inflacion    0.611
L1.tasa        -0.156
Name: inflacion, dtype: float64
```

Los coeficientes recuperan la estructura que sembramos: la inflación depende positivamente de su pasado y **negativamente de la tasa pasada**. Pero la forma más útil de leer un VAR no es la tabla de coeficientes sino la…

16.4.1. Función de impulso-respuesta (IRF)

La IRF responde la pregunta de política: *si hoy golpeo al sistema con un shock de una unidad en una variable, ¿cómo evolucionan todas las variables en los meses siguientes?* Es el experimento contrafáctico que el VAR permite trazar.

```
irf = resultado.irf(24)
irf.plot(orth=False, figsize=(10, 7))
plt.suptitle("Funciones de impulso-respuesta (24 meses)", y=1.02)
plt.tight_layout()
plt.show()
```

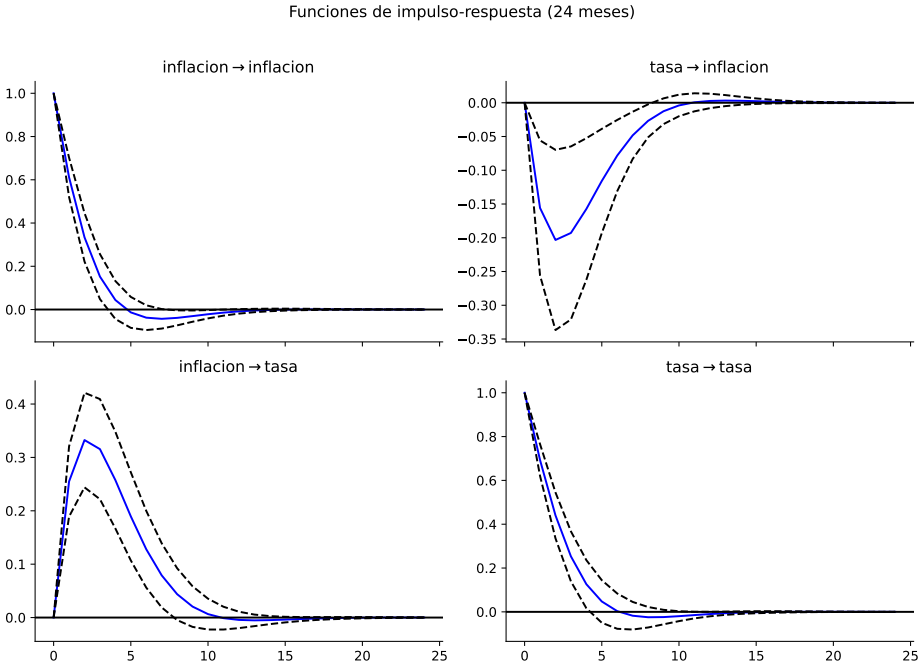


Figura 16.8.: Funciones de impulso-respuesta del VAR estimado, 24 meses. Cada panel muestra cómo responde la variable de la fila a un shock de una unidad en la variable de la columna, con su banda de confianza.

En la Figura 16.8, mirá el panel "tasa → inflacion": un shock de tasa deprime la inflación durante varios meses, con el efecto máximo no inmediato sino rezagado — exactamente la dinámica "la política monetaria actúa con rezagos" que los bancos centrales repiten. El panel "inflacion → tasa" muestra la respuesta de política: la tasa sube tras un shock inflacionario y luego decae.

Los VAR en la práctica

Los bancos centrales usan VARs (y variantes: SVAR, BVAR, FAVAR) como caballo de batalla para el análisis de política monetaria desde los trabajos de Sims (Nobel 2011). En finanzas se usan para sistemas tasa-inflación-actividad, para pronóstico macro que alimenta modelos de PPNR y stress testing (Capítulo 12), y para estudiar transmisión entre mercados.

16.5. Con datos reales: retornos sin memoria, volatilidad con memoria

El recuadro de la sección ARIMA afirmaba dos cosas sobre los retornos de acciones: que casi no se pueden pronosticar con su propio pasado, y que la *volatilidad*, en cambio, sí tiene inercia. Las dos se pueden verificar con la herramienta de identificación del capítulo, la ACF, aplicada a cinco años de retornos diarios del S&P 500 del dataset del libro (`datos/precios.csv`, ver Apéndice B).

```
precios = pd.read_csv("../datos/precios.csv", index_col="Date", parse_dates=True)
r_spy = precios["SPY"].pct_change().dropna()

fig, axes = plt.subplots(1, 2, figsize=(11, 3.8), sharey=True)
plot_acf(r_spy, lags=40, ax=axes[0], title="Retornos: casi sin memoria")
plot_acf(r_spy.abs(), lags=40, ax=axes[1], title="|Retornos|: memoria larga")
for ax in axes:
    ax.set_xlabel("Rezago (días)")
    ax.set_ylim(-0.15, 0.3)
```

```
plt.tight_layout()
plt.show()
```

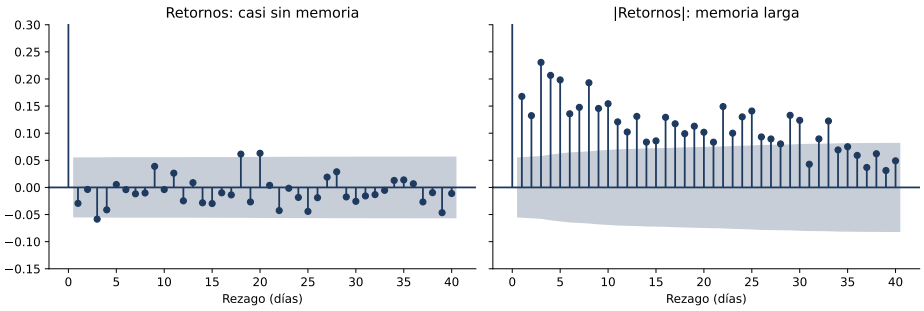


Figura 16.9.: ACF de los retornos diarios del S&P 500 (izquierda) y de su valor absoluto (derecha), 40 rezagos. Los retornos quedan casi siempre dentro de la banda de no significancia; su tamaño, en cambio, está correlacionado con el de semanas y meses anteriores.

El contraste de la Figura 16.9 es uno de los hechos más robustos de las finanzas empíricas. A la izquierda, las autocorrelaciones de los retornos son minúsculas: unas pocas rozan la banda, apenas por encima de lo que se espera por azar en 40 rezagos, y ninguna supera el 0.07 en valor absoluto. Saber que ayer el mercado subió no dice casi nada sobre hoy: es la eficiencia débil del recuadro, ahora medida. A la derecha, el **valor absoluto** de los retornos (su tamaño, sin el signo) tiene autocorrelaciones positivas en casi todos los rezagos, durante dos meses: un día agitado anticipa días agitados. Es el *volatility clustering* que modelan la familia GARCH y los modelos de volatilidad estocástica del Capítulo 20.

16.6. Errores comunes

⚠ Trampas frecuentes

- **Modelar en niveles sin testear estacionariedad**: la regresión espuria del inicio. ADF primero, modelo después.
- **Sobrediferenciar**: si la serie ya era estacionaria, diferenciarla "por las dudas" introduce un MA espurio y arruina el pronóstico. La "d" correcta es la mínima necesaria.
- **Elegir p y q maximizando el ajuste dentro de la muestra**: más rezagos siempre ajustan mejor *in-sample*; usá AIC/BIC o validación fuera de muestra (el tema del próximo capítulo).
- **Leer causalidad en un VAR**: que la tasa "anticipe" a la inflación (precedencia temporal, causalidad de Granger) no prueba causalidad económica; puede haber una tercera variable moviendo a ambas.
- **Ignorar los residuos**: un buen modelo deja residuos sin autocorrelación. Si la ACF de los residuos muestra estructura, el modelo se quedó corto.

16.7. Practicá en el navegador

Podés experimentar con el código del capítulo acá mismo: la celda ejecuta Python **en tu navegador**, sin instalar nada (la primera ejecución tarda unos segundos mientras carga el entorno; los paquetes que importes se instalan solos).

```
#| label: practica-navegador
import numpy as np
import statsmodels.api as sm
from statsmodels.tsa.arima.model import ARIMA

# Tu código acá...
```

16.8. Referencias

17. Capítulo 17: Backtesting de Estrategias

In-sample, out-of-sample, p-values y walk-forward

Cualquier estrategia parece brillante evaluada sobre los datos con los que fue diseñada. Este capítulo construye la disciplina para saber si funciona de verdad: separación in-sample / out-of-sample, el peligro del sobreajuste y del data snooping, cómo calcular e interpretar el p-value de una estrategia, el análisis walk-forward como estándar profesional, y las métricas correctas para juzgar el desempeño fuera de muestra.

Introducción

Cerramos el módulo con la pregunta que une todo lo anterior: tenés una estrategia —una regla que decide cuándo comprar y vender— y su simulación histórica da ganancias. **¿Eso prueba algo?**

Casi siempre, no. El problema es estructural: la estrategia se diseñó *mirando* esos mismos datos. Ajustaste parámetros, probaste variantes, descartaste lo que no andaba… y lo que sobrevivió, sobrevivió *por construcción*. Es como rendir un examen habiendo visto las respuestas: la nota no mide conocimiento.

El **backtesting** riguroso es el conjunto de prácticas para separar la señal de la casualidad. Es, en el fondo, el método científico aplicado al trading: hipótesis primero, evidencia después, y una paranoia saludable hacia los resultados demasiado lindos.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(83)
```

17.1. El pecado original: evaluar donde diseñaste

17.1.1. In-sample vs. out-of-sample

La división fundamental:

- **In-sample (IS)**: los datos usados para *diseñar* la estrategia (elegir reglas, calibrar parámetros). Ahí la estrategia siempre luce bien — la elegiste porque lucía bien ahí.
- **Out-of-sample (OOS)**: datos que la estrategia **nunca vio**. La única evidencia admisible sobre su capacidad real.

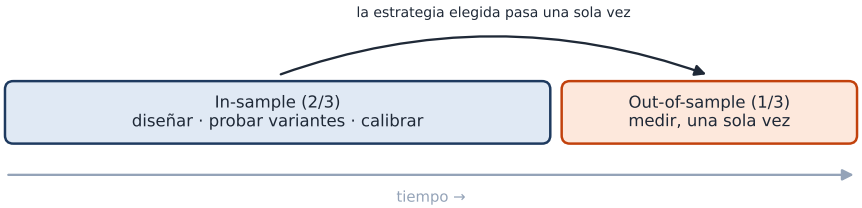


Figura 17.1.: La partición básica de un backtest. Todo el diseño (probar reglas, elegir parámetros) ocurre en el tramo in-sample; el tramo out-of-sample queda guardado y se abre una sola vez, al final, para medir.

La demostración más contundente de por qué esto importa: vamos a "descubrir" una estrategia excelente **en puro ruido**.

```
# 10 años de retornos diarios SIN NINGUNA señal: puro azar con drift cero
n_dias = 2520
retornos = rng.normal(0.0, 0.012, n_dias)
train, test = retornos[:1680], retornos[1680:]   # ~2/3 diseño, 1/3 evaluación

def estrategia_momentum(r, ventana):
```

```
    """Compra si la media de los últimos `ventana` días fue positiva."""
    señal = np.array([r[max(0, t-ventana):t].mean() > 0 for t in range(len(r))])
    return np.where(señal, r, 0.0)   # invertido o en cash

def sharpe(r):
    return r.mean() / r.std() * np.sqrt(252) if r.std() > 0 else 0.0

# "Investigación": probamos 60 ventanas distintas y elegimos la mejor IN-SAMPLE
ventanas = range(2, 62)
sharpes_is = {v: sharpe(estrategia_momentum(train, v)) for v in ventanas}
mejor_v = max(sharpes_is, key=sharpes_is.get)

print(f"Ventanas probadas: {len(list(ventanas))}")
print(f"Mejor ventana in-sample: {mejor_v} días → Sharpe IS = {sharpes_is[mejor_v]:.
print(f"La misma estrategia out-of-sample:   Sharpe OOS = "
      f"{sharpe(estrategia_momentum(test, mejor_v)):.2f}")
```

```
Ventanas probadas: 60
Mejor ventana in-sample: 43 días → Sharpe IS = 0.79
La misma estrategia out-of-sample:   Sharpe OOS = -0.51
```

La Figura 17.2 muestra las 60 variantes a la vez: arriba, el Sharpe de cada ventana en el tramo de diseño; abajo, el de esa misma ventana en el tramo nunca visto.

Los datos son **ruido puro por construcción** — no hay nada que descubrir. Y sin embargo, probando 60 variantes, alguna siempre luce bien in-sample. Fuera de muestra, el castillo se derrumba. Ese mecanismo tiene nombre: **data snooping** (o *p-hacking*): probar hasta que algo "funcione" garantiza encontrar espejismos.

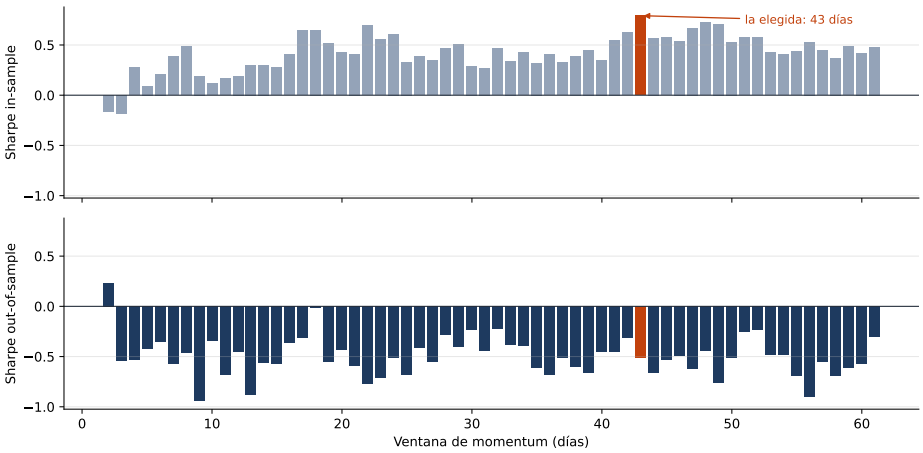


Figura 17.2.: Las 60 ventanas de momentum sobre ruido puro. Arriba, Sharpe in-sample, con la ventana elegida en rojo; abajo, el Sharpe out-of-sample de cada una. La mejor in-sample no es mejor afuera: el ranking del diseño no predice nada.

> ⚠ La regla de oro
>
> Los datos out-of-sample se usan **una sola vez**. Si evaluás en OOS, no te gusta el resultado, retocás la estrategia y volvés a evaluar en el mismo OOS… el OOS acaba de convertirse en in-sample. Esta regla es incómoda y por eso casi nadie la cumple — y por eso la mayoría de los backtests publicados no se replican.

17.1.2. Sobreajuste: la curva que memoriza

El data snooping es la versión "entre estrategias" de un fenómeno general: el **sobreajuste** (*overfitting*). Un modelo con suficientes parámetros puede memorizar cualquier historia — incluyendo su ruido. Cuantos más parámetros libres tenga tu estrategia (ventanas, umbrales, filtros, horarios), más capacidad tiene de ajustarse a accidentes del pasado que no se repetirán.

Regla mental de López de Prado: **la probabilidad de que un backtest sea falso crece con el número de configuraciones probadas** — y quien lo publica rara vez te dice cuántas probó.

17.2. El p-value de una estrategia

¿Cómo cuantificar si un resultado OOS es "suerte o señal"? Con la misma lógica inferencial del Capítulo 15. La hipótesis nula es dura pero justa:

> H_0: la estrategia no tiene ninguna habilidad (retorno medio verdadero = 0).

Bajo H_0, el estadístico t del retorno medio nos dice cuán sorprendente es lo observado:

$$t = \frac{\bar{r}}{s/\sqrt{n}} \qquad \text{y de hecho} \qquad t \approx \text{Sharpe anualizado} \times \sqrt{\text{años}}$$

Esa equivalencia es una de las fórmulas más útiles del oficio: **el t-stat de una estrategia es su Sharpe por la raíz de los años observados**. Un Sharpe de 1.0 necesita ~4 años para ser significativo al 5 %; un Sharpe de 0.5, ¡16 años!

La Figura 17.3 dibuja las dos reglas que gobiernan este capítulo: cuánto historial hace falta para que un Sharpe sea creíble (izquierda), y cuán rápido se vuelve casi seguro encontrar un falso positivo cuando se prueban muchas variantes (derecha, el tema del recuadro que viene).

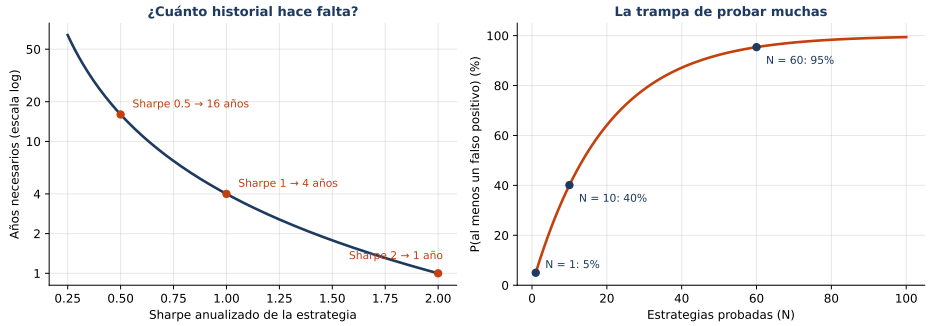


Figura 17.3.: Izquierda: años de track record necesarios para que un Sharpe dado sea significativo al 5 %, de t = Sharpe × √años 2. Derecha: probabilidad de que al menos una de N estrategias sin ninguna habilidad pase el umbral del 5 % por azar, 1 − 0.95^N.

```
def evaluar(r, nombre):
    n = len(r)
```

```
    t_stat = r.mean() / (r.std(ddof=1) / np.sqrt(n))
    p_value = 2 * (1 - stats.t.cdf(abs(t_stat), df=n - 1))
    print(f"{nombre:<28} Sharpe: {sharpe(r):5.2f} | t: {t_stat:5.2f} | "
          f"p-value: {p_value:.3f}")

# La estrategia "descubierta" en ruido, evaluada honestamente en OOS
evaluar(estrategia_momentum(test, mejor_v), "Momentum sobre ruido (OOS)")

# Una estrategia con señal genuina (le sembramos un edge real, pequeño)
señal_real = rng.normal(0.0004, 0.012, 840)   # drift positivo verdadero
evaluar(señal_real, "Estrategia con edge real")
```

```
Momentum sobre ruido (OOS)   Sharpe: -0.51 | t: -0.93 | p-
value: 0.352
Estrategia con edge real     Sharpe:  1.30 | t:  2.37 | p-
value: 0.018
```

El p-value con trampa múltiple

Cuidado: ese p-value asume que evaluaste **una** estrategia. Si probaste 60, la probabilidad de que *alguna* pase el umbral del 5 % por puro azar es enorme ($1 - 0.95^{60} \approx 95\,\%$). Las correcciones (Bonferroni: exigir α/N; o el *Deflated Sharpe Ratio* de López de Prado) suben la vara según cuántas configuraciones miraste. La honestidad estadística empieza por **contar tus intentos**.

Simulador interactivo

En la versión web de este capítulo hay un simulador donde podés cambiar cuántas variantes probás y cuántos años de datos tenés, y ver cómo se fabrica un falso descubrimiento.

17.3. Walk-forward: el estándar profesional

La partición simple IS/OOS tiene una debilidad: la evaluación depende de *un* corte particular. Además, las estrategias reales se recalibran periódicamente. El **análisis walk-forward** resuelve ambas cosas imitando la operatoria real:

1. **Entrená** (elegí parámetros) con una ventana de datos.
2. **Operá** con esos parámetros el período siguiente (nunca visto).
3. **Rodá** la ventana hacia adelante y repetí.
4. Concatená todos los tramos operados: eso es tu track record **100 % out-of-sample**.

La Figura 17.4 dibuja el esquema con los números que vamos a usar: dos años para entrenar, medio año para operar, y la ventana que avanza medio año por vez.

```
def walk_forward(retornos, n_train, n_test, ventanas):
    """Reoptimiza la ventana de momentum en cada tramo y opera el siguiente."""
    oos, elecciones = [], []
    inicio = 0
    while inicio + n_train + n_test <= len(retornos):
        tramo_train = retornos[inicio : inicio + n_train]
        tramo_test = retornos[inicio + n_train : inicio + n_train + n_test]

        # Optimización SOLO dentro del tramo de entrenamiento
        v_opt = max(ventanas, key=lambda v: sharpe(estrategia_momentum(tramo_train,
        elecciones.append(v_opt)

        # Operamos el tramo siguiente con lo elegido
        oos.append(estrategia_momentum(tramo_test, v_opt))
```

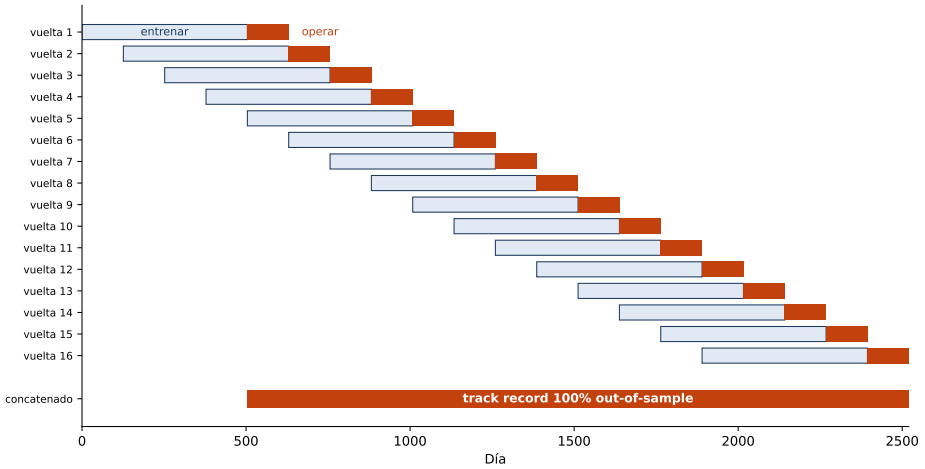


Figura 17.4.: Walk-forward sobre los 10 años de datos: cada fila es una vuelta, con 504 días de entrenamiento (azul) seguidos de 126 días de operación (rojo). La ventana rueda 126 días por vuelta. La fila de abajo concatena todos los tramos rojos: un track record de 8 años en el que ningún día fue usado para elegir los parámetros con los que se operó.

```
        inicio += n_test    # la ventana rueda

    return np.concatenate(oos), elecciones

# Sobre el ruido puro: el walk-forward no se deja engañar
oos_ruido, elecciones = walk_forward(retornos, n_train=504, n_test=126,
                                     ventanas=range(2, 62))
print(f"Tramos operados: {len(elecciones)} | Ventanas elegidas: {elecciones}")
evaluar(oos_ruido, "Walk-forward sobre ruido")
```

```
Tramos operados: 16 | Ventanas elegidas: [43, 17, 47, 43, 52, 7, 22, 10, 8, 2, 2, 2,
Walk-forward sobre ruido     Sharpe:  0.27 | t:  0.77 | p-
value: 0.440
```

Dos observaciones valiosas: las **ventanas elegidas saltan erráticamente** de tramo en tramo — señal inequívoca de que no hay estructura real que estimar, solo ruido que cambia de disfraz. Y el desempeño concatenado es mediocre, como debe ser. Un walk-forward con parámetros *estables* entre tramos y desempeño OOS consistente es de las mejores señales de que una estrategia es genuina.

17.4. Métricas para juzgar el track record OOS

Con el track record fuera de muestra en la mano, ¿qué mirar? Ninguna métrica sola alcanza; el tablero mínimo:

Métrica	Qué mide	Talón de Aquiles
Sharpe anualizado	Retorno en exceso por unidad de riesgo	Ciego a la asimetría y a las rachas
Máximo drawdown	La peor caída pico-a-valle	Una sola observación; muy sensible al período
Hit ratio	% de períodos ganadores	Ignora tamaños: podés acertar 60 % y quebrar
Profit factor	Ganancias brutas / pérdidas brutas	Inestable con pocas operaciones

```python
def reporte(r, nombre):
    equity = np.cumprod(1 + r)
    picos = np.maximum.accumulate(equity)
    drawdown = equity / picos - 1
    hit = (r[r != 0] > 0).mean() if (r != 0).any() else np.nan
    print(f"- {nombre}")
    print(f"  Sharpe:         {sharpe(r):6.2f}")
    print(f"  Retorno anual: {r.mean() * 252:6.1%}")
    print(f"  Máx. drawdown: {drawdown.min():6.1%}")
    print(f"  Hit ratio:      {hit:6.1%}")
    return equity, drawdown

equity_r, dd_r = reporte(oos_ruido, "Walk-forward sobre ruido")
print()
equity_e, dd_e = reporte(señal_real, "Estrategia con edge real")
```

```
- Walk-forward sobre ruido
  Sharpe:          0.27
  Retorno anual:   3.6%
  Máx. drawdown: -36.6%
```

```
  Hit ratio:      50.3%

- Estrategia con edge real
  Sharpe:          1.30
  Retorno anual:  25.0%
  Máx. drawdown: -24.1%
  Hit ratio:      54.3%
```

```python
fig, axes = plt.subplots(2, 1, figsize=(10, 6), sharex=False,
                         gridspec_kw={"height_ratios": [2, 1]})

ax = axes[0]
ax.plot(equity_r, label="Walk-forward sobre ruido")
ax.plot(equity_e, label="Estrategia con edge real")
ax.set_ylabel("Equity (base 1)")
ax.set_title("Curvas de capital out-of-sample")
ax.legend()
ax.grid(True, alpha=0.3)

ax = axes[1]
ax.fill_between(range(len(dd_r)), dd_r * 100, 0, alpha=0.5, label="Ruido")
ax.fill_between(range(len(dd_e)), dd_e * 100, 0, alpha=0.5, label="Edge real")
ax.set_ylabel("Drawdown (%)")
ax.set_xlabel("Día de operación OOS")
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

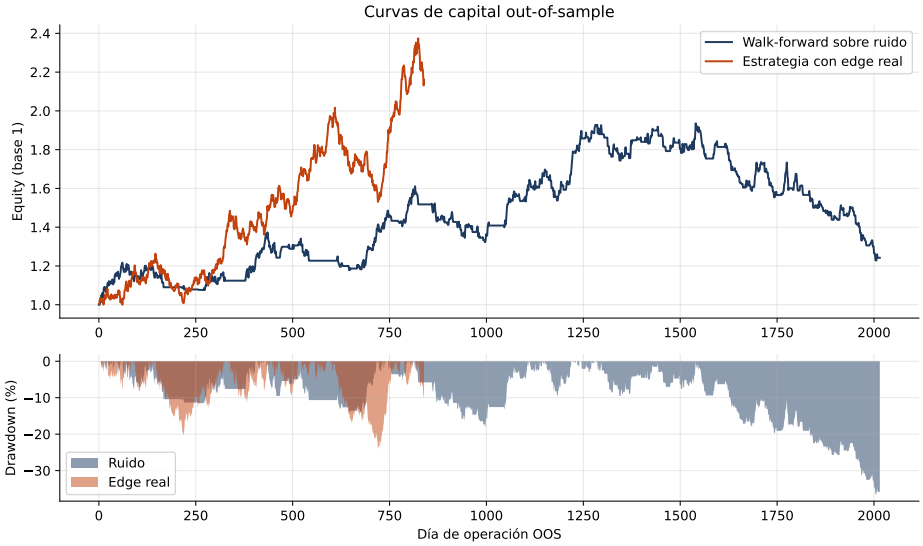


Figura 17.5.: Arriba: curvas de capital out-of-sample del walk-forward sobre ruido y de la estrategia con edge real. Abajo: el drawdown de cada una, la caída desde el máximo previo.

Como en la Figura 17.5, leé siempre la curva de capital junto con la de drawdown: el Sharpe resume, pero la trayectoria muestra si las ganancias vinieron parejas o de un golpe de suerte, y cuánto dolor (drawdown) hubo que aguantar en el camino. Un inversor real abandona la estrategia en el fondo del drawdown — si ese fondo es -40 %, el Sharpe final es teoría.

17.4.1. Los costos que el backtest olvida

Antes de creer cualquier número OOS, restale la realidad:

- **Costos de transacción y slippage**: una estrategia que rota mucho puede morir solo de comisiones. Simulá con costos por operación.

- **Sesgo de supervivencia**: si tu universo actual excluye las empresas que quebraron, el pasado luce mejor de lo que fue.
- **Look-ahead bias**: usar información que no estaba disponible en el momento de la decisión (balances publicados con retraso, precios de cierre para decidir "durante" el día).
- **Capacidad**: el backtest asume que tus órdenes no mueven el precio; a mayor escala, deja de ser cierto.

```
costo_por_operacion = 0.0005   # 5 puntos básicos por cambio de posición

señal = np.array([test[max(0, t-mejor_v):t].mean() > 0 for t in range(len(test))])
cambios = np.abs(np.diff(señal.astype(int), prepend=0))
r_neto = np.where(señal, test, 0.0) - cambios * costo_por_operacion

print(f"Operaciones (cambios de posición): {cambios.sum()}")
print(f"Sharpe bruto: {sharpe(np.where(señal, test, 0.0)):.2f}")
print(f"Sharpe neto de costos: {sharpe(r_neto):.2f}")
```

```
Operaciones (cambios de posición): 66
Sharpe bruto: -0.51
Sharpe neto de costos: -0.60
```

17.5. El protocolo, en síntesis

Un backtest confiable sigue esta secuencia — y el orden importa:

1. **Hipótesis primero**: por qué debería funcionar (fundamento económico), antes de mirar datos.
2. **Diseño in-sample**, con la menor cantidad de parámetros posible.

3. **Contá tus intentos**: cada variante probada infla el umbral de significancia exigible.
4. **Walk-forward** para generar un track record 100 % OOS con recalibración realista.
5. **Costos, siempre**: comisiones, slippage, y los sesgos de datos.
6. **Inferencia honesta**: t-stat/p-value del track record OOS, corregido por múltiples pruebas.
7. **El OOS se usa una vez.** Si volvés a diseñar, necesitás datos nuevos (o esperar a que el mercado los genere).

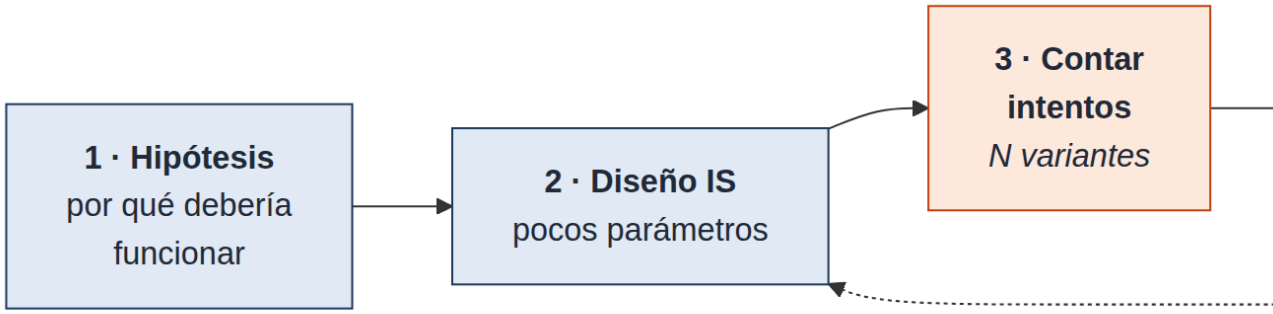


Figura 17.6.: El protocolo de backtesting. La flecha punteada es la tentación que hay que resistir: volver a diseñar después de mirar el resultado out-of-sample lo convierte en in-sample, y entonces hacen falta datos nuevos.

17.6. Con datos reales

Todo el capítulo trabajó sobre ruido fabricado, donde sabíamos la respuesta de antemano. El examen final es con datos donde **nadie** sabe la respuesta:

corremos el walk-forward completo sobre cinco años del SPY, del dataset del libro (`datos/precios.csv`, ver Apéndice B).

```
precios = pd.read_csv("../datos/precios.csv", index_col="Date", parse_dates=True)
r_spy = precios["SPY"].pct_change().dropna().values

oos_spy, elec_spy = walk_forward(r_spy, n_train=504, n_test=126, ventanas=range(2, 6

print(f"Tramos operados: {len(elec_spy)} | Ventanas elegidas: {elec_spy}")
evaluar(oos_spy, "Momentum SPY (walk-forward)")
evaluar(r_spy[-len(oos_spy):], "Buy & hold SPY (mismo período)")

_ = reporte(oos_spy, "Momentum SPY (OOS)")
print()
_ = reporte(r_spy[-len(oos_spy):], "Buy & hold SPY")
```

```
Tramos operados: 5 | Ventanas elegidas: [16, 10, 11, 9, 12]
Momentum SPY (walk-forward)  Sharpe:  0.92 | t:  1.45 | p-
value: 0.149
Buy & hold SPY (mismo período) Sharpe:  1.31 | t:  2.07 | p-
value: 0.039
- Momentum SPY (OOS)
  Sharpe:          0.92
  Retorno anual:   9.1%
  Máx. drawdown: -15.1%
  Hit ratio:      56.7%

- Buy & hold SPY
  Sharpe:          1.31
  Retorno anual:  20.8%
  Máx. drawdown: -18.8%
  Hit ratio:      57.5%
```

Este resultado es una clase entera de honestidad estadística, así que leelo despacio:

- **Las ventanas elegidas son estables entre tramos** — nada del salto errático que vimos sobre ruido puro. Eso sugiere que el momentum del índice tiene *algo* de estructura real.
- El Sharpe out-of-sample es positivo, pero el **p-value no baja del umbral**: con ~2.5 años de track record OOS, la fórmula $t \approx \text{Sharpe} \times \sqrt{\text{años}}$ hace su trabajo implacable. Un Sharpe cercano a 1 con dos años y medio de datos **no es evidencia suficiente** — verificalo vos mismo con la fórmula.
- Y la comparación que un backtest siempre debe incluir: en el mismo período, **comprar y no hacer nada rindió más**. El período OOS fue mayormente alcista, y toda estrategia que pasa tiempo en cash paga ese costo. El consuelo del momentum es un drawdown máximo menor — es una estrategia *defensiva*, y su valor aparecería en un bear market largo, no en este tramo. Elegir el benchmark correcto es parte del protocolo.

Ah, y esto es *antes* de costos de transacción. La conclusión metodológica —no sobre el momentum sino sobre el oficio— es que un backtest honesto casi nunca da un veredicto rotundo: da matices, condiciones y un p-value incómodo. Desconfiá de los que no.

17.7. Errores comunes

⚠ Trampas frecuentes

- **Re-usar el out-of-sample**: la más común y la más letal. Cada iteración de "ajusto y vuelvo a probar" lo degrada a in-sample.
- **Reportar el mejor de N sin decir N**: el Sharpe del mejor de 100 intentos es una estadística de orden, no una medida de habilidad.
- **Ignorar la estabilidad de los parámetros**: si el óptimo salta de 5 a 60 entre tramos del walk-forward, no estás estimando nada real.
- **Backtests demasiado cortos**: recordá $t \approx \text{Sharpe} \times \sqrt{\text{años}}$; con 2 años de datos casi nada es demostrable.
- **Confundir el drawdown máximo del backtest con el peor caso posible**: el futuro no está obligado a repetir el peor pasado; es un piso, no un techo.

17.8. Practicá en el navegador

Podés experimentar con el código del capítulo acá mismo: la celda ejecuta Python **en tu navegador**, sin instalar nada (la primera ejecución tarda unos segundos mientras carga el entorno; los paquetes que importes se instalan solos).

```
#| label: practica-navegador
import numpy as np

# Para usar el dataset real del libro (requiere conexión):
```

```
# import pandas as pd
# from pyodide.http import open_url
# url = "https://diegoonna.github.io/libro_finanzas_computacionales/datos/precios.cs
# precios = pd.read_csv(open_url(url), index_col="Date", parse_dates=True)

# Tu código acá...
```

17.9. Referencias

Parte IV.

Módulo 4: Derivados y Volatilidad

Módulo 4: Derivados y Volatilidad

El examen final: los derivados usan todo el libro a la vez

Los derivados tienen fama de ser el tema más difícil de las finanzas, y es una fama merecida — pero no porque las ideas sean oscuras, sino porque **usan todo a la vez**: probabilidad, simulación, métodos numéricos, optimización y estructura de mercado, operando juntos en un mismo instrumento. Por eso este módulo va al final: es el examen integrador de todo lo que construiste en los tres anteriores.

La historia arranca con la pregunta más elegante de las finanzas del siglo XX: *¿cuánto vale el derecho de comprar algo en el futuro?* La respuesta de **Black-Scholes** (Capítulo 18) es sorprendente: no depende de lo que nadie opine sobre el mercado, porque la opción se puede *replicar* operando el activo subyacente. Implementamos la fórmula, la damos vuelta para extraer la **volatilidad implícita** de los precios reales, y nos topamos con el descubrimiento que cambió la industria: la **sonrisa de volatilidad**, la evidencia de que el mercado no le cree del todo al modelo.

Después bajamos al llano: el **delta hedging** (Capítulo 19) es la réplica puesta en práctica, simulada día a día como la viviría un trader — con su error de cobertura, sus costos y el gamma mordiendo cerca del vencimiento. Es, en el fondo, la demostración experimental de Black-Scholes. Y el cierre (Capítulo 20) enfrenta lo que la sonrisa venía gritando: la volatilidad no es una constante, es un proceso con vida propia. Los modelos de **Heston** y **SABR** la tratan como tal, reproducen la sonrisa desde los fundamentos, y son la puerta de entrada al pricing profesional moderno.

La Figura 17.1 muestra cómo se encadenan los tres capítulos y qué traen del módulo anterior.

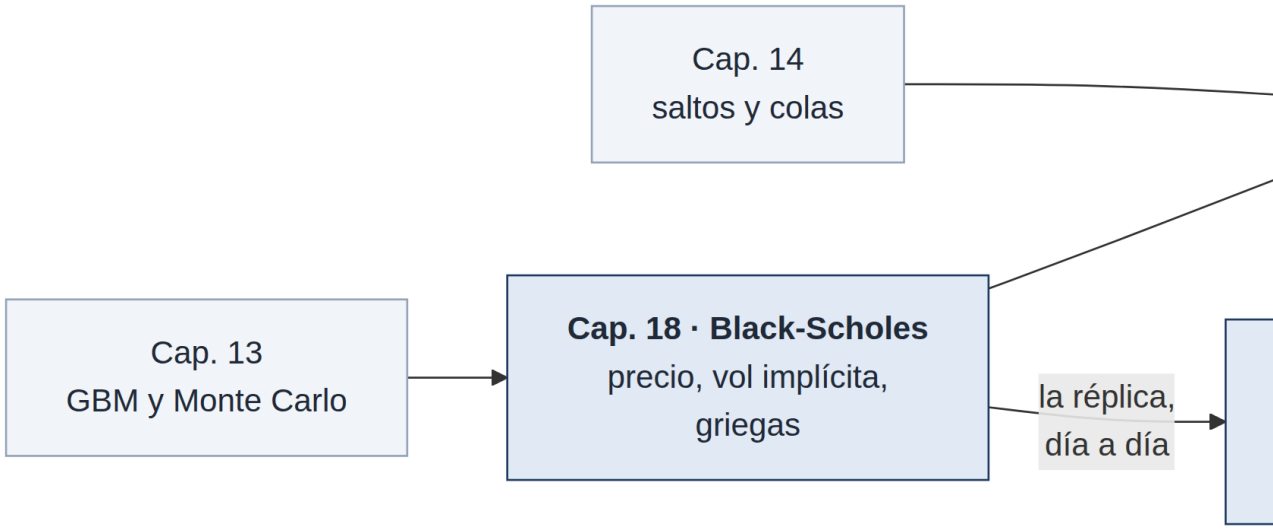


Figura 17.1.: El arco del módulo. Black-Scholes se apoya en el GBM; la réplica que lo justifica se pone a prueba en el delta hedging; la sonrisa que contradice la volatilidad constante lleva a los modelos de volatilidad estocástica.

Qué necesitás tener fresco

Este módulo apoya todo su peso sobre la simulación de Monte Carlo y el GBM (Capítulo 13), los procesos con saltos (Capítulo 14) y las nociones de colas y curtosis (Capítulos 7 y 11). Si venís directo, perfecto; si no, un repaso rápido de esos capítulos hace que este módulo fluya.

Objetivos de Aprendizaje

Al completar este módulo, vas a ser capaz de:

1. **Implementar la fórmula de Black-Scholes** para calls y puts europeas.
2. **Extraer volatilidad implícita** de precios de mercado usando Newton-Raphson.
3. **Calcular las griegas** (Delta, Gamma, Vega, Theta) analítica y numéricamente.
4. **Simular delta hedging dinámico** y analizar su P&L y su error de cobertura.
5. **Modelar la volatilidad estocástica** con Heston y SABR para capturar la sonrisa.

Recorrido Capítulo a Capítulo

Capítulo	Tema	Python	Finanzas
18	Derivados y Vol Implícita	`scipy.stats.norm`, iteración numérica	Black-Scholes, smile, griegas
19	Delta Hedging	Simulación de trading, DataFrames	Cobertura dinámica, P&L, riesgo gamma
20	Volatilidad Estocástica	Simulación de procesos acoplados	Heston, SABR, superficie de volatilidad

---

Herramientas y Librerías

SciPy

`norm.cdf`, `norm.pdf`

NumPy

Simulación vectorizada

Pandas

Logging de operaciones

Matplotlib

Payoffs, smile, P&L

Concepto Clave: Black-Scholes

! La fórmula de **Black-Scholes** para una call europea:

$$C = S_0\, N(d_1) - K\, e^{-rT}\, N(d_2)$$

$$d_1 = \frac{\ln(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}$$

Black-Scholes asume volatilidad constante, pero el mercado revela una **sonrisa de volatilidad** que contradice ese supuesto. El módulo entero puede leerse como esa tensión: primero la fórmula, después su uso real, y al final los modelos que reparan lo que le falta.

Lecturas Recomendadas

- Hull, J. (2018). *Options, Futures, and Other Derivatives*. Pearson.
- Wilmott, P. (2006). *Paul Wilmott on Quantitative Finance*. Wiley.
- Taleb, N. N. (1997). *Dynamic Hedging*. Wiley.
- Gatheral, J. (2006). *The Volatility Surface: A Practitioner's Guide*. Wiley.

18. Capítulo 18: Valuación de Opciones y Volatilidad Implícita

Black-Scholes, Newton-Raphson, la sonrisa y las griegas

Las opciones son el laboratorio perfecto de las finanzas cuantitativas: juntan probabilidad, métodos numéricos y estructura de mercado en un solo instrumento. Implementamos la fórmula de Black-Scholes, invertimos el problema para extraer la volatilidad implícita con Newton-Raphson, descubrimos la sonrisa de volatilidad —la evidencia de que el mercado no le cree a Black-Scholes— y calculamos las griegas, los tableros de sensibilidad que gobiernan la gestión de un book de opciones.

Introducción

Una **opción de compra** (*call*) da el derecho —no la obligación— de comprar un activo a un precio pactado K (el *strike*) en una fecha futura T. Una *put*, el derecho de vender. La pregunta que define al capítulo: **¿cuánto vale hoy ese derecho?**

Antes de valuar, conviene tener a la vista qué paga cada contrato al vencimiento. La Figura 18.1 muestra las cuatro posiciones básicas: el comprador paga una prima hoy a cambio de un pago asimétrico; el vendedor cobra la prima y asume ese pago.

La respuesta tiene una historia célebre. En 1973, Fischer Black, Myron Scholes y Robert Merton demostraron que —bajo ciertos supuestos— el precio de una opción **no depende de la expectativa de nadie** sobre si el activo va a subir o bajar. Depende de algo más sutil: de que la opción se puede *replicar* comprando y vendiendo dinámicamente el activo subyacente. Y si dos cosas tienen los mismos flujos, deben valer lo mismo (no-arbitraje). De esa lógica sale una fórmula cerrada, la más famosa de las finanzas.

En este capítulo la implementamos, y enseguida la damos vuelta: en el mercado real, la fórmula no se usa para calcular precios (los precios se *observan* en pantalla) sino para traducirlos a un lenguaje común: la **volatilidad implícita**.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
```

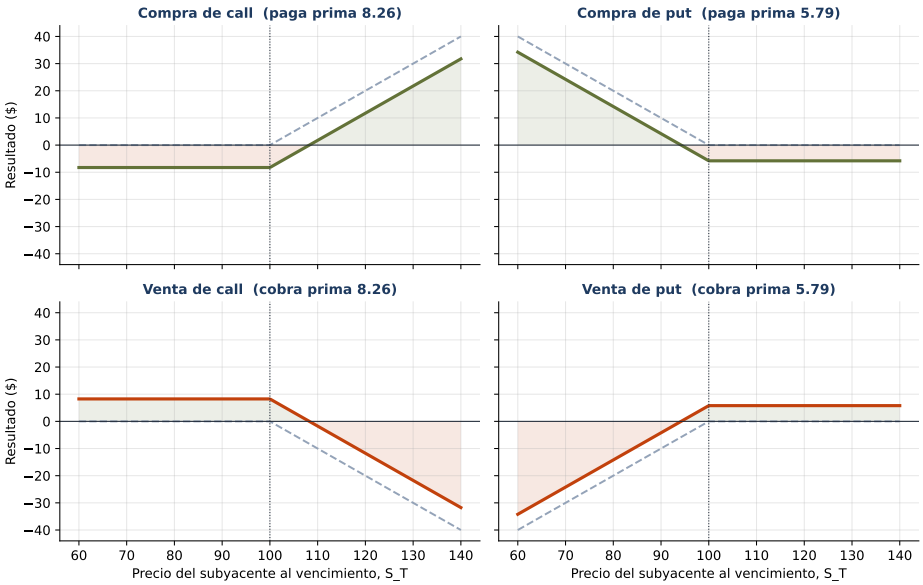


Figura 18.1.: Resultado al vencimiento de las cuatro posiciones básicas (strike K = 100, prima de Black-Scholes con 6 meses y vol 25 %). La línea punteada es el pago bruto; la llena, el resultado neto de la prima. El comprador de una opción pierde como mucho la prima; el vendedor gana como mucho la prima.

18.1. La fórmula de Black-Scholes

Para una call europea sobre un activo que sigue un GBM (Capítulo 13) con volatilidad σ, con tasa libre de riesgo r:

$$C = S_0\,N(d_1) - Ke^{-rT}N(d_2)$$

$$d_1 = \frac{\ln(S_0/K) + (r + \sigma^2/2)\,T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}$$

donde $N(\cdot)$ es la función de distribución normal estándar. La fórmula tiene una lectura económica directa que ayuda a memorizarla: es **lo que recibís menos lo que pagás, ponderado por probabilidades** (en el mundo neutral al riesgo): $N(d_2)$ es la probabilidad de ejercer, $Ke^{-rT}N(d_2)$ el valor presente esperado de lo que pagás si ejercés, y $S_0N(d_1)$ el valor esperado de lo que recibís.

```
def bs_call(S, K, T, r, sigma):
    """Precio Black-Scholes de una call europea."""
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)

def bs_put(S, K, T, r, sigma):
    """Precio Black-Scholes de una put europea."""
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    return K * np.exp(-r * T) * norm.cdf(-d2) - S * norm.cdf(-d1)
```

```python
# Un contrato concreto: call al dinero, 6 meses, vol 25%
S, K, T, r, sigma = 100, 100, 0.5, 0.05, 0.25

precio_call = bs_call(S, K, T, r, sigma)
precio_put = bs_put(S, K, T, r, sigma)

print(f"Call: ${precio_call:.4f}")
print(f"Put:  ${precio_put:.4f}")
print(f"\nVerificación de paridad put-call:")
print(f"  C - P           = {precio_call - precio_put:.4f}")
print(f"  S - K·e^(-rT)   = {S - K * np.exp(-r * T):.4f}")
```

```
Call: $8.2600
Put:  $5.7910

Verificación de paridad put-call:
  C - P           = 2.4690
  S - K·e^(-rT)   = 2.4690
```

La **paridad put-call** ($C - P = S - Ke^{-rT}$) que verificamos al final no depende de Black-Scholes ni de ningún modelo: sale de puro no-arbitraje. Por eso es el primer chequeo de cordura de cualquier implementación — si tu código la viola, está mal.

> **Los supuestos que después importarán**
>
> Black-Scholes supone: volatilidad σ **constante y conocida**, trayectorias continuas (sin saltos — ya sabemos por el Capítulo 14 que es falso), tasa constante, sin costos de transacción y posibilidad de operar continuamente. Cada supuesto se viola en la realidad, y la genialidad práctica del mercado fue inventar una forma de *medir* esas violaciones: la sonrisa de volatilidad que veremos enseguida.

18.1.1. ¿Qué mueve el precio de una opción?

Antes de invertir la fórmula, desarrollemos intuición sobre cómo responde el precio a sus inputs:

```
S_grid = np.linspace(60, 140, 100)

fig, axes = plt.subplots(1, 2, figsize=(11, 4))

# Panel 1: valor de la call vs precio del subyacente, a distintos plazos
ax = axes[0]
for T_i, estilo in [(0.05, ":"), (0.25, "--"), (1.0, "-")]:
    ax.plot(S_grid, bs_call(S_grid, K, T_i, r, sigma), estilo,
            label=f"T = {T_i} años")
ax.plot(S_grid, np.maximum(S_grid - K, 0), "k-", alpha=0.3, linewidth=3,
        label="Payoff al vencimiento")
ax.set_xlabel("Precio del subyacente S")
ax.set_ylabel("Valor de la call")
ax.set_title("El tiempo 'redondea' el payoff")
ax.legend()
ax.grid(True, alpha=0.3)

# Panel 2: valor vs volatilidad
ax = axes[1]
vol_grid = np.linspace(0.05, 0.60, 100)
for K_i in [90, 100, 110]:
    ax.plot(vol_grid * 100, bs_call(S, K_i, T, r, vol_grid), label=f"K = {K_i}")
ax.set_xlabel("Volatilidad  (%)")
ax.set_ylabel("Valor de la call")
ax.set_title("Más volatilidad, más valor - siempre")
ax.legend()
ax.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```

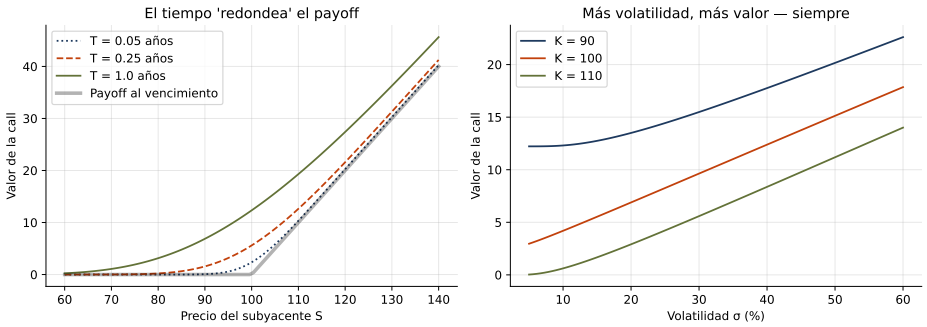


Figura 18.2.: Izquierda: valor de la call contra el precio del subyacente para tres plazos; cuanto más cerca del vencimiento, más se parece al payoff. Derecha: valor de la call contra la volatilidad para tres strikes; siempre creciente.

El panel derecho de la Figura 18.2 contiene la clave de todo el capítulo: **el precio de una opción crece monótonamente con la volatilidad**. Tiene sentido económico: la opción te protege de la baja (perdés como mucho la prima) y te deja toda la suba — más dispersión solo puede favorecerte. Esa monotonía significa que la relación precio volatilidad es **uno a uno**: dado un precio, existe una única σ que lo genera. Podemos invertir la fórmula.

18.2. Volatilidad implícita: leer la fórmula al revés

En el mercado, el precio de la opción se **observa** — lo pone la oferta y la demanda. De los cinco inputs de Black-Scholes, cuatro son observables (S, K, T, r). El único invisible es σ. Entonces el uso real de la fórmula es el inverso: *¿qué volatilidad está "implicando" este precio de mercado?*

$$\sigma_{impl}: \quad C_{BS}(S, K, T, r, \sigma_{impl}) = C_{mercado}$$

La Figura 18.3 muestra los dos usos de la misma fórmula: de ida, para valuar; de vuelta, para traducir precios observados a volatilidad.

No hay fórmula cerrada para despejar σ; hay que resolver numéricamente. El método clásico es **Newton-Raphson**: arrancar de un σ tentativo y corregirlo usando la pendiente de la función,

$$\sigma_{n+1} = \sigma_n - \frac{C_{BS}(\sigma_n) - C_{mercado}}{\partial C / \partial \sigma}$$

donde la derivada del denominador tiene nombre propio —**vega**— y fórmula cerrada. Cada iteración pregunta: "¿me pasé o me quedé corto? ¿y cuán sensible es el precio acá?" y da un paso proporcional al error.

```
def vega(S, K, T, r, sigma):
    """Vega: derivada del precio respecto de la volatilidad."""
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    return S * np.sqrt(T) * norm.pdf(d1)

def implied_vol(precio_mercado, S, K, T, r, sigma0=0.3, tol=1e-8, max_iter=100):
    """Volatilidad implícita de una call por Newton-Raphson."""
    sigma = sigma0
    for _ in range(max_iter):
        diff = bs_call(S, K, T, r, sigma) - precio_mercado
```

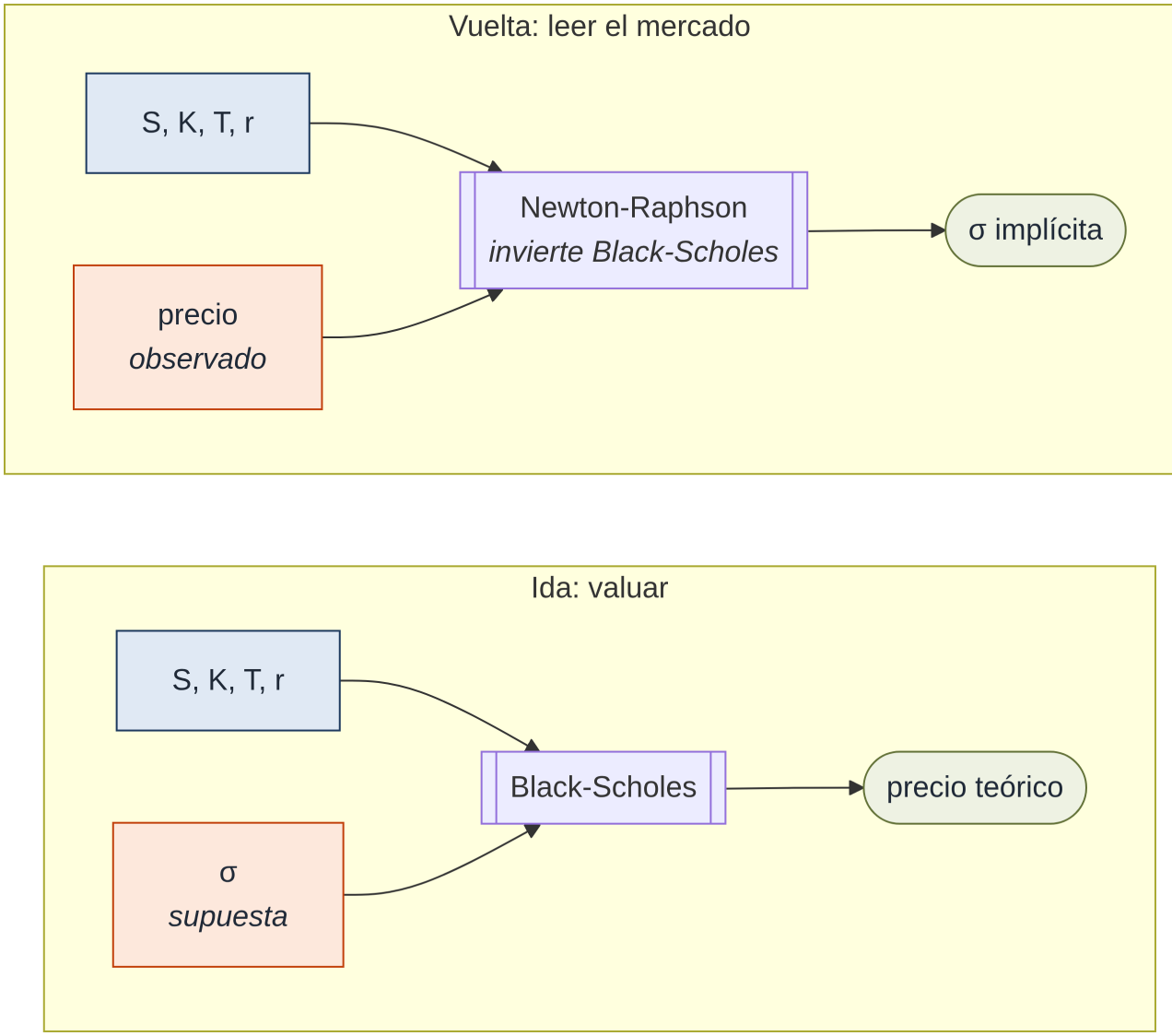


Figura 18.3.: La misma fórmula en dos direcciones. De ida, la volatilidad es un input y el precio es el resultado. De vuelta, el precio lo pone el mercado y lo que se despeja es la volatilidad implícita; como no hay fórmula cerrada, se resuelve con Newton-Raphson.

```
        if abs(diff) < tol:
            return sigma
        v = vega(S, K, T, r, sigma)
        if v < 1e-12:                        # vega ~0: Newton se queda sin pendiente
            raise RuntimeError("Vega demasiado chica; usar bisección/brentq")
        sigma -= diff / v
    raise RuntimeError("No convergió")

# Test de ida y vuelta: precio generado con  =30% → ¿recuperamos 30%?
precio_test = bs_call(100, 100, 0.5, 0.05, 0.30)
vol_recuperada = implied_vol(precio_test, 100, 100, 0.5, 0.05)
print(f"  original: 30.00%  →   recuperada: {vol_recuperada:.4%}")
```

```
  original: 30.00%  →   recuperada: 30.0000%
```

La Figura 18.4 muestra por qué Newton converge tan rápido. Cada paso reemplaza la curva por su recta tangente (cuya pendiente es el vega) y salta al punto donde esa recta alcanza el precio de mercado.

Newton converge en un puñado de iteraciones porque el precio es suave y monótono en σ. La volatilidad implícita es **el idioma en que se cotizan las opciones**: los operadores no dicen "la call vale $8.26", dicen "está en 30 de vol" — un número comparable entre strikes, plazos y subyacentes.

18.3. La sonrisa de volatilidad

Y acá viene el descubrimiento que cambió la industria. Si Black-Scholes fuera literalmente cierto, la volatilidad implícita sería **la misma para todos los strikes** — es un parámetro del activo, no del contrato. Grafiquemos qué se observa en el mercado real:

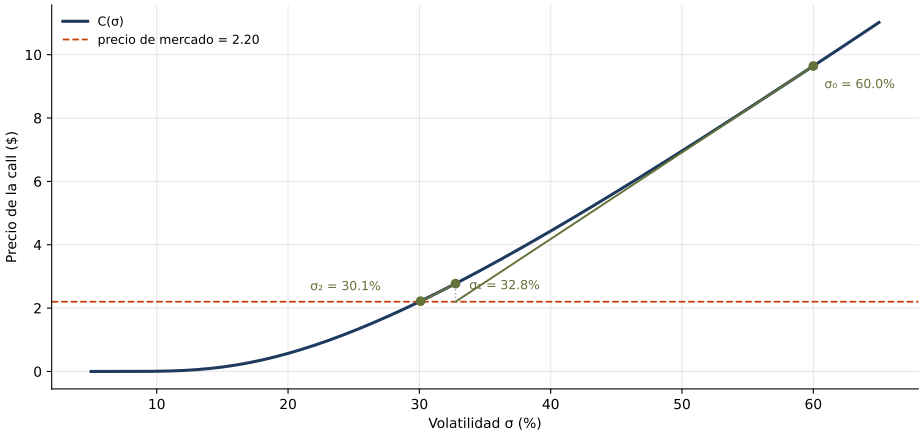


Figura 18.4.: Newton-Raphson buscando la volatilidad implícita de una call fuera del dinero (K = 125, 6 meses) cotizada a 30 % de vol, arrancando lejos, en = 60 %. Cada tangente (pendiente = vega) cae sobre el precio de mercado en el siguiente intento; al tercer paso el error ya es invisible a esta escala.

```
strikes = np.arange(80, 121, 2)
S, T, r = 100, 0.5, 0.05

# El mercado cotiza opciones fuera del dinero más "caras" (en vol):
# reproducimos precios de mercado con una vol que depende del strike
vols_mercado = 0.20 + 0.001 * (strikes - 100) ** 2 + 0.0015 * np.maximum(100 - strik
precios_mkt = [bs_call(S, K_i, T, r, v) for K_i, v in zip(strikes, vols_mercado)]

# Recuperamos la vol implícita de cada precio (así se construye el gráfico real)
vols_impl = [implied_vol(p, S, K_i, T, r) for p, K_i in zip(precios_mkt, strikes)]

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(strikes, np.array(vols_impl) * 100, "o-", color="#1e3a5f",
        label="Vol implícita observada")
ax.axhline(20, color="#c2410c", linestyle="--", alpha=0.6,
           label="Lo que Black-Scholes predice (  constante)")
ax.axvline(100, color="#94a3b8", linestyle=":", alpha=0.5)
ax.annotate("puts protectivas\n(caras)", xy=(84, 24.5), fontsize=9, ha="center")
ax.set_xlabel("Strike K")
ax.set_ylabel("Volatilidad implícita (%)")
ax.set_title("La sonrisa (smirk) de volatilidad")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

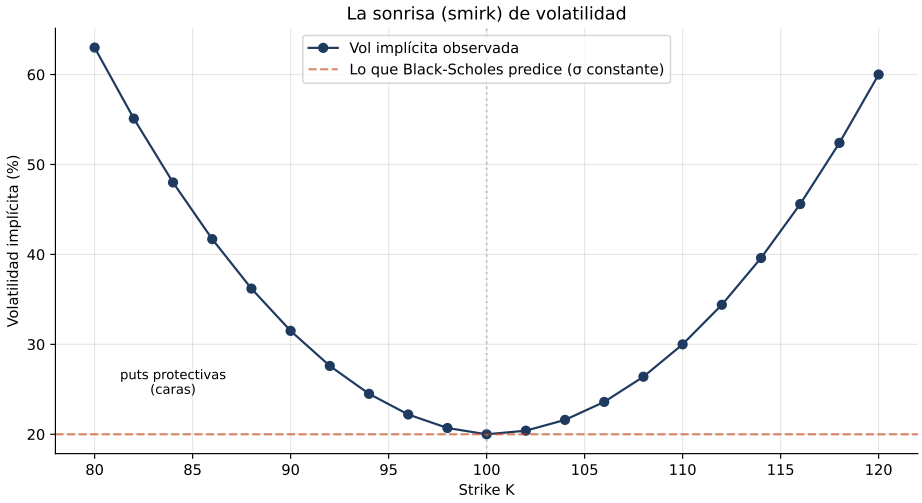


Figura 18.5.: Volatilidad implícita por strike, recuperada de precios de mercado. Si Black-Scholes fuera literalmente cierto, todos los puntos caerían sobre la línea horizontal.

La curva de la Figura 18.5 no es plana: las opciones **fuera del dinero se cotizan a volatilidades más altas**, con la rama izquierda (puts de strikes bajos) típicamente más empinada — por su forma, *sonrisa* o *mueca* (smirk). ¿Qué está diciendo el mercado?

- **Las colas existen**: los compradores pagan más (en vol) por las opciones que pagan en escenarios extremos, porque saben —desde el crash de 1987, cuando nació la sonrisa— que esos escenarios son más probables de lo que la normal admite. Es la curtosis del Capítulo 14 cotizada en pantalla.
- **La asimetría también**: la rama izquierda más cara refleja que los derrumbes son más violentos que los rallies, y que hay demanda estructural de protección (puts como seguro).

La Figura 18.6 muestra la cadena completa con una put fuera del dinero.

Si el mercado cree en una distribución con la cola izquierda más pesada que la lognormal de Black-Scholes, esa put vale más; y la única forma de que Black-Scholes reproduzca ese precio más alto es con una volatilidad más alta.

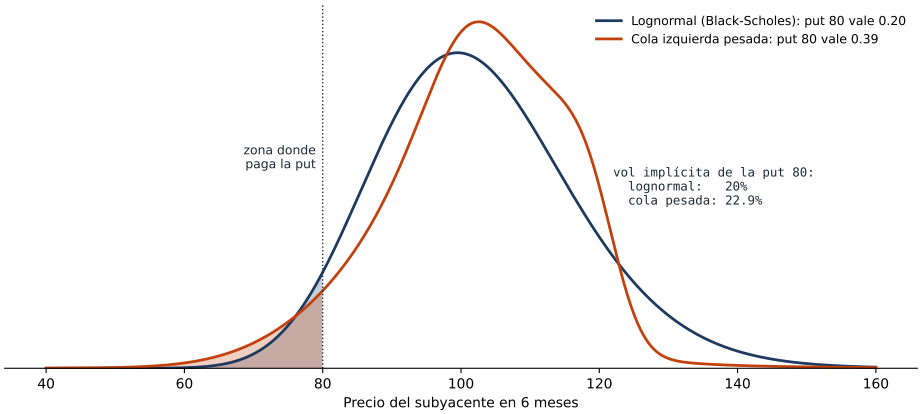


Figura 18.6.: Por qué existe la sonrisa. Dos distribuciones del precio a 6 meses con la misma media y la misma volatilidad (20 %): la lognormal de Black-Scholes y otra con la cola izquierda pesada. La put de strike 80 solo paga en la zona sombreada; con la cola pesada vale bastante más, y para reproducir ese precio Black-Scholes necesita una volatilidad implícita más alta.

! La sonrisa es el puente del libro

Todo converge acá: los retornos tienen colas pesadas y asimetría (Cap. 7), los modelos con saltos las generan (Cap. 14), y la sonrisa es el mercado **poniéndoles precio**. Black-Scholes no está "mal" — es la vara con la que el mercado mide sus propias desviaciones de la normalidad. Los modelos que *reproducen* la sonrisa (Heston, SABR)

son el tema del Capítulo 20.

18.4. Las griegas: el tablero de control

Quien tiene una posición en opciones necesita saber cómo cambia su valor cuando se mueve el mundo. Las **griegas** son esas derivadas parciales:

Griega	Derivada	Pregunta que responde
Delta (Δ)	$\partial C/\partial S$	¿Cuánto gano si el subyacente sube \$1?
Gamma (Γ)	$\partial^2 C/\partial S^2$	¿Cuán rápido cambia mi delta? (curvatura)
Vega	$\partial C/\partial \sigma$	¿Cuánto gano si la vol sube 1 punto?
Theta (Θ)	$\partial C/\partial t$	¿Cuánto pierdo por el paso de un día?

Para Black-Scholes existen fórmulas cerradas (delta de una call = $N(d_1)$), pero conviene dominar el método general de **diferencias finitas** —perturbar el input, repreciar, dividir— porque funciona con *cualquier* modelo de pricing, tenga fórmula o sea un Monte Carlo:

$$\Delta \approx \frac{C(S+h) - C(S-h)}{2h}$$

```python
def griegas_call(S, K, T, r, sigma, dS=0.01, dsig=0.0001, dT=1/365):
    """Griegas de una call por diferencias finitas centradas."""
    delta = (bs_call(S + dS, K, T, r, sigma) - bs_call(S - dS, K, T, r, sigma)) / (2
    gamma = (bs_call(S + dS, K, T, r, sigma) - 2 * bs_call(S, K, T, r, sigma)
             + bs_call(S - dS, K, T, r, sigma)) / dS**2
    vega_ = (bs_call(S, K, T, r, sigma + dsig) - bs_call(S, K, T, r, sigma - dsig))
    theta = (bs_call(S, K, T - dT, r, sigma) - bs_call(S, K, T, r, sigma)) / dT
    return {"Delta": delta, "Gamma": gamma, "Vega": vega_, "Theta (anual)": theta}

g = griegas_call(100, 100, 0.5, 0.05, 0.25)
for nombre, valor in g.items():
    print(f"  {nombre:>14}: {valor:>+10.4f}")

# Verificación contra la fórmula cerrada del delta
d1 = (np.log(100/100) + (0.05 + 0.5 * 0.25**2) * 0.5) / (0.25 * np.sqrt(0.5))
print(f"\n  Delta analítico N(d1) = {norm.cdf(d1):+.4f}  (coincide )")
```

```
           Delta:    +0.5909
           Gamma:    +0.0220
            Vega:   +27.4743
   Theta (anual):    -9.4199

  Delta analítico N(d1) = +0.5909  (coincide )
```

Interpretá los números: delta 0.59 (la call al dinero se mueve ~59 centavos por cada dólar del subyacente), gamma positivo (ese delta *crece* si el activo sube — la convexidad que hace valiosas a las opciones), vega positivo (la opción es una posición comprada en volatilidad) y theta negativo (cada

día que pasa, el derecho vale menos: el tiempo es el alquiler que paga el comprador).

Delta y gamma tienen una lectura geométrica directa, y la Figura 18.7 la muestra: el delta es la pendiente de la curva de precio en el punto actual; el gamma, su curvatura. La recta del delta alcanza para movimientos chicos; para movimientos grandes, hace falta la parábola que agrega el gamma.

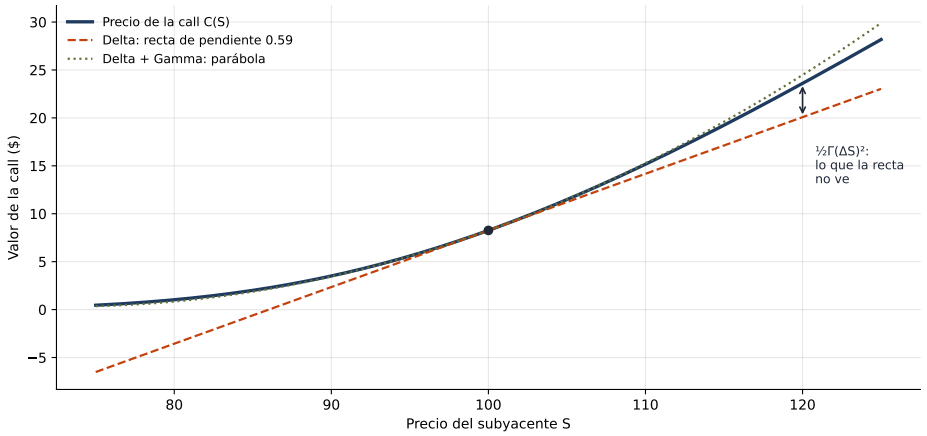


Figura 18.7.: Delta y gamma como geometría. La recta es la aproximación de primer orden (delta); la parábola agrega la curvatura (gamma). La distancia entre la recta y la curva, $\frac{1}{2}\Gamma(\Delta S)^2$, crece con el cuadrado del movimiento: es el término que el próximo capítulo convierte en error de cobertura.

```
S_grid = np.linspace(60, 140, 120)

fig, axes = plt.subplots(1, 2, figsize=(11, 4))

ax = axes[0]
for T_i, estilo in [(0.05, ":"), (0.5, "-")]:
    deltas = [griegas_call(s, 100, T_i, 0.05, 0.25)["Delta"] for s in S_grid]
```

```
    ax.plot(S_grid, deltas, estilo, label=f"T = {T_i}")
ax.set_xlabel("S")
ax.set_ylabel("Delta")
ax.set_title("Delta: de 0 a 1, brusco cerca del vencimiento")
ax.legend()
ax.grid(True, alpha=0.3)

ax = axes[1]
for T_i, estilo in [(0.05, ":"), (0.5, "-")]:
    gammas = [griegas_call(s, 100, T_i, 0.05, 0.25)["Gamma"] for s in S_grid]
    ax.plot(S_grid, gammas, estilo, label=f"T = {T_i}")
ax.set_xlabel("S")
ax.set_ylabel("Gamma")
ax.set_title("Gamma: se concentra en el strike")
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

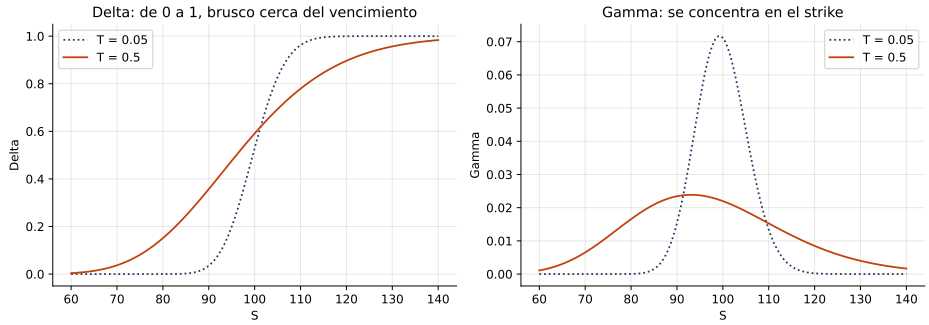


Figura 18.8.: Delta (izquierda) y gamma (derecha) de la call en función del precio del subyacente, a 6 meses y a 18 días del vencimiento. Cerca del vencimiento, el delta salta de 0 a 1 en un rango estrecho y el gamma se vuelve un pico alrededor del strike.

El gamma de la Figura 18.8, concentrado alrededor del strike y cada vez más puntiagudo al acercarse el vencimiento, es el protagonista del próximo capítulo: es lo que hace que cubrir una opción **cerca del vencimiento y cerca del strike** sea un deporte de riesgo.

> **Simulador interactivo**
>
> En la versión web de este capítulo hay un tablero donde podés mover S, K, T, σ y r y ver cómo responden el precio y las griegas.

18.5. Errores comunes

⚠ Trampas frecuentes

- **Unidades del theta**: la derivada es "por año"; dividí por 365 para el decaimiento diario que se reporta en las pantallas.
- **Newton sin red**: lejos del dinero, el vega es casi cero y Newton-Raphson diverge. Para uso robusto, combiná con bisección (`scipy.optimize.brentq`) que garantiza convergencia.
- **Volatilidad en las unidades equivocadas**: $\sigma = 0.25$, no 25. Un factor 100 en la vol produce precios absurdos silenciosamente.
- **Aplicar Black-Scholes a opciones americanas**: la fórmula es para ejercicio *europeo* (solo al vencimiento). Para americanas con dividendos, el ejercicio temprano vale y se necesitan árboles o métodos numéricos.
- **Paso de diferencias finitas mal elegido**: h demasiado grande sesga; demasiado chico amplifica el error de redondeo. Los valores del código (dS=0.01) son un buen punto medio para este problema.

18.6. Practicá en el navegador

Podés experimentar con el código del capítulo acá mismo: la celda ejecuta Python **en tu navegador**, sin instalar nada (la primera ejecución tarda unos segundos mientras carga el entorno; los paquetes que importes se instalan solos).

```
#| label: practica-navegador
import numpy as np
from scipy.stats import norm

# Tu código acá...
```

18.7. Referencias

19. Capítulo 19: Simulación de Cobertura Dinámica (Delta Hedging)

Replicar una opción operando el subyacente

El capítulo más integrador del libro: simulamos el día a día de un trader que vendió una call y se cubre recalculando su delta y rebalanceando acciones cada jornada. En el camino descubrimos la idea que fundamenta Black-Scholes —una opción se puede replicar—, medimos el error de cobertura, entendemos por qué el gamma lo gobierna, y estudiamos cómo la frecuencia de rebalanceo y los costos cambian el resultado.

Introducción

Imaginate que trabajás en la mesa de derivados de un banco y un cliente te compra una call. Cobraste la prima, pero ahora tenés un problema: si la acción sube fuerte, el payoff que vas a tener que pagar crece sin límite. **¿Cómo dormís tranquilo?**

La respuesta es la idea más profunda de todo el pricing de derivados: **cubrirse replicando**. Si la call tiene delta 0.63, su valor se mueve como 0.63 acciones; entonces comprás 0.63 acciones y tu posición combinada (call vendida + acciones) queda momentáneamente **insensible** al precio. El problema es que el delta cambia cuando el precio se mueve (eso es el gamma) y cuando pasa el tiempo — así que la cobertura hay que **rebalancearla continuamente**. De ahí el nombre: cobertura dinámica.

Y acá la conexión conceptual con el capítulo anterior: Black y Scholes derivaron su fórmula mostrando que esta estrategia de réplica, llevada al límite continuo, reproduce el payoff de la opción **exactamente**, sin importar hacia dónde vaya el mercado. El precio de la opción *es* el costo de replicarla. La simulación de este capítulo es, en el fondo, **la demostración experimental de Black-Scholes**.

Vamos a construirla completa, integrando todo el libro: GBM (Cap. 13), Black-Scholes y griegas (Cap. 18), y pandas para el registro de operaciones (Cap. 6).

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from scipy.stats import norm

rng = np.random.default_rng(42)

# El contrato que vendimos y el mercado
S0, K, T = 100, 100, 1.0     # call al dinero, 1 año
r, sigma = 0.05, 0.25
n_dias = 252
dt = T / n_dias

def bs_call(S, K, T, r, sigma):
    """Precio Black-Scholes de una call (maneja T=0 devolviendo el payoff)."""
    if T <= 0:
        return np.maximum(S - K, 0.0)
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)

def bs_delta(S, K, T, r, sigma):
    """Delta de la call: N(d1). En el vencimiento, escalón 0/1."""
    if T <= 0:
        return float(S > K)
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    return norm.cdf(d1)
```

La Figura 19.1 muestra qué significa "quedar insensible al precio". La call vendida pierde valor cuando el precio sube; las Δ acciones compradas lo ganan; la suma queda plana justo en el precio actual. Pero solo ahí: como la call es curva y las acciones son una recta, la suma se dobla hacia abajo apenas el precio se aleja. Esa curvatura es el gamma, y es la razón por la que hay que rebalancear.

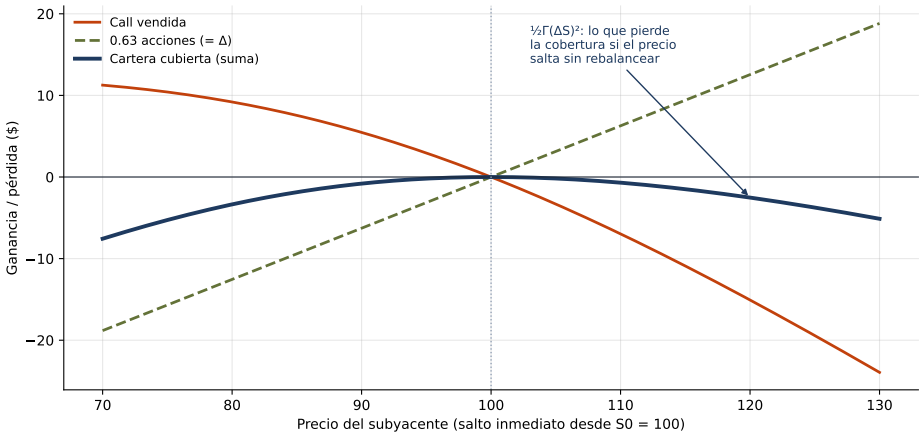


Figura 19.1.: Resultado inmediato (antes de que pase el tiempo) de cada pieza de la cobertura si el precio salta desde S0 = 100. La call vendida es cóncava; las Δ acciones, una recta tangente; la cartera cubierta es plana en S0 pero pierde ante movimientos grandes en cualquier dirección. Esa pérdida, $\frac{1}{2}\Gamma(\Delta S)^2$, es lo que el rebalanceo va corrigiendo.

19.1. La mecánica de un día de trading

La rutina diaria del trader cubierto, que vamos a codificar tal cual:

1. **Amanece**: el precio se movió. Recalculás el delta de la opción con el precio nuevo y el tiempo restante.
2. **Rebalanceás**: si el delta subió de 0.55 a 0.60, comprás 0.05 acciones más; si bajó, vendés. El cash de esas operaciones entra o sale de tu cuenta.
3. **El cash devenga interés** (o pagás por el financiamiento si estás en descubierto).
4. Al vencimiento, pagás el payoff al cliente y cerrás la posición en acciones.

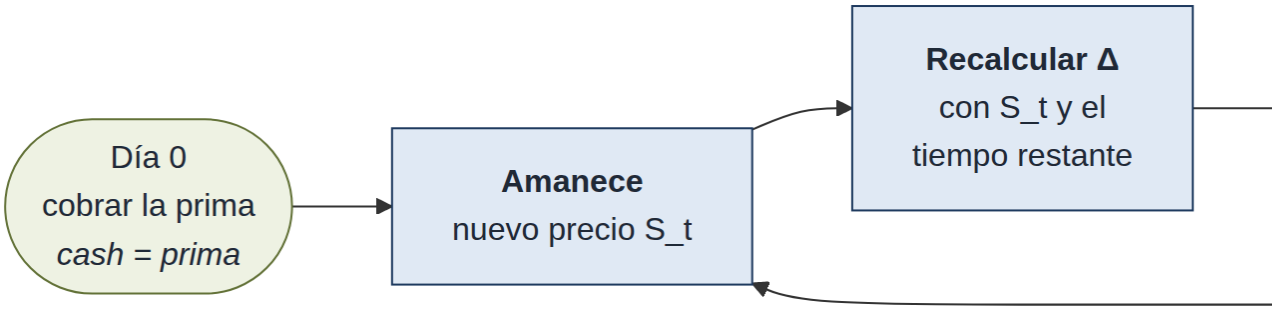


Figura 19.2.: El ciclo diario del delta hedging. El bucle se repite una vez por día; al vencimiento, lo que queda en la cartera replicante se compara con el payoff que hay que pagar, y la diferencia es el error de cobertura.

Si la réplica es buena, la prima cobrada al inicio + los resultados del trading deberían cubrir **casi exactamente** el payoff. La diferencia es el **error de cobertura**.

```
# Un escenario de mercado: el precio evoluciona como GBM
Z = rng.standard_normal(n_dias)
S = np.empty(n_dias + 1)
S[0] = S0
for t in range(1, n_dias + 1):
    S[t] = S[t-1] * np.exp((r - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z[t-1])

print(f"Precio final: ${S[-1]:.2f} → la call termina "
      f"{'DENTRO del dinero (habrá que pagar)' if S[-1] > K else 'fuera del dinero (
```

```
Precio final: $84.10 → la call termina fuera del dinero (expira sin valor)
```

```
prima = bs_call(S0, K, T, r, sigma)

registros = []
cash = prima          # arrancamos con la prima cobrada
acciones = 0.0        # y sin acciones

for t in range(n_dias + 1):
    tiempo_restante = T - t * dt
    delta = bs_delta(S[t], K, tiempo_restante, r, sigma)

    # Rebalanceo: llevamos la posición en acciones al delta de hoy
    trade = delta - acciones
    cash -= trade * S[t]          # comprar acciones consume cash (vender lo repone)
    acciones = delta

    # El cash devenga la tasa libre de riesgo hasta mañana
    if t < n_dias:
        cash *= np.exp(r * dt)

    # Mark-to-market: ¿cómo venimos vs. la opción que debemos?
```

```
    valor_opcion = bs_call(S[t], K, tiempo_restante, r, sigma)
    valor_cartera = acciones * S[t] + cash

    registros.append({
        "dia": t, "precio": S[t], "delta": delta, "trade": trade,
        "cash": cash, "valor_opcion": valor_opcion,
        "valor_cartera": valor_cartera,
        "pnl": valor_cartera - valor_opcion,
    })

df = pd.DataFrame(registros)

payoff_final = max(S[-1] - K, 0)
error_hedge = df.iloc[-1]["valor_cartera"] - payoff_final

print(f"Prima cobrada al inicio:  ${prima:8.2f}")
print(f"Payoff a pagar al cliente:${payoff_final:8.2f}")
print(f"Valor final de la réplica:${df.iloc[-1]['valor_cartera']:8.2f}")
print(f"Error de cobertura:       ${error_hedge:8.2f}  "
      f"({error_hedge / prima:+.1%} de la prima)")
```

```
Prima cobrada al inicio:  $   12.34
Payoff a pagar al cliente:$    0.00
Valor final de la réplica:$    0.91
Error de cobertura:       $    0.91  (+7.4% de la prima)
```

Leé el resultado con perspectiva: el precio se movió durante un año entero —quizás terminó 20 % arriba o abajo— y sin embargo el trader cubierto termina con un error de **centavos por cada 100 de prima**. No adivinó el mercado ni le hizo falta: la réplica transformó una posición especulativa en un negocio de fabricación. El banco "fabrica" la opción comprando y vendiendo el subyacente, y su ganancia es el margen sobre el costo de fabricación — no una apuesta direccional.

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

ax = axes[0, 0]
ax.plot(df["dia"], df["precio"], linewidth=1.2)
ax.axhline(K, color="#c2410c", linestyle="--", alpha=0.6, label=f"Strike K={K}")
ax.set_title("El escenario: precio del subyacente")
ax.set_ylabel("Precio ($)")
ax.legend()
ax.grid(True, alpha=0.3)

ax = axes[0, 1]
ax.plot(df["dia"], df["delta"], color="#637238", linewidth=1.2)
ax.set_title("La posición: delta = acciones en cartera")
ax.set_ylabel("Delta")
ax.grid(True, alpha=0.3)

ax = axes[1, 0]
ax.plot(df["dia"], df["valor_cartera"], label="Cartera replicante", linewidth=1.2)
ax.plot(df["dia"], df["valor_opcion"], "--", label="Valor de la opción", linewidth=1
ax.set_title("La réplica en acción: dos curvas casi idénticas")
ax.set_xlabel("Día")
ax.set_ylabel("Valor ($)")
ax.legend()
ax.grid(True, alpha=0.3)

ax = axes[1, 1]
ax.bar(df["dia"], df["trade"], color="#1e3a5f", alpha=0.7, width=1.5)
ax.set_title("Los rebalanceos diarios")
ax.set_xlabel("Día")
ax.set_ylabel("Acciones operadas")
ax.grid(True, alpha=0.3)

plt.tight_layout()
```

```
plt.show()
```

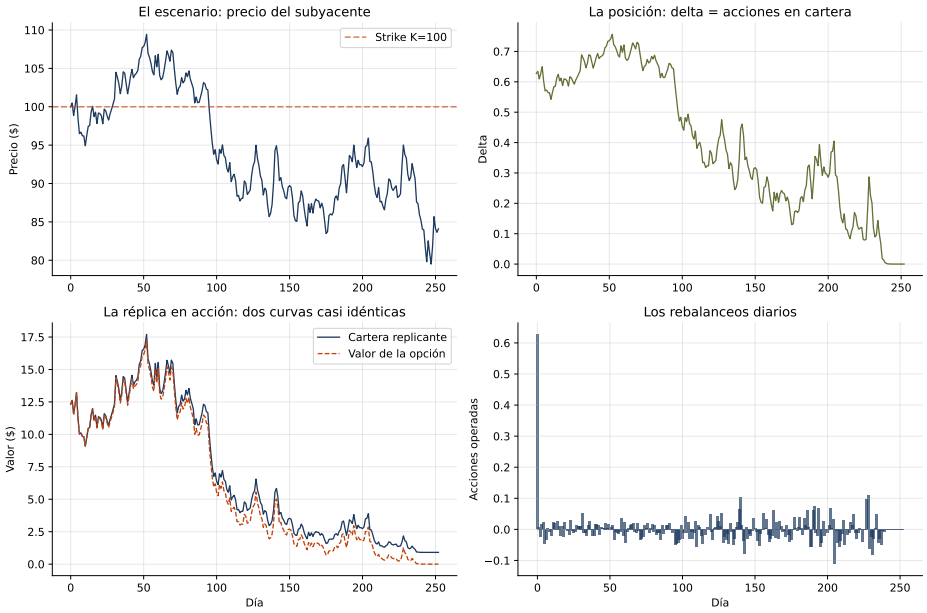


Figura 19.3.: Un año de delta hedging. Arriba: el escenario de precio y el delta (las acciones en cartera). Abajo: la cartera replicante contra el valor de la opción, y el tamaño de cada rebalanceo diario.

Fijate dos cosas en la Figura 19.3. Primero, el panel inferior izquierdo: la cartera replicante **persigue al valor de la opción** durante todo el año — esa es la réplica funcionando. Segundo, el panel del delta comparado con el del precio: cuando el precio cruza el strike, el delta se mueve rápido y los rebalanceos se agrandan. Ahí gobierna el **gamma**.

Comprar caro y vender barato, por diseño

Observá el patrón de los trades: cuando el precio **sube**, el delta sube y el trader **compra**; cuando **baja**, **vende**. El cubridor de una opción vendida compra en las subas y vende en las bajas — sistemáticamente pierde en cada ida y vuelta. Ese costo acumulado no es un defecto: **es exactamente el valor temporal de la opción** que cobró como prima. Si la volatilidad realizada termina siendo la que cotizó (σ), prima y costo de cobertura se cancelan. Si el mercado se mueve *más* que lo cotizado, el cubridor pierde; si se mueve *menos*, gana. Vender opciones cubiertas es, en el fondo, **vender volatilidad**.

La Figura 19.4 verifica el patrón con los datos de la simulación: cada punto es un día, con el movimiento del precio en el eje horizontal y la cantidad de acciones operadas en el vertical.

19.2. El error de cobertura: de dónde viene y cómo se achica

En el mundo ideal de Black-Scholes el rebalanceo es *continuo* y el error es cero. Nosotros rebalanceamos una vez por día, y entre rebalanceo y rebalanceo el delta queda desactualizado. ¿Cuánto cuesta esa discretización? La respuesta la da el **gamma**: entre dos rebalanceos, el error de la aproximación lineal es $\frac{1}{2}\Gamma(\Delta S)^2$ — cuadrático en el movimiento del precio.

La teoría predice algo verificable: el error de cobertura tiene media ~0 y su desvío **cae como** $1/\sqrt{n}$ con la frecuencia de rebalanceo. Comprobémoslo con un experimento masivo:

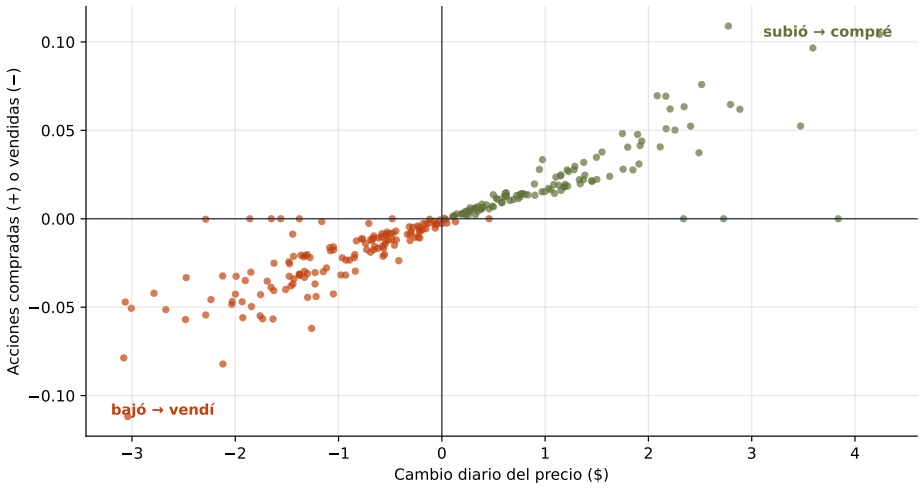


Figura 19.4.: Cada punto es un día de la simulación: cambio del precio contra acciones operadas ese día. Casi todos caen en los cuadrantes «subió y compré» o «bajó y vendí» (los que quedan en cero son días en que la opción estaba tan dentro o tan fuera del dinero que el delta casi no se movió): el cubridor de una call vendida compra caro y vende barato, de forma sistemática.

```
def error_hedging(n_rebalanceos, n_sims, rng):
    """Distribución del error de cobertura para una frecuencia dada."""
    dt_h = T / n_rebalanceos
    errores = np.empty(n_sims)

    for i in range(n_sims):
        Z = rng.standard_normal(n_rebalanceos)
        S_t, cash, acc = S0, prima, 0.0
        for t in range(n_rebalanceos):
            delta = bs_delta(S_t, K, T - t * dt_h, r, sigma)
            cash -= (delta - acc) * S_t
            acc = delta
            cash *= np.exp(r * dt_h)
            S_t *= np.exp((r - 0.5 * sigma**2) * dt_h + sigma * np.sqrt(dt_h) * Z[t]
        errores[i] = acc * S_t + cash - max(S_t - K, 0)
    return errores

frecuencias = {"Mensual (12)": 12, "Semanal (52)": 52, "Diaria (252)": 252}
resultados = {nombre: error_hedging(n, 400, np.random.default_rng(7))
              for nombre, n in frecuencias.items()}

print(f"{'Rebalanceo':<16}{'Error medio':>12}{'Desvío':>10}")
for nombre, err in resultados.items():
    print(f"{nombre:<16}{err.mean():>12.3f}{err.std():>10.3f}")
```

```
Rebalanceo       Error medio    Desvío
Mensual (12)           0.085     2.181
Semanal (52)           0.079     1.130
Diaria (252)          -0.009     0.567
```

```
fig, ax = plt.subplots(figsize=(9, 4.5))
for nombre, err in resultados.items():
```

```
    ax.hist(err, bins=40, alpha=0.55, label=nombre, density=True)
ax.axvline(0, color="black", alpha=0.4)
ax.set_xlabel("Error de cobertura ($)")
ax.set_ylabel("Densidad")
ax.set_title("Rebalancear más seguido angosta la distribución del error")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

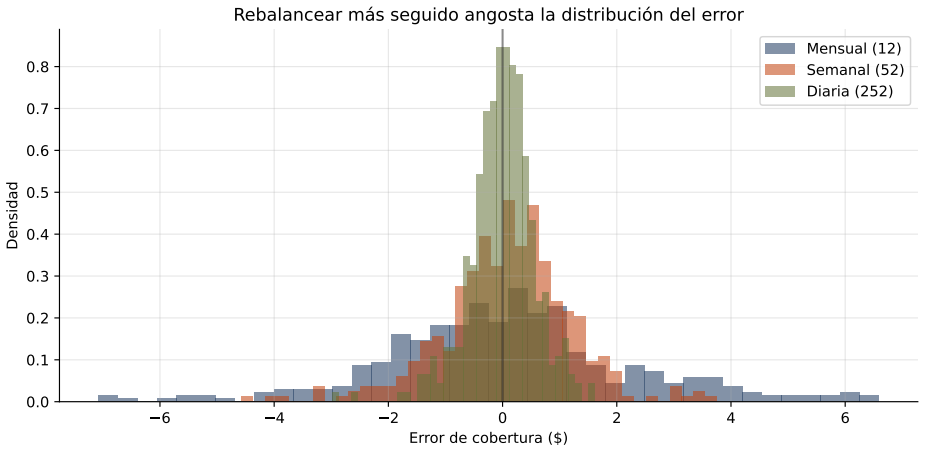


Figura 19.5.: Distribución del error de cobertura en 400 escenarios para tres frecuencias de rebalanceo. Las tres están centradas en cero; rebalancear más seguido las angosta.

En la Figura 19.5, la media es ~0 en todos los casos (la cobertura no está sesgada) y el desvío se achica con la raíz de la frecuencia: de mensual a diario hay un factor $\sqrt{252/12} \approx 4.6$. En el límite continuo, el error desaparece — **esa es la demostración experimental del argumento de Black-Scholes**.

19.2.1. El freno del mundo real: costos de transacción

¿Entonces por qué no rebalancear cada minuto? Porque cada operación **cuesta** (comisiones, spread). Más frecuencia = menos error de gamma pero más costos: hay un punto óptimo intermedio, y encontrarlo es un problema clásico de la microestructura de la cobertura.

```
def error_con_costos(n_rebalanceos, costo_bps, n_sims, rng):
    dt_h = T / n_rebalanceos
    errores = np.empty(n_sims)
    for i in range(n_sims):
        Z = rng.standard_normal(n_rebalanceos)
        S_t, cash, acc = S0, prima, 0.0
        for t in range(n_rebalanceos):
            delta = bs_delta(S_t, K, T - t * dt_h, r, sigma)
            trade = delta - acc
            cash -= trade * S_t + abs(trade) * S_t * costo_bps / 1e4  # costo propor
            acc = delta
            cash *= np.exp(r * dt_h)
            S_t *= np.exp((r - 0.5 * sigma**2) * dt_h + sigma * np.sqrt(dt_h) * Z[t]
        errores[i] = acc * S_t + cash - max(S_t - K, 0)
    return errores

print(f"Con costos de 5 puntos básicos por operación:")
print(f"{'Rebalanceo':<16}{'Error medio':>12}{'Desvío':>10}")
for nombre, n in frecuencias.items():
    err = error_con_costos(n, costo_bps=5, n_sims=400, rng=np.random.default_rng(7))
    print(f"{nombre:<16}{err.mean():>12.3f}{err.std():>10.3f}")
```

```
Con costos de 5 puntos básicos por operación:
Rebalanceo       Error medio    Desvío
Mensual (12)           0.002     2.186
Semanal (52)          -0.064     1.132
```

```
Diaria (252)          -0.289      0.588
```

Ahora el error medio es **negativo y empeora con la frecuencia**: los costos son un peaje que se paga en cada rebalanceo. El trader real equilibra: rebalancea por bandas ("solo si el delta se desvió más de X"), usa el gamma para decidir cuándo vale la pena, y cotiza la opción con un margen que cubra este peaje.

La Figura 19.6 junta los dos experimentos en un barrido más fino de frecuencias. A la izquierda, sin costos, el desvío cae siguiendo la recta $1/\sqrt{n}$ de la teoría. A la derecha, el peaje: como las dos funciones reciben la misma semilla, simulan exactamente los mismos escenarios, y la diferencia entre sus resultados es el costo de transacción puro, sin ruido de muestreo encima.

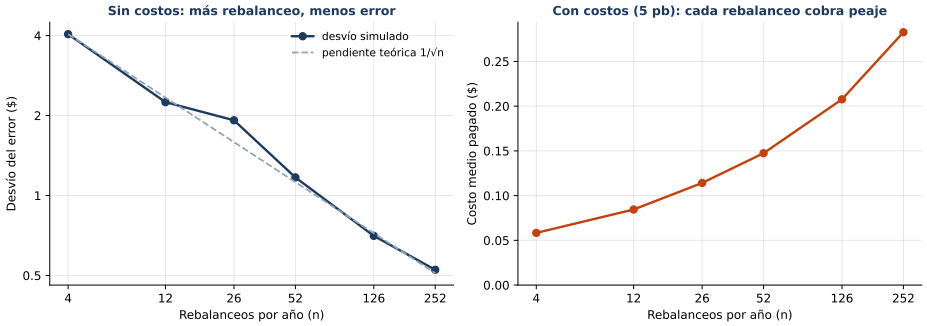


Figura 19.6.: Seis frecuencias de rebalanceo, 300 escenarios cada una. Izquierda (sin costos, escalas log): el desvío del error cae en paralelo a la recta $1/\sqrt{n}$. Derecha: costo de transacción medio pagado en el año con 5 pb por operación, que crece con la frecuencia. El rebalanceo óptimo equilibra ambos efectos.

> **Simulador interactivo**
>
> En la versión web de este capítulo hay un simulador donde podés elegir la frecuencia de rebalanceo y el costo de cada operación, y ver el error de cobertura en 300 escenarios.

19.3. Cuando el gamma muerde: cerca del strike, cerca del vencimiento

El talón de Aquiles de la cobertura delta es la zona donde el gamma explota: **strike cercano y poco tiempo restante** (el pico puntiagudo del capítulo anterior). Ahí el delta salta de casi-0 a casi-1 con movimientos chicos del precio, y ninguna frecuencia razonable de rebalanceo alcanza.

```
# Un escenario malicioso: el precio termina oscilando alrededor del strike
dias_final = 20
S_pin = np.concatenate([
    np.linspace(100, 99.5, n_dias + 1 - dias_final),          # tranquilo hasta el f
    100 + 1.5 * np.sin(np.linspace(0, 6 * np.pi, dias_final)) # y baila sobre el str
])

deltas_pin = [bs_delta(S_pin[t], K, max(T - t * dt, 1e-9), r, sigma)
              for t in range(n_dias + 1)]

fig, ax = plt.subplots(figsize=(9, 4))
ax2 = ax.twinx()
ax.plot(S_pin[-40:], color="#1e3a5f", label="Precio")
ax.axhline(K, color="#94a3b8", linestyle="--")
ax2.plot(deltas_pin[-40:], color="#c2410c", label="Delta")
```

```
ax.set_xlabel("Últimos 40 días")
ax.set_ylabel("Precio ($)", color="#1e3a5f")
ax2.set_ylabel("Delta", color="#c2410c")
ax.set_title("Pin risk: el delta se vuelve loco cuando el precio baila sobre el stri
plt.tight_layout()
plt.show()
```

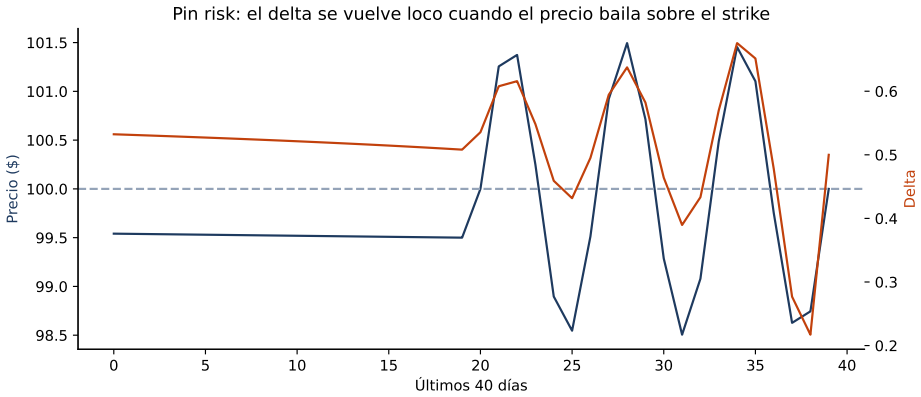


Figura 19.7.: Pin risk en los últimos 40 días: el precio oscila alrededor del strike justo antes del vencimiento, y el delta salta entre casi 0 y casi 1 con cada cruce.

En esos últimos días de la Figura 19.7, el trader compra arriba y vende abajo una y otra vez, sangrando en cada vuelta — el famoso **pin risk**. La respuesta profesional es cubrir también la curvatura: el **delta-gamma hedging** agrega *otra opción* a la cartera para neutralizar el gamma (las acciones solas no pueden: su gamma es cero). Es la puerta de entrada a la gestión de books de opciones reales.

19.4. Errores comunes

⚠ Trampas frecuentes

- **Olvidar el interés del cash**: en horizontes largos, el carry del cash es parte material del resultado; omitirlo sesga el error de cobertura.
- **Rebalancear con el delta de ayer**: el delta debe calcularse con el precio y el tiempo *de hoy*, antes de operar. Desfasar un día introduce un error sistemático.
- **Confundir el P&L del hedge con ganancia de trading**: el objetivo del cubridor es error ~0; un P&L grande (a favor o en contra) significa que la cobertura falló o que la vol realizada difirió de la cotizada.
- **Ignorar el caso T→0 en las funciones**: `bs_delta` con `T=0` divide por cero si no se maneja aparte; en el vencimiento el delta es un escalón.
- **Simular con drift para valuar**: la réplica funciona con cualquier drift (esa es la magia), pero el mark-to-market usa el precio Black-Scholes, que vive en el mundo neutral al riesgo.

19.5. Practicá en el navegador

Podés experimentar con el código del capítulo acá mismo: la celda ejecuta Python **en tu navegador**, sin instalar nada (la primera ejecución tarda unos segundos mientras carga el entorno; los paquetes que importes se instalan solos).

```
#| label: practica-navegador
import numpy as np
from scipy.stats import norm

# Tu código acá...
```

19.6. Referencias

20. Capítulo 20: Modelos de Volatilidad Estocástica

Heston y SABR: cuando la volatilidad tiene vida propia

Black-Scholes trata a la volatilidad como una constante; el mercado la cotiza distinta para cada strike y cada plazo. La salida es promover a la volatilidad a proceso estocástico: en Heston, la varianza sigue su propia dinámica con reversión a la media y correlación negativa con el precio; en SABR, la industria encontró su lenguaje estándar para interpolar sonrisas. Simulamos ambos, vemos cómo generan la sonrisa que Black-Scholes no puede producir, y cerramos el arco completo del libro.

Introducción

Cerramos el libro atacando la última gran mentira piadosa de Black-Scholes. En el Capítulo 18 vimos la evidencia: la **sonrisa de volatilidad**. Si la volatilidad fuera constante como supone el modelo, la vol implícita sería la misma en todos los strikes — y no lo es, en ningún mercado de opciones del planeta.

En el Capítulo 14 arreglamos una parte del problema agregando **saltos** al precio. Pero hay otro hecho estilizado que los saltos solos no explican: la volatilidad **cambia en el tiempo, y de manera persistente**. Hay semanas tranquilas y meses de pánico; la volatilidad se agrupa en rachas (*volatility clustering*), tiende a volver a un nivel normal cuando se dispara, y —el detalle crucial— **sube cuando el mercado cae**.

La respuesta natural: si la volatilidad se comporta como un proceso aleatorio… **modelémosla como un proceso aleatorio**. Eso son los modelos de volatilidad estocástica: dos procesos acoplados, uno para el precio y otro para su volatilidad.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from scipy.stats import norm

rng = np.random.default_rng(42)
```

20.1. La evidencia: la volatilidad no se queda quieta

Antes de los modelos, los hechos. Tres rasgos empíricos que cualquier serie financiera larga exhibe:

1. **Clustering**: los días agitados vienen en rachas; la volatilidad de mañana se parece a la de hoy.
2. **Reversión a la media**: los picos de pánico (VIX en 80) no duran; la vol vuelve a su zona normal.
3. **Efecto apalancamiento** (*leverage effect*): las caídas del precio vienen acompañadas de subas de volatilidad — la correlación entre retornos y cambios de vol es **negativa**.

```
# Ilustración con datos simulados de un proceso con clustering (GARCH-like)
n = 1500
vol = np.empty(n); vol[0] = 0.015
ret = np.empty(n)
for t in range(1, n):
    # la varianza de hoy hereda de ayer y reacciona al shock de ayer
    vol[t] = np.sqrt(0.02 * 0.015**2 + 0.90 * vol[t-1]**2 + 0.08 * ret[t-1]**2)
    ret[t] = vol[t] * rng.standard_normal()

fig, axes = plt.subplots(2, 1, figsize=(10, 5), sharex=True)
axes[0].plot(ret, linewidth=0.6)
axes[0].set_ylabel("Retorno diario")
axes[0].set_title("Volatility clustering: la agitación viene en rachas")
axes[1].plot(vol * np.sqrt(252) * 100, color="#c2410c", linewidth=1)
axes[1].set_ylabel("Vol anualizada (%)")
axes[1].set_xlabel("Día")
for ax in axes:
    ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

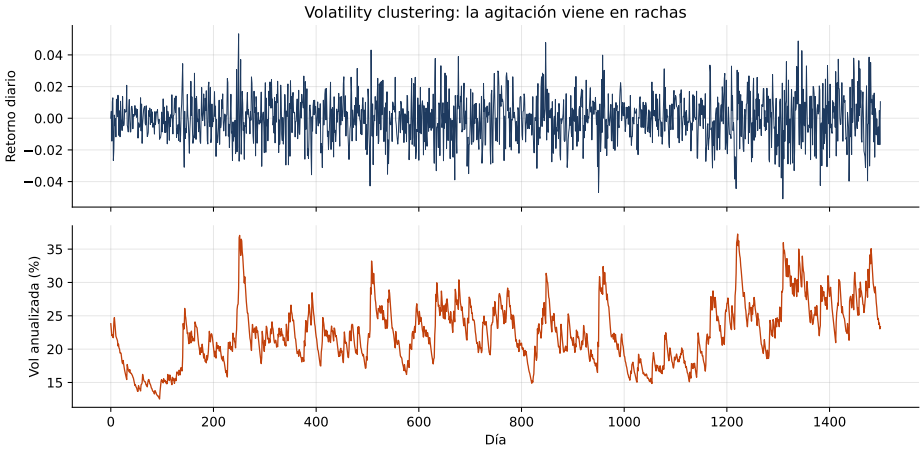


Figura 20.1.: Una serie con volatilidad que se agrupa. Arriba, los retornos diarios: hay tramos tranquilos y tramos agitados. Abajo, la volatilidad que los generó, que sube de golpe y vuelve despacio.

El GBM no puede producir el panel de arriba de la Figura 20.1: su volatilidad es una línea horizontal. Necesitamos que σ tenga su propia ecuación.

20.2. El modelo de Heston

Steven Heston (1993) propuso el modelo de volatilidad estocástica más usado de la historia, porque combina realismo con una virtud práctica enorme (tiene fórmulas semi-cerradas para valuar opciones). Son **dos procesos acoplados** — el precio y su varianza $v_t = \sigma_t^2$:

$$
\begin{aligned}
dS &= \mu S\,dt + \sqrt{v}\,S\,dW^S \\
dv &= \kappa(\theta - v)\,dt + \xi\sqrt{v}\,dW^v
\end{aligned}
\qquad \text{con } \operatorname{corr}(dW^S, dW^v) = \rho
$$

Cada parámetro tiene una interpretación económica limpia:

Parámetro	Nombre	Interpretación
θ	varianza de largo plazo	El "nivel normal" al que la varianza siempre vuelve
κ	velocidad de reversión	Cuán rápido vuelve ($\kappa = 2$: vida media 4 meses)
ξ	vol de la vol	Cuán violentos son los espasmos de la volatilidad
ρ	correlación	El efecto apalancamiento; en equity, $\rho < 0$ siempre
v_0	varianza inicial	Dónde arranca la vol hoy

La ecuación de la varianza es un proceso **CIR** (Cox-Ingersoll-Ross, el mismo que modela tasas de interés): el término $\kappa(\theta - v)\,dt$ la empuja hacia θ (reversión), y el $\sqrt{v}$ del ruido evita que se haga negativa.

La Figura 20.2 muestra cómo están conectadas las piezas: dos fuentes de ruido correlacionadas, una que mueve la varianza y otra que mueve el precio, con la varianza alimentando el tamaño de los movimientos del precio.

La parte determinística de la ecuación de la varianza se puede resolver a mano: el valor esperado es $E[v_t] = \theta + (v_0 - \theta)\,e^{-\kappa t}$. La Figura 20.3 lo

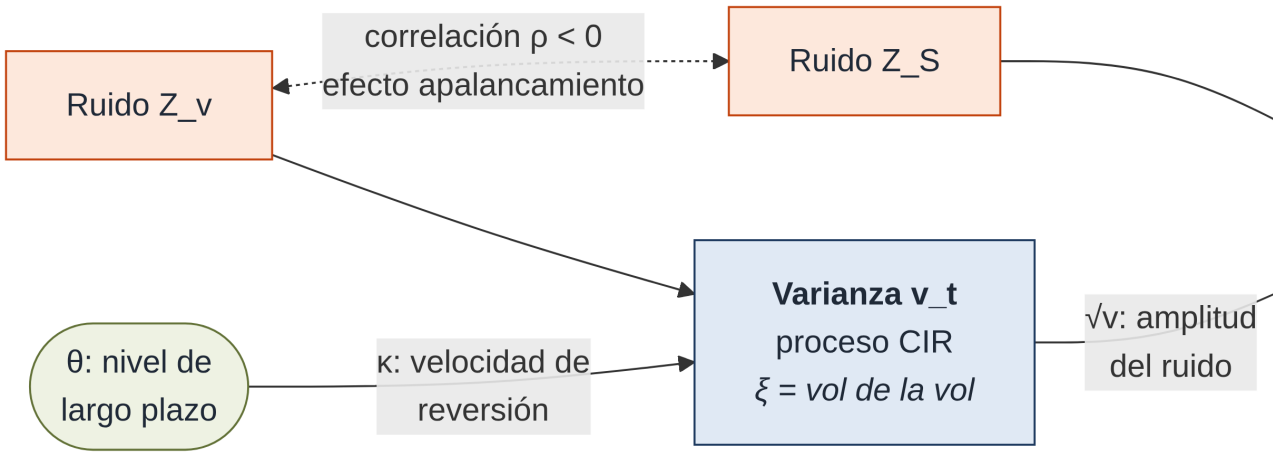


Figura 20.2.: El acople de Heston. El ruido Z_v mueve la varianza, que tira siempre hacia ; la raíz de la varianza fija la amplitud de los movimientos del precio. Como Z_S y Z_v están correlacionados (< 0), cuando el precio cae la varianza tiende a subir.

dibuja: desde cualquier punto de partida, la varianza esperada vuelve a θ, y κ decide qué tan rápido.

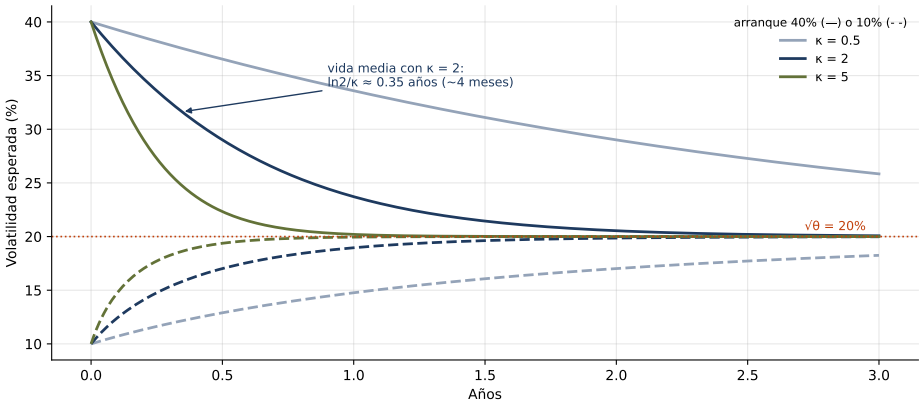


Figura 20.3.: Reversión a la media de la varianza esperada en Heston, expresada como volatilidad. Arrancando con la vol en 40 % (pánico) o en 10 % (calma), todas las curvas vuelven al nivel de largo plazo $\sqrt{\ } = 20\,\%$. Con $\ = 2$, la mitad de la distancia se recorre en ln 2 / 0.35 años, unos 4 meses.

20.2.1. Simulación

```python
def simular_heston(S0, v0, mu, kappa, theta, xi, rho, T, n_pasos, n_sims, rng):
    """Simula Heston con esquema de Euler (full truncation para v>=0)."""
    dt = T / n_pasos
    S = np.empty((n_pasos + 1, n_sims)); S[0] = S0
    v = np.empty((n_pasos + 1, n_sims)); v[0] = v0

    for t in range(1, n_pasos + 1):
        # Dos ruidos correlacionados: Z_v = rho*Z_s + sqrt(1-rho²)*Z_perp
```

```
        Z_s = rng.standard_normal(n_sims)
        Z_v = rho * Z_s + np.sqrt(1 - rho**2) * rng.standard_normal(n_sims)

        v_pos = np.maximum(v[t-1], 0)                    # truncamos en 0
        v[t] = v[t-1] + kappa * (theta - v_pos) * dt + xi * np.sqrt(v_pos * dt) * Z_
        S[t] = S[t-1] * np.exp((mu - 0.5 * v_pos) * dt + np.sqrt(v_pos * dt) * Z_s)

    return S, np.maximum(v, 0)

# Parámetros típicos de un índice de acciones
params = dict(S0=100, v0=0.04, mu=0.05, kappa=2.0, theta=0.04,
              xi=0.5, rho=-0.7, T=1.0, n_pasos=252)

S_h, v_h = simular_heston(**params, n_sims=5000, rng=rng)

print(f"Vol inicial:            {np.sqrt(params['v0']):.0%}")
print(f"Vol de largo plazo ( ): {np.sqrt(params['theta']):.0%}")
print(f"Vol promedio simulada:  {np.sqrt(v_h.mean()):.1%}")
print(f"Correlación ret-Δvol:   "
      f"{np.corrcoef(np.diff(np.log(S_h[:,0])), np.diff(v_h[:,0]))[0,1]:.2f} ( ={para
```

```
Vol inicial:            20%
Vol de largo plazo ( ): 20%
Vol promedio simulada:  19.9%
Correlación ret-Δvol:   -0.70 ( =-0.7)
```

La correlación que imprime la última línea es el efecto apalancamiento medido día a día. La Figura 20.4 la muestra punto por punto: los días en que el precio cae son, en promedio, días en que la varianza sube.

```
fig, axes = plt.subplots(2, 1, figsize=(10, 6), sharex=True)
```

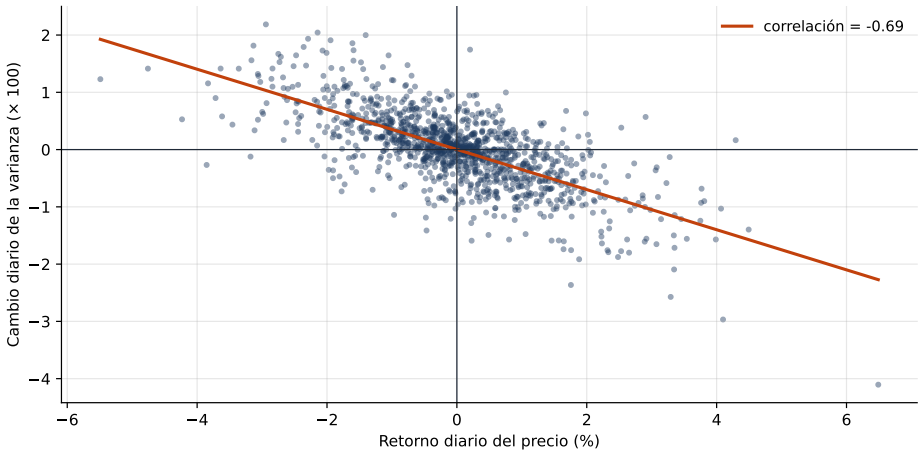


Figura 20.4.: Efecto apalancamiento en la simulación de Heston: retorno diario contra cambio diario de la varianza, en cinco trayectorias. La pendiente negativa es trabajando: caídas del precio y subas de la volatilidad vienen juntas.

```
axes[0].plot(S_h[:, :5], linewidth=1)
axes[0].set_ylabel("Precio ($)")
axes[0].set_title("Heston: el precio y su volatilidad, acoplados")
axes[1].plot(np.sqrt(v_h[:, :5]) * 100, linewidth=1)
axes[1].axhline(np.sqrt(params["theta"]) * 100, color="black", linestyle="--",
                alpha=0.6, label="Nivel de largo plazo √θ")
axes[1].set_ylabel("Volatilidad (%)")
axes[1].set_xlabel("Día")
axes[1].legend()
for ax in axes:
    ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

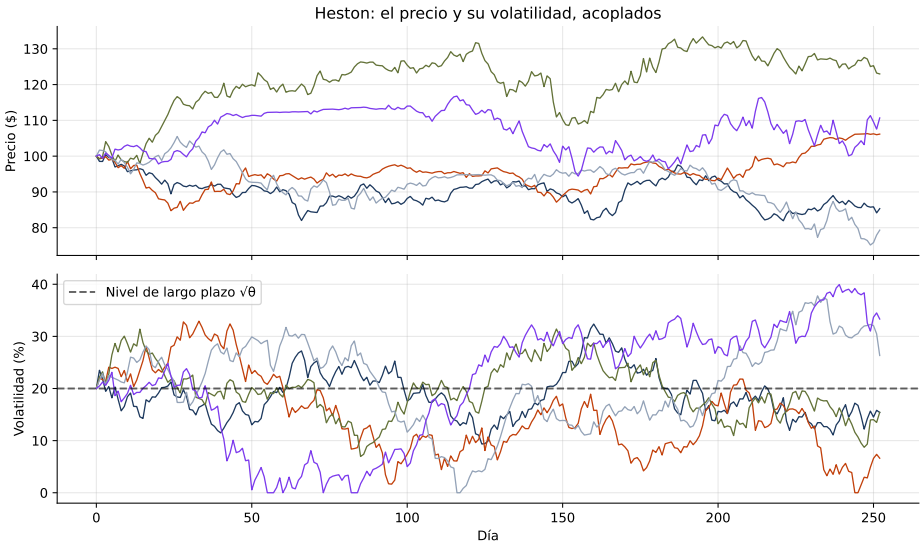


Figura 20.5.: Cinco trayectorias de Heston. Arriba, el precio; abajo, su volatilidad, cada una con su propia historia de calma y turbulencia alrededor del nivel de largo plazo.

En la Figura 20.5, compará las trayectorias de abajo con la línea horizontal que sería la vol del GBM: acá cada camino tiene su propia historia de calma y turbulencia, siempre gravitando alrededor de $\sqrt{\theta}$. Y mirá las trayectorias del precio junto a las de su vol: cuando el precio cae, su vol tiende a dispararse — el ρ negativo trabajando.

20.2.2. Heston genera la sonrisa

La prueba de fuego: valuemos opciones bajo Heston (por Monte Carlo, con drift r) y preguntémosle a Black-Scholes qué volatilidad implícita "ve" en cada strike. Si el modelo es bueno, debería aparecer la sonrisa del Capítulo 18 — sin que se la hayamos puesto a mano.

```
r = 0.03

# Simulación neutral al riesgo (drift = r) con muchas trayectorias
S_rn, _ = simular_heston(**{**params, "mu": r}, n_sims=100_000,
                         rng=np.random.default_rng(7))
ST = S_rn[-1]

def bs_call(S, K, T, r, sigma):
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d1 - sigma * np.sqrt(T))

def vol_implicita(precio, S, K, T, r):
    """Vol implícita por bisección (robusta en todos los strikes)."""
    lo, hi = 1e-4, 3.0
    for _ in range(80):
        mid = 0.5 * (lo + hi)
        if bs_call(S, K, T, r, mid) < precio:
            lo = mid
        else:
            hi = mid
```

```
    return mid

strikes = np.arange(70, 131, 5)
smile = []
for K in strikes:
    precio_heston = np.exp(-r * params["T"]) * np.maximum(ST - K, 0).mean()
    smile.append(vol_implicita(precio_heston, params["S0"], K, params["T"], r))

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(strikes, np.array(smile) * 100, "o-", color="#1e3a5f",
        label="Vol implícita de precios Heston")
ax.axhline(np.sqrt(params["theta"]) * 100, color="#c2410c", linestyle="--", alpha=0.
           label="Vol 'constante' de referencia (√ )")
ax.set_xlabel("Strike K")
ax.set_ylabel("Volatilidad implícita (%)")
ax.set_title("Heston produce la sonrisa (smirk) por sí solo")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

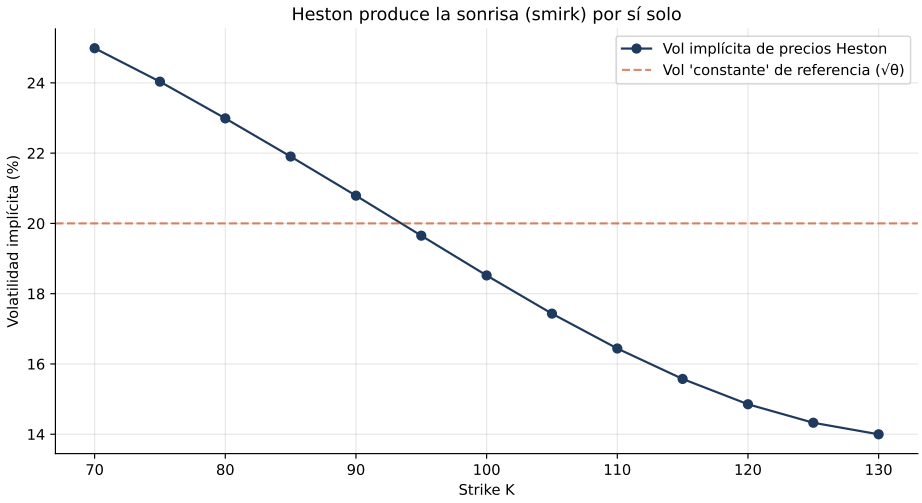


Figura 20.6.: Volatilidad implícita por strike de los precios que genera Heston (100.000 trayectorias, 1 año). Sin haberle impuesto ninguna sonrisa, el modelo produce la mueca típica de las acciones.

Ahí está, en la Figura 20.6: la curva desciende de izquierda a derecha —el *smirk* de los mercados de acciones— generada **endógenamente** por la dinámica del modelo. La mecánica: el ρ negativo hace que los escenarios de caída vengan con volatilidad alta, engordando la cola izquierda de la distribución de S_T; las puts de strikes bajos pagan justo ahí, valen más, y su vol implícita sube. La forma de la sonrisa es la huella digital de los parámetros: ρ controla la **inclinación**, ξ la **curvatura**, y κ, θ, v_0 el nivel y su estructura temporal.

La Figura 20.7 muestra esa mecánica directamente sobre los precios simulados: la distribución de retornos de Heston frente a una normal con la misma media y el mismo desvío.

En la práctica profesional, el flujo es el inverso al nuestro: se **calibra** — se

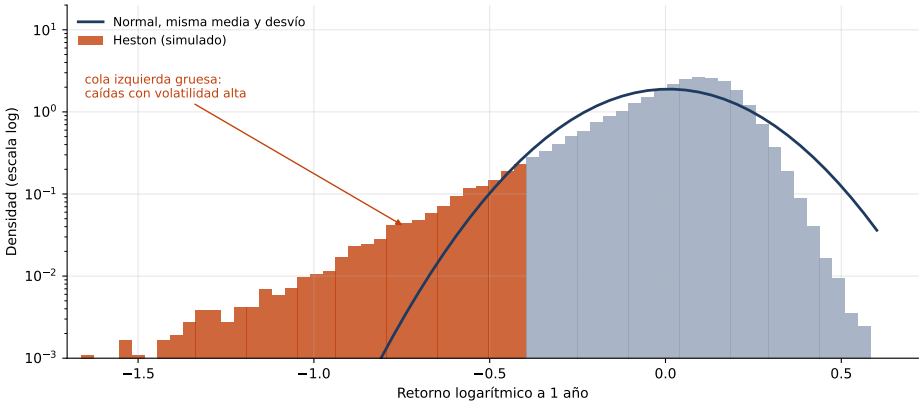


Figura 20.7.: Distribución del retorno logarítmico a un año bajo Heston (100.000 trayectorias neutrales al riesgo) contra una normal con la misma media y desvío, en escala logarítmica. La cola izquierda de Heston es mucho más gruesa: los escenarios de caída llegan con volatilidad alta. Ahí pagan las puts de strike bajo, y por eso su vol implícita sube.

buscan los parámetros de Heston que reproducen los precios de opciones observados hoy — y con el modelo calibrado se valúan productos exóticos, se calculan griegas consistentes con la sonrisa y se arman coberturas.

20.3. SABR: el estándar de la industria para la sonrisa

En los mercados de tasas de interés (swaptions, caps/floors), el modelo dominante es otro: **SABR** (*Stochastic Alpha, Beta, Rho*; Hagan et al., 2002):

$$\begin{aligned} dF &= \alpha F^{\beta}\, dW^{F} \\ d\alpha &= \nu\, \alpha\, dW^{\alpha} \end{aligned} \qquad \operatorname{corr}(dW^{F}, dW^{\alpha}) = \rho$$

donde F es el precio *forward* y α su volatilidad. Diferencias de diseño respecto de Heston: la vol es **lognormal y sin reversión** (más simple), y el exponente $\beta \in [0, 1]$ interpola entre dinámica normal ($\beta = 0$, usada en tasas) y lognormal ($\beta = 1$, como acciones).

¿Por qué conquistó la industria un modelo *menos* realista que Heston? Por una razón práctica imbatible: Hagan y coautores derivaron una **fórmula aproximada cerrada** para la volatilidad implícita en función de $(K, F, \alpha, \beta, \rho, \nu)$. Sin simulaciones, sin transformadas: la sonrisa entera en una expresión evaluable al instante. SABR no se usa tanto para *simular* mercados como para **describir e interpolar sonrisas**: con 3-4 parámetros ajustás los puntos cotizados y leés cualquier strike intermedio.

```
def sabr_vol(K, F, T, alpha, beta, rho, nu):
    """Vol implícita SABR (aprox. de Hagan 2002), vectorizada en K."""
    K = np.atleast_1d(K).astype(float)
    vol = np.empty_like(K)
    atm = np.isclose(K, F)

    FK_beta = (F * K) ** ((1 - beta) / 2)
    logFK = np.log(F / K, where=~atm, out=np.zeros_like(K))
    z = (nu / alpha) * FK_beta * logFK
    x_z = np.log((np.sqrt(1 - 2 * rho * z + z**2) + z - rho) / (1 - rho),
                 where=~atm, out=np.ones_like(K))

    # término de corrección temporal (común a ambos casos)
    corr = (1 + T * ((1 - beta)**2 / 24 * alpha**2 / (F * K) ** (1 - beta)
                     + rho * beta * nu * alpha / (4 * FK_beta)
                     + (2 - 3 * rho**2) / 24 * nu**2))

    vol = np.where(
        atm,
        alpha / F ** (1 - beta) * corr,
        alpha / (FK_beta * (1 + (1 - beta)**2 / 24 * logFK**2
                            + (1 - beta)**4 / 1920 * logFK**4)) * (z / x_z) * corr,
    )
    return vol

F, T_s = 100, 1.0
strikes_s = np.linspace(70, 130, 60)

fig, ax = plt.subplots(figsize=(9, 5))
escenarios = [
    (-0.6, 0.4, " =-0.6,  =0.4 (equity: smirk)"),
    (0.0, 0.4, " =0,  =0.4 (sonrisa simétrica)"),
    (0.0, 0.1, " =0,  =0.1 (vol de vol baja: casi plana)"),
```

```
]
for rho_i, nu_i, etiqueta in escenarios:
    vols = sabr_vol(strikes_s, F, T_s, alpha=0.20, beta=1.0, rho=rho_i, nu=nu_i)
    ax.plot(strikes_s, vols * 100, linewidth=2, label=etiqueta)

ax.set_xlabel("Strike K")
ax.set_ylabel("Volatilidad implícita (%)")
ax.set_title("SABR: cada parámetro esculpe la sonrisa")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

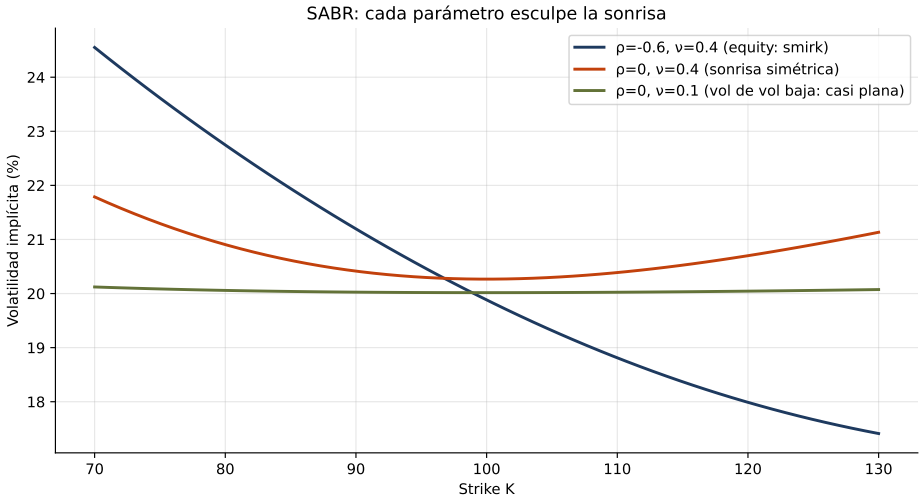


Figura 20.8.: Sonrisas de SABR (= 1, = 20 %) para tres combinaciones de parámetros: inclina la curva y la curva.

La Figura 20.8 es el manual de instrucciones del modelo: ν (vol de vol) **curva** la sonrisa, ρ la **inclina**, y α fija el nivel. Un operador de swaptions

calibra estos parámetros a las cotizaciones de la mañana y tiene toda la superficie descripta para el resto del día.

Simulador interactivo

En la versión web de este capítulo hay un simulador donde podés mover los parámetros de SABR y ver cómo cambian la sonrisa y la densidad que implica, incluido el punto en que la aproximación deja de ser libre de arbitraje.

¿Heston o SABR? El criterio profesional

No compiten: se usan para cosas distintas. **SABR** es el estándar para *cotizar e interpolar* sonrisas (sobre todo en tasas) por su fórmula instantánea. **Heston** es el caballo de batalla para *simular y valuar* productos dependientes de la trayectoria (exóticos, estructurados) porque tiene una dinámica temporal completa y consistente. Y la familia sigue: local vol (Dupire), modelos local-estocásticos (LSV), rough volatility — la frontera actual de la investigación.

20.4. El arco completo del libro

Detengámonos un momento en el camino recorrido, porque este capítulo lo cierra:

1. **GBM** (Cap. 13): precio aleatorio, vol constante. Elegante, insuficiente.
2. **Saltos** (Cap. 14): el precio puede moverse discontinuamente → colas pesadas.

3. **La sonrisa** (Cap. 18): el mercado cotiza las violaciones de Black-Scholes, cada día, en cada strike.
4. **Volatilidad estocástica** (este capítulo): la vol tiene dinámica propia → clustering, leverage effect, y la sonrisa reproducida desde los fundamentos.

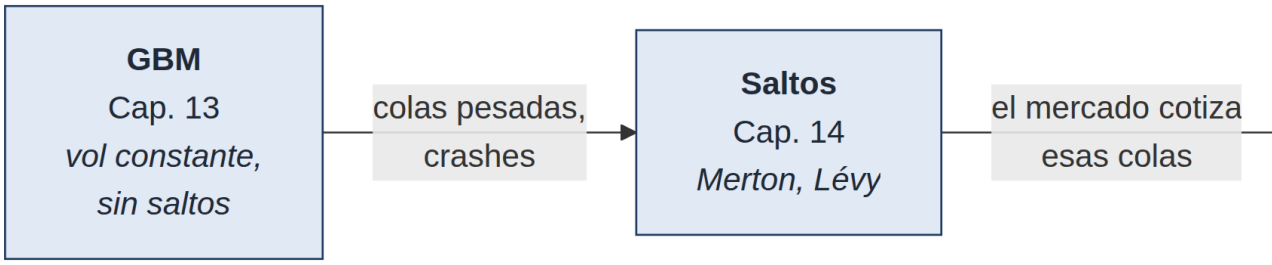


Figura 20.9.: El arco de los modelos de precio del libro. Cada flecha es un hecho empírico que el modelo anterior no podía explicar y que obligó a construir el siguiente.

Cada modelo nace de un hecho empírico que el anterior no explicaba. Esa es la lección metodológica más importante que este libro puede darte: **los modelos no son verdades, son herramientas con dominio de validez** — y el trabajo del analista cuantitativo es conocer ese dominio, medir lo que queda afuera (Módulo 3 entero) y elegir la herramienta adecuada para cada pregunta.

20.5. Errores comunes

⚠ Trampas frecuentes

- **Varianza negativa en la simulación de Heston**: el esquema de Euler ingenuo produce $v < 0$ y `sqrt` explota. Usá *full truncation* (como en el código) o esquemas exactos (QE de Andersen).
- **Ignorar la condición de Feller**: si $2\kappa\theta < \xi^2$, la varianza toca cero con probabilidad positiva y la simulación exige más cuidado (nuestros parámetros la violan levemente — es lo habitual al calibrar equity — y por eso el truncamiento importa).
- **Usar la fórmula de Hagan fuera de su dominio**: es una aproximación asintótica; con T grande, ν alto o strikes extremos puede dar vols absurdas (incluso densidades negativas). Verificá contra Monte Carlo.
- **Calibrar demasiados parámetros a pocos datos**: con 5 strikes cotizados y 5 parámetros libres, el ajuste es perfecto y no significa nada — el sobreajuste del Capítulo 17 aplica también acá.
- **Olvidar el drift neutral al riesgo al valuar**: para pricing, el drift del precio es r; el μ real solo importa para proyecciones.

20.6. Practicá en el navegador

Podés experimentar con el código del capítulo acá mismo: la celda ejecuta Python **en tu navegador**, sin instalar nada (la primera ejecución tarda unos segundos mientras carga el entorno; los paquetes que importes se instalan solos).

```
#| label: practica-navegador
import numpy as np

rng = np.random.default_rng(42)

# Tu código acá...
```

20.7. Referencias

A. Introducción a Python y Puesta en Marcha

Python es un lenguaje de programación de alto nivel, interpretado y de propósito general que se ha convertido en el estándar de facto para las finanzas computacionales y la ciencia de datos. Este apéndice tiene dos partes: una **guía práctica** para dejar tu entorno funcionando (lo que necesitás antes del Capítulo 1), y una explicación breve de **qué es y cómo funciona** el lenguaje.

A.1. Parte 1: Poner a punto tu entorno

Hay tres caminos, del más simple al más profesional. Si estás empezando, el primero alcanza de sobra.

A.1.1. Opción A: Google Colab (sin instalar nada)

Google Colab es un Jupyter Notebook que corre en la nube: entrás con tu cuenta de Google y ya podés ejecutar Python en el navegador. Las librerías del libro (`numpy`, `pandas`, `scipy`, `matplotlib`, `statsmodels`) vienen preinstaladas.

- **Ventajas**: cero instalación, funciona igual en cualquier máquina, fácil de compartir.

- **Desventajas**: necesita internet, y las sesiones se reinician (guardá tu trabajo en Drive).

Es la opción recomendada para las primeras semanas del curso.

A.1.2. Opción B: Instalación local con Python oficial

Para trabajar en tu máquina, sin depender de internet:

1. **Instalar Python**: descargalo de python.org/downloads (versión 3.11 o superior). En Windows, marcá la casilla *"Add Python to PATH"* durante la instalación.
2. **Crear un entorno virtual**. Un *venv* es una instalación de librerías aislada por proyecto — evita que los paquetes de un proyecto rompan otro. Desde la terminal (PowerShell en Windows, Terminal en Mac/Linux):

```
# Crear el entorno (una sola vez)
python -m venv .venv

# Activarlo (cada vez que abras una terminal nueva)
# Windows:
.venv\Scripts\activate
# Mac/Linux:
source .venv/bin/activate
```

3. **Instalar las librerías del libro**:

```
pip install numpy pandas scipy matplotlib statsmodels jupyter
```

4. **Abrir Jupyter** y empezar a trabajar:

```
jupyter notebook
```

A.1.3. Opción C: VS Code (el flujo profesional)

Visual Studio Code con la extensión oficial de Python combina lo mejor de ambos mundos: notebooks interactivos + un editor serio con autocompletado y debugging. Instalá VS Code, agregá las extensiones **Python** y **Jupyter**, y abrí cualquier archivo `.ipynb`. VS Code te va a pedir elegir el intérprete: seleccioná el del venv que creaste en la Opción B.

A.1.4. Cómo ejecutar el código del libro

Los bloques de código del libro son de dos tipos, y se distinguen a simple vista:

- **Bloques interactivos** (capítulos iniciales): tienen un botón *"Run code"* y se ejecutan **directamente en tu navegador** — podés editarlos y volver a correrlos ahí mismo, sin instalar nada. Usalos para experimentar mientras leés.
- **Bloques con salida estática** (capítulos con cómputo pesado): muestran el código y su resultado ya ejecutado. Para reproducirlos, copiá el código a tu entorno (Colab, Jupyter o VS Code).

> **El hábito que más rinde**
>
> No leas el código: **ejecutalo**. Cambiá un número, rompé algo a propósito, mirá el error, arreglalo. Veinte minutos de experimentación valen más que dos horas de lectura pasiva.

A.2. Parte 2: ¿Qué es Python?

Python fue creado por Guido van Rossum y lanzado por primera vez en 1991. Su filosofía de diseño enfatiza la legibilidad del código, con su notable uso de sangría significativa.

A.2.1. Características Principales

- **Sintaxis Clara:** Python es conocido por ser fácil de leer y escribir — el código bien escrito se lee casi como pseudocódigo.
- **Interpretado:** El código se ejecuta línea por línea, lo que facilita la experimentación y la depuración.
- **Tipado Dinámico:** No es necesario declarar el tipo de una variable al crearla.
- **Ecosistema de Librerías:** Cuenta con una colección enorme de librerías para finanzas y ciencia de datos (`pandas`, `numpy`, `scipy`, `matplotlib`, `statsmodels`), que son precisamente las que usa este libro.

A.2.2. ¿Cómo funciona Python?

Cuando ejecutás un programa en Python, ocurren varios procesos bajo el capó:

1. **Código Fuente (`.py`):** Escribís tus instrucciones en archivos de texto.
2. **Compilación a Bytecode:** Python traduce tu código a una forma intermedia llamada bytecode (archivos `.pyc`).
3. **Máquina Virtual de Python (PVM):** El bytecode es ejecutado por el intérprete, que traduce estas instrucciones a código de máquina que el procesador entiende.

A diferencia de los lenguajes compilados (como C++), donde el código se traduce completamente antes de ejecutarse, Python traduce y ejecuta en tiempo de ejecución. Esto permite un flujo de trabajo muy interactivo — ideal para el análisis de datos financieros, donde explorás, probás y ajustás constantemente.

A.2.3. ¿Por qué Python en Finanzas?

La simplicidad de Python permite a los profesionales de finanzas concentrarse en resolver problemas de modelado y análisis, en lugar de pelear con el lenguaje. A eso se suma el efecto red: los bancos, fondos y fintechs del mundo estandarizaron sus stacks cuantitativos sobre Python, así que las librerías, la documentación y las respuestas en foros están a un buscador de distancia. Cuando en el Capítulo 5 veas NumPy haciendo en milisegundos lo que un bucle tarda segundos, vas a entender por qué esta combinación —lenguaje simple, librerías rápidas— conquistó el sector.

B. Obtención de Datos Financieros

A lo largo del libro trabajamos sobre todo con datos simulados: son reproducibles, no dependen de internet y dejan el foco en los conceptos. Los complementa el dataset real congelado del repositorio (`datos/precios.csv`, descripto más abajo), que alimenta las secciones "Con datos reales". Pero tarde o temprano vas a querer tus propios datos — y en el trabajo profesional, conseguir datos limpios es la mitad del oficio. Este apéndice presenta las tres fuentes que cubren el 95 % de las necesidades: **yfinance** (precios de mercado, gratis), **FRED** (macro, gratis) y **Bloomberg** (el estándar institucional, pago).

> ⚠ Los bloques de este apéndice no se ejecutan en el libro
>
> A diferencia del resto del libro, el código de este apéndice está marcado como **no ejecutable** (`eval: false`): requiere conexión a internet, claves de API o —en el caso de Bloomberg— una terminal con licencia. Las salidas que se muestran son ejemplos ilustrativos. Copiá el código a tu entorno para correrlo de verdad.

B.1. El patrón profesional: descargar una vez, trabajar en local

Antes de las herramientas, el hábito. El error típico del principiante es poner la descarga **adentro** del análisis: cada corrida vuelve a pedir los datos a la API — lento, frágil (la API puede fallar o cambiar) e **irreproducible**

(los datos de hoy no son los de ayer si el proveedor corrige algo). El patrón correcto separa las dos etapas:

1. **Script de descarga**: pide los datos y los guarda en disco (CSV o Parquet).
2. **Análisis**: lee siempre del archivo local (`pd.read_csv`, como en el Capítulo 6).

```
# --- 1) descargar_datos.py: se corre UNA vez (o cuando querés actualizar) ---
import yfinance as yf

datos = yf.download("AAPL", start="2020-01-01", end="2024-12-31", auto_adjust=True)
datos.to_csv("datos/aapl.csv")

# --- 2) analisis.py: lee del disco, reproducible siempre ---
import pandas as pd
precios = pd.read_csv("datos/aapl.csv", index_col="Date", parse_dates=True)
```

Beneficios: tus resultados son reproducibles (el archivo no cambia), el análisis corre sin internet, y no castigás a la API gratuita con cien pedidos idénticos por día.

B.2. El dataset del libro: `datos/precios.csv`

El libro practica lo que predica: aplicamos este patrón una vez y **congelamos el resultado en el repositorio**, en `datos/precios.csv`. Es el archivo que usan las secciones "Con datos reales" de los capítulos 8, 9, 11 y 17 — todas trabajan sobre los mismos datos, y cualquiera puede reproducirlas sin claves de API ni conexión.

Qué contiene:

- **Precios diarios ajustados** (dividendos y splits incluidos — el retorno total del que habla el Capítulo 8), redondeados a 4 decimales.
- **Período**: julio 2021 a junio 2026 (~1250 ruedas). Incluye el bear market de 2022, el ciclo de suba de tasas y el boom de la IA — cinco años con de todo.
- **10 tickers** elegidos para cubrir sectores bien distintos: tecnología (AAPL, MSFT, NVDA), financiero (JPM), energía (XOM), consumo defensivo (KO, JNJ, WMT), medios (DIS), más el ETF **SPY** como proxy del mercado.

Para cargarlo, desde cualquier capítulo del libro (o desde tu clon del repositorio):

```
import pandas as pd

precios = pd.read_csv("datos/precios.csv", index_col="Date", parse_dates=True)
retornos = precios.pct_change().dropna()
```

El archivo se regeneró por última vez en **julio de 2026** con `yfinance` y `auto_adjust=True`, siguiendo exactamente el patrón de dos etapas de arriba. Si querés actualizarlo o cambiar el universo, el script es el de la sección anterior con la lista de tickers completa — pero recordá que al hacerlo tus números dejarán de coincidir con los impresos en el libro: ese es justamente el costo de la frescura que el congelamiento evita.

B.3. yfinance: precios de mercado, gratis

`yfinance` descarga datos de Yahoo Finance: acciones, ETFs, índices, monedas y cripto de casi cualquier mercado del mundo. Se instala con `pip install yfinance`.

```python
import yfinance as yf

# Un ticker: precios diarios ajustados
aapl = yf.download("AAPL", start="2023-01-01", end="2024-01-01", auto_adjust=True)
print(aapl.head(3))
#                  Open        High         Low       Close    Volume
# Date
# 2023-01-03  128.782447  129.395300  122.762418  123.632286  112117500
# 2023-01-04  125.283842  126.845998  123.809612  124.907707   89113600
# 2023-01-05  125.876932  126.263764  123.276929  123.583961   80962700

# Varios tickers a la vez: devuelve columnas jerárquicas (un nivel por ticker)
panel = yf.download(["AAPL", "MSFT", "SPY"], start="2023-01-01", auto_adjust=True)
retornos = panel["Close"].pct_change().dropna()

# Dividendos y splits de un activo
ticker = yf.Ticker("AAPL")
print(ticker.dividends.tail(3))
print(ticker.splits)
```

Tres puntos que importan:

- **`auto_adjust=True`** te da precios ajustados por dividendos y splits — el *retorno total* que el Capítulo 8 mostró que es el correcto para estimar μ. Sin ajustar, subestimás sistemáticamente el retorno de los activos de alto dividendo.
- El parámetro `interval` permite otras frecuencias: `"1wk"`, `"1mo"`, e intradiario (`"1h"`, `"5m"`) con historia limitada.
- Para **tickers argentinos** en BYMA se usa el sufijo `.BA`: `"GGAL.BA"`, `"YPFD.BA"` (pesos); los ADRs (`"GGAL"`, `"YPF"`) cotizan en dólares de NY — no los mezcles sin pensar en qué moneda estás.

> ⚠ Las limitaciones de lo gratuito
>
> yfinance usa una API **no oficial** de Yahoo: puede romperse tras cambios del sitio (actualizá la librería si deja de funcionar), tiene *rate limits* (descargas masivas → bloqueos temporales), y la calidad es de "análisis personal", no de auditoría: hay huecos y correcciones ocasionales. Además, ojo con el **sesgo de supervivencia** (Capítulo 17): si armás tu universo con los tickers *de hoy*, las empresas que quebraron no están — el pasado se ve mejor de lo que fue.

B.4. FRED: datos macroeconómicos, gratis

FRED (*Federal Reserve Economic Data*, de la Fed de St. Louis) es la referencia mundial de series macro: inflación, tasas, PIB, desempleo, spreads — más de 800.000 series. Es la fuente natural para alimentar los modelos ARIMA y VAR del Capítulo 16.

La vía clásica es pandas-datareader (`pip install pandas-datareader`), que también documenta el acceso a otras fuentes públicas:

```
import pandas_datareader.data as web
from datetime import datetime

inicio, fin = datetime(2010, 1, 1), datetime(2024, 12, 31)

# Cada serie tiene un código en fred.stlouisfed.org (buscalo en el sitio)
cpi = web.DataReader("CPIAUCSL", "fred", inicio, fin)      # inflación (índice CPI)
tasa_10y = web.DataReader("DGS10", "fred", inicio, fin)    # Treasury a 10 años
desempleo = web.DataReader("UNRATE", "fred", inicio, fin)  # tasa de desempleo

# Varias series de un tiro, alineadas por fecha (la alineación del Capítulo 6)
```

```
macro = web.DataReader(["CPIAUCSL", "DGS10", "UNRATE"], "fred", inicio, fin)
print(macro.tail(3))
```

La alternativa moderna es `fredapi`, que usa la API oficial con una **clave gratuita** (se pide en el sitio de FRED en dos minutos):

```
from fredapi import Fred

fred = Fred(api_key="TU_CLAVE_GRATUITA")
inflacion = fred.get_series("CPIAUCSL")

# Consejo: nunca escribas la clave en el código; guardala en una
# variable de entorno y leela con os.environ["FRED_API_KEY"]
```

El flujo típico para el Capítulo 16: descargar el CPI, convertirlo a inflación interanual (`cpi.pct_change(12)`), guardarlo a CSV, y ajustarle un ARIMA — ahora con datos reales.

B.5. Bloomberg: el estándar institucional

En bancos, fondos y mesas de trading, la fuente es la **Terminal Bloomberg** — datos de calidad auditada, cobertura universal (renta fija, derivados OTC, curvas) y el costo correspondiente (~USD 25.000/año por terminal). Si trabajás en una institución, tu acceso a datos será este.

Cómo funciona el acceso desde Python:

- La **Desktop API** (`blpapi`, la librería oficial de Bloomberg) permite pedir datos desde Python **solo si la Terminal está corriendo y logueada en la misma máquina** — la licencia es por usuario, y la API respeta eso. No hay versión gratuita ni "clave de prueba".

- Sobre `blpapi` (que es verbosa) la comunidad construyó wrappers pythónicos: `xbbg` es el más usado hoy. Sus funciones replican las que ya conocés de Excel: `bdh` (histórico), `bdp` (dato puntual), `bds` (bulk).

```
# Requiere: Terminal Bloomberg abierta + pip install blpapi xbbg
from xbbg import blp

# BDH: serie histórica (equivalente al =BDH() de Excel)
precios = blp.bdh(
    tickers=["AAPL US Equity", "GD30 Corp"],
    flds=["PX_LAST"],
    start_date="2023-01-01",
    end_date="2024-01-01",
)

# BDP: datos puntuales (equivalente a =BDP())
info = blp.bdp("GD30 Corp", flds=["YLD_YTM_MID", "DUR_ADJ_MID"])  # TIR y duración

# BDS: datos "bulk" (p.ej., el cronograma de flujos de un bono)
flujos = blp.bds("GD30 Corp", flds="DES_CASH_FLOW")
```

Notá la sintaxis de tickers de Bloomberg: `"AAPL US Equity"`, `"GD30 Corp"` — el sufijo indica la clase de activo (*yellow key*). Y un detalle contractual que importa: la licencia restringe la **redistribución** de los datos; podés analizarlos, no republicarlos.

B.6. ¿Cuál usar? Comparación y mapa de uso

	yfinance	**FRED**	**Bloomberg**
Costo	Gratis	Gratis (clave gratuita)	~USD 25k/año
Cobertura	Acciones, ETFs, FX, cripto	Macro (EEUUU y global)	Todo, incl. renta fija y OTC
Calidad	Buena para análisis personal	Oficial (la Fed)	Auditada, estándar de la industria
Confiabilidad de API	No oficial, puede romperse	Estable	Estable (con Terminal)
Uso en este libro	El dataset `datos/precios.csv` (secciones "Con datos reales")	Llevar el ARIMA/VAR del Cap. 16 a datos reales	El mundo profesional que te espera

La regla práctica: **yfinance para aprender y prototipar, FRED para macro, Bloomberg cuando alguien más paga la licencia**. Y sea cual sea la fuente: descargá una vez, guardá a disco, y analizá en local.

B.7. Referencias